

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 1\_Concept Builder

Attempt : 2

Total Mark : 20

Marks Obtained : 16

#### Section 1 : MCQs

1. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 3;  
        System.out.println(a / b);  
    }  
}
```

**Answer**

3

**Status :** Correct

**Marks :** 1/1

2. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int count = 8;  
        count = count ^ 1;  
  
        System.out.println(count);  
    }  
}
```

**Answer**

9

**Status :** Correct

**Marks :** 1/1

3. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int x = 5;  
        int X = 10;  
  
        int sum = x + X;  
        int bitwiseResult = x | X;  
  
        System.out.println(sum);  
        System.out.println(bitwiseResult);  
    }  
}
```

**Answer**

Compilation error

**Status :** Wrong

**Marks :** 0/1

4. What will be the output of the following code?

```
import java.util.*;
```

```
class TernaryOperatorExample {  
    public static void main(String[] args) {  
        int a = 5, b = 10;  
        int result = (a > b) ? a : b;  
        System.out.println(result);  
    }  
}
```

**Answer**

10

**Status :** Correct

**Marks :** 1/1

5. What is the output of the following code?

```
import java.util.*;  
  
class RelationalOperatorExample {  
    public static void main(String[] args) {  
        int x = 8, y = 4;  
        boolean result = (x != y);  
  
        System.out.println(result);  
    }  
}
```

**Answer**

true

**Status :** Correct

**Marks :** 1/1

6. What will be the output of the following code snippet?

```
import java.util.*;  
  
class OperatorPrecedenceExample {  
    public static void main(String[] args) {  
        int a = 5, b = 3, c = 2;
```

```
int result = a + b * c;  
  
    System.out.println(result);  
}  
}
```

**Answer**

11

**Status :** Correct

**Marks :** 1/1

7. What is the result of the following expression?

```
import java.util.*;  
  
class ComplexExpressionExample {  
    public static void main(String[] args) {  
        int a = 5, b = 2, c = 3, d = 4;  
        int result = a + b * c / d - b;  
  
        System.out.println(result);  
    }  
}
```

**Answer**

4

**Status :** Correct

**Marks :** 1/1

8. What will be the output of the following Java code snippet?

```
public class Main {  
    public static void main(String[] args) {  
        int score = 75;  
        if(score >= 90) {  
            System.out.println("Grade: A");  
        } else if(score >= 80) {  
            System.out.println("Grade: B");  
        } else if(score >= 70) {
```

```
        System.out.println("Grade: C");
    } else {
        System.out.println("Grade: D");
    }
}
}
```

**Answer**

Grade: C

**Status :** Correct

**Marks :** 1/1

9. What will be the output of the following Java code snippet?

```
public class Main {
    public static void main(String[] args) {
        int day = 4;
        String result = "";
        switch(day) {
            case 1:
                result = "Monday";
                break;
            case 2:
                result = "Tuesday";
                break;
            case 3:
                result = "Wednesday";
                break;
            default:
                result = "Other Day";
        }
        System.out.println(result);
    }
}
```

**Answer**

Compilation Error

**Status :** Wrong

**Marks :** 0/1

10. What will be the output of the following code?

```
class ConditionTest {  
    public static void main(String[] args) {  
        int x = 10;  
        if (x > 5)  
            System.out.print("High");  
    }  
}
```

**Answer**

Compilation error

**Status :** Wrong

**Marks :** 0/1

11. What will be the output of the following code?

```
class ConditionTest {  
    public static void main(String[] args) {  
        int a = 7;  
        if (a == 7)  
            System.out.print("Match");  
        else  
            System.out.print("No Match");  
    }  
}
```

**Answer**

Match

**Status :** Correct

**Marks :** 1/1

12. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int a = 4, b = 5;  
        if ((a + b) % 2 == 0)  
            System.out.print("Even");  
    }  
}
```

```
    else  
        System.out.print("Odd");  
    }  
}
```

**Answer**

Compilation error

**Status : Wrong**

**Marks : 0/1**

13. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int num = 15;  
        if (num > 10)  
            if (num % 3 == 0)  
                System.out.print("Divisible");  
            else  
                System.out.print("Not Divisible");  
        }  
    }  
}
```

**Answer**

Divisible

**Status : Correct**

**Marks : 1/1**

14. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int x = 5, y = 2;  
        if (x + y == 10)  
            System.out.print("Ten");  
        else if (x - y == 3)  
            System.out.print("Three");  
        else  
            System.out.print("None");  
    }  
}
```

**Answer**

Three

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        int sum = 0;  
        for(int i = 1; i <= 5; i++) {  
            sum += i;  
        }  
        System.out.println(sum);  
    }  
}
```

**Answer**

15

**Status :** Correct

**Marks :** 1/1

16. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        int i = 1;  
        while(i < 10) {  
            i += 2;  
        }  
        System.out.println(i);  
    }  
}
```

**Answer**

11



**Status :** Correct

**Marks :** 1/1

17. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        int i = 10;  
        do {  
            System.out.print(i + " ");  
            i -= 3;  
        } while(i > 0);  
    }  
}
```

**Answer**

10 7 4 1

**Status :** Correct

**Marks :** 1/1

18. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        for(int i = 1; i <= 20; i = i * 2) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

**Answer**

1 2 4 8 16

**Status :** Correct

**Marks :** 1/1

19. What will be the output of the following code?

```
class Main {  
    public static void main(String[] args) {
```

```
for (int i = 5; i > 0; i--) {  
    System.out.print(i + " ");  
}  
}
```

**Answer**

5 4 3 2 1

**Status :** Correct

**Marks :** 1/1

20. What will be the output of the following code?

```
class LoopTest {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.print(i + " ");  
            i *= 2;  
        } while (i <= 8);  
    }  
}
```

**Answer**

1 2 4 8

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 1\_Skill Builder\_Updated

Attempt : 1

Total Mark : 50

Marks Obtained : 30

#### **Section 1 : COD**

##### **1. Problem Statement**

In a high-precision manufacturing plant, Alex is an engineer responsible for creating a specialized lubricant blend using three components: Component X, Component Y, and Component Z, mixed in specific ratios to ensure optimal performance.

The total volume of the lubricant blend must be divided such that the volume of Component Y is one-sixth of the total volume. The volume of Component X should be double that of Component Y, and the remaining volume should be allocated to Component Z.

Write a program to calculate the volumes of X, Y, and Z based on the total blend volume.

***Input Format***

The input consists of a double value  $n$ , representing the total volume of the lubricant.

### **Output Format**

The first line of output displays "Lubricant X: A ml" where A is the calculated volume of Component X, as a double value, rounded to one decimal place.

The second line displays "Lubricant Y: B ml", where B is the calculated volume of Component Y, as a double value, rounded to one decimal place.

The third line displays "Lubricant Z: C ml ", where C is the calculated volume of Component Z, as a double value, rounded to one decimal place.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 120.0

Output: Lubricant X: 40.0 ml

Lubricant Y: 20.0 ml

Lubricant Z: 60.0 ml

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        double n=sc.nextDouble();
        double y=n/6;
        double x=2*y;
        double z=n-(x+y);
        System.out.printf("Lubricant X: %.1f ml\n",x);
        System.out.printf("Lubricant Y: %.1f ml\n",y);
        System.out.printf("Lubricant Z: %.1f ml\n",z);
    }
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Gwen is studying bitwise and arithmetic operations as part of her computer science course. To practice, she wants to perform two operations on a number:

Right shift the number by 2 bits. Add 10 to the original number as part of an arithmetic test.

She needs a program that accepts an integer input, performs both operations, and displays the results.

### ***Input Format***

The input consists of an integer N, which represents the number to be processed.

### ***Output Format***

The first line of output prints an integer representing the result after right shifting the number by 2 bits.

The second line of output prints an integer representing the result after adding 10 to the original number.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 16

Output: 4

26

### ***Answer***

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int rightShift=n>>2;
        int addTen=n+10;
        System.out.println(rightShift);
```

```
System.out.println(addTen);
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Samantha is a diligent math student who is exploring the world of programming. She is learning Java and has recently studied conditional statements. One day, her teacher gives her an interesting problem to solve, which takes a number as input and checks whether it is a multiple of 5 or 7.

Help her complete the task.

#### ***Input Format***

The input consists of a single integer N, representing the number to be checked.

#### ***Output Format***

If the number is a multiple of 5 but not 7, the output prints "N is a multiple of 5".

If the number is a multiple of 7, the output prints "N is a multiple of 7".

Otherwise the output prints "N is neither multiple of 5 nor 7" where N is an entered integer.

Refer to the sample output for formatting specifications.

#### ***Sample Test Case***

Input: 10

Output: 10 is a multiple of 5

#### ***Answer***

```
import java.util.Scanner;  
public class Main {
```

```

public static void main(String[] args) {
    Scanner scanner=new Scanner(System.in);
    int N=scanner.nextInt();
    scanner.close();
    if(N%5==0 && N%7!=0) {
        System.out.println(N+"is a multiple of 5");
    } else if(N%7==0) {
        System.out.println(N+"is a multiple of 7");
    } else {
        System.out.println(N+"is neither multiple of 5 nor 7");
    }
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Arun is working on a project to automate the process of determining whether a student has passed or failed based on their subject marks.

He aims to create a simple program that takes positive integers as marks for five subjects from the user. If the average of the marks is greater than or equal to 50, the student has passed the exam. Otherwise, the student has failed.

Help Arun to implement the project.

##### ***Input Format***

The input consists of five space-separated integers, representing the marks in five subjects.

##### ***Output Format***

The first line of output prints "Average score: " followed by an integer representing the average score.

The second line prints one of the following:

1. If the condition is satisfied, print "The student has passed".
2. Otherwise, the output prints "The student has failed".

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 50 60 70 80 90

Output: Average score: 70

The student has passed

### **Answer**

```
import java.util.Scanner;
public class studentresult {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int sum=0;
        for(int i=0;i<5;i++) {
            int mark=sc.nextInt();
            sum+=mark;
        }
        int average=sum/5;
        System.out.println("Average score:"+average);
        if(average>=50) {
            System.out.println("The student has passed");
        } else {
            System.out.println("The student has failed");
        }
        sc.close();
    }
}
```

**Status : Wrong**

**Marks : 0/10**

## **5. Problem Statement**

Subha is tasked with developing a program to analyze and categorize a large dataset of numerical data points. As a part of her program, she needs to identify and extract all the even numbers from the dataset, which ranges from 2 to a specified integer(inclusive).



Help Subha to write this program using a 'for' loop.

**Input Format**

The input consists of an integer N, representing the upper limit of the range.

**Output Format**

The output displays the even numbers in the dataset separated by a space, up to the specified integer.

Refer to the sample output for formatting specifications.

**Sample Test Case**

Input: 17

Output: 2 4 6 8 10 12 14 16

**Answer**

```
import java.util.Scanner;
public class EvenNumbers {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int N=sc.nextInt();
        for(int i=2; i<=N; i+=2) {
            System.out.println(i);
            if(i+2<=N) {
                System.out.print(" ");
            }
        }
        sc.close();
    }
}
```

**Status : Wrong**

**Marks : 0/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 1\_CY\_Updated

Attempt : 1

Total Mark : 50

Marks Obtained : 22.5

### Section 1 : COD

#### 1. Problem Statement

Jack is exploring bitwise and arithmetic operations in his programming course. He needs a program to:

Find the result of the bitwise AND between a given integer and 15. Multiply the integer by 2 as part of an arithmetic operation.

You need to create a program that accepts an integer input, performs both operations, and displays the results.

#### ***Input Format***

The input consists of an integer N, which represents the number to be processed.

#### ***Output Format***

The first line of output prints an integer representing the result of the bitwise

AND operation between the input and 15.

The second line of output prints an integer representing the result after multiplying the original number by 2.

Refer to the sample output for format specifications.

### **Sample Test Case**

Input: 25

Output: 9

50

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        int N=scanner.nextInt();
        scanner.close();
        int bitwiseAndResult=N&15;
        int multiplicationResult=N*2;
        System.out.println(bitwiseAndResult);
        System.out.println(multiplicationResult);
    }
}
```

**Status : Correct**

**Marks : 10/10**

## **2. Problem Statement:**

Miles is working on a program that involves analyzing two integers. He wants to check if either one of the integers is both:

Less than or equal to zero, andOdd.Can you help him create a program that identifies whether either of the integers meets these conditions?

### **Input Format**

The input consists of two integers on separate lines, denoted as 'input1' and 'input2'.

### **Output Format**

A single line with a boolean result (either 'true' or 'false') indicating whether either 'input1' or 'input2' is both less than or equal to zero and odd.

Refer to the sample output for format specifications

### **Sample Test Case**

Input: -45

10

Output: true

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        int input1=scanner.nextInt();
        int input2=scanner.nextInt();
        boolean result=checkCondition(input1)||checkCondition(input2);
        System.out.println(result);
    }
    public static boolean checkCondition(int num) {
        return num<=0 && num%2!=0;
    }
}
```

**Status :** Correct

**Marks :** 10/10

## **3. Problem Statement**

John is learning programming and wants to practice a program to test his understanding of the concept of loops.

He is given the task of writing a program using a 'for' loop that calculates

the sum of all integers from 1 up to and including a given positive integer provided by the user.

### ***Input Format***

The input consists of a positive integer N.

### ***Output Format***

The output prints the sum of numbers from 1 to the given integer N as "Sum of numbers from 1 to N: <sum>".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5

Output: Sum of numbers from 1 to 5: 15

### ***Answer***

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int N=sc.nextInt();
        int sum=0;
        for(int i=1;i<=N;i++) {
            sum+=1;
        }
        System.out.print("Sum of numbers from 1 to"+N+"."+sum);
    }
}
```

**Status :** Wrong

**Marks :** 0/10

## **4. Problem Statement**

Arun is a mathematics enthusiast who enjoys solving unique mathematical puzzles. Recently, he came across a problem that involves counting even digits in an integer and checking if the count itself is a divisor of the

original number.

Your task is to create a program for Arun that counts the even digits in an integer and checks for divisibility using a 'while' loop.

### ***Input Format***

The input consists of a single integer N.

### ***Output Format***

If there are no even digits in N, print "There are no even digits in the number."

If the count of even digits is a divisor of the original number, "<Count> is a divisor of <original number>" Where

<Count> represents the total number of even digits in the given number, and <original number> represents the input number provided by the user.

If the count of even digits is not a divisor of the original number, "<Count> is not a divisor of <original number>" Where

<Count> represents the total number of even digits in the given number, and <original number> represents the input number provided by the user.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 20

Output: 2 is a divisor of 20

### ***Answer***

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        int N=scanner.nextInt();
        scanner.close();
        int count=0;
        int originalNumber=N;
        while (count>0||N>0) {
```

```

int digit=N%10;
if(digit%2==0) {
    count++;
}
N/=10;
}
if(count==0) {
    System.out.println("There are no even digits in the number.");
} else if(originalNumber%count==0) {
    System.out.println(count+"is a divisor of"+originalNumber);
} else {
    System.out.println(count+"is not a divisor of"+originalNumber);
}
}
}

```

**Status :** Partially correct

**Marks :** 2.5/10

## 5. Problem Statement

John is a fitness trainer, and he wants to use the BMI calculator to assess the body mass index of his clients. He has a list of clients based on their height and weight.

John plans to write a program to quickly determine the BMI and provide a classification for each client.

If BMI is less than 18.5, the program will classify it as "Underweight" If BMI is between 18.6 and 24.9, the program will classify it as "Normal Weight" If BMI is between 25.0 and 29.9, the program will classify it as "Overweight" If BMI is 30.0 or higher, the program will classify it as "Obese"

Note: Formula to calculate BMI =  $\text{weight}/(\text{height} \times \text{height})$

### **Input Format**

The first line of input consists of a double value, representing the height of the person in meters.

The second line consists of a double value, representing the weight of the person in kilograms.

### Output Format

The first line of output prints "BMI: " followed by a double (rounded to two decimal places) representing the calculated BMI.

The second line prints "Classification: " followed by a string indicating the BMI category (Underweight, Normal Weight, Overweight, or Obese).

Refer to the sample output for formatting specifications.

### Sample Test Case

Input: 1.2

45.2

Output: BMI: 31.39

Classification: Obese

### Answer

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        double weight=sc.nextDouble();
        double height=sc.nextDouble();
        double bmi=weight/(height*height);
        System.out.printf("BMI: %2f\n",bmi);
        if(bmi<18.5) {
            System.out.println("Classification:Underweight");
        } else if(bmi<25) {
            System.out.println("Classificatuion:Normal");
        } else if(bmi<30) {
            System.out.println("Classification:Overweight");
        } else {
            System.out.println("Classification:Obese");
        }
        sc.close();
    }
}
```

Status : Wrong

Marks : 0/10



# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 1\_PAH\_Updated

Attempt : 1

Total Mark : 50

Marks Obtained : 17

### Section 1 : COD

#### 1. Problem Statement

In a coding competition, you are tasked with a unique problem. You need to write a program that calculates the sum of the squares of the odd digits in a given integer.

#### *Input Format*

The input consists of a single integer N, representing the number whose odd digits will be squared and summed.

#### *Output Format*

The output prints an integer, representing the sum of the squares of the odd digits in the given integer.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 12345

Output: 35

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int sum=0;
        while(n>0) {
            int digit=n%10;
            if(digit %2!=0) {
                sum=sum+(digit*digit);
            }
            n=n/10;
        }
        System.out.println(sum);
    }
}
```

**Status :** Correct

**Marks :** 10/10

## **2. Problem Statement**

Rohit is tasked with designing a program to analyze the digits of a given integer.

Write a program to help Rohit that takes an integer as input and identifies the minimum odd digit and the maximum even digit present in the number. If no odd or even digits are present, display appropriate messages.

Implement the solution using a 'while' loop to iterate through the digits of the given number.

### **Input Format**

The input consists of an integer n.

### **Output Format**

The first line of output prints the message "Minimum odd digit: " followed by an integer representing the smallest odd digit found in the input number.

If no odd digit exists, it prints "There are no odd digits in the number."

The second line of output prints the message "Maximum even digit: " followed by an integer representing the largest even digit found in the input number.

If no even digit exists, it prints "There are no even digits in the number."

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 3465

Output: Minimum odd digit: 3

Maximum even digit: 6

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int minOdd=9;
        int maxEven=0;
        boolean oddFound=false;
        boolean evenFound=false;
        while(n>0) {
            int digit=n%10;
            if(digit %2==1) {
                oddFound=true;
                if(digit<minOdd) {
                    minOdd=digit;
                }
            } else {
                evenFound=true;
```

```

        if(digit>maxEven) {
            maxEven=digit;
        }
    }
    n=n/10;

}
if(oddFound) {
    System.out.println("Minimum odd digit:"+minOdd);
} else {
    System.out.println("There are no odd digits in the number");
}
if(evenFound) {
    System.out.println("Maximum even digit:"+maxEven);
} else {
    System.out.println("There are no even digits in the number");
}
    sc.close();
}
}

```

**Status :** Partially correct

**Marks :** 7/10

### 3. Problem Statement

Akash is tasked with developing a program that calculates and categorizes blood pressure based on the given systolic and diastolic readings.

The program should use the following classifications:

Low Blood Pressure: Systolic < 90 mm Hg or Diastolic < 60 mm Hg  
 Normal Blood Pressure: Systolic ≤ 120 mm Hg and Diastolic ≤ 80 mm Hg  
 Prehypertension: Systolic ≤ 140 mm Hg and Diastolic ≤ 90 mm Hg  
 Stage 1 Hypertension: Systolic ≤ 160 mm Hg and Diastolic ≤ 100 mm Hg  
 Stage 2 Hypertension: Otherwise

Write a program to assist Akash in computing and classifying blood pressure levels based on input readings. The program should use a 'switch-case' to deliver a tailored blood pressure category.

**Input Format**

The input consists of two space-separated integers, representing the systolic blood pressure value S and diastolic blood pressure value D, respectively.

### **Output Format**

The output displays "Blood Pressure Category: " followed by the blood pressure category based on the provided input.

Refer to the sample output for the exact text and format.

### **Sample Test Case**

Input: 50 85

Output: Blood Pressure Category: Low Blood Pressure

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        int systolic=scanner.nextInt();
        int diastolic=scanner.nextInt();
        int category=getBloodPressureCategory(systolic,diastolic);
        System.out.println("Blood Pressure
Category"+getCategoryName(category));
    }
    private static int getBloodPressureCategory(int systolic,int diastolic) {
        if(systolic<90||diastolic<60)return 1;
        if(systolic<=120 && diastolic<=80)return 2;
        if(systolic<=140 && diastolic<90)return 3;
        if(systolic<=160 && diastolic<100)return 4;
        return 5;
    }
    private static String getCategoryName(int category) {
        switch(category) {
            case 1:return "low Blood Pressure";
            case 2:return "Normal Blood Pressure";
            case 3:return "Prehypertension";
            case 4:return "Stage 1 Hypertension";
            case 5:return "Stage 2 Hypertension";
            default:return "Unknown";
        }
    }
}
```

**Status : Wrong**

**Marks : 0/10**

#### 4. PROBLEM STATEMENT:

John, a student, is working on a program to determine the day of the week and provide activity suggestions based on a user's input. He needs to create a simple program that takes a number from 1 to 7 as input, where each number corresponds to a day of the week from Sunday to Saturday. The program should then output the corresponding day of the week and suggest activities commonly associated with that day which were listed below:

1) Sunday

Activity suggestions: Relax, spend time with family, go for a walk

2) Monday

Activity suggestions: Start the week with a positive attitude, plan your tasks

3) Tuesday

Activity suggestions: Work on your projects, attend meetings

4) Wednesday

Activity suggestions: Midweek check-in, exercise, catch up on tasks

5) Thursday

Activity suggestions: Focus on important tasks, prepare for upcoming events

6) Friday

Activity suggestions: Wrap up the week's work, plan for the weekend

7) Saturday

Activity suggestions: Relax, pursue hobbies, spend time with friends

### ***Input Format***

The input is a single integer, ranging from 1 to 7, representing the day of the week.

### ***Output Format***

The output should display the day of the week along with activity suggestions, adhering to the following format:

"[Day of the week]"

"Activity suggestions: [Suggestions for activities]"

Here, [Day of the week] should be replaced with the actual day name, and [Suggestions for activities] should be replaced with a list of activities typically associated with that day.

If the day of the week is not between 1 and 7, then display "Invalid input. Please enter a number from 1 to 7"

Refer to the sample output for the precise formatting specifications.

### ***Sample Test Case***

Input: 1

Output: Sunday

Activity suggestions: Relax, spend time with family, go for a walk

### ***Answer***

```
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        System.out.println("Enter a number from 1 to 7:");
        int dayNumber=scanner.nextInt();
        scanner.close();
        String[]
days={"Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"};
        String[] activities={
            "Relax,spend time with family,go for a walk",
            "Start the week with a positive attitude,plan your tasks",
            "Work on your project,attend meetings",
            "Midweek check-in,exercise,catch up on tasks",
            "Focus on important tasks,prepare for upcoming events",
            "Wrap up the weeks work,plan for the weekend",
            "Relax,pursue hobbies,spend time with friends"
        };
        if(dayNumber>=1 && dayNumber<=7) {
            System.out.println(days[dayNumber-1]);
            System.out.println("Activity suggestions:"+activities [dayNumber-1]);
        } else {
            System.out.println("Invalid input,Please enter a number from 1 to 7");
        }
    }
}

```

**Status : Wrong**

**Marks : 0/10**

## 5. Problem Statement

A number theorist is analyzing relationships between numbers for a mathematical journal. You're asked to find all pairs of friendly numbers less than a number N, where two numbers are friendly if the sum of proper divisors of one equals the other and vice versa.

Note: Use nested for loops to solve.

### **Input Format**

The first line contains an integer N, which represents the upper limit



### **Output Format**

The first line contains an integer K, representing the number of friendly pairs.

The next K lines each contain a pair of integers (a, b).

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 300

Output: 1

220 284

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static int sumOfDivisors(int num) {
        int sum=0;
        for(int i=1;i<num;i++) {
            if(num%i==0) {
                sum+=i;
            }
        }
        return sum;
    }
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int N=sc.nextInt();
        int count=0;
        for(int i=2;i<N;i++) {
            for(int j=i+1;j<N;j++) {
                if(sumOfDivisors(i)==j && sumOfDivisors(j)==i) {
                    System.out.println(i+" "+j);
                }
            }
        }
        sc.close();
    }
}
```

Status : Wrong

Marks : 0/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 2\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 15.83

### Section 1 : MCQ

1. What will be the output of the following code?

```
class Test {  
    private int value;  
    Test(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Test obj = new Test(10);  
        System.out.println(obj.value);  
    }  
}
```

```
}
```

**Answer**

Compile-time error

**Status :** Correct

**Marks :** 1/1

2. What is the output of the following code?

```
class Box {  
    int height;  
    Box(int height) {  
        this.height = height;  
    }  
    void modifyHeight(Box b) {  
        b.height += 10;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box b1 = new Box(20);  
        b1.modifyHeight(b1);  
        System.out.println(b1.height);  
    }  
}
```

**Answer**

30

**Status :** Correct

**Marks :** 1/1

3. What will be the output of the following code?

```
class Person {  
    String name;  
    void setName(String n) {  
        name = n;  
    }  
    void printName() {
```

```
        System.out.println(name);
    }
}
```

```
class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.printName();
    }
}
```

**Answer**

null

**Status :** Correct

**Marks :** 1/1

4. What will be the output of the following code?

```
class Sample {
    int x = 10;

    void display() {
        System.out.println("x = " + x);
    }

    public static void main(String[] args) {
        Sample s = new Sample();
        s.display();
    }
}
```

**Answer**

x = 10

**Status :** Correct

**Marks :** 1/1

5. What will be the output of the following code?

```
class Box {
```

```
int length = 5;  
int width = 4;
```

```
int area() {  
    return length * width;  
}
```

```
public static void main(String[] args) {  
    Box b = new Box();  
    System.out.println("Area = " + b.area());  
}
```

**Answer**

Area = 20

**Status :** Correct

**Marks :** 1/1

6. What will be the output of the following code?

```
class A {  
    int x = 50;  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.x = 100;  
        System.out.println(obj1.x);  
    }  
}
```

**Answer**

100

**Status :** Correct

**Marks :** 1/1

7. What will be the output of the following code?

```
class A {  
    int y = 30;  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        A a1 = new A();  
        A a2 = new A();  
        a1.y = 50;  
        System.out.println(a2.y);  
    }  
}
```

**Answer**

30

**Status :** Correct

**Marks :** 1/1

8. What will be the output of the following code?

```
class A {  
    int p = 5;  
    int q = 2;  
}  
  
class Main {  
    public static void main(String[] args) {  
        A obj = new A();  
        System.out.println(obj.p + obj.q);  
    }  
}
```

**Answer**

Compilation error

**Status :** Wrong

**Marks :** 0/1

9. What will be the output of the following code?

```
class A {  
    int val = 20;  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.val += 5;  
        System.out.println(obj1.val);  
    }  
}
```

**Answer**

25

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following code?

```
class Person {  
    int age = 18;  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.age += 2;  
        System.out.println("Age: " + p.age);  
    }  
}
```

**Answer**

Compilation error

**Status :** Wrong

**Marks :** 0/1

11. What will be the output of the following code?



```
class Ball {  
    int size = 11;  
}
```

```
class Game {  
    public static void main(String[] args) {  
        Ball b1 = new Ball();  
        Ball b2 = new Ball();  
        b2.size = 10;  
        System.out.println(b1.size);  
    }  
}
```

**Answer**

11

**Status : Correct**

**Marks : 1/1**

12. What will be the output of the following code?

```
class Demo {  
    void printMessage() {  
        System.out.println("Hello from Demo");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.printMessage();  
    }  
}
```

**Answer**

Hello from Demo

**Status : Correct**

**Marks : 1/1**

13. What will be the output of the following code?

```
class Box {  
    int volume(int l, int b, int h) {  
        return l * b * h;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box b = new Box();  
        System.out.println(b.volume(2, 3, 4));  
    }  
}
```

**Answer**

24

**Status :** Correct

**Marks :** 1/1

14. What will be the output of the following code?

```
class Alpha {  
    void greet(String name) {  
        System.out.println("Hello " + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Alpha obj = new Alpha();  
        obj.greet("Anu");  
    }  
}
```

**Answer**

Hello Anu

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following code?

```
class MathUtils {  
    int add(int x) {  
        return x + x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathUtils m = new MathUtils();  
        System.out.println(m.add(5));  
    }  
}
```

**Answer**

10

**Status :** Correct

**Marks :** 1/1

16. Which of the following statements about classes and objects in Java are correct?

**Answer**

A class is a blueprint for creating objects.  
An object is an instance of a class.

**Status :** Correct

**Marks :** 1/1

17. Which of the following statements about returning a value from a method in Java are correct?

**Answer**

A method with a return type of void cannot return a value.  
A method must always return a value.

**Status :** Partially correct

**Marks :** 0.33/1

18. Which of the following access modifiers allows access to a class member only within the same class?

**Answer**

private

**Status :** Correct

**Marks :** 1/1

19. What happens when a method returns an object?

**Answer**

A copy of the object is returned.

**Status :** Wrong

**Marks :** 0/1

20. Which statement is true about passing objects to methods in Java?

**Answer**

Objects are passed by value, but the reference itself is passed.

**Status :** Partially correct

**Marks :** 0.5/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 2\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

You are working as a developer for CityBank, which wants to build a basic account management system.

Each customer at the bank has:

An Account Number (integer) A Customer Name (string) An Initial Balance (double)

The bank allows two types of transactions:

Deposit – increases the balance. Withdrawal – decreases the balance only if enough funds are available.

If the withdrawal amount is greater than the balance, the withdrawal should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for account details. A constructor to initialize account details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's account details after all transactions.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

### ***Output Format***

For each customer, print the details in the following format:

1. Account Number: <account\_number>
2. Customer Name: <customer\_name>
3. Final Balance: <final\_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

**Answer**

```
import java.util.Scanner;

class BankAccount {
    private int accountNumber;
    private String customerName;
    private double balance;

    // Constructor
    public BankAccount(int accountNumber, String customerName, double
balance) {
        this.accountNumber = accountNumber;
        this.customerName = customerName;
        this.balance = balance;
    }

    // Deposit method
    public void deposit(double amount) {
        balance += amount;
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        }
    }

    // Getter methods
    public int getAccountNumber() {
        return accountNumber;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double getBalance() {
        return balance;
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {

            int accNo = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            double initialBalance = sc.nextDouble();
            double depositAmount = sc.nextDouble();
            double withdrawAmount = sc.nextDouble();

            BankAccount account = new BankAccount(accNo, name, initialBalance);

            account.deposit(depositAmount);
            account.withdraw(withdrawAmount);

            System.out.println("Account Number: " + account.getAccountNumber());
            System.out.println("Customer Name: " + account.getCustomerName());
            System.out.printf("Final Balance: %.1f\n", account.getBalance());
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

You are working as a developer for CityCab, a taxi service company that wants to build a ride fare management system.



Each customer booking has:

A Booking ID (integer) A Customer Name (string) A Distance Travelled in km (double)

The fare calculation rules are:

Base Fare = 50 units (flat charge for every ride). Per km charge = 10 units/km. If the distance is greater than 20 km, a 10% discount is applied on the total fare.

You are required to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customer rides.

Finally, display each booking's details and final fare.

### ***Input Format***

The first line of input contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the booking ID (integer).
- The following line contains the customer's name (string).
- The next line contains the distance travelled (double).

### ***Output Format***

For each booking, print the details in the following format:

1. Booking ID: <booking\_id>
2. Customer Name: <customer\_name>
3. Final Fare: <final\_fare> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1234

Rahul Sharma

15

Output: Booking ID: 1234

Customer Name: Rahul Sharma

Final Fare: 200.0

### **Answer**

```
import java.util.Scanner;
```

```
class Booking {
```

```
    private int bookingId;
```

```
    private String customerName;
```

```
    private double distance;
```

```
    // Constructor
```

```
    public Booking(int bookingId, String customerName, double distance) {
```

```
        this.bookingId = bookingId;
```

```
        this.customerName = customerName;
```

```
        this.distance = distance;
```

```
    }
```

```
    // Getter methods
```

```
    public int getBookingId() {
```

```
        return bookingId;
```

```
    }
```

```
    public String getCustomerName() {
```

```
        return customerName;
```

```
    }
```

```
    public double getDistance() {
```

```
        return distance;
```

```
    }
```

```
    // Setter methods
```

```
    public void setCustomerName(String name) {
```

```
        this.customerName = name;
```

```
    }
```

```
public void setDistance(double distance) {  
    this.distance = distance;  
}
```

```
// Method to calculate final fare
```

```
public double calculateFare() {  
    double fare = 50 + (distance * 10);  
  
    if (distance > 20) {  
        fare = fare - (fare * 0.10); // 10% discount  
    }  
}
```

```
return fare;  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        sc.nextLine();
```

```
        for (int i = 0; i < n; i++) {  
            int id = sc.nextInt();  
            sc.nextLine();
```

```
            String name = sc.nextLine();  
            double distance = sc.nextDouble();
```

```
            Booking b = new Booking(id, name, distance);
```

```
            double finalFare = b.calculateFare();
```

```
            System.out.println("Booking ID: " + b.getBookingId());  
            System.out.println("Customer Name: " + b.getCustomerName());  
            System.out.println("Final Fare: " + String.format("%.1f", finalFare));
```

```
        }
```

```
        sc.close();  
    }
```

}

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Ram is working as a developer for BrightEdu Coaching Center, which wants to build a student fee management system.

Each student's enrollment has:

An Enrollment ID (integer) A Student Name (string) The Number of Subjects (integer)

The fee calculation rules are:

Registration Fee = 1000 units (flat for every student). Per Subject Fee = 800 units. If the student enrolls in more than 5 subjects, a 20% scholarship (discount) is applied on the total fee.

Ram has been asked to implement this system using:

A class with attributes for student details. A constructor to initialize student details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent student enrollments.

Finally, display each student's details and final fee.

#### ***Input Format***

The first line of input contains an integer N, representing the number of students.

For each student:

- The next line contains the Enrollment ID (integer).
- The following line contains the student's name (string).
- The next line contains the Number of subjects (integer).

#### ***Output Format***

For each student, print the details in the following format:

- Enrollment ID: <enrollment\_id>
- Student Name: <student\_name>
- Final Fee: <final\_fee> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1234

Ravi Kumar

3

Output: Enrollment ID: 1234

Student Name: Ravi Kumar

Final Fee: 3400.0

### **Answer**

```
import java.util.Scanner;
```

```
class Student {  
    private int enrollmentId;  
    private String name;  
    private int subjects;
```

```
    // Constructor
```

```
    Student(int enrollmentId, String name, int subjects) {  
        this.enrollmentId = enrollmentId;  
        this.name = name;  
        this.subjects = subjects;  
    }
```

```
    // Setter methods
```

```
    public void setEnrollmentId(int enrollmentId) {  
        this.enrollmentId = enrollmentId;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
public void setSubjects(int subjects) {  
    this.subjects = subjects;  
}
```

```
// Getter methods  
public int getEnrollmentId() {  
    return enrollmentId;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public int getSubjects() {  
    return subjects;  
}
```

```
// Method to calculate final fee  
public double calculateFee() {  
    double fee = 1000 + (subjects * 800);
```

```
    if (subjects > 5) {  
        fee = fee - (fee * 0.20); // 20% discount  
    }
```

```
    return fee;
```

```
}  
}  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        sc.nextLine(); // clear buffer
```

```
        for (int i = 0; i < n; i++) {  
            int id = sc.nextInt();  
            sc.nextLine();
```

```
            String name = sc.nextLine();
```

```

        int subjects = sc.nextInt();

        Student s = new Student(id, name, subjects);

        System.out.println("Enrollment ID: " + s.getEnrollmentId());
        System.out.println("Student Name: " + s.getName());
        System.out.println("Final Fee: " + s.calculateFee());
    }

    sc.close();
}

```

**Status :** Correct

**Marks : 10/10**

#### 4. Problem Statement

Raj is working as a developer for CityFood Restaurant, which wants to build a billing system for family orders.

Each family's order has:

An Order ID (integer) A Family Name (string) The Number of Dishes Ordered (dish name along with their count)

The restaurant has a fixed menu with rates:

Pizza – 200 units Burger – 150 units Pasta – 180 units Sandwich – 120 units Coffee – 100 units

The calculation rules are:

Total Amount = sum of (dish price × quantity). If the family orders more than 5 dishes in total, a 10% discount is applied on the total amount.

Raj has been asked to implement this system using:

A class with attributes for order details. A constructor to initialize order details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent family orders.

Finally, display each family's order details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of families.

For each family:

1. The next line contains the Order ID (integer).
2. The following line contains the Family Name (string).
3. The next line contains an integer M (number of different dishes ordered).

For the next M lines, each line contains:

1. Dish Name (string)
2. Quantity (integer)

### ***Output Format***

For each family, print the details in the following format:

- Order ID: <order\_id>
- Family Name: <family\_name>
- Final Bill: <final\_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Kumar Family

1

Pizza

3

Output: Order ID: 1001

Family Name: Kumar Family

Final Bill: 600.0

### ***Answer***

```
import java.util.Scanner;
```



```

class Order {
    int orderId;
    String familyName;
    double total;

    Order(int id, String name) {
        orderId = id;
        familyName = name;
        total = 0;
    }

    void addDish(String dish, int qty) {
        int price = 0;

        if (dish.equals("Pizza")) price = 200;
        else if (dish.equals("Burger")) price = 150;
        else if (dish.equals("Pasta")) price = 180;
        else if (dish.equals("Sandwich")) price = 120;
        else if (dish.equals("Coffee")) price = 100;

        total += price * qty;
    }

    double getBill(int totalQty) {
        if (totalQty > 5) {
            total = total - total * 0.10;
        }
        return total;
    }
}

```

```

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {

            int id = sc.nextInt();

```

```

sc.nextLine();

String name = sc.nextLine();

Order o = new Order(id, name);

int m = sc.nextInt();

int totalQty = 0;

for (int j = 0; j < m; j++) {
    String dish = sc.next();
    int qty = sc.nextInt();

    o.addDish(dish, qty);
    totalQty += qty;
}

System.out.println("Order ID: " + o.orderId);
System.out.println("Family Name: " + o.familyName);
System.out.println("Final Bill: " + o.getBill(totalQty));

if (i < n - 1) {
    System.out.println();
}
}
}
}
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Neha is working as a developer for CityElectricity Board, which wants to build a household electricity billing system.

Each customer's electricity account has:

A Customer ID (integer) A Customer Name (string) Units Consumed (double)

The electricity bill is calculated based on these rules:

For the first 100 units 5 units charge per unit  
For the next 100 units (101 – 200) 7 units charge per unit  
For units above 200 10 units charge per unit  
If the total bill exceeds 2000 units, a 5% discount is applied on the final bill.

Neha has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

80

Output: Customer ID: 1001  
Customer Name: Ravi Kumar  
Final Bill: 400.0

**Answer**

```
import java.util.Scanner;

class Customer {
    private int id;
    private String name;
    private double units;

    // Constructor
    Customer(int id, String name, double units) {
        this.id = id;
        this.name = name;
        this.units = units;
    }

    // Setter methods
    public void setId(int id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setUnits(double units) {
        this.units = units;
    }

    // Getter methods
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

```

// Method to calculate bill
public double calculateBill() {
    double bill;

    if (units <= 100) {
        bill = units * 5;
    } else if (units <= 200) {
        bill = (100 * 5) + ((units - 100) * 7);
    } else {
        bill = (100 * 5) + (100 * 7) + ((units - 200) * 10);
    }

    if (bill > 2000) {
        bill = bill - (bill * 0.05);
    }

    return bill;
}
}

```

```

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {

            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            double units = sc.nextDouble();

            Customer c = new Customer(id, name, units);

            System.out.println("Customer ID: " + c.getId());
            System.out.println("Customer Name: " + c.getName());
            System.out.println("Final Bill: " + c.calculateBill());
        }
    }
}

```

```
    sc.close();  
  }  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 2\_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed (double)

The water bill is calculated based on these rules:

For the first 500 liters    2 per liter For the next 500 liters (501–1000)    3 per liter  
For liters above 1000    5 per liter If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

### ***Answer***

```
import java.util.Scanner;
```



```
class Customer {
    int customerId;
    String customerName;
    double litersConsumed;

    // Constructor
    Customer(int id, String name, double liters) {
        this.customerId = id;
        this.customerName = name;
        this.litersConsumed = liters;
    }

    // Setter methods
    void setCustomerId(int id) {
        this.customerId = id;
    }

    void setCustomerName(String name) {
        this.customerName = name;
    }

    void setLitersConsumed(double liters) {
        this.litersConsumed = liters;
    }

    // Getter methods
    int getCustomerId() {
        return customerId;
    }

    String getCustomerName() {
        return customerName;
    }

    double getLitersConsumed() {
        return litersConsumed;
    }

    // Method to calculate bill
    double calculateBill() {
        double bill = 0;
    }
}
```

```

        if (litersConsumed <= 500) {
            bill = litersConsumed * 2;
        } else if (litersConsumed <= 1000) {
            bill = (500 * 2) + (litersConsumed - 500) * 3;
        } else {
            bill = (500 * 2) + (500 * 3) + (litersConsumed - 1000) * 5;
        }

        // Apply discount
        if (bill > 3000) {
            bill = bill - (bill * 0.10);
        }

        return bill;
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine(); // consume newline

            String name = sc.nextLine();
            double liters = sc.nextDouble();

            Customer c = new Customer(id, name, liters);

            double finalBill = c.calculateBill();

            System.out.println("Customer ID: " + c.getId());
            System.out.println("Customer Name: " + c.getName());
            System.out.printf("Final Bill: %.1f\n", finalBill);
        }

        sc.close();
    }
}

```

**Status : Correct**

**Marks : 10/10**

## 2. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units    4 per unit For the next 100 units (51–150)    6 per unit  
For units above 150    8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

### ***Input Format***

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

### ***Output Format***

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Bill: <final\_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

### **Answer**

```
import java.util.Scanner;
```

```
class Customer {  
    int customerId;  
    String customerName;  
    double units;  
  
    // Constructor  
    Customer(int id, String name, double units) {  
        this.customerId = id;  
        this.customerName = name;  
        this.units = units;  
    }  
  
    // Setters  
    void setCustomerId(int id) {  
        this.customerId = id;  
    }  
}
```

```
void setCustomerName(String name) {  
    this.customerName = name;  
}
```

```
void setUnits(double units) {  
    this.units = units;  
}
```

```
// Getters  
int getCustomerId() {  
    return customerId;  
}
```

```
String getCustomerName() {  
    return customerName;  
}
```

```
double getUnits() {  
    return units;  
}
```

```
// Bill calculation  
double calculateBill() {  
    double bill = 0;
```

```
    if (units <= 50) {  
        bill = units * 4;  
    } else if (units <= 150) {  
        bill = (50 * 4) + (units - 50) * 6;  
    } else {  
        bill = (50 * 4) + (100 * 6) + (units - 150) * 8;  
    }  
}
```

```
// Apply discount  
if (bill > 2000) {  
    bill = bill - (bill * 0.15);  
}
```

```
    return bill;
```

```
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine(); // consume newline

            String name = sc.nextLine();
            double units = sc.nextDouble();

            Customer c = new Customer(id, name, units);

            double finalBill = c.calculateBill();

            System.out.println("Customer ID: " + c.getId());
            System.out.println("Customer Name: " + c.getName());
            System.out.printf("Final Bill: %.1f\n", finalBill);
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Anjali is working as a developer for the City Basketball Association, which wants to build a system to track and find the top scorer among basketball players.

Each player's record has:

Player ID (integer) Player Name (string) An array of points scored in 5 matches (integers)

The system must calculate:

The total score of each player (sum of all match points). Identify the highest scorer among all players. If two or more players have the same total score, the one with the lower Player ID is considered the top scorer.

Anjali has been asked to implement this system using:

A class with attributes for player details. A constructor to initialize player details. Getter and Setter methods to retrieve and update player details if required. A method to calculate the total score. Objects of the class to represent players.

Finally, display each player's details and announce the Top Scorer.

### ***Input Format***

The first line of input contains an integer N (number of players).

For each player:

- The next line contains the Player ID (integer).
- The following line contains the Player Name (string).
- The next line contains 5 integers separated by spaces (points scored in 5 matches).

### ***Output Format***

For each player the output prints the following details:

- Player ID: <player\_id>
- Player Name: <player\_name>
- Total Score: <total\_score>

Finally, print "Top Scorer: <player\_name> with <total\_score> points"

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

10 20 30 40 50

Output: Player ID: 1001

Player Name: Ravi Kumar

Total Score: 150

Top Scorer: Ravi Kumar with 150 points

### **Answer**

```
import java.util.Scanner;
```

```
class Player {  
    int playerId;  
    String playerName;  
    int[] points = new int[5];  
  
    // Constructor  
    Player(int id, String name, int[] pts) {  
        this.playerId = id;  
        this.playerName = name;  
        this.points = pts;  
    }  
  
    // Getters  
    int getPlayerId() {  
        return playerId;  
    }  
  
    String getPlayerName() {  
        return playerName;  
    }  
  
    // Method to calculate total score  
    int getTotalScore() {  
        int sum = 0;  
        for (int i = 0; i < 5; i++) {  
            sum += points[i];  
        }  
        return sum;  
    }  
}
```



```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        Player[] players = new Player[n];

        // Input
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            int[] pts = new int[5];
            for (int j = 0; j < 5; j++) {
                pts[j] = sc.nextInt();
            }

            players[i] = new Player(id, name, pts);
        }

        // Display and find top scorer
        int maxScore = -1;
        Player topPlayer = null;

        for (int i = 0; i < n; i++) {
            int total = players[i].getTotalScore();

            System.out.println("Player ID: " + players[i].getPlayerId());
            System.out.println("Player Name: " + players[i].getPlayerName());
            System.out.println("Total Score: " + total);

            // Find top scorer
            if (total > maxScore) {
                maxScore = total;
                topPlayer = players[i];
            } else if (total == maxScore) {
                if (players[i].getPlayerId() < topPlayer.getPlayerId()) {
```

```

        topPlayer = players[i];
    }
}

// Final output
System.out.println("Top Scorer: " + topPlayer.getPlayerName() +
    " with " + maxScore + " points");

sc.close();
}
}

```

**Status :** Correct

**Marks : 10/10**

#### 4. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:

Runner ID (integer) Runner Name (string) An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5). Identify the fastest runner (the one with the lowest average time). If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details. A constructor to initialize runner details. Getter and Setter methods to retrieve and update runner details if required. A method to calculate the average time. Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

### ***Input Format***

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

### ***Output Format***

For each runner the output prints the following details:

- Runner ID: <runner\_id>
- Runner Name: <runner\_name>
- Average Time: <average\_time>

Finally, print "Fastest Runner: <runner\_name> with <average\_time> minutes"

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

240 250 245 255 260

Output: Runner ID: 1001

Runner Name: Ravi Kumar

Average Time: 250

Fastest Runner: Ravi Kumar with 250 minutes

### ***Answer***

```
import java.util.Scanner;
```

```
class Runner {
```

```
int runnerId;  
String runnerName;  
int[] times = new int[5];
```

```
// Constructor  
Runner(int id, String name, int[] t) {  
    this.runnerId = id;  
    this.runnerName = name;  
    this.times = t;  
}
```

```
// Getters  
int getRunnerId() {  
    return runnerId;  
}
```

```
String getRunnerName() {  
    return runnerName;  
}
```

```
// Method to calculate average time  
int getAverageTime() {  
    int sum = 0;  
    for (int i = 0; i < 5; i++) {  
        sum += times[i];  
    }  
    return sum / 5; // integer average  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
  
        int n = sc.nextInt();  
        sc.nextLine(); // consume newline
```

```
  
        Runner[] runners = new Runner[n];
```

```
  
        // Input  
        for (int i = 0; i < n; i++) {  
            int id = sc.nextInt();
```

```

        sc.nextLine();

        String name = sc.nextLine();

        int[] t = new int[5];
        for (int j = 0; j < 5; j++) {
            t[j] = sc.nextInt();
        }

        runners[i] = new Runner(id, name, t);
    }

    // Display and find fastest runner
    int minAvg = Integer.MAX_VALUE;
    Runner fastest = null;

    for (int i = 0; i < n; i++) {
        int avg = runners[i].getAverageTime();

        System.out.println("Runner ID: " + runners[i].getRunnerId());
        System.out.println("Runner Name: " + runners[i].getRunnerName());
        System.out.println("Average Time: " + avg);

        // Find fastest (lowest average)
        if (avg < minAvg) {
            minAvg = avg;
            fastest = runners[i];
        } else if (avg == minAvg) {
            if (runners[i].getRunnerId() < fastest.getRunnerId()) {
                fastest = runners[i];
            }
        }
    }

    // Final output
    System.out.println("Fastest Runner: " + fastest.getRunnerName() +
        " with " + minAvg + " minutes");

    sc.close();
}
}

```

Status : Correct

Marks : 10/10

## 5. Problem Statement

You are working as a developer for CityMobile, which wants to build a basic mobile data usage management system.

Each customer has:

A Customer ID (integer)  
A Customer Name (string)  
An Initial Data Balance (in GB, double)

The company allows two types of operations:

Recharge – increases the data balance.  
Usage – decreases the data balance only if enough data is available.

If the usage amount is greater than the available data balance, the usage should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for customer details.  
A constructor to initialize customer details.  
Setter methods to update details if needed.  
Getter methods to retrieve details.  
Objects of the class to represent customers.

Finally, display each customer's details after all operations.

### **Input Format**

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Initial Data Balance (double).
- The next line contains the Recharge Amount in GB (double).
- The next line contains the Usage Amount in GB (double).

### **Output Format**

For each customer, print the details in the following format:

Customer ID: <customer\_id>

Customer Name: <customer\_name>

Final Data Balance: <final\_data\_balance> GB (The final balance must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1234

Ravi Kumar

5.0

2.0

3.0

Output: Customer ID: 1234

Customer Name: Ravi Kumar

Final Data Balance: 4.0 GB

### **Answer**

```
import java.util.Scanner;
```

```
class Customer {  
    int customerId;  
    String customerName;  
    double balance;
```

```
    // Constructor
```

```
    Customer(int id, String name, double bal) {  
        this.customerId = id;  
        this.customerName = name;  
        this.balance = bal;  
    }
```

```
    // Setters
```

```
    void setBalance(double bal) {
```

```

        this.balance = bal;
    }

    // Getters
    int getCustomerId() {
        return customerId;
    }

    String getCustomerName() {
        return customerName;
    }

    double getBalance() {
        return balance;
    }

    // Recharge method
    void recharge(double amount) {
        balance += amount;
    }

    // Usage method
    void useData(double amount) {
        if (amount <= balance) {
            balance -= amount;
        }
        // else ignore usage
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

```



```
double initial = sc.nextDouble();  
double recharge = sc.nextDouble();  
double usage = sc.nextDouble();
```

```
Customer c = new Customer(id, name, initial);
```

```
// Perform operations  
c.recharge(recharge);  
c.useData(usage);
```

```
// Output  
System.out.println("Customer ID: " + c.getId());  
System.out.println("Customer Name: " + c.getName());  
System.out.printf("Final Data Balance: %.1f GB\n", c.getBalance());
```

```
}  
sc.close();
```

```
}  
}
```

**Status :** Correct

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Veltech\_Java\_Week 2\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Ravi is working as a developer for SecureLogin Systems, which wants to build a system to evaluate the strength of user passwords.

Each user record has:

User ID (integer) User Name (string) Password (string)

The system must calculate whether a password is strong or weak.

A password is considered strong if it meets all of the following conditions:

At least 8 characters long. Contains at least one uppercase letter. Contains at least one lowercase letter. Contains at least one digit. Contains at least one special character (from !@#\$%^&\*).

Ravi has been asked to implement this system using:

A class with attributes for user details. A constructor to initialize user details. Getter and setter methods to retrieve or update user details. A method to check whether the password is strong. Objects of the class to represent users.

Finally, display each user's details and indicate whether their password is Strong or Weak.

### ***Input Format***

The first line contains an integer N, representing the number of users.

For each user:

The next line contains the User ID (integer).

The next line contains the User Name (string).

The next line contains the Password (string).

### ***Output Format***

For each user, print the details in the following format:

User ID: <user\_id>

User Name: <user\_name>

Password: <password>

Password Strength: <Strong/Weak>

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

Abc@1234

Output: User ID: 1001

User Name: Ravi Kumar

Password: Abc@1234

Password Strength: Strong

### **Answer**

```
import java.util.Scanner;
```

```
class User {
```

```
    int userId;
```

```
    String userName;
```

```
    String password;
```

```
    // Constructor
```

```
    User(int id, String name, String pass) {
```

```
        this.userId = id;
```

```
        this.userName = name;
```

```
        this.password = pass;
```

```
    }
```

```
    // Getters
```

```
    int getUserId() {
```

```
        return userId;
```

```
    }
```

```
    String getUserName() {
```

```
        return userName;
```

```
    }
```

```
    String getPassword() {
```

```
        return password;
```

```
    }
```

```
    // Method to check password strength
```

```
    String checkStrength() {
```

```
        boolean hasUpper = false;
```

```
        boolean hasLower = false;
```

```
        boolean hasDigit = false;
```

```
        boolean hasSpecial = false;
```

```
        if (password.length() < 8) {
```

```

        return "Weak";
    }

    for (int i = 0; i < password.length(); i++) {
        char ch = password.charAt(i);

        if (Character.isUpperCase(ch)) {
            hasUpper = true;
        } else if (Character.isLowerCase(ch)) {
            hasLower = true;
        } else if (Character.isDigit(ch)) {
            hasDigit = true;
        } else if ("!@#$%^&*".indexOf(ch) != -1) {
            hasSpecial = true;
        }
    }

    if (hasUpper && hasLower && hasDigit && hasSpecial) {
        return "Strong";
    } else {
        return "Weak";
    }
}
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();
            String pass = sc.nextLine();

            User u = new User(id, name, pass);

            System.out.println("User ID: " + u.getUserId());
        }
    }
}

```

```
        System.out.println("User Name: " + u.getUserName());
        System.out.println("Password: " + u.getPassword());
        System.out.println("Password Strength: " + u.checkStrength());
    }

    sc.close();
}
}
```

**Status :** Correct

**Marks : 10/10**

## 2. Problem Statement

Neha is working as a developer for CityMovie Theatre, which wants to build a system to calculate total ticket cost for movie-goers based on the number of tickets and type of seats booked.

Each customer's booking has:

Booking ID (integer) Customer Name (string) Number of Tickets (integer) Seat Type (string: "Standard", "Premium", "VIP")

The ticket prices are:

Standard – 250 units per ticket Premium – 400 units per ticket VIP – 600 units per ticket

The calculation rules:

Total Amount = Number of Tickets × Seat Price

If a customer books more than 4 tickets, they get a 10% discount on the total amount.

If the booking is for VIP seats and the total amount exceeds 3000 units, a 5% luxury tax is added after any discount.

Neha has been asked to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Getter and Setter methods to retrieve and update booking

details if required. A method to calculate the final ticket cost. Objects of the class to represent bookings.

Finally, display each customer's details and final ticket amount.

### ***Input Format***

The first line contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the Booking ID (integer).
- The next line contains the Customer Name (string).
- The next line contains Number of Tickets (integer).
- The next line contains Seat Type ("Standard", "Premium", or "VIP").

### ***Output Format***

For each booking, print:

- Booking ID: <booking\_id>
- Customer Name: <customer\_name>
- Final Ticket Amount: <final\_amount> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

3

Standard

Output: Booking ID: 1001

Customer Name: Ravi Kumar

Final Ticket Amount: 750.0

### ***Answer***

```
import java.util.Scanner;
```

```
class Booking {
```

```
int bookingId;
String customerName;
int tickets;
String seatType;

// Constructor
Booking(int id, String name, int tickets, String type) {
    this.bookingId = id;
    this.customerName = name;
    this.tickets = tickets;
    this.seatType = type;
}

// Getters
int getBookingId() {
    return bookingId;
}

String getCustomerName() {
    return customerName;
}

// Method to calculate final amount
double calculateAmount() {
    double price = 0;

    if (seatType.equals("Standard")) {
        price = 250;
    } else if (seatType.equals("Premium")) {
        price = 400;
    } else if (seatType.equals("VIP")) {
        price = 600;
    }

    double total = tickets * price;

    // Discount for more than 4 tickets
    if (tickets > 4) {
        total = total - (total * 0.10);
    }

    // Luxury tax for VIP
```



```

        if (seatType.equals("VIP") && total > 3000) {
            total = total + (total * 0.05);
        }

        return total;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();
            int tickets = sc.nextInt();
            sc.nextLine();

            String type = sc.nextLine();

            Booking b = new Booking(id, name, tickets, type);

            double amount = b.calculateAmount();

            System.out.println("Booking ID: " + b.getBookingId());
            System.out.println("Customer Name: " + b.getCustomerName());
            System.out.printf("Final Ticket Amount: %.1f\n", amount);
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks : 10/10**

### 3. Problem Statement

Anjali is working as a developer for CityFitness Gym, which wants to build a system to calculate monthly membership fees for gym members based on the type of membership and the number of personal training sessions booked.

Each member's record has:

Member ID (integer) Member Name (string) Membership Type (string: "Basic", "Premium", "Elite") Number of Personal Training Sessions (integer)

The monthly fees are:

Basic – 1000 units Premium – 1500 units Elite – 2000 units

The cost of personal training sessions is 500 units per session.

The calculation rules:

Total Amount = Membership Fee + (Number of Personal Training Sessions × 500) If the number of sessions is more than 5, a 10% discount is applied on the total amount. If the member has Elite membership and the total amount exceeds 4000, an additional 5% service tax is added after discount.

Anjali has been asked to implement this system using:

A class with attributes for member details. A constructor to initialize member details. Getter and Setter methods to retrieve and update member details if required. A method to calculate the final monthly fee. Objects of the class to represent members.

Finally, display each member's details and the final monthly fee.

### ***Input Format***

The first line contains an integer N, representing the number of members.

For each member:

- Next line contains Member ID (integer)
- Next line contains Member Name (string)
- Next line contains Membership Type ("Basic", "Premium", "Elite")
- Next line contains Number of Personal Training Sessions (integer)

### **Output Format**

For each member, print:

- Member ID: <member\_id>
- Member Name: <member\_name>
- Final Monthly Fee: <final\_fee> (The final fee must be rounded to one decimal place)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

1001

Ravi Kumar

Basic

3

Output: Member ID: 1001

Member Name: Ravi Kumar

Final Monthly Fee: 2500.0

### **Answer**

```
import java.util.*;
```

```
class Member {  
    private int memberId;  
    private String memberName;  
    private String membershipType;  
    private int sessions;
```

```
    // Constructor
```

```
    public Member(int memberId, String memberName, String membershipType,  
int sessions) {  
        this.memberId = memberId;  
        this.memberName = memberName;  
        this.membershipType = membershipType;  
        this.sessions = sessions;  
    }  
}
```

```

// Method to calculate fee
public double calculateFee() {
    double membershipFee = 0;

    if (membershipType.equals("Basic")) {
        membershipFee = 1000;
    } else if (membershipType.equals("Premium")) {
        membershipFee = 1500;
    } else if (membershipType.equals("Elite")) {
        membershipFee = 2000;
    }

    double total = membershipFee + (sessions * 500);

    // Discount
    if (sessions > 5) {
        total = total * 0.9;
    }

    // Elite tax
    if (membershipType.equals("Elite") && total > 4000) {
        total = total * 1.05;
    }

    return total;
}

public int getMemberId() {
    return memberId;
}

public String getMemberName() {
    return memberName;
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
    }
}

```

```

for (int i = 0; i < n; i++) {
    int id = sc.nextInt();
    sc.nextLine(); // FIX: consume newline

    String name = sc.nextLine();
    String type = sc.nextLine();
    int sessions = sc.nextInt();

    Member m = new Member(id, name, type, sessions);

    double fee = m.calculateFee();

    System.out.println("Member ID: " + m.getMemberId());
    System.out.println("Member Name: " + m.getMemberName());
    System.out.printf("Final Monthly Fee: %.1f\n", fee);
}

sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Neha is working as a developer for CityQuiz Platform, which wants to build a system to calculate quiz scores and identify top scorers among participants.

Each participant's record has:

Participant ID (integer) Participant Name (string) An array of scores in 5 quiz rounds (integers, each between 0 and 100)

The system must calculate:

Total Score = sum of scores in all 5 rounds. Average Score = Total Score ÷ 5. If a participant scores above 80 in all rounds, a bonus of 10 points is added to the total score. Identify the Top Scorer among all participants. If two participants have the same total score, the one with the lower

Participant ID is considered the top scorer.

Neha has been asked to implement this system using:

A class with attributes for participant details. A constructor to initialize participant details. Getter and setter methods to retrieve or update participant details. A method to calculate total score and average score (including bonus if applicable). Objects of the class to represent participants.

Finally, display each participant's details and announce the Top Scorer.

### ***Input Format***

The first line of input contains an integer N, representing the number of participants.

For each participant:

- Next line: Participant ID (integer)
- Next line: Participant Name (string)
- Next line: 5 integers separated by spaces (scores for 5 quiz rounds)

### ***Output Format***

For each participant:

- Participant ID: <participant\_id>
- Participant Name: <participant\_name>
- Total Score: <total\_score>
- Average Score: <average\_score>

Finally, print "Top Scorer: <participant\_name> with <total\_score> points"

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

1001

Ravi Kumar

85 90 88 92 87

Output: Participant ID: 1001

Participant Name: Ravi Kumar

Total Score: 452

Average Score: 90

Top Scorer: Ravi Kumar with 452 points

### **Answer**

```
import java.util.*;
```

```
class Participant {  
    private int id;  
    private String name;  
    private int[] scores;  
    private int total;  
    private int average;
```

```
    // Constructor
```

```
    public Participant(int id, String name, int[] scores) {  
        this.id = id;  
        this.name = name;  
        this.scores = scores;  
        calculateScore();  
    }
```

```
    // Method to calculate total and average (with bonus)
```

```
    public void calculateScore() {  
        total = 0;  
        boolean bonus = true;
```

```
        for (int s : scores) {  
            total += s;  
            if (s <= 80) {  
                bonus = false;  
            }  
        }  
    }
```

```
        if (bonus) {  
            total += 10;
```

```
}  
    average = total / 5;  
}  
  
// Getters  
public int getId() {  
    return id;  
}  
  
public String getName() {  
    return name;  
}  
  
public int getTotal() {  
    return total;  
}  
  
public int getAverage() {  
    return average;  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        int n = Integer.parseInt(sc.nextLine());  
        Participant[] p = new Participant[n];  
  
        for (int i = 0; i < n; i++) {  
            int id = Integer.parseInt(sc.nextLine());  
            String name = sc.nextLine();  
  
            int[] scores = new int[5];  
            String[] input = sc.nextLine().split(" ");  
            for (int j = 0; j < 5; j++) {  
                scores[j] = Integer.parseInt(input[j]);  
            }  
  
            p[i] = new Participant(id, name, scores);  
        }  
    }  
}
```



```
// Find top scorer
Participant top = p[0];

for (int i = 0; i < n; i++) {
    System.out.println("Participant ID: " + p[i].getId());
    System.out.println("Participant Name: " + p[i].getName());
    System.out.println("Total Score: " + p[i].getTotal());
    System.out.println("Average Score: " + p[i].getAverage());
}

for (int i = 1; i < n; i++) {
    if (p[i].getTotal() > top.getTotal() ||
        (p[i].getTotal() == top.getTotal() && p[i].getId() < top.getId())) {
        top = p[i];
    }
}

System.out.println("Top Scorer: " + top.getName() + " with " + top.getTotal()
+ " points");
}
```

**Status :** Correct

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Arrays\_Week 3\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 18

#### Section 1 : MCQ

1. What will be the output of the following code?

```
class Student {  
    String name;  
    int age;  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    void display() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student student = new Student("Alice", 20);
```

```
        student.display();  
    }  
}
```

**Answer**

Name: Alice, Age: 20

**Status :** Correct

**Marks :** 1/1

2. What will be the output of the following code?

```
class Test {  
    Test() {  
        System.out.println("Constructor Called");  
    }  
    protected void finalize() {  
        System.out.println("Finalize Called");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Test testObj = new Test();  
        testObj = null;  
        System.gc();  
    }  
}
```

**Answer**

Constructor Called

**Status :** Wrong

**Marks :** 0/1

3. What will be the output of the following code?

```
class Test {  
    int x;  
    Test(int x) {  
        this.x = x;  
    }  
}
```

```
}  
void display() {  
    System.out.println("Value of x: " + x);  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Test testObj = new Test(10);  
        testObj.display();  
    }  
}
```

**Answer**

Value of x: 10

**Status :** Correct

**Marks :** 1/1

4. What will happen if a class does not define a constructor explicitly?

**Answer**

A default constructor will be provided automatically.

**Status :** Correct

**Marks :** 1/1

5. What will be the output of the following code?

```
class Counter {  
    static int total = 0;  
    int value;  
  
    Counter(int value) {  
        this.value = value;  
        total += this.value;  
    }  
}
```

```
public static void main(String[] args) {  
    new Counter(2);  
}
```

```
        new Counter(3);  
        new Counter(5);  
        System.out.println(total);  
    }  
}
```

**Answer**

10

**Status :** Correct

**Marks :** 1/1

6. What will be the output of the following code?

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name;  
    }  
    void print() {  
        System.out.println(this.name);  
    }  
    public static void main(String[] args) {  
        Student s = new Student("Alice");  
        s.print();  
    }  
}
```

**Answer**

Alice

**Status :** Correct

**Marks :** 1/1

7. What will be the output of the following code?

```
class Demo {  
    static int x = 10;  
    int y;  
    Demo(int y) {
```

```
this.y = y;  
x += this.y;  
}
```

```
public static void main(String[] args) {  
    new Demo(5);  
    new Demo(10);  
    System.out.println(x);  
}  
}
```

**Answer**

25

**Status :** Correct

**Marks :** 1/1

8. What is the output of the following code?

```
class OuterClass {  
    static class StaticClass {  
        int num;  
        StaticClass(int num) {  
            this.num = num;  
        }  
        void display() {  
            System.out.println("Number: " + num);  
        }  
    }  
    public static void main(String[] args) {  
        StaticClass sc = new StaticClass(10);  
        sc.display();  
    }  
}
```

**Answer**

Number: 10

**Status :** Correct

**Marks :** 1/1

9. What is the main characteristic of a static class in Java?

**Answer**

It can be instantiated without the need for an object of the outer class

**Status : Correct**

**Marks : 1/1**

10. Can a static class in Java access the non-static members of the outer class?

**Answer**

Yes, by creating an instance of the outer class

**Status : Wrong**

**Marks : 0/1**

11. What happens when you attempt to create an object of a static class in Java from outside the outer class?

**Answer**

It will compile and run fine

**Status : Correct**

**Marks : 1/1**

12. Which of the following is true for a static class in Java?

**Answer**

A static class can have both static and non-static members

**Status : Correct**

**Marks : 1/1**

13. Static nested classes doesn't have access to \_\_\_\_\_ from enclosing class.

**Answer**

Any other members

**Status : Correct**

**Marks : 1/1**

14. What will be the output of the given code?

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 5};  
        int n = arr.length;  
        int temp = arr[0];  
  
        for (int i = 0; i < n - 1; i++) {  
            arr[i] = arr[i + 1];  
        }  
        arr[n - 1] = temp;  
  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
}
```

**Answer**

2 3 4 5 1

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following code?

```
class Sample {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6}  
        };  
        System.out.println(matrix[1][2]);  
    }  
}
```

**Answer**

6

**Status :** Correct

**Marks :** 1/1



16. What will be the output of the following code?

```
class Sample {  
    public static void main(String[] args) {  
        int[][] data = {  
            {1, 2},  
            {3, 4}  
        };  
        int sum = 0;  
  
        for (int[] row : data) {  
            for (int val : row) {  
                sum += val;  
            }  
        }  
  
        System.out.println("Sum = " + sum);  
    }  
}
```

**Answer**

Sum = 10

**Status : Correct**

**Marks : 1/1**

17. What will be the output of the following code?

```
class M {  
    public static void main(String[] args) {  
        int[][] arr = {  
            {1, 2},  
            {3, 4},  
            {5, 6}  
        };  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i][0] + " ");  
        }  
    }  
}
```

```
}
```

**Answer**

1 3 5

**Status :** Correct

**Marks :** 1/1

18. What will be the output of the following code?

```
class ReverseArray {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3, 4};  
        for (int i = 0; i < a.length / 2; i++) {  
            int temp = a[i];  
            a[i] = a[a.length - 1 - i];  
            a[a.length - 1 - i] = temp;  
        }  
        for (int i : a)  
            System.out.print(i + " ");  
    }  
}
```

**Answer**

4 3 2 1

**Status :** Correct

**Marks :** 1/1

19. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[][] arr = {  
            {5, 6, 7},  
            {8, 9, 10}  
        };  
        System.out.println(arr[0][2]);  
    }  
}
```

**Answer**

7

**Status :** Correct

**Marks :** 1/1

20. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[][] a = {  
            {1, 2},  
            {3, 4}  
        };  
        int sum = 0;  
        for (int i = 0; i < a.length; i++)  
            for (int j = 0; j < a[0].length; j++)  
                sum += a[i][j];  
        System.out.println(sum);  
    }  
}
```

**Answer**

10

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Arrays\_Week 3\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 30

### Section 1 : Coding

#### 1. Problem Statement

John is curious to know how many leap years he has lived through. He wants a program where he can input his birth year and the current year, and the program calculates the total number of leap years between them.

Leap Year Conditions: A year is considered a leap year if:

It is divisible by 4, and It is not divisible by 100 unless it is also divisible by 400.

Implement a class AgeCalculatorFunctions with a constructor AgeCalculatorFunctions(int birthYear, int currentYear) to initialize the birth year. The class should contain a method to count the number of leap years from the birth year to the current year.

**Input Format**

The first line contains an integer birthYear, representing Alex's birth year.

The second line contains an integer currentYear, representing the current year.

### **Output Format**

The output displays an integer, representing the total number of leap years Alex has experienced.

Refer to the sample output for format specifications.

### **Sample Test Case**

Input: 1990

2023

Output: 8

### **Answer**

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class AgeCalculatorFunctions {  
    int birthYear, currentYear;
```

```
    // Constructor
```

```
    AgeCalculatorFunctions(int birthYear, int currentYear) {  
        this.birthYear = birthYear;  
        this.currentYear = currentYear;  
    }
```

```
    // Method to count leap years
```

```
    public int calculate.caluculateLeapYears() {  
        int count = 0;
```

```
        for (int year = birthYear; year <= currentYear; year++) {  
            if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {  
                count++;  
            }  
        }
```

```
        return count;
```

```

    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int birthYear = scanner.nextInt();
        int currentYear = scanner.nextInt();
        AgeCalculatorFunctions calculator = new AgeCalculatorFunctions(birthYear,
currentYear);
        System.out.println(calculator.calculateLeapYears());
        scanner.close();
    }
}

```

**Status : Wrong**

**Marks : 0/10**

## 2. Problem Statement

Gadha is preparing to buy groceries at the local market but is unsure about the currency denominations to use when paying the shopkeeper. Create a static class named CurrencyBreakdown and static method calculateBreakdown that helps calculate the optimal way to break down the total bill into various currency denominations. denominations are 100, 50, 20, 10, 5, and 1.

### **Input Format**

The first line consists of the amount to be paid as an integer.

### **Output Format**

The first line should display the message indicating the amount for which the currency breakdown is being provided:

"Currency Denomination Breakdown for [amount]:"

For each denomination that is used (i.e., when the count of that denomination is greater than 0), output the denomination and the count: "denomination(s): count" (ex : 100s: 1)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 250

Output: Currency Denomination Breakdown for 250:

100s: 2

50s: 1

### **Answer**

```
import java.util.Scanner;
```

```
public class Main {
```

```
    static class CurrencyBreakdown {
```

```
        public static void calculateBreakdown(int amount) {
```

```
            int count;
```

```
            System.out.println("Currency Denomination Breakdown for " + amount +  
            ":");
```

```
            count = amount / 100;
```

```
            if (count > 0) {
```

```
                System.out.println("100s: " + count);
```

```
                amount %= 100;
```

```
            }
```

```
            count = amount / 50;
```

```
            if (count > 0) {
```

```
                System.out.println("50s: " + count);
```

```
                amount %= 50;
```

```
            }
```

```
            count = amount / 20;
```

```
            if (count > 0) {
```

```
                System.out.println("20s: " + count);
```

```
                amount %= 20;
```

```
            }
```

```

        count = amount / 10;
        if (count > 0) {
            System.out.println("10s: " + count);
            amount %= 10;
        }

        count = amount / 5;
        if (count > 0) {
            System.out.println("5s: " + count);
            amount %= 5;
        }

        count = amount / 1;
        if (count > 0) {
            System.out.println("1s: " + count);
        }
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int amount = scanner.nextInt();
    scanner.close();

    System.out.println("Currency Denomination Breakdown for " + amount + ":");
    CurrencyBreakdown.calculateBreakdown(amount);
}
}

```

**Status : Wrong**

**Marks : 0/10**

### 3. Problem Statement

Meet Ryan, a seasoned stock market enthusiast who takes his investments seriously. Ryan has been meticulously tracking the stock prices of a tech company over a specific time period, eager to understand the percentage change in stock prices. To assist him in making well-informed investment decisions, you are tasked with creating a "Percentage Change Calculator" utility using the static method, which will take the initial and final stock prices as input and provide Ryan with the percentage change, ensuring that he can continue to make informed investment



decisions confidently.

### ***Input Format***

The input consists of two decimal numbers of 'double' data type, "initialValue" and "finalValue" separated by spaces.

### ***Output Format***

If "initialValue" is zero, the program should print an error message: "Initial value cannot be zero"

If the percentage change is calculated successfully, the program should print the percentage change as a decimal number of 'double' data type with a precision of two decimal places and the "%" symbol.

The output format should be: "Percentage Change: {value}%"

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5992.92 10000.00

Output: Percentage Change: 66.86%

### ***Answer***

```
import java.util.Scanner;
public class Main {
    // Static method to calculate percentage change
    public static double calculatePercentageChange(double initialValue, double
finalValue) {
        return ((finalValue - initialValue) / initialValue) * 100;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double initialValue = sc.nextDouble();
        double finalValue = sc.nextDouble();
```

```

// Check if initial value is zero
if (initialValue == 0) {
    System.out.println("Initial value cannot be zero");
} else {
    double result = calculatePercentageChange(initialValue, finalValue);
    System.out.printf("Percentage Change: %.2f%%", result);
}

sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Rosh is intrigued by numerical patterns. Today, she stumbled upon a puzzle while working with arrays. She wants to compute the sum of the third-largest and second-smallest elements from a list of integers. She seeks your help to implement a program that solves this for her efficiently.

##### ***Input Format***

The first line of input is an integer N, representing the size of the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

##### ***Output Format***

The output displays a single integer representing the sum of the third-largest and second-smallest elements in the array.

Refer to the sample output for the formatting specifications.

##### ***Sample Test Case***

Input: 10  
10 20 30 40 50 60 70 80 90 100

Output: 100

**Answer**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        // Read array elements
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Sort the array
        Arrays.sort(arr);

        // Second smallest and third largest
        int secondSmallest = arr[1];
        int thirdLargest = arr[n - 3];

        // Output result
        System.out.print(secondSmallest + thirdLargest);
    }
}
```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement:

Emma, a budding computer vision enthusiast, is working on a challenging image processing project. She has a square image represented as a 2D matrix of integers. As part of a special filter operation, she needs to rotate the image by 90 degrees clockwise, but there's a twist — she must perform the rotation in-place, using no extra space.

This means Emma has to rotate the matrix without creating a new one.

Your task is to help her implement a Java program that takes this square matrix as input and rotates it within the same structure.

Can you help Emma efficiently rotate the image so that her project can move to the next stage?

### ***Input Format***

The first line of input contains a single integer  $n$ , representing the number of rows and columns of the square matrix (i.e., the matrix is of size  $n \times n$ ).

The next  $n$  lines each contain  $n$  space-separated integers, representing the elements of each row of the 2D array.

### ***Output Format***

The first line of output prints "Rotated 2D Array:"

The next  $n$  lines of output print the rotated matrix.

Each line contains  $n$  space-separated integers representing a row of the rotated matrix.

Refer to the sample output for format specification.

### ***Sample Test Case***

Input: 3

1 2 3

4 5 6

7 8 9

Output: Rotated 2D Array:

7 4 1

8 5 2

9 6 3

### ***Answer***

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();  
int[][] mat = new int[n][n];
```

```
// Input matrix  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        mat[i][j] = sc.nextInt();  
    }  
}
```

```
// Step 1: Transpose the matrix  
for (int i = 0; i < n; i++) {  
    for (int j = i; j < n; j++) {  
        int temp = mat[i][j];  
        mat[i][j] = mat[j][i];  
        mat[j][i] = temp;  
    }  
}
```

```
// Step 2: Reverse each row  
for (int i = 0; i < n; i++) {  
    int left = 0, right = n - 1;  
    while (left < right) {  
        int temp = mat[i][left];  
        mat[i][left] = mat[i][right];  
        mat[i][right] = temp;  
        left++;  
        right--;  
    }  
}
```

```
// Output  
System.out.println("Rotated 2D Array:");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.print(mat[i][j] + " ");  
    }  
    System.out.println();  
}
```

```
    sc.close();  
  }  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Arrays\_Week 3\_Yourself

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Yong is a real estate developer who wants to calculate property taxes based on the value and location of a property. The tax rates are as follows:

Urban: 3% Suburban: 2% Rural: 1% He needs a simple program where users can enter the property value and location, and get the correct tax amount.

To keep the program efficient, use:

A static class named: TaxAssessorA static method inside it named: calculatePropertyTax This method should take the property value and location as input and return the tax amount.

Formula:

property tax = Property Value \* tax Rate;

### ***Input Format***

The first line of input contains a double value 'd' representing the value of the property.

The second line contains a string value 's' which represents the location of the property, which can be one of the following:

"urban," "suburban," or "rural."

### ***Output Format***

The output displays the calculated property tax as follows: Property Tax:  
<tax\_amount>

Here the tax\_amount represents the amount to be paid rounded to two decimal places.

If the location is invalid, the output prints "Invalid location." and does not display a property tax value.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 100000.25

urban

Output: Property Tax: 3000.01

### ***Answer***

```
import java.util.Scanner;
public class Main {
    static class TaxAssessor {

        // Static method
        public static double calculatePropertyTax(double value, String location) {
            location = location.toLowerCase();

            if (location.equals("urban")) {
                return value * 0.03;
```



```

    } else if (location.equals("suburban")) {
        return value * 0.02;
    } else if (location.equals("rural")) {
        return value * 0.01;
    } else {
        return -1; // invalid
    }
}
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    double propertyValue = scanner.nextDouble();
    scanner.nextLine();

    String location = scanner.nextLine();

    double propertyTax = TaxAssessor.calculatePropertyTax(propertyValue,
location);

    if (propertyTax >= 0) {
        System.out.printf("Property Tax: %.2f%n", propertyTax);
    } else {
        System.out.println("Invalid location.");
    }

    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement:

Emma loves playing with words and has created a static method to reverse the order of words in a sentence without reversing the characters of each word. Write a program to help Emma in implementing the same.

**Input Format**

The input consists of a sentence which is a string containing words separated by spaces.

### **Output Format**

The output prints a single line containing the sentence with the words in reverse order, separated by a single space.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: The quick brown fox jumps over the lazy dog.

Output: dog. lazy the over jumps fox brown quick The

### **Answer**

```
import java.util.Scanner;

class SentenceReverser {
    public static String reverseWords(String sentence) {
        String[] words = sentence.split(" ");
        String result = "";

        for (int i = words.length - 1; i >= 0; i--) {
            result += words[i];
            if (i != 0) {
                result += " ";
            }
        }
        return result;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String input = sc.nextLine();
        System.out.println(SentenceReverser.reverseWords(input));
    }
}
```

```
        sc.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Michael, a computer science student, is working on a project to develop a GPA calculator. He wants to create a program that accepts grades for multiple subjects and calculates the Grade Point Average (GPA) for a student.

Additionally, he wants to implement a scholarship eligibility check, where a student qualifies for a scholarship if their GPA is 3.5 or higher and they have taken at least 5 subjects.

To achieve this, create a GPACalculator class that encapsulates the logic for GPA calculation and scholarship eligibility. The Main class should handle user input and output. Ensure that the program utilizes this keyword and constructor for proper object-oriented design.

#### **Input Format**

The first line of input consists of an integer N (the number of subjects).

The second line consists of N double values, representing the grade for each subject, each separated by space.

#### **Output Format**

The first line of output prints the calculated average GPA for the student, as a decimal number rounded to two decimal places.

The second line prints the eligibility status for a scholarship.

1. If the student's GPA is 3.5 or higher and they've taken at least 5 subjects, the program should output "Eligible for a scholarship"
2. Otherwise, it should output "Not eligible for a scholarship"

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 3

3.5 4.0 3.0

Output: GPA: 3.50

Not eligible for a scholarship

### **Answer**

```
import java.util.Scanner;
```

```
// You are using Java  
class GPACalculator {
```

```
    int n;  
    double[] grades;
```

```
    // Constructor matching your Main  
    GPACalculator(double[] grades) {  
        this.grades = grades;  
        this.n = grades.length;  
    }
```

```
    double getGPA() {  
        double sum = 0;  
        for (int i = 0; i < n; i++) {  
            sum += grades[i];  
        }  
        return sum / n;  
    }
```

```
    boolean isEligibleForScholarship() {  
        return (getGPA() >= 3.5 && n >= 5);  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        int numSubjects = scanner.nextInt();  
        double[] studentGrades = new double[numSubjects];  
        for (int i = 0; i < numSubjects; i++) {
```

```

        studentGrades[i] = scanner.nextDouble();
    }
    GPA Calculator gpaCalculator = new GPA Calculator(studentGrades);
    System.out.printf("GPA: %.2f\n", gpaCalculator.getGPA());
    if (gpaCalculator.isEligibleForScholarship()) {
        System.out.println("Eligible for a scholarship");
    } else {
        System.out.println("Not eligible for a scholarship");
    }
    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

You are developing a warehouse management system for a shipping company. The system uses an integer array to represent the weights of packages in a specific order. To verify that the weight capacity is not exceeded, the program needs to calculate the sum of the weights of the first and last packages in the list.

Task:

Write a code to calculate the sum of the weights of the first and last packages in the list. The program should take an integer array as input and return the total weight of the first and last packages.

##### **Input Format**

The first line of the input is an integer N representing the size of the array.

The second line of the input is N space-separated integer values.

##### **Output Format**

The output is displayed in the following format:

"Sum of the first and last elements: <<Sum>>"

Refer to the sample output for formatting specifications.

**Sample Test Case**

Input: 5

10 20 30 40 50

Output: Sum of the first and last elements: 60

**Answer**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input size
        int n = sc.nextInt();

        int[] arr = new int[n];

        // Input array elements
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Calculate sum of first and last elements
        int sum = arr[0] + arr[n - 1];

        // Output
        System.out.println("Sum of the first and last elements: " + sum);

        sc.close();
    }
}
```

**Status :** Correct

**Marks :** 10/10

**5. Problem Statement**

Sesha is developing a weather monitoring system for a region with

multiple weather stations. Each weather station collects temperature data hourly and stores it in a 2D array.

Write a program that can add the temperature data from two different weather stations to create a combined temperature record for the region.

### ***Input Format***

The first line of input consists of two space-separated integers N and M, representing the number of rows and columns of the matrices, respectively.

The next N lines consist of M space-separated integers, representing the values of the first matrix.

The following N lines consist of M space-separated integers, representing the values of the second matrix.

### ***Output Format***

The output prints the addition of the two matrices in N rows and M columns, representing the combined temperature record.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 3 3

1 2 3

4 5 6

7 8 9

1 1 1

2 2 2

3 3 3

Output: 2 3 4

6 7 8

10 11 12

### ***Answer***

```
import java.util.Scanner;
```

```
public class Main {
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);
```

```
    // Input size
```

```
    int n = sc.nextInt();
```

```
    int m = sc.nextInt();
```

```
    int[][] A = new int[n][m];
```

```
    int[][] B = new int[n][m];
```

```
    int[][] C = new int[n][m];
```

```
    // Input first matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            A[i][j] = sc.nextInt();
```

```
        }
```

```
    }
```

```
    // Input second matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            B[i][j] = sc.nextInt();
```

```
        }
```

```
    }
```

```
    // Addition of matrices
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            C[i][j] = A[i][j] + B[i][j];
```

```
        }
```

```
    }
```

```
    // Output result
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            System.out.print(C[i][j] + " ");
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
    sc.close();
```

```
}
```



}

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Arrays\_Week 3\_Home

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Arun is assigned the task of developing a program that calculates the product of two numbers using a static method. The program should be designed to allow users to input two numbers, and the static method should perform the multiplication operation.

Help Arun to complete this program.

#### ***Input Format***

The input consists of two space-separated integers.

#### ***Output Format***

The output prints the product of the given two integers.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 2 3

Output: 6

### **Answer**

```
import java.util.Scanner;

public class Main {

    public static int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
        int b = sc.nextInt();

        System.out.println(multiply(a, b));
    }
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Euclid, the renowned mathematician, was working on right-angled triangles when his apprentice, Hypatia, asked a simple yet intriguing question. Hypatia said, "Master Euclid, I have a right-angled triangle where the two legs measure 3 inches and 4 inches. Can you help me determine the length of the hypotenuse?"

To answer Hypatia's question, create a static class PythagorasSolver with a static method calculateHypotenuse. This method should take the

lengths of the two legs (a and b) of a right-angled triangle and use the Pythagorean theorem to calculate and return the length of the hypotenuse.

Additionally, declare a static variable inside the class to store the computed hypotenuse length.

Formula:  $\text{hypotenuse} = \sqrt{a^2 + b^2}$

Where:

a is the length of the first side (leg) b is the length of the second side (leg)

#### ***Input Format***

The input consists of two space-separated integers, representing the lengths of the two shorter sides of the right-angled triangle.

#### ***Output Format***

The output displays a double value up to two decimal places representing the value of the hypotenuse side.

Refer to the sample output for the formatting specifications.

#### ***Sample Test Case***

Input: 5 12

Output: 13.00

#### ***Answer***

```
import java.util.Scanner;
```

```
public class Main {
```

```
    static class PythagorasSolver {
```

```
        // Static variable
```

```
        static double hypotenuse;
```

```
        // Static method
```

```
        static double calculateHypotenuse(int a, int b) {
```

```

        hypotenuse = Math.sqrt(a * a + b * b);
        return hypotenuse;
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    int sideA = scanner.nextInt();
    int sideB = scanner.nextInt();

    double hypotenuse = PythagorasSolver.calculateHypotenuse(sideA, sideB);

    System.out.printf("%.2f%n", hypotenuse);

    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Sharon is creating a program that finds the first repeated element in an integer array. The program should efficiently identify the first element that appears more than once in the given array. If no such element is found, it should appropriately display a message.

Help Sharon to complete the program.

#### ***Input Format***

The first line of input consists of an integer  $n$ , representing the number of elements in the array.

The second line consists of  $n$  space-separated integers, representing the array elements.

#### ***Output Format***

If a repeated element is found, print the first element that appears more than once.

If no repeated element is found, print "No repeated element found in the array".

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 8

12 21 13 14 21 36 47 21

Output: 21

### **Answer**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        // Read array elements
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Find first repeated element
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println(arr[i]);
                    return;
                }
            }
        }

        // If no repeated element found
        System.out.println("No repeated element found in the array");
    }
}
```

Status : Correct

Marks : 10/10

#### 4. Problem Statement

Monica is interested in finding a treasure but the key to opening is to get the sum of the main diagonal elements and secondary diagonal elements.

Write a program to help Monica find the diagonal sum of a square 2D array.

Note: The main diagonal of the array consists of the elements traversing from the top-left corner to the bottom-right corner. The secondary diagonal includes elements from the top-right corner to the bottom-left corner.

#### **Input Format**

The first line of input consists of an integer N, representing the number of rows and columns.

The following N lines consist of N space-separated integers, representing the 2D array elements.

#### **Output Format**

The first line of output prints "Sum of the main diagonal: " followed by an integer, representing the sum of the main diagonal.

The second line prints "Sum of the secondary diagonal: " followed by an integer, representing the sum of the secondary diagonal.

Refer to the sample output for formatting specifications.

#### **Sample Test Case**

Input: 3

1 2 3

4 5 6

7 8 9

Output: Sum of the main diagonal: 15

Sum of the secondary diagonal: 15

**Answer**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[][] arr = new int[n][n];

        // Read matrix
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        int mainSum = 0;
        int secondarySum = 0;

        // Calculate diagonal sums
        for (int i = 0; i < n; i++) {
            mainSum += arr[i][i];           // main diagonal
            secondarySum += arr[i][n - 1 - i]; // secondary diagonal
        }

        // Output
        System.out.println("Sum of the main diagonal: " + mainSum);
        System.out.println("Sum of the secondary diagonal: " + secondarySum);
    }
}
```

**Status : Correct****Marks : 10/10**



# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Inheritance\_Week 4\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 18

#### Section 1 : MCQ

1. What will be the output of the following program?

```
class A {  
    public int i;  
    private int j;  
}  
class B extends A {  
    void display() {  
        super.j = super.i + 1;  
        System.out.println(super.i + " " + super.j);  
    }  
}  
class inheritance {  
    public static void main(String args[]) {  
        B obj = new B();  
        obj.i=1;  
    }  
}
```

```
obj.j=2;  
obj.display();  
}  
}
```

**Answer**

Compile Time Error

**Status :** Correct

**Marks :** 1/1

2. Select the correct keyword for implementing inheritance through the class.

**Answer**

extends

**Status :** Correct

**Marks :** 1/1

3. Which of the following is the correct way for class B to inherit from class A?

**Answer**

class B inherits class A {}

**Status :** Wrong

**Marks :** 0/1

4. What will be the output of the following program?

```
class Vehicle {  
    String type = "Vehicle";  
}
```

```
class Car extends Vehicle {  
    String type = "Car";  
}
```

```
class Test {  
    public static void main(String[] args) {
```

```
Car c = new Car();  
System.out.println(c.type);  
}  
}
```

**Answer**

Car

**Status :** Correct

**Marks :** 1/1

5. What will be the output of the following program?

```
class Parent {  
    Parent() {  
        System.out.println("Parent Constructor");  
    }  
}  
class Child extends Parent {  
    Child() {  
        System.out.println("Child Constructor");  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        Child c = new Child();  
    }  
}
```

**Answer**

Parent Constructor Child Constructor

**Status :** Correct

**Marks :** 1/1

6. Which of the following is an example of Multilevel Inheritance?

**Answer**

A B C

**Status :** Correct

**Marks :** 1/1

7. How many classes are involved at a minimum in Multilevel Inheritance?

**Answer**

3

**Status :** Correct

**Marks :** 1/1

8. In multilevel inheritance, how does the constructor execution happen?

**Answer**

Parent class constructor is called first

**Status :** Correct

**Marks :** 1/1

9. What will be the output of the following Java program?

```
class A {  
    void display() {  
        System.out.println("Class A");  
    }  
}
```

```
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}
```

```
class C extends B {  
    void print() {  
        System.out.println("Class C");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
    }  
}
```

```
obj.show();  
obj.print();  
}  
}
```

**Answer**

Class AClass BClass C

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following code?

```
class Parent {  
    Parent() {  
        System.out.println("Parent Constructor");  
    }  
}
```

```
class Child extends Parent {  
    Child() {  
        System.out.println("Child Constructor");  
    }  
}
```

```
class GrandChild extends Child {  
    GrandChild() {  
        System.out.println("GrandChild Constructor");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        GrandChild obj = new GrandChild();  
    }  
}
```

**Answer**

Parent ConstructorChild ConstructorGrandChild Constructor

**Status :** Correct

**Marks :** 1/1

11. What will be the output of the following program?

```
class A {  
    int x = 10;  
}  
  
class B extends A {  
    int x = 20;  
}  
  
class C extends B {  
    int x = 30;  
  
    void display() {  
        System.out.println(x);  
        System.out.println(super.x);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
    }  
}
```

**Answer**

3020

**Status :** Correct

**Marks :** 1/1

12. What will be the output of the following code?

```
class A {  
    A() {  
        System.out.println("A Constructor");  
    }  
}
```

```
class B extends A {  
    B() {  
        System.out.println("B Constructor");  
    }  
}
```

```
class C extends B {  
    C() {  
        System.out.println("C Constructor");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
    }  
}
```

**Answer**

A ConstructorB ConstructorC Constructor

**Status :** Correct

**Marks :** 1/1

13. What will be the output of the following code?

```
class A {  
    void display() {  
        System.out.println("Display A");  
    }  
}
```

```
class B extends A {  
    void display() {  
        System.out.println("Display B");  
    }  
}
```

```
class C extends B {  
    void display() {
```

```

    super.display();
}
}

class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
    }
}

```

**Answer**

Display B

**Status :** Correct

**Marks :** 1/1

14. What will be the output of the following code?

```

class A {
    void show(int x) {
        System.out.println("A: " + x);
    }
}

class B extends A {
    void show(double x) {
        System.out.println("B: " + x);
    }
}

class Test {
    public static void main(String[] args) {
        B obj = new B();
        obj.show(5);
    }
}

```

**Answer**

A: 5



Status : Correct

Marks : 1/1

15. What will be the output of the following program?

```
class A {  
    A() {  
        System.out.println("A Constructor");  
    }  
}  
  
class B extends A {  
    B() {  
        System.out.println("B Constructor");  
    }  
}  
  
class C extends B {  
    C() {  
        System.out.println("C Constructor");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        B obj = new B();  
    }  
}
```

Answer

A ConstructorB Constructor

Status : Correct

Marks : 1/1

16. What will be the output of the following code?

```
class A {  
    int calculate(int x) {  
        return x * 2;  
    }  
}
```

```

    }
}

class B extends A {
    int calculate(int x) {
        return super.calculate(x) + 3;
    }
}

class C extends B {
    int calculate(int x) {
        return super.calculate(x) - 1;
    }
}

class Test {
    public static void main(String[] args) {
        C obj = new C();
        System.out.println(obj.calculate(5));
    }
}

```

**Answer**

14

**Status :** Wrong

**Marks :** 0/1

17. What will be the output of the following code?

```

class A {
    int sum(int x) {
        return x + 2;
    }
}

class B extends A {
    int sum(int x) {
        return super.sum(x) * 2;
    }
}

```

```

    }
    class C extends B {
        int sum(int x) {
            return super.sum(x) - 3;
        }
    }

    class Test {
        public static void main(String[] args) {
            C obj = new C();
            System.out.println(obj.sum(4));
        }
    }

```

**Answer**

9

**Status :** Correct

**Marks :** 1/1

18. What is the output of the given code?

```

class Language {
    void type() {
        System.out.print("Programming ");
    }
}

```

```

class Java extends Language {
    void type() {
        System.out.print("Java ");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Language obj = new Java();
        obj.type();
    }
}

```

```
}
```

**Answer**

Java

**Status :** Correct

**Marks :** 1/1

19. What is the output of the given code?

```
class Fruit {  
    String name = "Fruit";  
}
```

```
class Apple extends Fruit {  
    String name = "Apple";  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Fruit obj = new Apple();  
        System.out.print(obj.name);  
    }  
}
```

**Answer**

Fruit

**Status :** Correct

**Marks :** 1/1

20. What is the output of the given code?

```
class Animal {  
    final void sound() {  
        System.out.print("Animal");  
    }  
}
```

```
class Dog extends Animal {  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        new Dog().sound();  
    }  
}
```

**Answer**

Animal

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Inheritance\_Week 4\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 20

### **Section 1 : Coding**

#### **1. Problem Statement**

Design a Java program to model a basic vehicle system using inheritance.

Implement a base class Vehicle with attributes for distance and time.

Then, create a derived class Car that includes a brand name, calculates speed using the formula  $\text{Speed} = \text{Distance} / \text{Time}$ , and adds extra utility.

In the Car class, implement:

A method `calc()` to calculate the speed. A final method `display()` to print the brand and speed of the car (rounded to 2 decimal places) and categorize it as Slow (speed < 20), Average (20–60), or Fast (> 60). An additional method `predictDistance(double newTime)` to calculate how much distance the car would cover in a different given time. An additional method `predictTime(double newDistance)` to estimate how much time the car would take to cover a new distance at the current speed.

The brand name attribute in the Car class should be declared with the final keyword to prevent modification after initialization.

### ***Input Format***

The first line of input contains two double values separated by a space representing the Original distance and Original time (for calculating the initial speed).

The second line contains one double value representing the New time (for predicting distance covered).

The third line contains one double value representing the New distance (for predicting time taken).

### ***Output Format***

The first line of output prints "Brand: " followed by a string representing the brand name of the car.

The second line of output prints the speed of the car rounded to two decimal places, followed by the speed category in parentheses. The format is: Speed: <value> km/hr (Slow/Average/Fast) where the category is "Slow" if speed < 20, "Average" if speed is between 20 and 60 (inclusive), and "Fast" if speed > 60.

The third line of output prints the distance the car is predicted to cover in the given new time, rounded to two decimal places, in the format: Distance covered in <newTime> hours: <distance> km

The fourth line of output prints the estimated time the car will take to cover the specified new distance, rounded to two decimal places, in the format: Time taken to cover <newDistance> km: <time> hours

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 50.0 10.0

5.0

100.0

Output: Brand: Toyota

Speed: 5.00 km/hr (Slow)  
Distance covered in 5.0 hours: 25.00 km  
Time taken to cover 100.0 km: 20.00 hours

**Answer**

```
import java.util.Scanner;

class Vehicle {
    double distance, time;

    Vehicle(double distance, double time) {
        this.distance = distance;
        this.time = time;
    }
}

class Car extends Vehicle {
    final String brand; // final variable
    double speed;

    // Constructor with brand
    Car(String brand, double distance, double time) {
        super(distance, time);
        this.brand = brand;
    }

    // Calculate speed
    void calc() {
        speed = distance / time;
    }

    // Final display method
    final void display() {
        String category;

        if (speed < 20)
            category = "Slow";
        else if (speed <= 60)
            category = "Average";
        else
            category = "Fast";

        System.out.println("Brand: " + brand);
    }
}
```



```

        System.out.printf("Speed: %.2f km/hr (%s)\n", speed, category);
    }

    // Predict distance
    double predictDistance(double newTime) {
        return speed * newTime;
    }

    // Predict time
    double predictTime(double newDistance) {
        return newDistance / speed;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double distance = scanner.nextDouble();
        double time = scanner.nextDouble();
        double newTime = scanner.nextDouble();
        double newDistance = scanner.nextDouble();

        Car car = new Car("Toyota", distance, time);
        car.calc();
        car.display();

        System.out.printf("Distance covered in %.1f hours: %.2f km\n", newTime,
            car.predictDistance(newTime));
        System.out.printf("Time taken to cover %.1f km: %.2f hours\n", newDistance,
            car.predictTime(newDistance));

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Alice is managing an online store and wants to implement a program using

inheritance to calculate the selling price of products after applying discounts.

Guide her by following the instructions:

Create a base class called Product with a public double attribute price. Create a subclass called DiscountedProduct, which extends Product and includes a private double attribute discount rate. This subclass has a method called calculateSellingPrice() to determine the final selling price after applying the discount.

Formula: Discounted selling price = price \* (1 - discount rate)

### ***Input Format***

The first line of input consists of a double value p, the initial price of the product.

The second line consists of a double value d, the discount rate.

### ***Output Format***

The output prints "Rs. X", where X is a double value, representing the calculated discounted selling price, rounded off to two decimal places.

If the discount rate is greater than 1, print "Not applicable".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 50.00

0.20

Output: Rs. 40.00

### ***Answer***

```
import java.util.Scanner;
```

```
class Product {  
    public double price;  
}
```

```
class DiscountedProduct extends Product {
```

```

private double discount;
// Constructor
DiscountedProduct(double price, double discount) {
    this.price = price;
    this.discount = discount;
}
// Method to calculate selling price
double calculateSellingPrice() {
    return price * (1 - discount);
}

// Method to check validity of discount
boolean isValid() {
    return discount <= 1;
}
}

class ProductPricing {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double initialPrice = scanner.nextDouble();
        double discountRate = scanner.nextDouble();
        DiscountedProduct discountedProduct = new
DiscountedProduct(initialPrice, discountRate);
        double sellingPrice = discountedProduct.calculateSellingPrice();

        if (sellingPrice >= 0) {
            System.out.printf("Rs. %.2f%n", sellingPrice);
        } else {
            System.out.println("Not applicable");
        }
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem statement

Write a program that contains a class named "Length" that has the public

member length of a cuboid. It is then multilevel inheritance by the class "Width" which has a public member width of the cuboid. Then it is inherited by the class "Height" to read all the inputs, including height, and it also has a method to return the volume of the cuboid.

### ***Input Format***

The input consists of three integers representing the length, width, and height of the cuboid.

### ***Output Format***

The output displays the volume of the cuboid in the following format: "Volume of the cuboid is: [volume]"

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 10 20 30

Output: Volume of the cuboid is: 6000

### ***Answer***

```
import java.util.Scanner;

class Length {
    public int length;
}

class Width extends Length {
    public int width;
}

class Height extends Width {
    public int height;

    // Method to read inputs (no arguments)
    void getInputs() {
        Scanner sc = new Scanner(System.in);
        length = sc.nextInt();
        width = sc.nextInt();
    }
}
```

```

        height = sc.nextInt();
    }

    // Method to calculate volume
    int getVolume() {
        return length * width * height;
    }
}

public class Main {
    public static void main(String[] args) {
        Height cuboid = new Height();
        cuboid.getInputs();
        cuboid.getVolume();
    }
}

```

**Status : Wrong**

**Marks : 0/10**

#### 4. Problem Statement

Maria, a software developer, is building a grading automation system for Computer Science students. The program calculates the Grade Point Average (GPA) and determines class standing based on the number of credits completed.

The GPA is calculated using the formula:

$$\text{GPA} = 4.0 \times (\text{creditsCompleted} / 120.0)$$
Where 120 credits are required for graduation, and the maximum possible GPA is 4.0.

The class standing is determined as follows:

Freshman: 0-29 credits  
Sophomore: 30-59 credits  
Junior: 60-89 credits  
Senior: 90 or more credits

The program consists of three classes:

Student – Stores the student's name and age, and has a method `displayInfo()` to print these details.  
Undergraduate – Inherits from Student, adds `creditsCompleted`, and includes `calculateGPA()` to compute the GPA and `getClassStanding()` to determine class standing and `displayInfo()` to

display the info.ComputerScienceMajor – Extends Undergraduate, representing a Computer Science major without adding extra functionality. Help Maria implement this system to calculate the GPA and class standing for Computer Science students.

### ***Input Format***

The first line contains the name of the student as a string (at most 20 characters).

The second line contains the age of the student as an integer.

The third line contains the number of credits completed by the student as a double.

### ***Output Format***

The first line of output prints: "Name: " followed by student\_name (the name of the student).

The second line of output prints: "Age: " followed by student\_age (the student's age).

The third line of output prints: "Credits Completed: " followed by credits\_completed (the number of credits completed rounded to one decimal place).

The fourth line of output prints: "GPA: " followed by calculated\_GPA (the GPA rounded to one decimal place).

The fifth line of output prints: "Class Standing: " followed by class\_standing (the determined class standing based on credits completed).

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: John Doe

20

15.00

Output: Name: John Doe  
Age: 20  
Credits Completed: 15.0  
GPA: 0.5  
Class Standing: Freshman

**Answer**

```
import java.util.Scanner;

class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Undergraduate extends Student {
    double creditsCompleted;

    Undergraduate(String name, int age, double creditsCompleted) {
        super(name, age);
        this.creditsCompleted = creditsCompleted;
    }

    double calculateGPA() {
        return 4.0 * (creditsCompleted / 120.0);
    }

    String getClassStanding() {
        if (creditsCompleted < 30)
            return "Freshman";
        else if (creditsCompleted < 60)
            return "Sophomore";
        else if (creditsCompleted < 90)
            return "Junior";
```

```

        else
            return "Senior";
    }

    void displayInfo() {
        super.displayInfo();
        System.out.printf("Credits Completed: %.1f\n", creditsCompleted);
        System.out.printf("GPA: %.1f\n", calculateGPA());
        System.out.println("Class Standing: " + getClassStanding());
    }
}

class ComputerScienceMajor extends Undergraduate {
    ComputerScienceMajor(String name, int age, double creditsCompleted) {
        super(name, age, creditsCompleted);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String name = scanner.nextLine();
        int age = scanner.nextInt();
        double creditsCompleted = scanner.nextDouble();
        ComputerScienceMajor student = new ComputerScienceMajor(name, age,
creditsCompleted);
        student.displayInfo();
        double gpa = student.calculateGPA();
        System.out.printf("GPA: %.1f\n", gpa);
        String classStanding = student.getClassStanding();
        System.out.println("Class Standing: " + classStanding);
    }
}

```

**Status : Wrong**

**Marks : 0/10**

## 5. Problem Statement

Shasha, an HR manager in a multinational corporation, needs a program to compute bonuses for developers and designers using hierarchical inheritance. Implement three classes:



Employee with a method `setBaseSalary(double baseSalary)`. Developer (inherits from Employee) with methods `setWorkHours(int workHours)` and `developerFinalIncome()` and assign Bonus Percentages as 0.10. Designer (inherits from Employee) with methods `setWorkHours(int workHours)` and `designerFinalIncome()` and assign Bonus Percentages as 0.05.

Shasha inputs the base salary and work hours for both roles, and the program computes and displays their final incomes using:

$$\text{Final Income} = \text{Base Salary} + (\text{Base Salary} * \text{Bonus Percentage} * \text{Work Hours} / 100)$$

### ***Input Format***

The first line of input consists of the base salary (double) and working hours (int) of the developer as space-separated values.

The second line consists of the base salary (double) and working hours (int) of the designer as space-separated values.

### ***Output Format***

The first line prints a double value, representing the developer's final income, rounded to two decimal places, in the format: "Rs. X.XX", where X.XX denotes the final calculated income.

The second line prints a double value, representing the designer's final income, rounded to two decimal places, in the format: "Rs. X.XX", where X.XX denotes the final calculated income.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 10000.0 10

10000.0 10

Output: Rs. 10100.00

Rs. 10050.00

### ***Answer***

```
import java.util.Scanner;
```

// You are using Java

```
class Employee {  
    double baseSalary;
```

```
    void setBaseSalary(double baseSalary) {  
        this.baseSalary = baseSalary;  
    }  
}
```

```
class Developer extends Employee {  
    int workHours;
```

```
    void setWorkHours(int workHours) {  
        this.workHours = workHours;  
    }
```

```
    double developerFinalIncome() {  
        return baseSalary + (baseSalary * 0.10 * workHours / 100);  
    }  
}
```

```
class Designer extends Employee {  
    int workHours;
```

```
    void setWorkHours(int workHours) {  
        this.workHours = workHours;  
    }
```

```
    double designerFinalIncome() {  
        return baseSalary + (baseSalary * 0.05 * workHours / 100);  
    }  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);
```

```
        Developer developer = new Developer();  
        double devBaseSalary = scanner.nextDouble();  
        developer.setBaseSalary(devBaseSalary);  
        int devWorkHours = scanner.nextInt();
```

```
developer.setWorkHours(devWorkHours);
```

```
Designer designer = new Designer();  
double desBaseSalary = scanner.nextDouble();  
designer.setBaseSalary(desBaseSalary);  
int desWorkHours = scanner.nextInt();  
designer.setWorkHours(desWorkHours);
```

```
scanner.close();
```

```
developer.developerFinalIncome();
```

```
designer.designerFinalIncome();
```

```
}  
}
```

**Status : Wrong**

**Marks : 0/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Inheritance\_Week 4\_Yourself

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### **Section 1 : Coding**

#### **1. Problem Statement**

Raj, an insurance agent, needs a program to streamline the calculation and updating of insurance prices for two-wheelers and four-wheelers using hierarchical inheritance. Implement three classes:

Vehicle with a method setBasePrice(double basePrice) and showPrice(). TwoWheeler (inherits from Vehicle) with methods calculateFinalPriceTwoWheeler() to calculate the updated price for two-wheelers. FourWheeler (inherits from Vehicle) with methods calculateFinalPriceFourWheeler() to calculate the updated price for four-wheelers.

Raj inputs the base prices of the vehicles, and the program calculate and display the original and updated prices, incorporating a 20% (0.20) increase for two-wheelers and a 30% (0.30) increase for four-wheelers.

Formula: Updated price = base price + (base price \* increase);

NOTE: Fixed values were passed as parameters.

### ***Input Format***

The first line of input is a double value T, representing the insurance base price of the two-wheeler.

The second line is a double value F, representing the insurance base price of the four-wheeler.

### ***Output Format***

The first part of the output prints the original and updated price (double values) of two-wheelers.

The next part prints the original and updated price (double values) of four-wheelers.

Round off with two decimal points.

Refer to the sample outputs for the exact text and format.

### ***Sample Test Case***

Input: 20000.00  
30000.00

Output: Original and Updated Price  
Rs. 20000.00  
Rs. 24000.00  
Original and Updated Price  
Rs. 30000.00  
Rs. 39000.00

### ***Answer***

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class Vehicle {  
    protected double basePrice;
```

```
public void setBasePrice(double basePrice) {
    this.basePrice = basePrice;
}

public void showPrice() {
    System.out.println("Original and Updated Price");
    System.out.printf("Rs. %.2f%n", basePrice);
}
}
```

```
class TwoWheeler extends Vehicle {
    public void calculateFinalPriceTwoWheeler() {
        double updated = basePrice + (basePrice * 0.20);
        System.out.printf("Rs. %.2f%n", updated);
    }
}
```

```
class FourWheeler extends Vehicle {
    public void calculateFinalPriceFourWheeler() {
        double updated = basePrice + (basePrice * 0.30);
        System.out.printf("Rs. %.2f%n", updated);
    }
}
```

```
public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        TwoWheeler bike = new TwoWheeler();
        double bikeBasePrice = scanner.nextDouble();
        bike.setBasePrice(bikeBasePrice);

        FourWheeler car = new FourWheeler();
        double carBasePrice = scanner.nextDouble();
        car.setBasePrice(carBasePrice);

        scanner.close();

        bike.showPrice();
        bike.calculateFinalPriceTwoWheeler();
    }
}
```

```
        car.showPrice();
        car.calculateFinalPriceFourWheeler();
    }
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Dinesh, a dedicated fitness enthusiast, decides to track his daily running sessions using a fitness app. He uses the app to record his running duration and speed. Write a program that takes his running duration and speed as input and calculates the calories burned during the run.

Refer to the below class diagram:

Formula: calories burned = met value\*(duration/60)\*3.5\*(speed/3.0),  
where met value = 7.0

### ***Input Format***

The first line of input consists of a double value d, representing the running duration.

The second line consists of a double value s, representing the speed.

### ***Output Format***

The output prints a double value, representing the total calories burned, rounded off to two decimal places.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 45.3  
6.5

Output: 40.08

**Answer**

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        double d = sc.nextDouble(); // duration
        double s = sc.nextDouble(); // speed

        // Constant MET value
        double met = 7.0;

        // Formula calculation
        double calories = met * (d / 60.0) * 3.5 * (s / 3.0);

        // Output (rounded to 2 decimal places)
        System.out.printf("%.2f", calories);

        sc.close();
    }
}
```

**Status :** Correct

**Marks : 10/10**

### 3. Problem Statement

Arun wants to calculate the age gap between the grandfather and the son and determine the father's age after 5 years.

Your task is to assist him in developing a program using three classes: GrandFather, Father, and Son, where the GrandFather stores the grandfather's age, the Father extends GrandFather to include the father's age and calculates his age after 5 years, and Son extends Father to include the son's age and calculate the age difference between the grandfather and the son.



### ***Input Format***

The input consists of three integers representing the ages of the grandfather, father, and son, one per line.

### ***Output Format***

The first line of output prints "Grandfather and son's age gap:" followed by an integer representing the age gap between the grandfather and the son, ending with "years".

The second line prints "Father's Age:" followed by an integer representing the father's age after 5 years, ending with "years".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 50

30

3

Output: Grandfather and son's age gap: 47 years

Father's Age: 35 years

### ***Answer***

```
import java.util.Scanner;
```

```
class GrandFather {  
    int grandFatherAge;
```

```
    void setGrandfatherAge(int age) { // FIXED NAME  
        this.grandFatherAge = age;  
    }  
}
```

```
class Father extends GrandFather {  
    int fatherAge;
```

```
    void setFatherAge(int age) {  
        this.fatherAge = age;  
    }  
}
```

```

        int calculateFatherAgeAfter5Years() {
            return fatherAge + 5;
        }
    }

    class Son extends Father {
        int sonAge;

        void setSonAge(int age) {
            this.sonAge = age;
        }

        int calculateGrandfatherSonAgeDifference() {
            return grandfatherAge - sonAge;
        }
    }

    public class Main {
        public static void main(String[] args) {
            Scanner scanner = new Scanner(System.in);
            Son son = new Son();

            int grandfatherAge = scanner.nextInt();
            son.setGrandfatherAge(grandfatherAge);

            int fatherAge = scanner.nextInt();
            son.setFatherAge(fatherAge);

            int sonAge = scanner.nextInt();
            son.setSonAge(sonAge);

            System.out.println("Grandfather and son's age gap: "+
            son.calculateGrandfatherSonAgeDifference() + " years");

            int fatherAgeAfter5Years = son.calculateFatherAgeAfter5Years();
            System.out.println("Father's Age: " + fatherAgeAfter5Years + " years");
        }
    }
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Elsa subscribes to a premium service with a base monthly cost, a service tax and an extra feature cost. Assist her in writing an inheritance program that takes input for these values and calculates the total monthly cost.

Refer to the below class diagram:

##### ***Input Format***

The first line of input consists of a double value, representing the base monthly cost.

The second line consists of a double value, representing the service tax.

The third line consists of a double value, representing the extra feature cost.

##### ***Output Format***

The output prints "Rs. X" where X is a double value, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

##### ***Sample Test Case***

Input: 10.0

2.5

5.0

Output: Rs. 17.50

##### ***Answer***

```
import java.util.Scanner;
```

```
class BaseSubscription {  
    double baseMonthlyCost;
```

```
    BaseSubscription(double baseMonthlyCost) {  
        this.baseMonthlyCost = baseMonthlyCost;  
    }  
}
```

```
}
```

```
class TaxSubscription extends BaseSubscription {  
    double serviceTax;
```

```
    TaxSubscription(double baseMonthlyCost, double serviceTax) {  
        super(baseMonthlyCost);  
        this.serviceTax = serviceTax;  
    }  
}
```

```
class PremiumSubscription extends TaxSubscription {  
    double extraFeatureCost;
```

```
    PremiumSubscription(double baseMonthlyCost, double serviceTax, double  
extraFeatureCost) {  
        super(baseMonthlyCost, serviceTax);  
        this.extraFeatureCost = extraFeatureCost;  
    }  
}
```

```
// FIXED METHOD NAME
```

```
double calculateMonthlyCost() {  
    return baseMonthlyCost + serviceTax + extraFeatureCost;  
}  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);
```

```
        double baseMonthlyCost = scanner.nextDouble();  
        double serviceTax = scanner.nextDouble();  
        double extraFeatureCost = scanner.nextDouble();
```

```
        PremiumSubscription premiumSubscription = new  
PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);
```

```
        double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();
```

```
        System.out.printf("Rs. %.2f%n", totalMonthlyCost);
```

```
        scanner.close();
```

```
    }
```

}

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Ravi owns a courier service and wants to calculate the shipping costs for regular and fragile parcels. A regular parcel has only a basic shipping cost, whereas a fragile parcel has additional costs for extra packing and insurance.

Write a program to compute the shipping cost for both types of parcels and display them using single inheritance.

### **Input Format**

The first line contains an integer B, representing the basic cost of a regular parcel.

The second line contains an integer E, representing the extra packing cost for a fragile parcel.

The third line contains an integer I, representing the insurance cost for a fragile parcel.

### **Output Format**

The first line prints "Regular Parcel Cost: " followed by the cost of the regular parcel.

The second line prints "Fragile Parcel Cost: " followed by the cost of the fragile parcel.

Refer to the sample outputs for the format specifications.

### **Sample Test Case**

Input: 10

5

3

Output: Regular Parcel Cost: 10 rupees  
Fragile Parcel Cost: 18 rupees

**Answer**

```
import java.util.Scanner;

class Parcel {
    int basicCost;

    Parcel(int basicCost) {
        this.basicCost = basicCost;
    }

    int getBasicCost() {
        return basicCost;
    }
}

class FragileParcel extends Parcel {
    int extraPackingCost;
    int insuranceCost;

    FragileParcel(int basicCost, int extraPackingCost, int insuranceCost) {
        super(basicCost);
        this.extraPackingCost = extraPackingCost;
        this.insuranceCost = insuranceCost;
    }

    int getTotalCost() {
        return basicCost + extraPackingCost + insuranceCost;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int basicCost = scanner.nextInt();
        int extraPackingCost = scanner.nextInt();
        int insuranceCost = scanner.nextInt();

        Parcel regularParcel = new Parcel(basicCost);
        FragileParcel fragileParcel = new FragileParcel(basicCost, extraPackingCost,
insuranceCost);
    }
}
```

```
        System.out.println("Regular Parcel Cost: " + regularParcel.getBasicCost() + "  
rupees");  
        System.out.println("Fragile Parcel Cost: " + fragileParcel.getTotalCost() + "  
rupees");  
  
        scanner.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

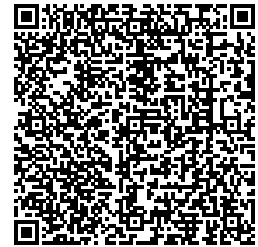
Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Inheritance\_Week 4\_Home

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Kishore, an IT employee, wishes to apply for a loan. Write a program using single level inheritance to determine his loan eligibility and negotiate the interest rate. Create a base class named LoanDetails to store credit score, annual income, and interest rate. Then, create a derived class LoanEligibilityChecker that inherits from LoanDetails and checks if Kishore is eligible for a loan. A person is eligible if their credit score is 700 or more and annual income is at least 30,000.

Next, create another derived class InterestRateNegotiator which also inherits from LoanDetails. This class calculates the final interest rate based on the eligibility result. If the person is eligible, the interest rate is reduced to 90% of the original; otherwise, it stays the same. Display the eligibility status and final interest rate (rounded to one decimal) as output.



### ***Input Format***

The first line of input represents the credit score as an integer.

The second line of input represents the annual income as an integer.

The last line of input represents the rate of interest as double.

### ***Output Format***

The first line of output prints "Loan eligibility: <status> " followed by a string representing whether the person is eligible or not (Eligible / Not eligible).

The second line of output prints "Final interest rate: <rate>" followed by a double value representing the final interest rate rounded to one decimal place.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 700

30000

10.0

Output: Loan eligibility: Eligible

Final interest rate: 9.0

### ***Answer***

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class LoanDetails {
```

```
    int creditScore;
```

```
    double income;
```

```
    double interestRate;
```

```
    public void setDetails(int creditScore, double income, double interestRate) {
```

```
        this.creditScore = creditScore;
```

```
        this.income = income;
```

```
        this.interestRate = interestRate;
```

```
    }
```

```
}
```

```
class LoanEligibilityChecker extends LoanDetails {  
    public boolean isEligibleForLoan() {  
        return (creditScore >= 700 && income >= 30000);  
    }  
}
```

```
class InterestRateNegotiator extends LoanDetails {  
    public double negotiateInterestRate(boolean isEligible) {  
        if (isEligible) {  
            return interestRate * 0.9;  
        } else {  
            return interestRate;  
        }  
    }  
}
```

```
class LoanApplication {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        int creditScore = scanner.nextInt();  
        double income = scanner.nextDouble();  
        double baseInterestRate = scanner.nextDouble();  
  
        LoanEligibilityChecker eligibilityChecker = new LoanEligibilityChecker();  
        eligibilityChecker.setDetails(creditScore, income, baseInterestRate);  
  
        boolean isEligible = eligibilityChecker.isEligibleForLoan();  
  
        InterestRateNegotiator negotiator = new InterestRateNegotiator();  
        negotiator.setDetails(creditScore, income, baseInterestRate);  
  
        double finalInterestRate = negotiator.negotiateInterestRate(isEligible);  
  
        System.out.println("Loan eligibility: " + (isEligible ? "Eligible" : "Not eligible"));  
        System.out.println("Final interest rate: " + finalInterestRate);  
  
        scanner.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Sharon, a software developer, is working on a library system program.

She has created a class hierarchy for different types of books:

class Book (with title and author attributes), class Fiction (which extends the Book class and includes an attribute called "genre" for which string "Fantasy" is passed as a default value through the constructor), class Fantasy (which extends the Fiction class and includes an attribute called daysLate).

Now, she needs to calculate the late fees for borrowed fantasy books. The late fee is determined by multiplying the number of days a book is late by 0.50.

Write a program that takes input for a borrowed fantasy book's title, author, and days late. The program should then calculate and display the late fee.

### ***Input Format***

The first line of input consists of a string, representing the title of the book.

The second line consists of a string, representing the author of the book.

The third line consists of an integer, representing the number of days the book is late.

### ***Output Format***

The output prints a double value, representing the late fee for the book which is a double value.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: Harry Potter and the Philosopher's Stone

J.K. Rowling

1

Output: 0.5

### Answer

```
import java.util.Scanner;

// You are using Java
class Book {
    String title;
    String author;
    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }
}

class Fiction extends Book {
    String genre;
    public Fiction(String title, String author, String genre) {
        super(title, author);
        this.genre = genre;
    }
}

class Fantasy extends Fiction {
    int daysLate;
    public Fantasy(String title, String author, String genre, int daysLate) {
        super(title, author, genre);
        this.daysLate = daysLate;
    }
    public double calculateLateFee() {
        return daysLate * 0.50;
    }
}

class LibrarySystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String title = scanner.nextLine();
        String author = scanner.nextLine();
        int daysLate = scanner.nextInt();
        Fantasy book = new Fantasy(title, author, "Fantasy", daysLate);
        System.out.println(book.calculateLateFee());
    }
}
```

Status : Correct

Marks : 10/10

### 3. Problem Statement

A painter needs to determine the cost to paint different shapes based on their surface area. The program should be designed to handle the area of a sphere and calculate the total painting cost using the following formulas:

Area of sphere:  $\text{Area} = 4 * \pi * r^2$  where  $\pi = 3.14$   
Total painting cost:  $\text{Cost} = \text{cost per square meter} * \text{area of sphere}$

The program will consist of three classes:

Shape class: This class should set the shape type and radius.

Area class: This class should extend Shape to calculate the area.  
Cost class: This class should extend Area to calculate the total painting cost.

#### **Input Format**

The input consists of a string representing the shape type, a double value representing the radius, and another double value representing the cost per square meter on each line.

#### **Output Format**

For a valid shape type of "Sphere":

- The first line prints: "Area of Sphere is: <calculated\_area>" rounded to two decimal places.
- The second line prints: "Cost to paint the shape is: <total\_painting\_cost>" rounded to two decimal places.

For any other shape types, print: "Invalid type".

Refer to the sample output for formatting specifications.

#### **Sample Test Case**

Input: Sphere

3.4

5.8

Output: Area of Sphere is: 145.19  
Cost to paint the shape is: 842.12

**Answer**

```
import java.util.Scanner;

class Shape {
    String shapeType;
    double radius;
    public void setShape(String s, Scanner sc) {
        this.shapeType = s;
        this.radius = sc.nextDouble();
    }
}

class Area extends Shape {
    double area;
    public void calculateArea() {
        if (!shapeType.equals("Sphere")) {
            System.out.println("Invalid type");
            return;
        }
        area = 4 * 3.14 * radius * radius;
        System.out.printf("Area of Sphere is: %.2f\n", area);
    }
}

class Cost extends Area {
    double costPerSqm;
    public void setCost(double cost) {
        this.costPerSqm = cost;
    }
    public void calculateCost() {
        if (!shapeType.equals("Sphere")) {
            return;
        }
        double totalCost = costPerSqm * area;
        System.out.printf("Cost to paint the shape is: %.2f", totalCost);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}
```

```
String s = scanner.next();
Cost shape = new Cost();
shape.setShape(s, scanner);
double costToPaint = scanner.nextDouble();
shape.calculateArea();
shape.setCost(costToPaint);
shape.calculateCost();
}
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Joy, a house-building constructor, seeks assistance from his friend, a developer.

He wants to design a program using hierarchical inheritance, where a base class Property contains dimensions for length and width. Two derived classes, TwoBHK and FiveBHK, extend Property and calculate the square root of the product of length and width for two-bedroom and five-bedroom houses, respectively.

Joy inputs dimensions for both types of houses, and the program outputs their square roots.

Note: Square root of the area =  $\sqrt{(\text{length} * \text{width})}$

##### **Input Format**

The first line of input consists of a space-separated double-point number, representing dimensions of 2BHK (length, width)

The second line consists of a space-separated double-point number, representing dimensions of 5BHK (length, width).

##### **Output Format**

The output displays the square root of the area for 2BHK (double) and 5BHK (double) with two decimal places in the following format:

"2BHK Square root: X.XX"

5BHK Square root: X.XX"

Refer to the sample output for format specifications.

### **Sample Test Case**

Input: 1.0 1.0

1.0 1.0

Output: 2BHK Square root: 1.00

5BHK Square root: 1.00

### **Answer**

```
import java.util.Scanner;

// You are using Java
class Property {
    double length;
    double width;
    public void setDimensions(double length, double width) {
        this.length = length;
        this.width = width;
    }
}

class TwoBHK extends Property {
    public double TBHKSquareRoot() {
        return Math.sqrt(length * width);
    }
}

class FiveBHK extends Property {
    public double FBHKSquareRoot() {
        return Math.sqrt(length * width);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```



```
TwoBHK twoBHK = new TwoBHK();  
double twoBHKLength = scanner.nextDouble();  
double twoBHKWidth = scanner.nextDouble();  
twoBHK.setDimensions(twoBHKLength, twoBHKWidth);
```

```
FiveBHK fiveBHK = new FiveBHK();  
double fiveBHKLength = scanner.nextDouble();  
double fiveBHKWidth = scanner.nextDouble();  
fiveBHK.setDimensions(fiveBHKLength, fiveBHKWidth);
```

```
scanner.close();
```

```
System.out.printf("2BHK Square root: %.2f%n", twoBHK.TBHKSquareRoot());  
System.out.printf("5BHK Square root: %.2f%n", fiveBHK.FBHKSquareRoot());
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MO\_Week 5\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 19

### Section 1 : MCQ

1. What will be the output of the following Java program?

```
class Parent {  
    void show() {  
        System.out.println("Parent class");  
    }  
}  
class Child extends Parent {  
    void show() {  
        System.out.println("Child class");  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        Parent obj = new Child();  
        obj.show();  
    }  
}
```

**Answer**

Child class

**Status :** Correct

**Marks :** 1/1

2. Which of the following is true about method overriding in Java?

**Answer**

The method must have the same name, same parameters, and must be in different classes with an inheritance relationship

**Status :** Correct

**Marks :** 1/1

3. What will be the output of the following Java program?

```
class Vehicle {  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
}  
class Car extends Vehicle {  
    void start() {  
        System.out.println("Car starts");  
    }  
}  
class ElectricCar extends Car {  
    void start() {  
        System.out.println("Electric Car starts silently");  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        Vehicle v = new ElectricCar();  
        v.start();  
    }  
}
```

**Answer**

Electric Car starts silently

**Status :** Correct

**Marks :** 1/1

4. What will be the output of the following Java program?

```
class A {  
    int value = 10;  
    void display() {  
        System.out.println("A's display: " + value);  
    }  
}  
class B extends A {  
    int value = 20;  
    void display() {  
        System.out.println("B's display: " + value);  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.display();  
        System.out.println("Value: " + obj.value);  
    }  
}
```

**Answer**

B's display: 20 Value: 10

**Status :** Correct

**Marks :** 1/1

5. Which access modifiers allow function overriding in Java?

**Answer**

Methods with public, protected, or package-private (default) access can be

overridden

**Status :** Correct

**Marks :** 1/1

6. What will be the output of the following Java program?

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Vehicle engine started");  
    }  
}
```

```
class Car extends Vehicle {  
    void startEngine() {  
        System.out.println("Car engine started");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Vehicle myVehicle = new Car();  
        myVehicle.startEngine();  
    }  
}
```

**Answer**

Car engine started

**Status :** Correct

**Marks :** 1/1

7. What will be the output of the following Java program?

```
class Music {  
    void play() {  
        System.out.println("Playing music");  
    }  
}
```

```
class Jazz extends Music {
```

```
void play() {  
    System.out.println("Playing jazz music");  
}  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Music sound = new Jazz();  
        Jazz jazzMusic = (Jazz) sound;  
        Music music = jazzMusic;  
        music.play();  
    }  
}
```

**Answer**

Playing jazz music

**Status :** Correct

**Marks :** 1/1

8. What will be the output of the following Java program?

```
class Employee {  
    void calculateSalary() {  
        System.out.println("Basic salary calculation");  
    }  
}  
  
class Manager extends Employee {  
    void calculateSalary() {  
        super.calculateSalary();  
        System.out.println("Adding management bonus");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Manager mgr = new Manager();  
        mgr.calculateSalary();  
    }  
}
```

```
}
```

**Answer**

Basic salary calculation Adding management bonus

**Status :** Correct

**Marks :** 1/1

9. What will be the output of the following Java program?

```
class BankAccount {  
    void calculateInterest() {  
        System.out.println("Standard interest rate");  
    }  
}  
  
class SavingsAccount extends BankAccount {  
    void calculateInterest() {  
        System.out.println("Savings account interest");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        BankAccount account;  
        account = new SavingsAccount();  
        account.calculateInterest();  
    }  
}
```

**Answer**

Savings account interest

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following Java program?

```
class Phone {  
    void ring() {  
        System.out.println("Phone is ringing");  
    }  
}
```

```

    }
}

class SmartPhone extends Phone {
    void ring() {
        System.out.println("SmartPhone playing ringtone");
    }

    void ring(String tone) {
        System.out.println("Playing: " + tone);
    }
}

class Main {
    public static void main(String[] args) {
        Phone device = new SmartPhone();
        device.ring();
    }
}

```

**Answer**

SmartPhone playing ringtone

**Status :** Correct

**Marks :** 1/1

11. What will be the output of the following Java program?

```

class Television {
    void changeChannel() {
        System.out.println("TV channel changed");
    }
}

class SmartTV extends Television {
    void changeChannel() {
        System.out.println("Smart TV channel changed");
    }
}

```



```
class Main {  
    public static void main(String[] args) {  
        Television tv = new SmartTV();  
        ((SmartTV)tv).changeChannel();  
    }  
}
```

**Answer**

Smart TV channel changed

**Status :** Correct

**Marks :** 1/1

12. What will be the output of the following Java program?

```
class Transport {  
    void move() {  
        System.out.println("Transport is moving");  
    }  
}
```

```
class Bicycle extends Transport {  
    void move() {  
        System.out.println("Bicycle is pedaling");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Transport vehicle;  
        vehicle = new Transport();  
        vehicle.move();  
        vehicle = new Bicycle();  
        vehicle.move();  
    }  
}
```

**Answer**

Transport is movingBicycle is pedaling

**Status :** Correct

**Marks :** 1/1

13. What will be the output of the following Java program?

```
class Weather {  
    void forecast() {  
        System.out.println("Weather forecast");  
    }  
}  
  
class RainyWeather extends Weather {  
    void forecast() {  
        System.out.println("It will rain today");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Weather today = new RainyWeather();  
        RainyWeather rainy = new RainyWeather();  
        today.forecast();  
        rainy.forecast();  
    }  
}
```

**Answer**

It will rain todayIt will rain today

**Status :** Correct

**Marks :** 1/1

14. Private methods in a parent class:

**Answer**

Are not inherited and cannot be overridden

**Status :** Correct

**Marks :** 1/1

15. What is the correct rule for method overriding in Java?

**Answer**

The overriding method must have the same or more restrictive access modifier

**Status :** Wrong

**Marks :** 0/1

16. Which of the following methods CANNOT be overridden in Java?

**Answer**

static methods

**Status :** Correct

**Marks :** 1/1

17. What happens when a final method is attempted to be overridden?

**Answer**

A compile-time error occurs

**Status :** Correct

**Marks :** 1/1

18. What happens if you try to override a method with a more restrictive access modifier?

```
class Parent {  
    protected void method() {}  
}  
class Child extends Parent {  
    private void method() {} // What happens here?  
}
```

**Answer**

Compile-time error occurs

**Status :** Correct

**Marks :** 1/1

19. In method overriding, which of the following about the method signature is true?

**Answer**

Both method name and parameters must be exactly the same

**Status :** Correct

**Marks :** 1/1

20. Which statement about constructors and method overriding is correct?

**Answer**

Constructors cannot be overridden

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MO\_Week 5\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Teena's retail store has implemented a Loyalty Points System to reward customers based on their spending. The program calculates and displays the loyalty points based on whether the customer is a regular or a premium customer.

For regular customers (class Customer), the loyalty points are calculated as:

$\text{Loyalty points} = \text{amount spent} / 10$

For premium customers (class PremiumCustomer, which inherits from Customer), the loyalty points are calculated as:

$\text{Loyalty points} = 2 * (\text{amount spent} / 10)$

The program should use method overriding for premium customers to

calculate their loyalty points. The method that needs to be overridden is calculateLoyaltyPoints in the Customer class.

### ***Input Format***

The first line of input consists of an integer representing the amount spent by the customer.

The second line consists of a string representing the premium customer status:

- "yes" if the customer is a premium customer.
- "no" if the customer is not a premium customer.

### ***Output Format***

The output should display the loyalty points earned based on the amount spent and the customer type.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 50

yes

Output: 10

### ***Answer***

```
import java.util.Scanner;

// You are using Java
class Customer {
    public int calculateLoyaltyPoints(int amountSpent) {
        return amountSpent / 10;
    }
}

class PremiumCustomer extends Customer {
    @Override
    public int calculateLoyaltyPoints(int amountSpent) {
        return 2 * (amountSpent / 10);
    }
}
```

```

}
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int amountSpent = scanner.nextInt();

        String isPremium = scanner.next().toLowerCase();

        Customer customer;

        if (isPremium.equals("yes")) {
            customer = new PremiumCustomer();
        } else {
            customer = new Customer();
        }

        int loyaltyPoints = customer.calculateLoyaltyPoints(amountSpent);

        System.out.println(loyaltyPoints);
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Mary is managing a business and wants to analyze its profitability. She operates both a regular business model and a seasonal business model. To assess profitability, she uses a program that calculates and compares the profit margins for both models based on revenue and cost.

The program defines:

BusinessUtility class with a method calculateMargin(double revenue, double cost). SeasonalBusinessUtility (inherits from BusinessUtility) and overrides calculateMargin(double revenue, double cost), adding a seasonal adjustment of 10% to the base margin. ProfitabilityChecker class with a method checkProfitability(double regularMargin), which prints "Business is profitable." if the regular margin is 10% or more, otherwise prints "Business

is not profitable."

Mary inputs revenue and cost, and the program compute and display the regular and seasonal margins using:

$$\text{Margin} = ((\text{Revenue} - \text{Cost}) / \text{Revenue}) \times 100$$

$$\text{Seasonal Margin} = \text{Margin} + 10$$

### ***Input Format***

The first line of input consists of a double value *r*, representing the revenue.

The second line consists of a double value *c*, representing the cost.

### ***Output Format***

The first line prints a double value, representing the regular profit margin, rounded to two decimal places, in the format: "Regular Margin: X. XX%", where X.XX denotes the calculated regular margin.

The second line prints a double value, representing the seasonal profit margin, rounded to two decimal places, in the format: "Seasonal Margin: X. XX%", where X.XX denotes the calculated seasonal margin.

The third line prints a string, indicating whether the business is profitable or not profitable, based on the regular margin.

If the regular margin is less than 10, print "Business is not profitable.". If it is 10 or greater, print "Business is profitable."

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 1000.0  
800.0

Output: Regular Margin: 20.00%  
Seasonal Margin: 30.00%  
Business is profitable.

### ***Answer***



```

import java.util.Scanner;

class BusinessUtility {
    public double calculateMargin(double revenue, double cost) {
        return ((revenue - cost) / revenue) * 100;
    }
}

class SeasonalBusinessUtility extends BusinessUtility {
    @Override
    public double calculateMargin(double revenue, double cost) {
        double baseMargin = super.calculateMargin(revenue, cost);
        return baseMargin + 10;
    }
}

class ProfitabilityChecker {
    public void checkProfitability(double regularMargin) {
        if (regularMargin >= 10) {
            System.out.print("Business is profitable.");
        } else {
            System.out.print("Business is not profitable.");
        }
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double revenue = scanner.nextDouble();
        double cost = scanner.nextDouble();
        BusinessUtility business = new BusinessUtility();
        SeasonalBusinessUtility seasonalBusiness = new
SeasonalBusinessUtility();
        double regularMargin = business.calculateMargin(revenue, cost);
        double seasonalMargin = seasonalBusiness.calculateMargin(revenue,
cost);

        System.out.printf("Regular Margin: %.2f%%\n", regularMargin);
        System.out.printf("Seasonal Margin: %.2f%%\n", seasonalMargin);

        ProfitabilityChecker checker = new ProfitabilityChecker();
        checker.checkProfitability(regularMargin);
        scanner.close();
    }
}

```

}

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Shreya is a student who needs to calculate her exam percentage. Her regular percentage is calculated using the formula (obtained marks / total marks) \* 100. However, as a scholarship student, she receives an additional 5% bonus on her calculated percentage.

Write a program that uses an overridden method calculatePercentage to compute both her regular and scholarship percentages.

#### **Input Format**

The first line of input consists of an integer n, which represents the total marks in the exam.

The second line consists of an integer m, which represents the marks obtained by the student.

#### **Output Format**

The first line of output displays a double value, representing the percentage of marks obtained by the regular student, rounded to two decimal places in the format: "Student Percentage: <value>%".

The second line displays a double value, representing the percentage of marks obtained by the scholarship student, rounded to two decimal places in the format: "Scholarship Student Percentage: <value>%".

Refer to the sample output for formatting specifications.

#### **Sample Test Case**

Input: 500  
450

Output: Student Percentage: 90.00%

Scholarship Student Percentage: 95.00%

**Answer**

```
import java.util.Scanner;

// You are using Java
class Student {
    public double calculatePercentage(int totalMarks, int obtainedMarks) {
        return ((double) obtainedMarks / totalMarks) * 100;
    }
}

class ScholarshipStudent extends Student {
    @Override
    public double calculatePercentage(int totalMarks, int obtainedMarks) {
        double base = super.calculatePercentage(totalMarks, obtainedMarks);
        return base + 5;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        Student student = new Student();
        ScholarshipStudent scholarshipStudent = new ScholarshipStudent();

        int totalMarks = scanner.nextInt();
        int obtainedMarks = scanner.nextInt();

        double studentPercentage = student.calculatePercentage(totalMarks,
            obtainedMarks);
        double scholarshipStudentPercentage =
            scholarshipStudent.calculatePercentage(totalMarks, obtainedMarks);

        System.out.printf("Student Percentage: %.2f%%\n", studentPercentage);
        System.out.printf("Scholarship Student Percentage: %.2f%%\n",
            scholarshipStudentPercentage);

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

#### 4. Problem Statement

Rithish is developing a simple pizza ordering system. He needs a program that calculates the price of a pizza based on its base price, topping cost, and the number of toppings added.

The program defines:

Pizza class with:

A constructor `Pizza(double basePrice, double toppingCost)`, initializing the base price and topping cost. A method `calculatePrice(int numberOfToppings)`, which computes the total price as:  $\text{Total Price} = \text{Base Price} + (\text{Topping Cost} \times \text{Number of Toppings})$ .

DiscountedPizza class (inherits from Pizza) with:

A constructor `DiscountedPizza(double basePrice, double toppingCost)`, which initializes the base price and topping cost by calling the parent constructor. A method `calculatePrice(int numberOfToppings)`, which calculates the total price and applies a 10% discount if more than 3 toppings are added.

The program prompts the user to:

Input the base price and topping cost of a pizza. Input the number of toppings.

It then calculates and displays both regular price and discounted price (if applicable), formatted to two decimal places.

Example 1

Input:

9.5

1.25

3

Output:

Price without discount: Rs.13.25

Price with discount: Rs.13.25

Explanation:

Rithish orders a pizza with a base price of Rs. 9.5, a topping cost of Rs. 1.25, and selects 3 toppings. The price is calculated as  $9.5 + (1.25 * 3) = 13.25$ . The regular and discounted prices are both Rs. 13.25, as no discount has been applied.

Example 2

Input:

11.0

2.0

7

Output:

Price without discount: Rs.25.00

Price with discount: Rs.22.50

Explanation:

Rithish orders another pizza with a higher base price of Rs. 11.0, a topping cost of Rs. 2.0, and chooses 7 toppings.

Regular Price:  $11.0 + (2.0 * 7) = \text{Rs. } 25.00$ .

Discounted Price: The discounted price is calculated as 90% of the regular price, i.e.,  $0.9 * 25.00 = \text{Rs. } 22.50$ .

### ***Input Format***

The first line of input consists of a double value, representing the base price of the pizza.

The second line consists of a double value, representing the cost per topping.

The third line consists of an integer, representing the number of toppings chosen

for the pizza.

### **Output Format**

The first line of output prints the price without discount, rounded off to two decimal places.

The second line prints the price with the discount, rounded off to two decimal places.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 9.5

1.25

3

Output: Price without discount: Rs.13.25

Price with discount: Rs.13.25

### **Answer**

```
import java.util.Scanner;
import java.text.DecimalFormat;

// You are using Java
class Pizza {
    double basePrice;
    double toppingCost;
    public Pizza(double basePrice, double toppingCost) {
        this.basePrice = basePrice;
        this.toppingCost = toppingCost;
    }
    public double calculatePrice(int numberOfToppings) {
        return basePrice + (toppingCost * numberOfToppings);
    }
}

class DiscountedPizza extends Pizza {
    public DiscountedPizza(double basePrice, double toppingCost) {
        super(basePrice, toppingCost);
    }
    @Override
```

```

    public double calculatePrice(int numberOfToppings) {
        double price = super.calculatePrice(numberOfToppings);
        if (numberOfToppings > 3) {
            return price * 0.9;
        }
        return price;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double basePrice = scanner.nextDouble();
        double toppingCost = scanner.nextDouble();
        int numberOfToppings = scanner.nextInt();

        Pizza pizza = new Pizza(basePrice, toppingCost);
        DiscountedPizza discountedPizza = new DiscountedPizza(basePrice,
            toppingCost);

        DecimalFormat df = new DecimalFormat("#.00");

        double regularPrice = pizza.calculatePrice(numberOfToppings);
        double discountedPrice =
            discountedPizza.calculatePrice(numberOfToppings);

        System.out.println("Price without discount: Rs." + df.format(regularPrice));
        System.out.println("Price with discount: Rs." + df.format(discountedPrice));

        scanner.close();
    }
}

```

**Status :** Correct

**Marks : 10/10**

## 5. Problem Statement

Adam is a college student who is part of a coding club that organizes weekly coding challenges. This week, the challenge is to create a program that can calculate the greatest common divisor (GCD) and the least

common multiple (LCM) of two numbers. Adam decides to take on this challenge and starts by creating a class called "NumberUtils" with a method gcd(int a, int b) that calculates the GCD of two numbers.

Excited by the challenge, Adam decides to go a step further and create an "AdvancedNumberUtils" subclass. In this subclass, he plans to override the gcd method to not only calculate the GCD but also compute the LCM of two numbers. Your task is to help Adam successfully complete this coding challenge and impress his peers in the coding club.

Note:

Use Math.abs() to ensure the calculation of GCD and LCM handles negative inputs correctly by converting them to absolute (positive) values.

#### ***Input Format***

The first line consists of an integer value 'a'.

The second line consists of an integer value 'b'.

#### ***Output Format***

The first line displays the GCD (Greatest Common Divisor) of "a" and "b". The GCD result is printed as "GCD: {gcd\_value}".

The second line displays the LCM (Least Common Multiple) of "a" and "b". The LCM result is printed as "LCM: {lcm\_value}".

Refer to the sample output for the formatting specifications.

#### ***Sample Test Case***

Input: 12

16

Output: GCD: 4

LCM: 48

#### ***Answer***

```
import java.util.Scanner;
```



```

class NumberUtils {
    public int gcd(int a, int b) {
        a = Math.abs(a);
        b = Math.abs(b);

        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }
}

class AdvancedNumberUtils extends NumberUtils {
    @Override
    public int gcd(int a, int b) {
        return super.gcd(a, b);
    }
    public int lcm(int a, int b) {
        a = Math.abs(a);
        b = Math.abs(b);
        int gcdValue = super.gcd(a, b);
        return (a / gcdValue) * b;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int a = scanner.nextInt();
        int b = scanner.nextInt();

        AdvancedNumberUtils utils = new AdvancedNumberUtils();
        int gcdValue = utils.gcd(a, b);
        int lcmValue = utils.lcm(a, b);

        System.out.println("GCD: " + gcdValue);
        System.out.println("LCM: " + lcmValue);
    }
}

```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MO\_Week 5\_Yourself

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Raj is a software developer tasked with creating a train ticket pricing system for a local railway company using method overriding. The system needs to consider different pricing rules for regular passengers and senior citizens. Senior citizens, aged 60 and above, are eligible for a 10% discount on the regular ticket price.

The calculatePrice method in the SeniorCitizenTicket class overrides the same method in the Ticket class to provide a different implementation for senior citizens. The method public double calculatePrice(int distance) calculates the ticket price based on the distance traveled.

Formula:

regular price = distance \* 0.05

senior discounted price = regular price \* 0.9

Example

Input:

120 65

Output:

Senior Citizen Ticket Price: 5.4

Explanation:

Since the passenger's age is 65, which is greater than 60, the person is a senior citizen. The regular ticket price is calculated as  $120 * 0.05 = 6.0$ . The discounted price is then calculated as 90% of the regular price, i.e.,  $6.0 * 0.9 = 5.4$

### ***Input Format***

The input consists of two space-separated integers, representing the distance and age respectively.

### ***Output Format***

If the passenger is a regular passenger (age < 60), prints "Regular Passenger Ticket Price: <price>", where price is a double value formatted to one decimal places.

If the passenger is a senior citizen (age >= 60), prints "Senior Citizen Ticket Price: <price>" where price is a double value formatted to one decimal places.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 120 65

Output: Senior Citizen Ticket Price: 5.4

### ***Answer***

```
import java.util.Scanner;
```

```

class Ticket {
    public double calculatePrice(int distance) {
        return distance * 0.05;
    }
}

class SeniorCitizenTicket extends Ticket {
    @Override
    public double calculatePrice(int distance) {
        double regularPrice = super.calculatePrice(distance);
        return regularPrice * 0.9;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int distance = scanner.nextInt();
        int age = scanner.nextInt();

        Ticket ticketUtility;
        if (age >= 60) {
            ticketUtility = new SeniorCitizenTicket();
        } else {
            ticketUtility = new Ticket();
        }

        double ticketPrice = ticketUtility.calculatePrice(distance);

        if (age < 60) {
            System.out.printf("Regular Passenger Ticket Price: %.1f\n", ticketPrice);
        } else {
            System.out.printf("Senior Citizen Ticket Price: %.1f\n", ticketPrice);
        }
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Implement a program to calculate the driving range for two types of cars: a "Regular Car" and an "Eco Car." The program should calculate the range based on the fuel amount and mileage for each car. The Eco Car has a special feature that adds 20 miles to the calculated range, but only if the range is above 50 miles.

The program should use method overriding for calculating the range for each car type.

CarUtility with a method calculateRange(double fuelAmount, double mileage) which calculates the range for the regular car using the formula:  $\text{Range} = \text{Fuel Amount} * \text{Mileage}$  It also prints a "Low Range Warning!" if the range is below 50 miles. EcoCarUtility (inherits from CarUtility) with a method calculateRange(double fuelAmount, double mileage) which overrides the method in the parent class and adds 20 miles to the range if the range is above 50 miles.

#### ***Input Format***

The first line of the input has two space-separated integers 'f' representing the fuel amount (in gallons) and 'm' representing the mileage (in miles per gallon) of a Regular Car.

The second line of the input has two space-separated integers 'f' representing the fuel amount (in gallons) and 'm' representing the mileage (in miles per gallon) of an Eco Car.

#### ***Output Format***

If the range of both cars is less than 50 miles, then the "Low Range Warning" is displayed above "Regular Car Range" and "Eco Car Range".

If any one of the cars has a range of less than 50 miles, then the "Low Range Warning" is displayed for the particular car range.

Else the first line of the input displays "Regular Car Range: XX miles".

The second line of the input displays "Eco Car Range: XX miles".

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 10.0 25.0

10.0 25.0

Output: Regular Car Range: 250.0 miles

Eco Car Range: 270.0 miles

### **Answer**

```
import java.util.Scanner;
```

```
class CarUtility {  
    double calculateRange(double fuelAmount, double mileage) {  
        double range = fuelAmount * mileage;  
        if (range < 50) {  
            System.out.println("Low Range Warning!");  
        }  
    }  
}
```

```
    return range;  
}
```

```
}
```

```
class EcoCarUtility extends CarUtility {
```

```
    @Override
```

```
    double calculateRange(double fuelAmount, double mileage) {
```

```
        double range = fuelAmount * mileage;
```

```
        if (range > 50) {
```

```
            range += 20;
```

```
        } else {
```

```
            System.out.println("Low Range Warning!");
```

```
        }
```

```
        return range;
```

```
    }  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        double regularFuel = scanner.nextDouble();
```

```
        double regularMileage = scanner.nextDouble();
```

```
double ecoFuel = scanner.nextDouble();
double ecoMileage = scanner.nextDouble();

CarUtility car = new CarUtility();
double regularRange = car.calculateRange(regularFuel, regularMileage);
System.out.println("Regular Car Range: " + regularRange + " miles");

EcoCarUtility ecoCar = new EcoCarUtility();
double ecoRange = ecoCar.calculateRange(ecoFuel, ecoMileage);
System.out.println("Eco Car Range: " + ecoRange + " miles");

scanner.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Jamie is a computer science student who is currently studying number conversions. She is interested in converting numbers between different bases. To help her with this, you are tasked with writing a program to convert a decimal number to a specified base and vice versa.

#### **Input Format**

The first line of input consists of an integer 'decimalNumber', which represents the decimal number to be converted.

The second line of input consists of an integer 'base', which represents the target base for conversion.

#### **Output Format**

The first line of output displays the result of converting the 'decimalNumber' to the specified 'base' in the following format:

Decimal {decimalNumber} in base {base}: {result}

The second line of output displays the result of converting the 'convertedNumber' back to decimal in the following format:

Converted back to decimal: {converted value}

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 42

8

Output: Decimal 42 in base 8: 52

Converted back to decimal: 42

### **Answer**

```
import java.util.Scanner;

class NumberConverter {
    String convertToBase(int decimalNumber, int base) {
        if (decimalNumber == 0) return "0";
        StringBuilder result = new StringBuilder();
        while (decimalNumber > 0) {
            int remainder = decimalNumber % base;
            if (remainder < 10) {
                result.append(remainder);
            } else {
                result.append((char) ('A' + remainder - 10));
            }

            decimalNumber /= base;
        }
        return result.reverse().toString();
    }
}

class AdvancedNumberConverter extends NumberConverter {
    int convertToDecimal(String number, int base) {
        int decimalValue = 0;
        for (int i = 0; i < number.length(); i++) {
            char ch = number.charAt(i);
            int value;
            if (Character.isDigit(ch)) {
                value = ch - '0';
            } else {
```



```

        value = ch - 'A' + 10;
    }
    decimalValue = decimalValue * base + value;
}
return decimalValue;
}
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int decimalNumber = scanner.nextInt();
        int base = scanner.nextInt();

        NumberConverter numberConverter = new AdvancedNumberConverter();

        String convertedNumber =
numberConverter.convertToBase(decimalNumber, base);
        System.out.println("Decimal " + decimalNumber + " in base " + base + ": " +
convertedNumber);

        AdvancedNumberConverter advancedConverter =
(AdvancedNumberConverter) numberConverter;
        int convertedDecimal =
advancedConverter.convertToDecimal(convertedNumber, base);
        System.out.println("Converted back to decimal: " + convertedDecimal);

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement:

Mariya, a software developer, is working on a project to create a Statistical Calculator. She wants to design a simple program that takes an array of integers as input and provides statistical information as output.

Specifically, she wants to calculate the mean, median, and mode of the

given array of integers.

### ***Input Format***

The first line contains an integer N, representing the number of integers in the array.

The second line contains N space-separated integers, representing the values of the array.

### ***Output Format***

The output displays the mean, median, and mode of the array in the following format:

Mean: <<mean>>

Median:<<median>>

Mode:<<mode>>

Note: Mean and Median are formatted to single decimal places.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5

3 5 2 2 6

Output: Mean: 3.6

Median: 3.0

Mode: 2

### ***Answer***

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
```

```

        arr[i] = scanner.nextInt();
    }
    double sum = 0;
    for (int num : arr) {
        sum += num;
    }
    double mean = sum / n;
    Arrays.sort(arr);
    double median;
    if (n % 2 == 0) {
        median = (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
    } else {
        median = arr[n / 2];
    }
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int num : arr) {
        map.put(num, map.getDefault(num, 0) + 1);
    }
    int mode = arr[0];
    int maxCount = 0;
    for (int key : map.keySet()) {
        int count = map.get(key);
        if (count > maxCount || (count == maxCount && key < mode)) {
            maxCount = count;
            mode = key;
        }
    }
    System.out.printf("Mean: %.1f\n", mean);
    System.out.printf("Median: %.1f\n", median);
    System.out.println("Mode: " + mode);
    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

You are tasked with developing a loan management system for a financial institution. The system should include a LoanCalculator class to calculate monthly loan payments, track the total interest paid, and monitor the

remaining balance for various loan accounts. To enhance this system, you need to implement an AdvancedLoanCalculator subclass that offers additional features and benefits for loan management. For each month, it will provide monthly payments, principal payments, interest payments, and remaining balance. At the end, the total interest paid and monthly payment should be displayed.

Your goal is to design and implement the AdvancedLoanCalculator to extend the capabilities of the LoanCalculator in a way that provides more comprehensive insights into loan management.

### ***Input Format***

The first line of the input is a double value 'a', representing the Principal amount (loan amount) with two decimal places.

The second line of the input is a double value 'b', representing the Annual interest rate as a percentage with two decimal places.

The third line of the input is an integer value 'c', representing the Loan term in months.

### ***Output Format***

For each month of the loan term, the program will display the following details:

The first line of the output displays, the "Month " and is followed by the number (e.g., "Month 1:").

The second line of the output displays, the "Monthly Payment: " and is followed by the amount (formatted to two decimal places).

The third line of the output displays, the "Principal Payment: " and is followed by the amount (formatted to two decimal places).

The fourth line of the output displays, the "Interest Payment: " amount (formatted to two decimal places).

The fifth line of the output displays, the "Remaining Balance: " (formatted to two decimal places).

(between each month a blank line should be left)

At the end of all monthly outputs, display:

The output prints "Total Interest Paid: " followed by the total interest amount paid over the loan term, formatted to two decimal places.

The last line of output prints "Monthly Payment: " repeated again with the same monthly payment value (formatted to two decimal places) for summary.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 100000.00

10.00

2

Output: Month 1:

Monthly Payment: 50625.86

Principal Payment: 49792.53

Interest Payment: 833.33

Remaining Balance: 50207.47

Month 2:

Monthly Payment: 50625.86

Principal Payment: 50207.47

Interest Payment: 418.40

Remaining Balance: 0.00

Total Interest Paid: 1251.73

Monthly Payment: 50625.86

### **Answer**

```
import java.util.Scanner;
```

```
class LoanCalculator {
```

```
    double amortizationSchedule(double principal, double annualRate, int months)
```

```
{
```

```
    double monthlyRate = (annualRate / 100) / 12;
```

```

        double emi = (principal * monthlyRate * Math.pow(1 + monthlyRate,
months)) /
            (Math.pow(1 + monthlyRate, months) - 1);
        return emi;
    }
}

class AdvancedLoanCalculator extends LoanCalculator {
    @Override
    double amortizationSchedule(double principal, double annualRate, int months)
    {
        double monthlyRate = (annualRate / 100) / 12;
        double emi = (principal * monthlyRate * Math.pow(1 + monthlyRate,
months)) /
            (Math.pow(1 + monthlyRate, months) - 1);
        double balance = principal;
        double totalInterest = 0;
        for (int i = 1; i <= months; i++) {
            double interest = balance * monthlyRate;
            double principalPayment = emi - interest;
            balance -= principalPayment;
            if (balance < 0) balance = 0;
            totalInterest += interest;
            System.out.println("Month " + i + ":");
            System.out.printf("Monthly Payment: %.2f%n", emi);
            System.out.printf("Principal Payment: %.2f%n", principalPayment);
            System.out.printf("Interest Payment: %.2f%n", interest);
            System.out.printf("Remaining Balance: %.2f%n", balance);
            if (i != months) {
                System.out.println();
            }
        }
        System.out.printf("%nTotal Interest Paid: %.2f%n", totalInterest);
        return emi;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double principal = scanner.nextDouble();
        double rate = scanner.nextDouble();
        int months = scanner.nextInt();
    }
}

```

```
LoanCalculator calculator = new AdvancedLoanCalculator();  
double monthlyPayment = calculator.amortizationSchedule(principal, rate,  
months);
```

```
System.out.printf("Monthly Payment: %.2f%n", monthlyPayment);
```

```
scanner.close();
```

```
}  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MO\_Week 5\_Home

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Arsh wants to developing a program for an Employee Management System with two classes: Employee and Manager.

The Employee class holds attributes for name, employee ID, and basic salary, featuring methods for salary calculation and details display. The Manager class, a subclass of Employee, extends functionality by introducing department and bonus attributes and overrides the calculateSalary() and displayDetails() methods.

The program should input and display employee details and the total salary using the formula (basic salary + bonus).

#### ***Input Format***



The first line of input consists of a string, representing the employee name.

The second line consists of an integer, representing the employee ID.

The third line consists of a double value, representing the basic salary.

The fourth line consists of a string, representing the department, it contains lowercase and uppercase letters with spaces.

The fifth line consists of a double value, representing the bonus.

### ***Output Format***

The first line of output prints: "Employee ID: " followed by employeeId.

The second line prints: "Name: " followed by name.

The third line prints: "Basic Salary: " followed by basicSalary.

The fourth line prints: "Department: " followed by department.

The fifth line prints: "Bonus: " followed by bonus.

The sixth line prints: "Total Salary: " followed by totalSalary (rounded to two decimal places).

Refer to the sample output for the exact text and format.

### ***Sample Test Case***

Input: Paran

1001

50000.45

Sales force

10000.37

Output: Employee ID: 1001

Name: Paran

Basic Salary: 50000.45

Department: Sales force

Bonus: 10000.37

Total Salary: 60000.82

### Answer

```
import java.util.Scanner;

class Employee {
    String name;
    int employeeId;
    double basicSalary;
    Employee(String name, int employeeId, double basicSalary) {
        this.name = name;
        this.employeeId = employeeId;
        this.basicSalary = basicSalary;
    }
    double calculateSalary() {
        return basicSalary;
    }
    void displayDetails() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Name: " + name);
        System.out.println("Basic Salary: " + basicSalary);
    }
}

class Manager extends Employee {
    String department;
    double bonus;
    Manager(String name, int employeeId, double basicSalary, String department,
double bonus) {
        super(name, employeeId, basicSalary);
        this.department = department;
        this.bonus = bonus;
    }
    @Override
    double calculateSalary() {
        return basicSalary + bonus;
    }
    @Override
    void displayDetails() {
        super.displayDetails();
        System.out.println("Department: " + department);
        System.out.println("Bonus: " + bonus);
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String name = scanner.nextLine();
        int employeeId = scanner.nextInt();
        double basicSalary = scanner.nextDouble();
        scanner.nextLine();
        String department = scanner.nextLine();
        double bonus = scanner.nextDouble();

        Employee employee;

        employee = new Manager(name, employeeId, basicSalary, department,
            bonus);

        employee.displayDetails();
        System.out.printf("Total Salary: %.2f\n", employee.calculateSalary());
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Mr. Kapoor wants to create a program to calculate the volume of a Cuboid and a Cube using method overriding.

Implements a base class Cuboid with attributes for length, width, and height. Include a method calculateVolume() that computes the volume of the cuboid.

Extends the base class with a subclass Cube representing a cube, where all sides are equal. Override the calculateVolume() method in the Cube class to compute the volume of the cube.

The program should take user input for the dimensions of the cuboid and the side length of the cube and display the calculated volumes with two decimal places.

### ***Input Format***

The first line of input consists of 3 space-separated double values, representing the cuboid length, width, and height, respectively.

The second line consists of a double value, representing the side length of the cube.

### ***Output Format***

The first line of output prints the volume of the cuboid, rounded off to two decimal places.

The second line prints the volume of the cube, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 60.0 60.0 60.0  
50.0

Output: Volume of Cuboid: 216000.00  
Volume of Cube: 125000.00

### ***Answer***

```
import java.util.Scanner;

class Cuboid {
    double length, width, height;
    Cuboid(double length, double width, double height) {
        this.length = length;
        this.width = width;
        this.height = height;
    }
    double calculateVolume() {
        return length * width * height;
    }
}

class Cube extends Cuboid {
    Cube(double side) {
```

```

        super(side, side, side);
    }
    @Override
    double calculateVolume() {
        return length * length * length;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double cuboidLength = scanner.nextDouble();
        double cuboidWidth = scanner.nextDouble();
        double cuboidHeight = scanner.nextDouble();

        // Regular object instantiation for Cuboid
        Cuboid cuboid = new Cuboid(cuboidLength, cuboidWidth, cuboidHeight);
        System.out.printf("Volume of Cuboid: %.2f\n", cuboid.calculateVolume());

        double cubeSide = scanner.nextDouble();

        // Upcasting - Using superclass reference for subclass object (DMD)
        Cuboid cube = new Cube(cubeSide); // Upcasting
        System.out.printf("Volume of Cube: %.2f", cube.calculateVolume()); // Calls
        Cube's method dynamically

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Ram is designing a program to calculate the Body Mass Index (BMI). Your task is to assist him by following the given specifications.

Create a base class BMIcalculator with a method calculateBMI() to compute BMI using the formula  $\text{weight} / (\text{height} * \text{height})$ .

Extend the class with a subclass CustomBMICalculator that overrides the method calculateBMI() to calculate BMI based on custom criteria, assigning categories such as "Underweight," "Normal Weight," "Overweight," or "Obese."

BMI < 18.5, category = "Underweight" BMI >= 18.5 & < 24.9, category = "Normal Weight" BMI >= 25 & < 29.9, category = "Overweight" else category = "Obese"

Implement user input for weight and height and display both the standard and custom BMI calculations.

### ***Input Format***

The first line of input consists of a double value, representing the weight in kgs.

The second line consists of a double value, representing the height in meters.

### ***Output Format***

The first line of output prints: "Standard BMI Calculation:"

The second line of output prints: "BMI: " followed by the calculated BMI value (to two decimal places).

The third line of output prints: "Custom BMI Calculation:"

The fourth line of output prints: "Category: " followed by the BMI category.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 69.7

2.6

Output: Standard BMI Calculation:

BMI: 10.31

Custom BMI Calculation:

Category: Underweight

### ***Answer***

```

import java.util.Scanner;

class BMlcalculator {
    double weight, height;
    BMlcalculator(double weight, double height) {
        this.weight = weight;
        this.height = height;
    }
    double calculateBMI() {
        return weight / (height * height);
    }
    void displayBMI() {
        System.out.printf("BMI: %.2f\n", calculateBMI());
    }
}

class CustomBMlcalculator extends BMlcalculator {
    CustomBMlcalculator(double weight, double height) {
        super(weight, height);
    }
    @Override
    double calculateBMI() {
        return super.calculateBMI();
    }
    void displayCustomBMI() {
        double bmi = calculateBMI();
        String category;
        if (bmi < 18.5) {
            category = "Underweight";
        } else if (bmi < 24.9) {
            category = "Normal Weight";
        } else if (bmi < 29.9) {
            category = "Overweight";
        } else {
            category = "Obese";
        }
        System.out.println("Category: " + category);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}

```

```

double weight = scanner.nextDouble();
double height = scanner.nextDouble();

BMIcalculator bmiCalculator = new BMIcalculator(weight, height);
System.out.println("Standard BMI Calculation:");
bmiCalculator.displayBMI();

CustomBMIcalculator customBMIcalculator = new
CustomBMIcalculator(weight, height);
System.out.println("Custom BMI Calculation:");
customBMIcalculator.displayCustomBMI();

scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Janani aims to create a versatile palindrome checker capable of handling both numeric values and words.

The base class, PalindromeChecker, features a parameterized constructor that takes an integer as input. It includes a method, isPalindrome(), which determines whether the given integer is a palindrome. Include a method displayResult() to print the result of the palindrome check for numbers.

The subclass, WordPalindromeChecker, is derived from the base class. This subclass overrides the isPalindrome() method to accommodate word inputs, treating them case-insensitively. The overridden displayResult() method ensures that the outcome of the word palindrome check is appropriately printed.

Create instances of both classes in the main class and display the results.

#### **Input Format**

The first line of input consists of an integer.



The second line consists of a string, it contains lowercase and uppercase letters with spaces.

### **Output Format**

If the given integer is a palindrome, the first line of output prints "The number is a palindrome."

Else, print "The number is not a palindrome."

If the given string is a palindrome, the second line of output prints "The word is a palindrome."

Else, print "The word is not a palindrome."

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 121  
madam

Output: The number is a palindrome.  
The word is a palindrome.

### **Answer**

```
import java.util.Scanner;

class PalindromeChecker {
    int number;
    PalindromeChecker(int number) {
        this.number = number;
    }
    boolean isPalindrome() {
        int temp = number;
        int rev = 0;

        while (temp > 0) {
            int digit = temp % 10;
            rev = rev * 10 + digit;
            temp /= 10;
        }
    }
}
```

```

    }
    return rev == number;
}
void displayResult() {
    if (isPalindrome()) {
        System.out.println("The number is a palindrome.");
    } else {
        System.out.println("The number is not a palindrome.");
    }
}
}

class NumberPalindromeChecker extends PalindromeChecker {
    NumberPalindromeChecker(int number) {
        super(number);
    }
    @Override
    boolean isPalindrome() {
        return super.isPalindrome();
    }
    @Override
    void displayResult() {
        super.displayResult();
    }
}

class WordPalindromeChecker extends PalindromeChecker {
    String word;
    WordPalindromeChecker(String word) {
        super(0);
        this.word = word;
    }
    boolean isPalindromeWord() {
        String cleaned = word.replaceAll("\\s+", "").toLowerCase();
        int i = 0, j = cleaned.length() - 1;
        while (i < j) {
            if (cleaned.charAt(i) != cleaned.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }
}

```

```

@Override
void displayResult() {
    if (isPalindromeWord()) {
        System.out.println("The word is a palindrome.");
    } else {
        System.out.println("The word is not a palindrome.");
    }
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int number = scanner.nextInt();
        scanner.nextLine();
        String word = scanner.nextLine();

        PalindromeChecker checker;

        checker = new NumberPalindromeChecker(number);
        checker.displayResult();

        checker = new WordPalindromeChecker(word);
        checker.displayResult();

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Abstract\_Week 6\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 20

#### Section 1 : MCQ

1. what will be the output of the following code?

```
abstract class Device {  
    abstract void powerOn();  
}
```

```
class Computer extends Device {  
    void powerOn() {  
        System.out.println("Computer is powering on");  
    }  
}
```

```
class Laptop extends Computer {  
    void powerOn() {  
        System.out.println("Laptop is powering on");  
    }  
}
```

```
}  
public class Main {  
    public static void main(String[] args) {  
        Device d = new Laptop();  
        d.powerOn();  
    }  
}
```

**Answer**

Laptop is powering on

**Status : Correct**

**Marks : 1/1**

2. What is the key difference between an abstract class and an interface in Java?

**Answer**

An abstract class can have both abstract and non-abstract methods, while an interface can only have abstract methods.

**Status : Correct**

**Marks : 1/1**

3. If a class inheriting an abstract class does not define all of its function then it will be known as?

**Answer**

abstract

**Status : Correct**

**Marks : 1/1**

4. Which of these is not a correct statement?

**Answer**

Abstract class can be initiated by new operator

**Status : Correct**

**Marks : 1/1**

5. Which of the following is the correct way of implementing an interface salary by class manager?

**Answer**

class manager implements salary {}

**Status :** Correct

**Marks :** 1/1

6. What does an interface contain?

**Answer**

Method declaration

**Status :** Correct

**Marks :** 1/1

7. What type of methods an interface contain by default?

**Answer**

abstract

**Status :** Correct

**Marks :** 1/1

8. What type of variable can be defined in an interface?

**Answer**

public , static , final

**Status :** Correct

**Marks :** 1/1

9. what will be the output of the following code?

```
abstract class Shape {  
    abstract void draw();  
}
```

```
class Circle extends Shape {  
    public void draw() {
```

```
        System.out.println("Drawing Circle");
    }
}

class Square extends Shape {
    public void draw() {
        System.out.println("Drawing Square");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Shape s = new Square();
        s.draw();
    }
}
```

**Answer**

Drawing Square

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following code?

```
abstract class Animal {
    abstract void sound();
}

class Dog extends Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

```
}
```

**Answer**

Bark

**Status :** Correct

**Marks :** 1/1

11. What is the primary purpose of static methods in Java interfaces?

**Answer**

They allow an interface to provide helper methods without requiring an implementing class.

**Status :** Correct

**Marks :** 1/1

12. What happens when an implementing class does not override a default method from an interface?

**Answer**

The default method's implementation from the interface will be used.

**Status :** Correct

**Marks :** 1/1

13. Can a Java interface contain both default and static methods?

**Answer**

Yes, an interface can have both default and static methods.

**Status :** Correct

**Marks :** 1/1

14. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}
```



```
class B {  
    public void show() {  
        System.out.println("B's Method");  
    }  
}  
  
class C extends B implements A {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

**Answer**

B's Method

**Status :** Correct

**Marks :** 1/1

15. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}  
  
interface B {  
    default void show() {  
        System.out.println("B's Default Method");  
    }  
}  
  
class C implements A, B {  
    public void show() {  
        A.super.show();  
    }  
}
```

```
}  
}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

**Answer**

A's Default Method

**Status : Correct**

**Marks : 1/1**

16. If a class implements two interfaces that have the same default method, what must the class do?

**Answer**

The class must override the method to resolve ambiguity.

**Status : Correct**

**Marks : 1/1**

17. Consider a class implementing an interface and extending a class, both having a method with the same name. Which method gets called?

**Answer**

The method from the superclass

**Status : Correct**

**Marks : 1/1**

18. What is the output of the following code?

```
interface MathOperations {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathOperations.square(5));  
    }  
}
```

**Answer**

25

**Status :** Correct

**Marks :** 1/1

19. Which of the following statements about Java interfaces is true?

**Answer**

A class can implement multiple interfaces.

**Status :** Correct

**Marks :** 1/1

20. What is the output of the following code?

```
interface A {  
    static void display() {  
        System.out.println("Static method in A");  
    }  
}
```

```
class B implements A {  
    static void display() {  
        System.out.println("Static method in B");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        B.display();  
    }  
}
```

**Answer**

Static method in B

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Abstract\_Week 6\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Develop a vehicle rental system that calculates the rental cost for different types of vehicles: Car, Bike, and Truck. Each vehicle type has its own rental rate. The system should be able to calculate the total rental cost based on the type of vehicle selected and the number of rental days.

To implement this system, create the following classes:

**Vehicle:** An abstract class that provides the structure for different types of vehicles. It contains attributes for the rental rate and abstract methods `calculateRentalCost(int days)` for calculating the rental cost.

**Car:** A concrete subclass of Vehicle that represents a car. It implements the rental cost calculation specific to cars. **Bike:** A concrete subclass of Vehicle that represents a bike. It implements the rental cost calculation

specific to bikes. Truck: A concrete subclass of Vehicle that represents a truck. It implements the rental cost calculation specific to trucks.

Your system will allow users to input the rental rates for each type of vehicle and the number of rental days. Then, it will display the rental rate for the selected vehicle type and calculate and display the total rental cost based on the rental rate and the number of rental days.

### ***Input Format***

The first line of input contains an integer, carRate, representing the rental rate for a car.

The second line contains an integer, bikeRate, representing the rental rate for a bike.

The third line contains an integer, truckRate, representing the rental rate for a truck.

The fourth line contains an integer, rentalDays, representing the number of days for rental.

### ***Output Format***

For each type of vehicle, the program should display two lines:

The first line should include "Rental Rate: rate per day," where rate is the rental rate for the vehicle.

The second line should include "Total Rental Cost for Vehicle Type: totalCost," where Vehicle Type is the type of vehicle (Car, Bike, or Truck), and totalCost is the total rental cost based on the rental rate and the number of days.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 1000

150

750

7

Output: Rental Rate: 1000 per day  
Total Rental Cost for Car: 7000  
Rental Rate: 150 per day  
Total Rental Cost for Bike: 1050  
Rental Rate: 750 per day  
Total Rental Cost for Truck: 5250

**Answer**

```
import java.util.Scanner;
abstract class Vehicle {
    int rentalRate;
    Vehicle(int rentalRate) {
        this.rentalRate = rentalRate;
    }
    abstract int calculateRentalCost(int days);
    void display(int days, String type) {
        System.out.println("Rental Rate: " + rentalRate + " per day");
        System.out.println("Total Rental Cost for " + type + ": " +
            calculateRentalCost(days));
    }
}
class Car extends Vehicle {
    Car(int rentalRate) {
        super(rentalRate);
    }
    @Override
    int calculateRentalCost(int days) {
        return rentalRate * days;
    }
}
class Bike extends Vehicle {
    Bike(int rentalRate) {
        super(rentalRate);
    }
    @Override
    int calculateRentalCost(int days) {
        return rentalRate * days;
    }
}
class Truck extends Vehicle {
    Truck(int rentalRate) {
        super(rentalRate);
    }
}
```

```

    }
    @Override
    int calculateRentalCost(int days) {
        return rentalRate * days;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int carRate = scanner.nextInt();
        int bikeRate = scanner.nextInt();
        int truckRate = scanner.nextInt();
        int rentalDays = scanner.nextInt();
        Vehicle car = new Car(carRate);
        car.display(rentalDays, "Car");
        Vehicle bike = new Bike(bikeRate);
        bike.display(rentalDays, "Bike");
        Vehicle truck = new Truck(truckRate);
        truck.display(rentalDays, "Truck");
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Alice, a teacher seeking efficiency in analyzing student performance, has enlisted your help to create a program. Your task is to implement the StudentMarksCalculator class, extending the abstract class MarksCalculator.

This abstract class has three methods: calculateSum (int) for total marks, calculateAverage (double) for average marks, and findBestMark (double) for the highest mark. Your job is to complete the implementation of these methods in the StudentMarksCalculator class.

### **Input Format**

The first line of input is an integer representing the student ID.



The second line of input is an integer representing the number of subjects.

The third line of input is a space-separated integer representing the marks obtained by the student in each subject.

### ***Output Format***

The first line of output prints "Total Marks: " followed by an integer representing the sum of the marks.

The second line of output prints "Average Marks: " followed by a double value (rounded to two decimal places) representing the average marks.

The third line of output prints "Best Mark: " followed by an integer representing the highest mark obtained.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 101

3

90 85 88

Output: Total Marks: 263

Average Marks: 87.67

Best Mark: 90

### ***Answer***

```
import java.util.*;

abstract class MarksCalculator {
    abstract int calculateSum(int[] marks);
    abstract double calculateAverage(int[] marks);
    abstract int findBestMark(int[] marks);
}

class StudentMarksCalculator extends MarksCalculator {
    @Override
    int calculateSum(int[] marks) {
        int sum = 0;
        for (int mark : marks) {
            sum += mark;
        }
    }
}
```

```

    }
    return sum;
}
@Override
double calculateAverage(int[] marks) {
    int sum = calculateSum(marks);
    return (double) sum / marks.length;
}
@Override
int findBestMark(int[] marks) {
    int max = marks[0];
    for (int mark : marks) {
        if (mark > max) {
            max = mark;
        }
    }
    return max;
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int studentId = scanner.nextInt();

        int numSubjects = scanner.nextInt();

        int[] marks = new int[numSubjects];
        for (int i = 0; i < numSubjects; i++) {
            marks[i] = scanner.nextInt();
        }

        StudentMarksCalculator calculator = new StudentMarksCalculator();

        int totalMarks = calculator.calculateSum(marks);
        System.out.println("Total Marks: " + totalMarks);

        double averageMarks = calculator.calculateAverage(marks);
        System.out.println("Average Marks: " + String.format("%.2f",
            averageMarks));

        int bestMark = calculator.findBestMark(marks);
    }
}

```

```
System.out.println("Best Mark: " + bestMark);
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Bob is developing a geometric calculator to calculate the area and perimeter of different shapes. He wants to write a program using an abstract class named Geometry, which declares two abstract methods: calculateArea() and calculatePerimeter().

To implement the required functionality, create a class Shape that extends Geometry and provides concrete implementations for the area and perimeter calculations of a rectangle, square, and circle.

The program should take input values for:

length and width of the rectangle  
side of the square  
radius of the circle

These inputs should be read in the AppMain class, which contains the main() method. After setting the values, the program should display:

The area of the rectangle, square, and circle (each on a new line)  
The perimeter of the rectangle, square, and circle (each on a new line)

All outputs must be formatted to two decimal places.

Formula

Rectangle

Area = length \* width

Perimeter = 2 \* (length + width)

Square

Area = side \* side

$\text{Perimeter} = 4 * \text{side}$

Circle

$\text{Area} = \pi * r^2$

$\text{Perimeter} = 2 * \pi * \text{radius}$

where  $\pi = 3.14$

### ***Input Format***

The first line of input is an integer, length, representing the length of the rectangle.

The second line of input is an integer, width, representing the width of the rectangle.

The third line of input is an integer, side, representing the side length of the square.

The fourth line of input is a double radius, representing the radius of the circle.

### ***Output Format***

The first three lines of output display the area of the rectangle, square, and circle (each on a new line, rounded to two decimal places).

The last three lines of output display the perimeter of the rectangle, square, and circle (each on a new line, rounded to two decimal places).

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5

7

4

2.0

Output: 35.00

16.00

12.56

24.00  
16.00  
12.56

**Answer**

```
import java.util.Scanner;

abstract class Geometry {
    abstract void calculateArea();
    abstract void calculatePerimeter();
}

class Shape extends Geometry {
    int length, width, side;
    double radius;
    @Override
    void calculateArea() {
        double rectArea = length * width;
        double squareArea = side * side;
        double circleArea = 3.14 * radius * radius;
        System.out.printf("%.2f%n", rectArea);
        System.out.printf("%.2f%n", squareArea);
        System.out.printf("%.2f%n", circleArea);
    }
    @Override
    void calculatePerimeter() {
        double rectPerimeter = 2 * (length + width);
        double squarePerimeter = 4 * side;
        double circlePerimeter = 2 * 3.14 * radius;
        System.out.printf("%.2f%n", rectPerimeter);
        System.out.printf("%.2f%n", squarePerimeter);
        System.out.printf("%.2f%n", circlePerimeter);
    }
}

class AppMain {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Shape obj = new Shape();

        obj.length = scanner.nextInt();
        obj.width = scanner.nextInt();
        obj.side = scanner.nextInt();
        obj.radius = scanner.nextDouble();
    }
}
```

```
obj.calculateArea();
obj.calculatePerimeter();

    scanner.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Sophia is developing a matrix analysis tool for a data analytics company. The tool needs to analyze square matrices and extract insights from the matrix diagonals.

To organize the code properly, Sophia creates an interface named Matrix that declares a method for finding the smallest and largest elements along the principal and secondary diagonals of the matrix.

Sophia then creates a class named MatrixAnalyzer that implements the Matrix interface. This class provides the logic to process a given square matrix and print:

The smallest and largest elements in the principal diagonal (from top-left to bottom-right). The smallest and largest elements in the secondary diagonal (from top-right to bottom-left).

Your task is to implement the Matrix interface and the MatrixAnalyzer class. The main driver program (in the class Main) will read the input matrix, create an instance of MatrixAnalyzer, and invoke its method to display the results.

##### **Input Format**

The first line contains an integer n, representing the size of the square matrix.

The next n lines each contain n integers separated by spaces, representing the elements of the matrix.

##### **Output Format**

The output prints the four lines:

"Smallest Element - 1: <smallest element in the principal diagonal>" (integer)

"Largest Element - 1: <largest element in the principal diagonal>" (integer)

"Smallest Element - 2: <smallest element in the secondary diagonal>" (integer)

"Largest Element - 2: <largest element in the secondary diagonal>" (integer)

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 5

7 8 9 0 1

2 3 4 5 6

5 4 2 0 8

23 5 6 8 9

12 5 6 7 32

Output: Smallest Element - 1: 2

Largest Element - 1: 32

Smallest Element - 2: 1

Largest Element - 2: 12

### **Answer**

```
import java.util.Scanner;

interface Matrix {
    void diagonalsMinMax(int[][] matrix);
}

class MatrixAnalyzer implements Matrix {
    @Override
    public void diagonalsMinMax(int[][] matrix) {
        int n = matrix.length;
        int min1 = Integer.MAX_VALUE;
        int max1 = Integer.MIN_VALUE;
        int min2 = Integer.MAX_VALUE;
        int max2 = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
```

```

        min1 = Math.min(min1, matrix[i][i]);
        max1 = Math.max(max1, matrix[i][i]);
        min2 = Math.min(min2, matrix[i][n - i - 1]);
        max2 = Math.max(max2, matrix[i][n - i - 1]);
    }
    System.out.println("Smallest Element - 1: " + min1);
    System.out.println("Largest Element - 1: " + max1);
    System.out.println("Smallest Element - 2: " + min2);
    System.out.println("Largest Element - 2: " + max2);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] matrix = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }
        MatrixAnalyzer analyzer = new MatrixAnalyzer();
        analyzer.diagonalsMinMax(matrix);
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Raj is curious about how old he is in the current year.

He has asked you to create a simple program that calculates a person's age based on their birth year. You decide to implement this functionality using the AgeCalculator interface and the HumanAgeCalculator class.

Note: The current year is 2024. Calculate the current age by using the formula: current year - birth year.

**Input Format**



The input consists of an integer representing the birth year.

### **Output Format**

The output displays "You are X years old." where X is an integer representing the calculated age based on the entered birth year.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1934

Output: You are 90 years old.

### **Answer**

```
import java.util.Scanner;

interface AgeCalculator {
    int calculateAge(int birthYear);
}

class HumanAgeCalculator implements AgeCalculator {
    @Override
    public int calculateAge(int birthYear) {
        int currentYear = 2024;
        return currentYear - birthYear;
    }
}

class AgeCalculatorApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        AgeCalculator ageCalculator = new HumanAgeCalculator();

        int birthYear = scanner.nextInt();
        int age = ageCalculator.calculateAge(birthYear);

        System.out.println("You are " + age + " years old.");
    }
}
```

**Status : Correct**

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Abstract\_Week 6\_Yourself

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Jessica is designing a fantasy game character system. The system includes an abstract class GameCharacter with two abstract methods: attack(int attributeValue) and defend(int defense).

Two subclasses extend GameCharacter:

Warrior implements attack(int strength), where attack damage = strength × 3, and defend(int defense), which boosts defense. Wizard implements attack(int magicPower), where magic damage = magicPower × 2, and defend(int defense), which creates a magical barrier.

The program prompts the player to:

Choose a character type: 1 for Warrior or 2 for Wizard.

Input an integer representing either strength (for Warrior) or magic power

(for Wizard). The program dynamically instantiates the chosen character, execute the attack and defend actions, and display the corresponding messages.

### ***Input Format***

The first line of input consists of an integer, representing the choice of the character - 1 for Warrior and 2 for Wizard.

If the choice is 1, the second line consists of an integer N, representing the strength.

If the choice is 2, the second line consists of an integer M, representing the magic power.

### ***Output Format***

If the choice is 1, prints the actions of a warrior in the following format:

"Warrior Actions:

Attack: Powerful sword slash for [result] damage!

Defend: Raises shield, defence boosted by [N]!"

If the choice is 2, prints the actions of a wizard in the following format:

"Wizard Actions:

Attack: Casts spell, deals [result] magical damage!

Defend: Creates magical barrier, defence boosted by [M]!"

If any other choice is given, print "Invalid choice".

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 1

68

Output: Warrior Actions:

Attack: Powerful sword slash for 204 damage!

Defend: Raises shield, defence boosted by 68!

### **Answer**

```
import java.util.Scanner;

abstract class GameCharacter {
    abstract void attack(int attributeValue);
    abstract void defend(int defense);
}

class Warrior extends GameCharacter {
    @Override
    void attack(int strength) {
        System.out.println("Attack: Powerful sword slash for " + (strength * 3) + "
damage!");
    }
    @Override
    void defend(int defense) {
        System.out.println("Defend: Raises shield, defence boosted by " + defense +
"!");
    }
}

class Wizard extends GameCharacter {
    @Override
    void attack(int magicPower) {
        System.out.println("Attack: Casts spell, deals " + (magicPower * 2) + "
magical damage!");
    }
    @Override
    void defend(int defense) {
        System.out.println("Defend: Creates magical barrier, defence boosted by " +
defense + "!");
    }
}

class Main {
    public static void main(String[] args) {
```

```

Scanner scanner = new Scanner(System.in);
int choice = scanner.nextInt();
int attributeValue = scanner.nextInt();
GameCharacter character;
if (choice == 1) {
    character = new Warrior();
} else if (choice == 2) {
    character = new Wizard();
} else {
    System.out.println("Invalid choice");
    return;
}

System.out.println((choice == 1 ? "Warrior" : "Wizard") + " Actions:");
character.attack(attributeValue);
character.defend(attributeValue);

scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

In a travel expense management system, individuals have various expenses associated with their trips, including accommodation, transportation, and meals. Implement an abstract class `Expense` with an abstract method `calculateTotal()` and the following subclasses:

Accommodation with a method `calculateTotal()` that calculates the total accommodation cost by multiplying quantity and price. Transportation with a method `calculateTotal()` that calculates the total transportation cost by multiplying quantity and price. Meals with a method `calculateTotal()` that calculates the total meal cost by multiplying quantity and price.

The program inputs the quantity and price for each of these expense types. It should then calculate and display the total expenses for each category and the overall trip expenses. The results need to be rounded off to two decimal places.

### ***Input Format***

The first line of input consists of two space-separated values, quantity(an integer), and price per unit(a double) for accommodation.

The second line consists of two space-separated values, quantity(an integer), and price per unit(a double) for transportation.

The third line consists of two space-separated values, quantity(an integer), and price per unit(a double) for meals.

### ***Output Format***

The first line of output prints "Accommodation: ", followed by a double value, representing the expenses for accommodation.

The second line prints "Transportation: ", followed by a double value, representing the expenses for transportation.

The third line prints "Meals: ", followed by a double value, representing the expenses for meals.

The fourth line prints "Total Expenses: ", followed by a double value, representing the total expenses.

Round off all the values to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 2 450.50

4 1020.78

3 375.23

Output: Accommodation: Rs.901.00

Transportation: Rs.4083.12

Meals: Rs.1125.69

Total Expenses: Rs.6109.81

### Answer

```
import java.util.Scanner;

abstract class Expense {
    int quantity;
    double price;
    void inputDetails(Scanner scanner) {
        quantity = scanner.nextInt();
        price = scanner.nextDouble();
    }
    abstract double calculateTotal();
    abstract void displayDetails();
}

class Accommodation extends Expense {
    @Override
    double calculateTotal() {
        return quantity * price;
    }
    @Override
    void displayDetails() {
        System.out.printf("Accommodation: Rs.%.2f%n", calculateTotal());
    }
}

class Transportation extends Expense {
    @Override
    double calculateTotal() {
        return quantity * price;
    }
    @Override
    void displayDetails() {
        System.out.printf("Transportation: Rs.%.2f%n", calculateTotal());
    }
}

class Meals extends Expense {
    @Override
    double calculateTotal() {
        return quantity * price;
    }
    @Override
    void displayDetails() {
        System.out.printf("Meals: Rs.%.2f%n", calculateTotal());
    }
}
```

```

    }
}
class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        Expense accommodation = new Accommodation();
        accommodation.inputDetails(scanner);

        Expense transportation = new Transportation();
        transportation.inputDetails(scanner);

        Expense meals = new Meals();
        meals.inputDetails(scanner);

        accommodation.displayDetails();
        transportation.displayDetails();
        meals.displayDetails();

        double totalExpenses = accommodation.calculateTotal() +
        transportation.calculateTotal() + meals.calculateTotal();
        System.out.printf("Total Expenses: Rs.%.2f\n", totalExpenses);

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Enigma is tasked with developing a comprehensive retail management system for a multi-product store. The system needs to handle diverse products such as electronics and clothing, each with specific costs, prices, and additional costs like warranty or material expenses.

Create an abstract class Product with abstract methods calculateTotalCost() and calculateProfitMargin(). Subclasses ElectronicsProduct and ClothingProduct, implement these abstract methods to cater to the specific needs of each product type.



## Formula

Total cost of electronic product = cost + warranty cost

Total cost of clothing product = cost + material cost

Profit margin =  $((\text{price} - \text{cost}) / \text{price}) * 100$ .

## Input Format

The first line of input consists of three positive double values, representing the cost, price, and warranty cost of the electronic product, separated by space.

The second line consists of three positive double values, representing the cost, price, and material cost of the clothing product, separated by space.

## Output Format

The first line of output prints "Electronics Product:".

The second line prints "Total Cost: " followed by a double representing the total cost of the electronics product, formatted to two decimal places.

The third line prints "Profit Margin: " followed by a double representing the profit margin of the electronics product, formatted to two decimal places.

After a blank line, the output prints "Clothing Product:".

The next line prints "Total Cost: " followed by a double representing the total cost of the clothing product, formatted to two decimal places.

The final line prints "Profit Margin: " followed by a double representing the profit margin of the clothing product, formatted to two decimal places.

Refer to the sample output for formatting specifications.

## Sample Test Case

Input: 500.7 799.7 50.6

30.4 89.9 15.2

Output: Electronics Product:

Total Cost: 551.30

Profit Margin: 37.39

Clothing Product:

Total Cost: 45.60

Profit Margin: 66.18

**Answer**

```
import java.util.Scanner;
```

```
abstract class Product {  
    double cost;  
    double price;
```

```
    Product(double cost, double price) {  
        this.cost = cost;  
        this.price = price;  
    }
```

```
    abstract double calculateTotalCost();  
    abstract double calculateProfitMargin();  
}
```

```
class ElectronicsProduct extends Product {  
    double warrantyCost;
```

```
    ElectronicsProduct(double cost, double price, double warrantyCost) {  
        super(cost, price);  
        this.warrantyCost = warrantyCost;  
    }
```

```
    @Override  
    double calculateTotalCost() {  
        return cost + warrantyCost;  
    }
```

```
    @Override  
    double calculateProfitMargin() {  
        return ((price - cost) / price) * 100;  
    }  
}
```

```
class ClothingProduct extends Product {  
    double materialCost;
```

```

ClothingProduct(double cost, double price, double materialCost) {
    super(cost, price);
    this.materialCost = materialCost;
}

@Override
double calculateTotalCost() {
    return cost + materialCost;
}

@Override
double calculateProfitMargin() {
    return ((price - cost) / price) * 100;
}
}

class ProductMain {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input for ElectronicsProduct
        double electronicsCost = scanner.nextDouble();
        double electronicsPrice = scanner.nextDouble();
        double electronicsWarrantyCost = scanner.nextDouble();
        ElectronicsProduct electronicsProduct = new
        ElectronicsProduct(electronicsCost, electronicsPrice, electronicsWarrantyCost);

        // Input for ClothingProduct
        double clothingCost = scanner.nextDouble();
        double clothingPrice = scanner.nextDouble();
        double clothingMaterialCost = scanner.nextDouble();
        ClothingProduct clothingProduct = new ClothingProduct(clothingCost,
        clothingPrice, clothingMaterialCost);

        System.out.println("Electronics Product:");
        System.out.printf("Total Cost: %.2f\n",
        electronicsProduct.calculateTotalCost());
        System.out.printf("Profit Margin: %.2f\n",
        electronicsProduct.calculateProfitMargin());

        System.out.println("\nClothing Product:");
        System.out.printf("Total Cost: %.2f\n",
        clothingProduct.calculateTotalCost());
    }
}

```

```
System.out.printf("Profit Margin: %.2f\n",
clothingProduct.calculateProfitMargin());

    scanner.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Develop a program for managing employee information that caters to both full-time and part-time employees. The program should be capable of computing the salary for each category of employee and presenting their particulars. To achieve this, create two classes, FullTimeEmployee and PartTimeEmployee, that adhere to the Employee interface.

The program is expected to accept input data, including the name and monthly salary for full-time employees, as well as the name, hourly rate, and hours worked for part-time employees. Subsequently, it should calculate and exhibit the employee details and their respective salaries.

For Full-Time employees, the annual salary should be calculated as 12 times the monthly salary.

For Part-Time employees, the salary calculation should be based on the formula: hourly rate \* hours worked.

##### ***Input Format***

The first line of input should be a string representing the name of a full-time employee.

The second line of input should be an integer representing the monthly salary of the full-time employee.

The third line of input should be a string representing the name of a part-time employee.

The fourth line of input should be an integer representing the hourly rate of the

part-time employee.

The fifth line of input should be an integer representing the number of hours worked by the part-time employee.

### ***Output Format***

The output displays the following details:

Full-Time Employee Details:

Name: [Full-Time Employee Name] (string)

Monthly Salary: \$[Monthly Salary] (integer)

Annual Salary: \$[12 times Monthly Salary] (integer)

Part-Time Employee Details:

Name: [Part-Time Employee Name] (string)

Hourly Rate: \$[Hourly Rate] (integer)

Hours Worked: [Hours Worked] hours (integer)

Monthly Salary: \$[Calculated Monthly Salary] (integer)

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: John Smith

15000

Mary Johnson

100

100

Output: Full-Time Employee Details:

Name: John Smith

Monthly Salary: \$15000

Annual Salary: \$180000

Part-Time Employee Details:

Name: Mary Johnson

Hourly Rate: \$100

Hours Worked: 100 hours

Monthly Salary: \$10000

### **Answer**

```
import java.util.Scanner;
```

```
interface Employee {  
    void displayDetails();  
}
```

```
class FullTimeEmployee implements Employee {  
    String name;  
    int monthlySalary;  
    FullTimeEmployee(String name, int monthlySalary) {  
        this.name = name;  
        this.monthlySalary = monthlySalary;  
    }  
    @Override  
    public void displayDetails() {  
        int annualSalary = monthlySalary * 12;  
        System.out.println("Full-Time Employee Details:");  
        System.out.println("Name: " + name);  
        System.out.println("Monthly Salary: $" + monthlySalary);  
        System.out.println("Annual Salary: $" + annualSalary);  
    }  
}
```

```
class PartTimeEmployee implements Employee {  
    String name;  
    int hourlyRate;  
    int hoursWorked;  
    PartTimeEmployee(String name, int hourlyRate, int hoursWorked) {  
        this.name = name;  
        this.hourlyRate = hourlyRate;  
        this.hoursWorked = hoursWorked;  
    }  
}
```

```

    }
    @Override
    public void displayDetails() {
        int monthlySalary = hourlyRate * hoursWorked;

        System.out.println("Part-Time Employee Details:");
        System.out.println("Name: " + name);
        System.out.println("Hourly Rate: $" + hourlyRate);
        System.out.println("Hours Worked: " + hoursWorked + " hours");
        System.out.println("Monthly Salary: $" + monthlySalary);
    }
}

class EmployeeInheritanceDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String fullName = scanner.nextLine();
        int fullTimeSalary = scanner.nextInt();
        scanner.nextLine();
        String partTimeName = scanner.nextLine();
        int hourlyRate = scanner.nextInt();
        int hoursWorked = scanner.nextInt();
        FullTimeEmployee fullTimeEmployee = new FullTimeEmployee(fullName,
fullTimeSalary);
        PartTimeEmployee partTimeEmployee = new
PartTimeEmployee(partTimeName, hourlyRate, hoursWorked);
        fullTimeEmployee.displayDetails();
        System.out.println();
        partTimeEmployee.displayDetails();
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Jaheer is working on a health monitoring system to help individuals calculate their Body Mass Index (BMI). He has implemented a basic BMI calculator and an interface called HealthCalculator. It should have a

method called calculateBMI.

You are tasked with creating a program that takes weight and height as input, calculates the BMI using the BMICalculator class, and displays the result. If the height or weight is less than or equal to zero, then return -1.

Formula:  $BMI = \text{weight} / (\text{height} * \text{height})$

### ***Input Format***

The first line of input consists of a double value W, the person's weight in kilograms.

The second line consists of a double value H, the height of the person in meters.

### ***Output Format***

The output displays "BMI: " followed by a double value, representing the calculated BMI, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 70.0

1.75

Output: BMI: 22.86

### ***Answer***

```
import java.util.Scanner;
```

```
interface HealthCalculator {  
    double calculateBMI(double weight, double height);  
}
```

```
class BMICalculator implements HealthCalculator {
```

```
    @Override
```

```
    public double calculateBMI(double weight, double height) {  
        if (weight <= 0 || height <= 0) {  
            return -1;
```



```
}  
    return weight / (height * height);  
}  
}  
  
class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double weight = scanner.nextDouble();  
        double height = scanner.nextDouble();  
  
        BMICalculator bmiCalculator = new BMICalculator();  
  
        double bmi = bmiCalculator.calculateBMI(weight, height);  
  
        System.out.printf("BMI: %.2f\n", bmi);  
  
        scanner.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Abstract\_Week 6\_Home

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

In an educational institution, the administration streamlines the process of calculating students' CGPA. They want to create a program that takes into account the credit points and grade points for each subject, allowing students to easily calculate their CGPA.

The program consists of an abstract class GradeCalculator with an abstract method calculateGradePoints(), and a class Subject representing each academic subject.

Students can input the number of subjects, along with their respective credit scores and grade points. The program then calculates the CGPA based on the total credit scores and grade points.

Note: To calculate the CGPA, calculate the grade points for the subject by

multiplying credits by grade points. Divide the total grade points by the total credits.

### ***Input Format***

The first line of input consists of an integer N, representing the number of subjects.

The next N lines consist of two space-separated values representing credit score(an integer) and grade points(a double).

### ***Output Format***

The output displays a double value, representing the CGPA of the student, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 3

3 8.5

2 7.0

4 9.0

Output: CGPA: 8.39

### ***Answer***

```
import java.util.Scanner;

abstract class GradeCalculator {
    abstract double calculateGradePoints();
}

class Subject extends GradeCalculator {
    private int credits;
    private double gradePoints;
    Subject(int credits, double gradePoints) {
        this.credits = credits;
        this.gradePoints = gradePoints;
    }
    public int getCredits() {
        return credits;
    }
}
```

```

    }
    @Override
    double calculateGradePoints() {
        return credits * gradePoints;
    }
}

class CGPACalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int numSubjects = scanner.nextInt();
        Subject[] subjects = new Subject[numSubjects];
        for (int i = 0; i < numSubjects; i++) {
            int credits = scanner.nextInt();
            double gradePoints = scanner.nextDouble();
            subjects[i] = new Subject(credits, gradePoints);
        }

        double totalGradePoints = 0;
        int totalCredits = 0;
        for (Subject subject : subjects) {
            totalGradePoints += subject.calculateGradePoints();
            totalCredits += subject.getCredits();
        }

        double cgpa = totalGradePoints / totalCredits;
        System.out.printf("CGPA: %.2f\n", cgpa);
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Maria is developing a game where players explore different planets and collect resources. The availability of resources on each planet follows a geometric progression.

Create an abstract class Series with an abstract method nextTerm().

Implement a subclass GeometricSeries that calculates the next term in a geometric progression for the resource collection on planets. The GeometricSeries class should have the following constructor:

GeometricSeries(int firstTerm, int commonRatio, int numberOfTerms):  
Initializes the series with the first term, common ratio, and number of terms (representing the planets to explore). Allow players to input the initial resource level, resource growth ratio, and the number of planets they plan to explore.

Display the predicted resource levels on each planet.

Note:

$\text{term}_i = a \times r^i$

Where:

$a$  = firstTerm (the initial value of the series)

$r$  = commonRatio (the value multiplied at each step)

$i$  = currentTerm (which starts from 0 for the first term, 1 for second, and upto (number of terms -1) for last term)

### ***Input Format***

The first line contains an integer representing the first term of the geometric series (initial resource level).

The second line contains an integer representing the common ratio (resource growth ratio).

The third line contains an integer representing the number of terms (planets to explore).

### ***Output Format***

The output prints the resource levels of each planet (as integers), separated by space.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 2

3

5

Output: 2 6 18 54 162

### **Answer**

```
import java.util.Scanner;
```

```
abstract class Series {  
    abstract int nextTerm();  
}
```

```
class GeometricSeries extends Series {  
    int firstTerm;  
    int commonRatio;  
    int numberOfTerms;  
    int currentIndex = 0;
```

```
    GeometricSeries(int firstTerm, int commonRatio, int numberOfTerms) {  
        this.firstTerm = firstTerm;  
        this.commonRatio = commonRatio;  
        this.numberOfTerms = numberOfTerms;  
    }
```

```
    @Override  
    int nextTerm() {  
        int value = (int)(firstTerm * Math.pow(commonRatio, currentIndex));  
        currentIndex++;  
        return value;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        int firstTerm = scanner.nextInt();  
        int commonRatio = scanner.nextInt();  
        int numberOfTerms = scanner.nextInt();  
        GeometricSeries geometricSeries = new GeometricSeries(firstTerm,  
            commonRatio, numberOfTerms);
```

```
for (int i = 0; i < numberOfTerms; i++) {  
    System.out.print(geometricSeries.nextTerm() + " ");  
}  
  
    scanner.close();  
}  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

A financial analyst, Alex, needs a program to calculate simple interest for various financial transactions. He requires a straightforward tool that takes in the principal amount, interest rate, and time in years and computes the interest.

The formula to be used is:  $\text{Interest} = \text{Principal} \times \text{Rate} \times \text{Time} / 100$

Implement this functionality using the InterestCalculator interface and the SimpleInterestCalculator class.

#### **Input Format**

The first line of input consists of the principal amount P as a double value.

The second line of input consists of the annual interest rate r as a double value.

The third line of input consists of the number of years t as a positive integer, which is an integer value.

#### **Output Format**

The output displays the calculated simple interest in the following format: "Simple Interest: [interest\_value]", Here, [interest\_value] should be replaced with the actual interest value calculated by the program.

Refer to the sample output for the formatting specifications.

### Sample Test Case

Input: 1000.00

5.00

2

Output: Simple Interest: 100.0

### Answer

```
import java.util.Scanner;

interface InterestCalculator {
    double simpleInterest(double principal, double rate, int time);
}

class SimpleInterestCalculator implements InterestCalculator {
    @Override
    public double simpleInterest(double principal, double rate, int time) {
        return (principal * rate * time) / 100;
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double principal = scanner.nextDouble();

        double rate = scanner.nextDouble();

        int time = scanner.nextInt();

        InterestCalculator calculator = new SimpleInterestCalculator();

        double interest = calculator.simpleInterest(principal, rate, time);

        System.out.println("Simple Interest: " + interest);
    }
}
```

Status : Correct

Marks : 10/10

### 4. Problem Statement



Oviya is fascinated by automorphic numbers and wants to create a program to determine whether a given number is an automorphic number or not.

An automorphic number is a number whose square ends with the same digits as the number itself. For example,  $25 = (25)^2 = 625$

Oviya has defined two interfaces: NumberInput for taking user input and AutomorphicChecker for checking if a given number is automorphic. The class AutomorphicNumber implements both interfaces.

Help her complete the task.

### ***Input Format***

The input consists of a single integer n.

### ***Output Format***

If the input number is an automorphic number, print "n is an automorphic number". Otherwise, print "n is not an automorphic number".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 25

Output: 25 is an automorphic number

### ***Answer***

```
import java.util.Scanner;

interface NumberInput {
    int getInput();
}

interface AutomorphicChecker {
    boolean checkAutomorphic(int n);
}

class AutomorphicNumber implements NumberInput, AutomorphicChecker {
    Scanner scanner = new Scanner(System.in);
```

```

@Override
public int getInput() {
    return scanner.nextInt();
}
@Override
public boolean checkAutomorphic(int n) {
    int square = n * n;
    String numStr = Integer.toString(n);
    String squareStr = Integer.toString(square);
    return squareStr.endsWith(numStr);
}
}

public class Main {
    public static void main(String[] args) {
        AutomorphicNumber automorphicNumber = new AutomorphicNumber();
        int inputNumber = automorphicNumber.getInput();

        boolean isAutomorphic =
        automorphicNumber.checkAutomorphic(inputNumber);

        if (isAutomorphic) {
            System.out.println(inputNumber+" is an automorphic number");
        } else {
            System.out.println(inputNumber+" is not an automorphic number");
        }
    }
}

```

**Status : Correct**

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Exception\_Week 7\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 20

#### Section 1 : MCQ

1. Which of the following correctly throws a custom exception object in Java?

**Answer**

throw new CustomException();

**Status : Correct**

**Marks : 1/1**

2. Which class do all exception objects inherit from in Java?

**Answer**

Throwable

**Status : Correct**

**Marks : 1/1**

3. What is the return type of the getMessage() method of an exception object?

**Answer**

String

**Status : Correct**

**Marks : 1/1**

4. In the catch block, what type of object is assigned to the exception variable?

**Answer**

The exception object

**Status : Correct**

**Marks : 1/1**

5. Which of the following statements is incorrect about exception objects in Java?

**Answer**

Exception objects cannot be serialized.

**Status : Correct**

**Marks : 1/1**

6. Which class is the root of the exception hierarchy in Java?

**Answer**

Throwable

**Status : Correct**

**Marks : 1/1**

7. Which of the following is the root class for all exceptions in Java?

**Answer**

Throwable

**Status : Correct**

**Marks : 1/1**

8. Which of the following exceptions is an unchecked exception?

**Answer**

NullPointerException

**Status : Correct**

**Marks : 1/1**

9. Which of the following statements is true regarding the finally block in Java?

**Answer**

It is executed whether or not an exception occurs.

**Status : Correct**

**Marks : 1/1**

10. What is the purpose of the try block in Java exception handling?

**Answer**

To define the code that may throw an exception.

**Status : Correct**

**Marks : 1/1**

11. Which of the following is used to handle exceptions in Java?

**Answer**

try-catch

**Status : Correct**

**Marks : 1/1**

12. What type of exception is ArithmeticException?

**Answer**

Unchecked Exception

**Status : Correct**

**Marks : 1/1**

13. What is the purpose of a catch block?

**Answer**

To handle exceptions

**Status : Correct**

**Marks : 1/1**

14. When do exceptions occur during the execution of a Java program?

**Answer**

At runtime

**Status : Correct**

**Marks : 1/1**

15. Which of the following handles the exception if a catch block is not present?

**Answer**

default handler

**Status : Correct**

**Marks : 1/1**

16. What will be the output of the following code?

```
class MyException extends Exception {  
    public MyException(String message) {  
        super(message);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        try {  
            throw new MyException("Custom exception");  
        } catch (MyException e) {  
            System.out.println("Exception: " + e.getMessage());  
        } catch (Exception e) {  

```

```
        System.out.println("Some other exception");
    }
}
```

**Answer**

Exception: Custom exception

**Status :** Correct

**Marks :** 1/1

17. What will be the output of the following code?

```
class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException occurred");
        }
    }
}
```

**Answer**

ArithmeticException occurred

**Status :** Correct

**Marks :** 1/1

18. What will be the output of the following code?

```
class Main {
    public static void main(String[] args) {
        try {
            String str = null;
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException caught");
        }
    }
}
```

**Answer**

NullPointerException caught

**Status :** Correct

**Marks :** 1/1

19. What will be the output of the following code?

```
class MyException extends Exception {  
    public MyException(String message) {  
        super(message);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        try {  
            throw new MyException("User-defined exception occurred");  
        } catch (MyException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

**Answer**

User-defined exception occurred

**Status :** Correct

**Marks :** 1/1

20. What will be the output of the following code?

```
class Main {  
    public static void main(String[] args) {  
        try {  
            int a = 5;  
            int b = 0;  
            System.out.println(a / b);  
        } catch (ArithmeticException | NullPointerException e) {  
            System.out.println("Exception caught");  
        }  
    }  
}
```



```
}  
}  
}
```

**Answer**

Exception caught

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Exception\_Week 7\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Ashok wants to determine if a given integer is an Armstrong number. He requires your help in implementing a multi-catch block to handle potential issues during user input or calculation. An Armstrong number is a number that is equal to the sum of its digits, each raised to the power of the number of digits.

For example, if you take the digits of 153 individually and cube them:  $(1 \times 1 \times 1) + (5 \times 5 \times 5) + (3 \times 3 \times 3)$ , then add the results, you get 153, which is an Armstrong number

Create a method isArmstrongNumber that takes an integer input and returns true if it is an Armstrong number, and false otherwise. Throw an IllegalArgumentException if the input is negative. In the main method, input an integer. Utilize a multi-catch block to handle the following scenarios: If

the input is negative, catch `IllegalArgumentException` and print "Error: Input number must be non-negative." If the input is not a valid integer, catch `InputMismatchException` and print "Error: Input must be a valid integer." Finally, print the result.

### ***Input Format***

The input consists of an integer N.

### ***Output Format***

The output prints one of the following:

If N is an Armstrong number, print "N is Armstrong number."

If it is not an Armstrong number, print "N is not Armstrong number."

If  $N < 0$ , print "Error: Input number must be non-negative"

If N is anything other than an integer value, print "Error: Input must be a valid integer."

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 153

Output: 153 is Armstrong number.

### ***Answer***

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Main {
    public static boolean isArmstrongNumber(int n) {
        if (n < 0) {
            throw new IllegalArgumentException();
        }
        int original = n;
        int temp = n;
        int digits = 0;
        if (temp == 0) {
```

```

        digits = 1;
    } else {
        while (temp > 0) {
            digits++;
            temp /= 10;
        }
    }
    temp = n;
    int sum = 0;
    while (temp > 0) {
        int digit = temp % 10;
        sum += Math.pow(digit, digits);
        temp /= 10;
    }
    return sum == original;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    try {
        int n = sc.nextInt();
        if (isArmstrongNumber(n)) {
            System.out.println(n + " is Armstrong number.");
        } else {
            System.out.println(n + " is not Armstrong number.");
        }
    } catch (IllegalArgumentException | InputMismatchException e) {
        if (e instanceof IllegalArgumentException) {
            System.out.println("Error: Input number must be non-negative");
        } else {
            System.out.println("Error: Input must be a valid integer.");
        }
    } finally {
        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Alice is working on a program to calculate the factorial of a non-negative integer. However, she needs to implement error handling to address scenarios where the user inputs a negative number.

The program takes user input for a non-negative integer and calculates its factorial using a recursive method. The code includes exception handling to address potential issues: If the input is negative, an `IllegalArgumentException` is thrown with a corresponding error message, and If the input is not a valid integer, an `InputMismatchException` is caught. The program then prints the calculated factorial.

Assist Alice in this task.

### ***Input Format***

The input consists of an integer N.

### ***Output Format***

The output displays the result of the factorial calculation or an error message if applicable.

1. If the user enters a negative integer, an `IllegalArgumentException` is thrown, and the corresponding error message is: "Error: Input must be non-negative"
2. If the user enters a value that is not a valid integer, an `InputMismatchException` is caught, and the error message is: "Error: Input must be a valid integer."

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5

Output: 120

### ***Answer***

```
import java.util.InputMismatchException;  
import java.util.Scanner;  
public class Main {
```

```

public static long factorial(int n) {
    if (n < 0) {
        throw new IllegalArgumentException();
    }
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    try {
        int n = sc.nextInt();
        long result = factorial(n);
        System.out.print(result);
    } catch (IllegalArgumentException e) {
        System.out.print("Error: Input must be non-negative");
    } catch (InputMismatchException e) {
        System.out.print("Error: Input must be a valid integer.");
    } finally {
        sc.close();
    }
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

A school is conducting a digital math workshop where students are required to input a list of their test scores to calculate the average. The system is designed to accept only positive integer scores. However, students might accidentally enter negative numbers, non-integer characters, or misspelled words while typing their inputs.

To ensure accurate results, the system must validate the input, ignore invalid values with a warning, and calculate the average of all valid positive integers entered.

The student is prompted to enter positive integers separated by spaces. If

the number is negative, throw an `IllegalArgumentException` with the message "Negative numbers are not allowed." If a non-integer value is encountered, catch the `NumberFormatException` and print a warning message: "Warning: Ignoring non-integer value 'value'." If the number is positive, add it to the sum and increment the count. Calculate the average as  $\text{sum}/\text{count}$  and print it with two decimal places.

### ***Input Format***

The input consists of a string with N numeric values, representing the positive integers separated by a space.

### ***Output Format***

The output prints a double value representing the average of the entered integers, rounded off to two decimal places.

If negative integers are entered, it will print: "Error: Negative numbers are not allowed."

If non-integer values are encountered (e.g., letters or symbols), it will print a warning message: "Warning: Ignoring non-integer value 'value'" and print the average following the warning message in a newline.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 10 5 4 8 14

Output: 8.20

### ***Answer***

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            String line = sc.nextLine();
            String[] arr = line.split(" ");
            int sum = 0;
            int count = 0;
```

```

for (String s : arr) {
    try {
        int val = Integer.parseInt(s);

        if (val < 0) {
            throw new IllegalArgumentException();
        }
        sum += val;
        count++;
    } catch (NumberFormatException e) {
        System.out.println("Warning: Ignoring non-integer value '" + s + "'");
    }
}
double avg = (count == 0) ? 0.0 : (double) sum / count;
System.out.printf("%.2f", avg);
} catch (IllegalArgumentException e) {
    System.out.print("Error: Negative numbers are not allowed.");
} finally {
    sc.close();
}
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Aarav is developing a fitness tracking system for a wellness center. The system records three inputs from the user in this order:

The participant's weight (expected to be a valid positive double  $\leq 300$ ), The height in meters (expected to be a valid positive double  $\leq 2.5$ ), The workout duration in minutes (expected to be a valid integer between 1 and 300).

Each field might throw an error:

The weight may be non-numeric or out of range. The height may be non-numeric or out of range. The workout duration may be non-integer or out of range.

Aarav wants to validate each field using a separate try-catch block. If a



field is valid, print its value; otherwise, print a specific error message. Even if one field throws an exception, the validation for the other fields must continue.

If both weight and height are valid, the program must also calculate and print the Body Mass Index (BMI) using the formula:

$$\text{BMI} = \text{weight} / (\text{height} * \text{height})$$

The BMI must be printed rounded to 2 decimal places.

### ***Input Format***

The first line of input contains a double representing the participant's weight in kilograms.

The second line of input contains a double representing the participant's height in meters.

The third line of input contains an integer representing the workout duration in minutes.

### ***Output Format***

The output prints three lines:

Each line corresponds to one input field validation:

1. For weight, print either Weight: <weight> if valid, or Weight: Error - Invalid weight value
2. For height, print either Height: <height> if valid, or Height: Error - Invalid height value
3. For workout duration, print either Workout Duration: <minutes> if valid, or Workout Duration: Error - Invalid workout duration

If both weight and height are valid, print an additional line: BMI: <value> (rounded to 2 decimal places)

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 68.54

1.75

60

Output: Weight: 68.54

Height: 1.75

Workout Duration: 60

BMI: 22.38

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean weightValid = false;
        boolean heightValid = false;
        double weight = 0;
        double height = 0;
        try {
            String w = sc.nextLine();
            weight = Double.parseDouble(w);

            if (weight <= 0 || weight > 300) {
                System.out.println("Weight: Error - Invalid weight value");
            } else {
                System.out.println("Weight: " + weight);
                weightValid = true;
            }
        } catch (Exception e) {
            System.out.println("Weight: Error - Invalid weight value");
        }
        try {
            String h = sc.nextLine();
            height = Double.parseDouble(h);

            if (height <= 0 || height > 2.5) {
                System.out.println("Height: Error - Invalid height value");
            } else {
                System.out.println("Height: " + height);
                heightValid = true;
            }
        }
```

```

    }
} catch (Exception e) {
    System.out.println("Height: Error - Invalid height value");
}
try {
    String d = sc.nextLine();
    int duration = Integer.parseInt(d);
    if (duration < 1 || duration > 300) {
        System.out.println("Workout Duration: Error - Invalid workout duration");
    } else {
        System.out.println("Workout Duration: " + duration);
    }
} catch (Exception e) {
    System.out.println("Workout Duration: Error - Invalid workout duration");
}
if (weightValid && heightValid) {
    double bmi = weight / (height * height);
    System.out.printf("BMI: %.2f", bmi);
}
sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Nithya is creating a secure login validator for her mobile app. The system collects three inputs from the user: username, password, and mobile number. The username must be between 4 and 20 characters. The password should be at least 8 characters long and contain one uppercase letter and one digit. The mobile number must be exactly 10 digits, start with 6–9, and contain only numbers.

Nithya decides to validate each input separately using multi-try and multi-catch blocks. If any rule is broken, an `IllegalArgumentException` is thrown. If the mobile number contains non-digits, a `NumberFormatException` is used. Even when one input is invalid, the program should continue to check and report the others.

### ***Input Format***

The first line of input contains a string representing the username.

The second line of input contains a string representing the password.

The third line of input contains a string representing the mobile number.

### ***Output Format***

The output prints three lines:

For username:

1. Username: <username> if valid
2. Username: Error - Invalid username length (if invalid)

For password:

1. Password: <password> if valid
2. Password: Error - Must be at least 8 chars, include uppercase and digit (if invalid)

For mobile number:

1. Mobile: <mobile> if valid
2. Mobile: Error - Invalid mobile number (if invalid)

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: Alice

Pass1234

9876543210

Output: Username: Alice

Password: Pass1234

Mobile: 9876543210

### ***Answer***

```
import java.util.Scanner;
```

```

public class Main {
    public static boolean isValidPassword(String password) {
        if (password.length() < 8) return false;
        boolean hasUpper = false;
        boolean hasDigit = false;
        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
            if (Character.isDigit(c)) hasDigit = true;
        }
        return hasUpper && hasDigit;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String username = sc.nextLine();
        String password = sc.nextLine();
        String mobile = sc.nextLine();
        try {
            if (username.length() < 4 || username.length() > 20) {
                throw new IllegalArgumentException();
            }
            System.out.println("Username: " + username);
        } catch (IllegalArgumentException e) {
            System.out.println("Username: Error - Invalid username length");
        }
        try {
            if (!isValidPassword(password)) {
                throw new IllegalArgumentException();
            }
            System.out.println("Password: " + password);
        } catch (IllegalArgumentException e) {
            System.out.println("Password: Error - Must be at least 8 chars, include
uppercase and digit");
        }
        try {
            if (mobile.length() != 10) {
                throw new IllegalArgumentException();
            }
            if (mobile.charAt(0) < '6' || mobile.charAt(0) > '9') {
                throw new IllegalArgumentException();
            }
            for (char c : mobile.toCharArray()) {
                if (!Character.isDigit(c)) {

```

```
        throw new NumberFormatException();
    }
}
System.out.println("Mobile: " + mobile);
} catch (NumberFormatException e) {
    System.out.println("Mobile: Error - Invalid mobile number");
} catch (IllegalArgumentException e) {
    System.out.println("Mobile: Error - Invalid mobile number");
}
}
sc.close();
}
}
```

**Status :** Correct

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Exception\_Week 7\_Yourself

Attempt : 1

Total Mark : 50

Marks Obtained : 47.5

### Section 1 : Coding

#### 1. Problem Statement

Alice is tasked with creating a program that validates user input for time values (hours, minutes, and seconds). The program should check whether the input values are within the valid range. If any value falls outside the valid range, a custom exception, `InvalidHourException`, `InvalidMinuteException`, or `InvalidSecondException`, should be thrown, indicating which part of the time (hours, minutes, or seconds) is invalid.

If no exceptions occur, print "Correct Time - " followed by the time in the format "hours:minutes:seconds."

#### ***Input Format***

The first line of input consists of a positive integer, H, representing the hours.

The second line of input consists of a positive integer, M, representing the minutes.

The third line of input consists of a positive integer, S, representing the seconds.

### ***Output Format***

If the input is valid, the output prints "Correct Time - HH:MM:SS" where H, M, and S are hours, minutes, and seconds, respectively.

If the hour is invalid, the output prints "Exception occurred: InvalidHourException - hour is greater than 24".

If the minute is invalid, the output prints "Exception occurred: InvalidMinuteException - minute is greater than 60".

If the second is invalid, the output prints "Exception occurred: InvalidSecondsException - second is greater than 60".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 12

30

45

Output: Correct Time - 12:30:45

### ***Answer***

```
import java.util.Scanner;
class InvalidHourException extends Exception {
    public InvalidHourException(String msg) {
        super(msg);
    }
}
class InvalidMinuteException extends Exception {
    public InvalidMinuteException(String msg) {
        super(msg);
    }
}
class InvalidSecondException extends Exception {
```



```

    public InvalidSecondException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void validateTime(int h, int m, int s)
        throws InvalidHourException, InvalidMinuteException,
        InvalidSecondException {
        if (h > 24) {
            throw new InvalidHourException("hour is greater than 24");
        }
        if (m > 60) {
            throw new InvalidMinuteException("minute is greater than 60");
        }
        if (s > 60) {
            throw new InvalidSecondException("second is greater than 60");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            int h = sc.nextInt();
            int m = sc.nextInt();
            int s = sc.nextInt();
            validateTime(h, m, s);
            System.out.println("Correct Time - " + h + ":" + m + ":" + s);
        } catch (InvalidHourException e) {
            System.out.print("Exception occurred: InvalidHourException - " +
            e.getMessage());
        } catch (InvalidMinuteException e) {
            System.out.print("Exception occurred: InvalidMinuteException - " +
            e.getMessage());
        } catch (InvalidSecondException e) {
            System.out.print("Exception occurred: InvalidSecondsException - " +
            e.getMessage());
        }
        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Helen is an adventurous explorer on a quest to discover the hidden treasures marked on an ancient enchanted map. To unlock the map's secrets, you need to input two pairs of coordinates (x1, y1) and (x2, y2) as directed by the mystical map.

The enchanted map is guarded by the "Guardian of Arguments," a magical guardian who enforces the map's rules. The correct coordinates reveal the sum of squares leading to the treasure's location. Incorrect inputs warn adventurers.

Prompt the user to enter four integer coordinates separated by spaces in a single line. Split the input string into an array of arguments and check if exactly four coordinates are provided. If not, throw a custom exception named CheckArgument with the message "Guardian of Arguments awakens!" If four coordinates are provided, parse them as integers, calculate the sum of the squares of these coordinates, and display the sum of squares as the treasure's secret location. If an incorrect number of coordinates is provided or if there's an issue with parsing, catch the CheckArgument exception, and display "Be aware!" followed by the exception message. Use a finally block to close any resources used.

### ***Input Format***

The input consists of 4 space-separated integers representing the coordinates (x1, y1) and (x2, y2).

### ***Output Format***

If the input is valid, the output prints "Treasure's Secret Location: " followed by an integer representing the location of the treasure.

Otherwise, the output prints "Be aware!"

Guardian of Arguments awakens!" each in a new line.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1 -2 -3 4

Output: Treasure's Secret Location: 30

**Answer**

```
import java.util.Scanner;
class CheckArgument extends Exception {
    public CheckArgument(String msg) {
        super(msg);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            String line = sc.nextLine();
            String[] arr = line.trim().split("\\s+");
            if (arr.length != 4) {
                throw new CheckArgument("Guardian of Arguments awakens!");
            }
            int sum = 0;

            for (int i = 0; i < 4; i++) {
                int val = Integer.parseInt(arr[i]);
                sum += val * val;
            }
            System.out.print("Treasure's Secret Location: " + sum);

        } catch (CheckArgument e) {
            System.out.print("Be aware!\n" + e.getMessage());
        } catch (Exception e) {
            System.out.print("Be aware!\nGuardian of Arguments awakens!");
        } finally {
            sc.close();
        }
    }
}
```

**Status : Correct**

**Marks : 10/10**

### 3. Problem Statement

Sara is developing a program called WordReverser that reverses words. However, there are specific conditions to be met for the reversal to occur.

The program should take a word as input from the user. If the input word contains any numeric digits, throw an UnsupportedOperationException with the message "Cannot reverse a word containing numbers." If the input word is an empty string or a single-letter word, throw an UnsupportedOperationException with the message "Cannot reverse an empty string or a single-letter word." Reverse the valid input word using a StringBuilder. Display the reversed word.

Assist Sara in this task.

#### ***Input Format***

The input consists of a string S, representing the user input.

#### ***Output Format***

The output displays the string representing the reversed word if the input meets the conditions for reversal.

If the word contains numbers, the output displays "Error: Cannot reverse a word containing numbers."

If the word is a single letter, then the output displays "Error: Cannot reverse an empty string or a single-letter word."

Refer to the sample output for formatting specifications.

#### ***Sample Test Case***

Input: Hello!!

Output: !!olleH

#### ***Answer***

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
try {
    String s = sc.nextLine();
    if (s.length() <= 1) {
        throw new UnsupportedOperationException(
            "Cannot reverse an empty string or a single-letter word.");
    }
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            throw new UnsupportedOperationException(
                "Cannot reverse a word containing numbers.");
        }
    }
    StringBuilder sb = new StringBuilder(s);
    System.out.print(sb.reverse().toString());
} catch (UnsupportedOperationException e) {
    if (e.getMessage().equals("Cannot reverse a word containing numbers."))
    {
        System.out.print("Error: Cannot reverse a word containing numbers.");
    } else {
        System.out.print("Error: Cannot reverse an empty string or a single-
letter word.");
    }
} finally {
    sc.close();
}
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Ravi is developing a system to process loan applications. The system takes three inputs from the user in this order:

The applicant's credit score (expected to be a valid integer between 300 and 850), The applicant's loan type (should be either "personal", "home", or "auto"), The loan amount requested (expected to be a valid positive double).

Each input might throw an error:

The credit score may be non-integer or out of the specified range. The loan type might be null or invalid (not one of the allowed types). The loan amount might be non-numeric or non-positive.

Ravi wants to validate each field using a separate try-catch block. If a field is valid, print its value; otherwise, print a specific error message. Even if one field throws an exception, the validation for the other fields must continue.

After successful validation, calculate and print the estimated monthly payment assuming a fixed annual interest rate of 5% and loan term of 5 years, using the formula:

$P$  is the loan amount,  $r = 0.004167$  (monthly interest rate),  $n = 60$  (number of months).

If the loan amount input is invalid, skip this calculation.

### ***Input Format***

The first line of input contains an integer representing the applicant's credit score.

The second line contains a string representing the loan type.

The third line contains a double value representing the loan amount requested.

### ***Output Format***

The output Print three lines:

For credit score, print either

1. Credit Score: <score> (int) if valid, or
2. Credit Score: Error - Credit score must be between 300 and 850

For loan type, print either

1. Loan Type: <loan\_type> if valid, or
2. Loan Type: Error - Invalid loan type

For loan amount, print either

1. Loan Amount: <amount> (double) if valid, or
2. Loan Amount: Error - Invalid loan amount

If loan amount is valid, print the monthly payment calculated (rounded to 2 decimal places) as

Monthly Payment: <payment>

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 720

home

2500.9

Output: Credit Score: 720

Loan Type: home

Loan Amount: 2500.9

Monthly Payment: 47.20

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Integer creditScore = null;
        String loanType = null;
        Double loanAmount = null;
        boolean amountValid = false;
        try {
            String cs = sc.nextLine();
            creditScore = Integer.parseInt(cs);
            if (creditScore < 300 || creditScore > 850) {
                throw new Exception();
            }
        }
    }
}
```

```

    }
    System.out.println("Credit Score: " + creditScore);
} catch (Exception e) {
    System.out.println("Credit Score: Error - Credit score must be between
300 and 850");
}
try {
    loanType = sc.nextLine();
    if (loanType == null ||
        !(loanType.equals("personal") ||
          loanType.equals("home") ||
          loanType.equals("auto"))) {
        throw new Exception();
    }
    System.out.println("Loan Type: " + loanType);
} catch (Exception e) {
    System.out.println("Loan Type: Error - Invalid loan type");
}
try {
    String la = sc.nextLine();
    loanAmount = Double.parseDouble(la);
    if (loanAmount <= 0) {
        throw new Exception();
    }
    System.out.println("Loan Amount: " + loanAmount);
    amountValid = true;
} catch (Exception e) {
    System.out.println("Loan Amount: Error - Invalid loan amount");
}
if (amountValid) {
    double r = 0.004167;
    int n = 60;
    double p = loanAmount;
    double payment = (p * r * Math.pow(1 + r, n)) / (Math.pow(1 + r, n) - 1);

    System.out.printf("Monthly Payment: %.2f", payment);
}
sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10



## 5. Problem Statement

Ravi is developing a simple calculator app that takes three inputs from the user: two integers and an operator. The operator can be '+', '-', '\*', or '/'.

Ravi wants to validate each input separately using multi-try and multi-catch blocks:

The first and second inputs must be valid integers. If not, a `NumberFormatException` should be thrown. The operator must be one of the allowed characters. If not, an `IllegalArgumentException` should be thrown. If the operator is division ('/') and the second integer is zero, an `ArithmeticException` should be thrown.

Even if one input is invalid, Ravi wants the app to continue validating the others and print appropriate messages for each.

Write a program that reads the inputs (3 lines), performs validation, and prints the outcome for each input.

### ***Input Format***

The first line of input contains the first integer as a string.

The second line of input contains the second integer as a string.

The third line of input contains the operator character as a string.

### ***Output Format***

The output prints three lines:

For the first integer: print "First number: <value>" if valid, otherwise "First number: Error - Invalid integer"

For the second integer: print "Second number: <value>" if valid, otherwise "Second number: Error - Invalid integer"

For the operator: print "Operator: <operator>" if valid, otherwise print the corresponding error:

1. "Operator: Error - Invalid operator" if operator is invalid
2. "Operator: Error - Division by zero" if operator is '/' and second integer is zero

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 10

20

+

Output: First number: 10

Second number: 20

Operator: +

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Integer a = null;
        Integer b = null;
        String op = null;
        try {
            a = Integer.parseInt(sc.nextLine());
            System.out.println("First number: " + a);
        } catch (NumberFormatException e) {
            System.out.println("First number: Error - Invalid integer");
        }

        // Second number
        try {
            b = Integer.parseInt(sc.nextLine());
            System.out.println("Second number: " + b);
        } catch (NumberFormatException e) {
            System.out.println("Second number: Error - Invalid integer");
        }

        // Operator
        try {
            op = sc.nextLine();

            if (!(op.equals("+") || op.equals("-") || op.equals("*") || op.equals("/"))) {
```

```
        throw new IllegalArgumentException();
    }

    if (op.equals("/") && b != null && b == 0) {
        throw new ArithmeticException();
    }

    System.out.println("Operator: " + op);

    } catch (ArithmeticException e) {
        System.out.println("Operator: Error - Division by zero");
    } catch (IllegalArgumentException e) {
        System.out.println("Operator: Error - Invalid operator");
    }
    }

    sc.close();
}
}
```

**Status :** Partially correct

**Marks :** 7.5/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_Exception\_Week 7\_Home

Attempt : 1

Total Mark : 40

Marks Obtained : 37.5

### Section 1 : Coding

#### 1. Problem Statement

Buck is developing a program to validate phone numbers. He wants to ensure that the phone numbers entered by users are in the correct format. Buck's program should handle various cases, including situations where the input contains non-numeric characters or the phone number is not exactly 10 digits.

Check if the input contains only numeric digits. If the input is non-numeric, throw an `InputMismatchException` with a descriptive error message "Error: Input mismatch. Please enter a valid 10-digit phone number." Check if the length of the phone number is exactly 10 digits. If not, throw an `IllegalArgumentException` with a message like "Error: Invalid phone number: Must be a 10-digit number." If the input passes both validations, print "Valid" to indicate that the phone number is valid. Use the finally block to close the Scanner to prevent resource leaks.

### ***Input Format***

The input consists of a string value S, representing the phone number.

### ***Output Format***

If the input is a valid 10-digit phone number, the output displays "Valid"

If the input contains non-numeric characters, the output displays "Error: Input mismatch. Please enter a valid 10-digit phone number."

If the input is not exactly 10 digits long, the output displays "Error: Invalid phone number: Must be a 10-digit number."

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 7456984562

Output: Valid

### ***Answer***

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            String s = sc.nextLine();
            for (char c : s.toCharArray()) {
                if (!Character.isDigit(c)) {
                    throw new InputMismatchException();
                }
            }
            if (s.length() != 10) {
                throw new IllegalArgumentException();
            }

            System.out.print("Valid");
        } catch (InputMismatchException e) {
```

```
        System.out.print("Error: Input mismatch. Please enter a valid 10-digit  
phone number.");  
    } catch (IllegalArgumentException e) {  
        System.out.print("Error: Invalid phone number: Must be a 10-digit  
number.");  
    } finally {  
        sc.close();  
    }  
}  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Tim is tasked with writing a program that takes an integer input from the user and processes it. If the user enters a negative number, the program should throw a custom exception called `NegativeNumberException` and provide an appropriate error message. Otherwise, it should double the value of the positive number and display the result.

Use a try-catch block to handle the `NegativeNumberException`. If the exception is caught, print an error message that includes the exception message.

If no exception occurs, print "Value is Doubled: " followed by the processed result.

### ***Input Format***

The input consists of an integer, N.

### ***Output Format***

If the input is positive, the output prints "Value is Doubled: " followed by an integer representing the doubled value.

Otherwise, the output prints "Exception occurred: X is negative" where X is an entered integer.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 7

Output: Value is Doubled: 14

### **Answer**

```
import java.util.Scanner;
class NegativeNumberException extends Exception {
    public NegativeNumberException(String msg) {
        super(msg);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            int n = sc.nextInt();
            if (n < 0) {
                throw new NegativeNumberException(n + " is negative");
            }
            System.out.print("Value is Doubled: " + (n * 2));
        } catch (NegativeNumberException e) {
            System.out.print("Exception occurred: " + e.getMessage());
        }
        sc.close();
    }
}
```

**Status :** Correct

**Marks :** 10/10

### **3. Problem Statement**

Sara is assigned to develop a program for basic arithmetic operations. The program should accept two numeric inputs and an operation symbol (+, -, \*, /) to perform the corresponding operation.

Utilize a multi-catch block to address the following scenarios:

If the numeric inputs are not valid doubles, catch `NumberFormatException` and print "Invalid number format: " followed by the built-in exception message. If there is an attempt to divide by zero, catch `ArithmeticException` and print "Arithmetic exception: Cannot divide by zero.". If the operation symbol is not a valid character, catch `StringIndexOutOfBoundsException` and print "Invalid operation input: Invalid operation: [symbol]". Perform the specified operation and print the result with one decimal place.

Help Sara complete the program.

### ***Input Format***

The first two lines of the input consist of two double values, representing the numeric inputs.

The third line of input consists of the operation symbol.

### ***Output Format***

If the input is valid, the output prints "Result: " followed by a double value formatted to one decimal place, representing the result of the operation.

If the input is a string, the output prints "Invalid number format: For input string: " followed by the entered string.

If the user tries to divide the double value by 0, the output prints "Arithmetic exception: Cannot divide by zero."

If the operation is invalid, the output prints "Invalid operation input: Invalid operation: " followed by the operation.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5.3  
2.6  
+

Output: Result: 7.9

### ***Answer***



```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            String aStr = sc.nextLine();
```

```
            String bStr = sc.nextLine();
```

```
            String op = sc.nextLine();
```

```
            double a = Double.parseDouble(aStr);
```

```
            double b = Double.parseDouble(bStr);
```

```
            try {
```

```
                if (op.length() != 1) {
```

```
                    throw new StringIndexOutOfBoundsException();
```

```
                }
```

```
                char c = op.charAt(0);
```

```
                double result;
```

```
                if (c == '+') {
```

```
                    result = a + b;
```

```
                } else if (c == '-') {
```

```
                    result = a - b;
```

```
                } else if (c == '*') {
```

```
                    result = a * b;
```

```
                } else if (c == '/') {
```

```
                    if (b == 0) {
```

```
                        throw new ArithmeticException();
```

```
                    }
```

```
                    result = a / b;
```

```
                } else {
```

```
                    throw new StringIndexOutOfBoundsException();
```

```

    }

    System.out.printf("Result: %.1f", result);

    } catch (ArithmeticException e) {
        System.out.print("Arithmetic exception: Cannot divide by zero.");
    } catch (StringIndexOutOfBoundsException e) {
        System.out.print("Invalid operation input: Invalid operation: " + op);
    }

    } catch (NumberFormatException e) {
        System.out.print("Invalid number format: " + e.getMessage());
    } finally {
        sc.close();
    }
}
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Lavanya is building a train ticket booking assistant. The system asks the user to enter two inputs:

Train Number – which must be a 5-digit number. Journey Date – which must be in the format dd/MM/yyyy.

Lavanya wants to validate each field separately using multi-try and multi-catch blocks.

If the train number is not a valid 5-digit integer, a `NumberFormatException` should be thrown. If the date is not in the correct format, an `IllegalArgumentException` should be thrown.

Even if one input is invalid, the other should still be validated and reported accordingly.

Write a program that reads two lines of input and validates both using try-catch blocks.

### ***Input Format***

The first line of input contains a string representing the train number.

The second line of input contains a string representing the journey date in dd/MM/yyyy format.

### ***Output Format***

The output prints two lines:

For train number:

1. Print "Train Number: <value>" if valid.
2. Else, print "Train Number: Error - Invalid train number".

For journey date:

1. Print "Journey Date: <value>" if valid.
2. Else, print "Journey Date: Error - Invalid date format".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 12345

25/12/2025

Output: Train Number: 12345

Journey Date: 25/12/2025

### ***Answer***

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        // ----- Train Number Validation -----
```

```
        try {
```

```

String trainStr = sc.nextLine().trim();
int train = Integer.parseInt(trainStr);

if (train < 10000 || train > 99999) {
    throw new NumberFormatException();
}

System.out.println("Train Number: " + train);

} catch (NumberFormatException e) {
    System.out.println("Train Number: Error - Invalid train number");
}

// ----- Journey Date Validation -----
try {
    String date = sc.nextLine().trim();

    // basic format check dd/MM/yyyy
    if (date.length() != 10 || date.charAt(2) != '/' || date.charAt(5) != '/') {
        throw new IllegalArgumentException();
    }

    String dd = date.substring(0, 2);
    String mm = date.substring(3, 5);
    String yy = date.substring(6);

    Integer.parseInt(dd);
    Integer.parseInt(mm);
    Integer.parseInt(yy);

    System.out.println("Journey Date: " + date);

} catch (Exception e) {
    System.out.println("Journey Date: Error - Invalid date format");
}

sc.close();
}
}

```

**Status :** Partially correct

**Marks :** 7.5/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_String\_Week 8\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 17

#### Section 1 : MCQ

1. What will be the output of the following program?

```
class Main {  
    public static void main(String[] args) {  
        String greet = "Welcome\n";  
        System.out.print("String: " + greet);  
        int length = greet.length();  
        System.out.print("Length: " + length);  
    }  
}
```

**Answer**

String: welcomeLength: 8

**Status : Wrong**

**Marks : 0/1**

2. What will be the output of the following code?

```
class Main {  
    public static void main(String [] args) {  
        String s1 = "Work";  
        String s2 = "Work";  
        StringBuilder sb1 = new StringBuilder();  
        sb1.append("Wo").append("rk");  
        System.out.println(s1 == s2);  
        System.out.println(s1.equals(s2));  
        System.out.println(sb1.toString() == s1);  
        System.out.println(sb1.toString().equals(s1));  
    }  
}
```

**Answer**

true true false true

**Status :** Correct

**Marks :** 1/1

3. Predict the output for the following code.

```
public class Main {  
    public static void main(String[] args) {  
        String a = "java";  
        char temp = a.charAt(1);  
        System.out.println(temp);  
    }  
}
```

**Answer**

a

**Status :** Correct

**Marks :** 1/1

4. What will be the output of the following code snippet?

```
public class Main{  
    public static void main(String args[]){
```

```
String build=new StringBuilder("Hi");
build.append("Team");
System.out.print(build);
}
}
```

**Answer**

HiTeam

**Status : Correct**

**Marks : 1/1**

5. What will be the output of the following code?

```
class Main {
    public static void main(String args[]) {
        char c[] = {'j', 'a', 'v', 'a'};
        String s1 = new String(c);
        String s2 = new String(s1);
        System.out.println(s1);
        System.out.println(s2);
    }
}
```

**Answer**

javajava

**Status : Correct**

**Marks : 1/1**

6. What is the expected behavior of the below code?

```
public class Main
{
    public static void main(String[] args) {
        int a = 10.0;
        String temp = Integer.toString(a);
        System.out.println(temp);
    }
}
```

**Answer**

Error: incompatible types: possible lossy conversion from double to int

**Status :** Correct

**Marks :** 1/1

7. What will be the output of the following code?

```
class Main {  
    public static void main(String args[]) {  
        String s1 = "Hello";  
        String s2 = "Hello";  
        String s3 = "Good-bye";  
        String s4 = "HELLO";  
  
        System.out.println(s1 + " equals " + s2 + " -> " +  
            s1.equals(s2));  
        System.out.println(s1 + " equals " + s3 + " -> " +  
            s1.equals(s3));  
        System.out.println(s1 + " equals " + s4 + " -> " +  
            s1.equals(s4));  
        System.out.println(s1 + " equalsIgnoreCase " + s4 + " -> " +  
            s1.equalsIgnoreCase(s4));  
    }  
}
```

**Answer**

Hello equals Hello -> trueHello equals Good-bye -> falseHello equals HELLO  
-> falseHello equalsIgnoreCase HELLO -> true

**Status :** Correct

**Marks :** 1/1

8. What is the output of the following code?

```
class Main  
{  
    public static void main(String args[])  
    {  
        StringBuilder s1 = new StringBuilder("Hello");
```



```
StringBuilder s2 = s1.reverse();  
System.out.println(s2);  
}  
}
```

**Answer**

olleH

**Status :** Correct

**Marks :** 1/1

9. What will be the output of the following program?

```
class Main {  
    public static void main(String args[]) {  
        String name="Work Hard";  
        name.concat("Success");  
        System.out.println(name);  
    }  
}
```

**Answer**

Work Hard

**Status :** Correct

**Marks :** 1/1

10. What will be the output of the following code?

```
public class Main{  
    public static void main(String args[]){  
        StringBuilder obj=new StringBuilder("Programming");  
        obj.delete(1,3);  
        System.out.print(obj);  
    }  
}
```

**Answer**

Pgramming

**Status :** Correct

**Marks :** 1/1

11. Predict the output for the following code.

```
class Main {  
    public static void main(String[] fruits) {  
        String fruit1 = new String("apple");  
        String fruit2 = new String("orange");  
        String fruit3 = new String("pear");  
        fruit3 = fruit1;  
        fruit2 = fruit3;  
        fruit1 = fruit2;  
        System.out.println(fruit1);  
        System.out.println(fruit2);  
        System.out.println(fruit3);  
    }  
}
```

**Answer**

appleorangepear

**Status :** Wrong

**Marks :** 0/1

12. What will be the output of the following program?

```
class Main {  
    public static void main(String[] args) {  
        String s = new String("5");  
        System.out.println(1 + 1111 + s + 1 + 1010);  
    }  
}
```

**Answer**

1112511010

**Status :** Correct

**Marks :** 1/1

13. What will be the output of the following code?

```
class Main {  
    public static void main(String args[]) {
```

```
String c = "Hello i love java";  
boolean var;  
var = c.startsWith("hello");  
System.out.println(var);  
}  
}
```

**Answer**

false

**Status :** Correct

**Marks :** 1/1

14. What will be the output of the following code?

```
class Main {  
    public static void main(String args[]) {  
        String s = "This is a demo of the getChars method.";  
        int start = 10;  
        int end = 14;  
        char buf[] = new char[end - start];  
        s.getChars(start, end, buf, 0);  
        System.out.println(buf);  
    }  
}
```

**Answer**

demo

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following code?

```
public class Main{  
    public static void main(String args[]){  
        StringBuilder sb=new StringBuilder("Stock");  
        sb.replace(1,3,"Market");  
        System.out.print(sb);  
    }  
}
```

**Answer**

SMarketck

**Status :** Correct

**Marks :** 1/1

16. Predict the output for the following code:

```
public class Main {  
    public static void main(String[] args) {  
        float a = 10.0f;  
        String temp = Float.toString(a);  
        System.out.println(temp);  
    }  
}
```

**Answer**

10.0

**Status :** Correct

**Marks :** 1/1

17. What will be the output of the following program?

```
class Main {  
    public static void main(String[] args) {  
        String s1 = "EDUCATION";  
        String s2 = new String("EDUCATION");  
        String s3 = "EDUCATION";  
        if (s1 == s2) {  
            System.out.println("s1 and s2 equal");  
        }  
        else {  
            System.out.println("s1 and s2 not equal");  
        }  
        if (s1 == s3) {  
            System.out.println("s1 and s3 equal");  
        }  
        else {  
            System.out.println("s1 and s3 not equal");  
        }  
    }  
}
```

**Answer**

s1 and s2 not equals1 and s3 equal

**Status :** Correct

**Marks :** 1/1

18. What will be the output of the following code?

```
class Main {  
    public static void main(String args[]) {  
        String s1 = "Hello i love java";  
        String s2 = new String(s1);  
        System.out.println((s1 == s2) + " " + s1.equals(s2));  
    }  
}
```

**Answer**

false true

**Status :** Correct

**Marks :** 1/1

19. What will be the output for the following code?

```
class Main {  
    public static void main(String[] args) {  
        String languages[] = { "C", "C++", "Java", "Python", "Ruby"};  
        for (String sample: languages) {  
            System.out.println(sample);  
        }  
    }  
}
```

**Answer**

CC++JavaPythonRuby

**Status :** Correct

**Marks :** 1/1

20. Predict the output for the following code:

```
class Main {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer("I Java!");  
        sb.insert(5, "like ");  
        System.out.println(sb);  
    }  
}
```

**Answer**

I Java like!

**Status :** Wrong

**Marks :** 0/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_String\_Week 8\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Kunal is developing a software tool that requires him to efficiently determine the length of the longest substring without repeating characters from a given string. This capability is crucial for optimizing search functionality within large text data. The program reads a string and calculates the length of the longest substring that contains unique characters.

Help Kunal to accomplish his task.

#### ***Input Format***

The input consists of a string, representing the text from which the longest substring without repeating characters needs to be extracted.

#### ***Output Format***

The output prints an integer representing the length of the longest substring without repeating characters.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: abcabcb

Output: 3

### **Answer**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        Set<Character> set = new HashSet<>();
        int maxLength = 0;
        int left = 0;
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }
        System.out.print(maxLength);
    }
}
```

**Status :** Correct

**Marks : 10/10**

## **2. Problem Statement**

Consider a scenario where you are tasked with organizing the different possible subsequences of a word into sorted order. You are given a string, and your goal is to find and display all the contiguous subsequences



(subarrays) of that string.

For instance, if the input string is "fun", the output should list all possible contiguous subsequences such as f, fu, fun, n, u, un

The total number of subarrays for a string of length `n` can be calculated using the formula  $n(n+1)/2$ .

Write a program that generates all subarrays of the given string and prints them in alphabetically sorted order.

### ***Input Format***

The first line of input contains a string representing the input string

### ***Output Format***

The program should print all contiguous subarrays (substrings) of the input string in alphabetical order. Each substring should be printed on a separate line. The substrings must include all possible contiguous combinations, starting from the smallest (1 character) up to the full length of the string.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: examly

Output: a

am

aml

amly

e

ex

exa

exam

examl

examly

l

ly

m

ml  
mly  
x  
xa  
xam  
xaml  
xamly  
y

**Answer**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        List<String> substrings = new ArrayList<>();
        int n = str.length();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j <= n; j++) {
                substrings.add(str.substring(i, j));
            }
        }
        Collections.sort(substrings);
        for (String s : substrings) {
            System.out.println(s);
        }
    }
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Gowri needs to write a paragraph about modern history. While writing, she noticed that some words were repeated again and again. She started counting the number of times a particular word was repeated.

Write a program to get a string from the user, and count the number of times a word is present in the string.

### **Input Format**

The first line of input consists of a string, separated by a space.

The second line consists of a single word that needs to be counted in the string.

### **Output Format**

The output displays an integer representing the number of times the given word is present in the string.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: I felt happy because I saw the others were happy and because I knew I  
should feel happy  
happy

Output: 3

### **Answer**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sentence = sc.nextLine();
        String target = sc.nextLine();
        String[] words = sentence.split("[^a-zA-Z]+");
        int count = 0;
        for (String word : words) {
            if (word.equals(target)) {
                count++;
            }
        }
        System.out.print(count);
    }
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Ben is a passionate text enthusiast who loves making text more readable and cleaner. He's organized a "Text Cleanup Challenge" for his friends.

Participants are tasked with writing a program to clean up text by removing extra white spaces, both at the beginning and end, and within sentences or words.

Note :

For Ben's Text Cleanup Challenge, participants need to use StringBuffer to develop a program that removes excess whitespace from the text.

##### ***Input Format***

The input consists of a string representing a text string with potential white space issues.

##### ***Output Format***

The output prints the cleaned text with extra white spaces removed.

Refer to the sample output for the formatting specifications.

##### ***Sample Test Case***

Input: This is a test.

Output: This is a test.

##### ***Answer***

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();
        StringBuffer result = new StringBuffer();
        boolean spaceFound = false;
        input = input.trim();
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
```

```

        if (ch == ' ') {
            if (!spaceFound) {
                result.append(ch);
                spaceFound = true;
            }
        } else {
            result.append(ch);
            spaceFound = false;
        }
    }
    System.out.print(result.toString());
}
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Sarah wants to develop a tool that receives two strings as input, removes spaces, converts them to lowercase, and determines if they are anagrams. Display "Anagrams" if they are, and "Not Anagrams" if they're not.

Ensure the program handles case insensitivity and ignores spaces in both the strings.

### **Input Format**

The first line of input consists of a string, representing the first string.

The second line of input consists of a string, representing the second string.

### **Output Format**

The output prints "Anagrams" if the strings are anagrams.

The output prints "Not Anagrams" if the strings are not anagrams.

Refer to the sample output for the formatting specifications.

### Sample Test Case

Input: OpenAI

iApNeo

Output: Anagrams

### Answer

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str1 = sc.nextLine();
        String str2 = sc.nextLine();
        str1 = str1.replaceAll(" ", "").toLowerCase();
        str2 = str2.replaceAll(" ", "").toLowerCase();
        char[] arr1 = str1.toCharArray();
        char[] arr2 = str2.toCharArray();
        Arrays.sort(arr1);
        Arrays.sort(arr2);
        if (Arrays.equals(arr1, arr2)) {
            System.out.print("Anagrams");
        } else {
            System.out.print("Not Anagrams");
        }
    }
}
```

Status : Correct

Marks : 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_String\_Week 8\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Ashok is given a string consisting of some words separated by spaces. He wants to return the length of the last word in the string. The word is a maximal substring consisting of non-space characters only.

Help Ashok to complete the program.

#### ***Input Format***

The input consists of a string.

#### ***Output Format***

The output prints an integer representing the length of the last word in the string.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: Hello World

Output: 5

### **Answer**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        str = str.trim();
        int length = 0;
        for (int i = str.length() - 1; i >= 0; i--) {
            if (str.charAt(i) == ' ') {
                break;
            }
            length++;
        }
        System.out.print(length);
    }
}
```

**Status :** Correct

**Marks :** 10/10

## **2. Problem Statement**

Given a string *s* that consists of lowercase letters, return the length of the longest palindrome that can be built with those letters. Letters are case-sensitive. For example, "Aa" is not considered a palindrome here.

Input: *s* = "abccccdd"

Output: 7

Explanation: One longest palindrome that can be built is "dccaccd", whose length is 7.

### **Input Format**



The input consists of a string.

### **Output Format**

The output prints the length of the longest palindrome that can be built with the given letters.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: abccccdd

Output: 7

### **Answer**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int[] freq = new int[128];
        for (char c : s.toCharArray()) {
            freq[c]++;
        }

        int length = 0;
        boolean oddFound = false;
        for (int count : freq) {
            length += (count / 2) * 2;
            if (count % 2 == 1) {
                oddFound = true;
            }
        }
        if (oddFound) {
            length += 1;
        }
        System.out.print(length);
    }
}
```

**Status : Correct**

**Marks : 10/10**

### 3. Problem Statement

Maria, a software developer, is currently working on a program that sorts characters in a string based on their frequency. She wants to create a Java program that takes a string as input and rearranges its characters in ascending order of frequency.

However, she needs your help in refining the program. She wants you to modify the existing code to ensure it follows her specific requirements.

Note :

Use StringBuffer for building the final sorted string

#### ***Input Format***

The input contains a single string S, representing the text to be sorted based on character frequency.

#### ***Output Format***

The output displays a string "Sorted by frequency: [sortedOrder]" where [sortedOrder] is the result of sorting characters by their frequency.

if two characters have same frequency sort them by alphabetical order.

Refer to the sample output for the formatting specifications.

#### ***Sample Test Case***

Input: hello

Output: Sorted by frequency: eholl

#### ***Answer***

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        Map<Character, Integer> map = new HashMap<>();
```

```

    for (char c : str.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }
    List<Character> list = new ArrayList<>(map.keySet());
    Collections.sort(list, new Comparator<Character>() {
        public int compare(Character a, Character b) {
            int freqCompare = map.get(a) - map.get(b);
            if (freqCompare == 0) {
                return a - b;
            }
            return freqCompare;
        }
    });
    StringBuffer result = new StringBuffer();

    for (char c : list) {
        int count = map.get(c);
        for (int i = 0; i < count; i++) {
            result.append(c);
        }
    }

    System.out.print("Sorted by frequency: " + result.toString());
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Nick is tasked with developing a Text Replacement Engine that replaces all occurrences of a specified substring in a given text. The engine should take the original text, the substring to be replaced, and the replacement string as input and output the modified text after performing the replacement.

Help Nick complete the task using the StringBuilder class.

##### **Input Format**

The first line of input contains a string that represents the original string.

The second line of input contains a string that represents the substring to replace.

The third line of input contains a string that represents the replacement string.

### ***Output Format***

The first line of output prints a string representing the original string.

The second line prints a string representing the modified string after replacement.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: Hello, world! Hello, Java!

Hello

Hi

Output: Hello, world! Hello, Java!

Hi, world! Hi, Java!

### ***Answer***

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String original = sc.nextLine();
        String target = sc.nextLine();
        String replacement = sc.nextLine();
        StringBuilder sb = new StringBuilder(original);
        int index = sb.indexOf(target);
        while (index != -1) {
            sb.replace(index, index + target.length(), replacement);
            index = sb.indexOf(target, index + replacement.length());
        }
        System.out.println(original);
        System.out.print(sb.toString());
    }
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_String\_Week 8\_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Robin is fascinated by ancient Roman numerals and wants to create a program to convert decimal numbers to Roman numerals. He needs your help to complete the program.

Roman numerals are represented using the following symbols:

Given an integer, convert it to a Roman numeral.

#### ***Input Format***

The input consists of an integer, n.

#### ***Output Format***

The output prints the Roman numeral of the given integer.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 3

Output: III

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        String[] romans = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
        String result = "";
        for (int i = 0; i < values.length; i++) {
            while (n >= values[i]) {
                result += romans[i];
                n -= values[i];
            }
        }
        System.out.print(result);
    }
}
```

**Status :** Correct

**Marks :** 10/10

## **2. Problem Statement**

Jei loves playing with strings. She is interested in finding out whether a given string is a palindrome or not. A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward.

Write a program to help Jei determine if a given string is a palindrome or not and also print the reversed version of the string.

### **Input Format**

The input consists of a single line containing a string s.

### **Output Format**

The first line of output prints a string representing the reversed string.

The second line of output prints 'palindrome' if the given string is a palindrome otherwise print 'not a palindrome'

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: madam

Output: madam

palindrome

### **Answer**

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        String reversed = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            reversed += s.charAt(i);
        }
        System.out.println(reversed);
        if (s.equalsIgnoreCase(reversed)) {
            System.out.print("palindrome");
        } else {
            System.out.print("not a palindrome");
        }
    }
}
```

**Status :** Correct

**Marks :** 10/10



### 3. Problem Statement:

Sophia is working on a programming project that involves generating all possible permutations of a given string. She needs a program that can efficiently compute these permutations and help her analyze the different arrangements of characters in the string.

To achieve this, she is seeking assistance in creating a program that can produce all the permutations of a given string.

Example:

Sample input:

set

Sample output:

set

ste

est

ets

tes

tse

Explanation:

Permutations are all possible arrangements of characters where the order matters.

The input string is "set" which has 3 unique characters: 's', 'e', and 't'. To find all permutations, we rearrange the characters in every possible order. Since there are 3 characters, the total number of permutations is  $3! = 6$ .

The permutations of "set" are: set, ste, est, ets, tes, tse.

Each permutation is a unique arrangement where the order of characters is different. This is important because "set" and "tes" have the same characters, but they are in different positions — making them different

permutations.

### ***Input Format***

The input consists of the string.

### ***Output Format***

The output displays all possible permutations.

Refer to the sample output for a better understanding.

### ***Sample Test Case***

Input: test

Output: test

tets

tset

tste

ttse

ttes

etst

etts

estt

estt

etst

etts

sett

sett

stet

stte

stte

stet

test

tets

tset

tste

ttse

ttes

### ***Answer***

```

import java.util.Scanner;
public class Main {
    static String swap(String str, int i, int j) {
        char[] ch = str.toCharArray();
        char temp = ch[i];
        ch[i] = ch[j];
        ch[j] = temp;
        return String.valueOf(ch);
    }
    static void permute(String str, int l, int r) {
        if (l == r) {
            System.out.println(str);
            return;
        }
        for (int i = l; i <= r; i++) {
            str = swap(str, l, i);
            permute(str, l + 1, r);
            str = swap(str, l, i); // backtrack
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();
        permute(input, 0, input.length() - 1);
    }
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Lara is working on a text analysis tool and wants to count the occurrences of a specific character within a given string.

Write a program to determine how many times a specified character appears in a provided string. Ensure that the program handles both uppercase and lowercase letters distinctly.

Note: This program uses StringBuffer for efficient string handling while counting character occurrences.

### ***Input Format***

The first line consists of a string representing the text in which you need to count the occurrences of a character.

The second line contains a single character to count the occurrences in the text.

### ***Output Format***

The output prints a single integer, representing the count of occurrences of the specified character in the given text.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: Hello World

o

Output: 2

### ***Answer***

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char ch = sc.nextLine().charAt(0);

        StringBuffer sb = new StringBuffer(text);

        int count = 0;

        for (int i = 0; i < sb.length(); i++) {
            if (sb.charAt(i) == ch) {
                count++;
            }
        }
    }
}
```

```
        System.out.print(count);  
    }  
  
}
```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Sampad is given the project of developing a text utility that reverses a given string. To achieve this, he is required to use the String Builder concept, which provides efficient ways to manipulate strings.

You have to assist Sampad in writing a program that takes user input for a string, use the StringBuilder class to reverse the string, and then display the reversed string.

### ***Input Format***

The input consists of a string. The string may contain alphabets, spaces, and punctuation marks.

### ***Output Format***

The output prints the modified reversed string.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: Hello world!

Output: !dlrow olleH

### ***Answer***

```
import java.util.Scanner;  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
String input = sc.nextLine();
StringBuilder sb = new StringBuilder(input);
sb.reverse();
System.out.print(sb.toString());
    }
}
```

**Status :** Correct

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_FH\_Week 9\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 16

#### Section 1 : MCQ

1. Which class in Java is used to handle file input and output operations?

**Answer**

FileInputStream

**Status : Correct**

**Marks : 1/1**

2. What is the purpose of the File class in Java?

**Answer**

To create directories and manage file metadata

**Status : Correct**

**Marks : 1/1**

3. In Java, what does the BufferedReader class provide?

**Answer**

Methods for reading text from a character-input stream

**Status : Correct**

**Marks : 1/1**

4. What is the primary purpose of using the FileInputStream class in Java?

**Answer**

To read bytes from a file

**Status : Correct**

**Marks : 1/1**

5. What will be the output of the following program?

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            FileOutputStream outputStream = new
FileOutputStream("output.txt");
            String data = "Hello, World!";
            outputStream.write(data.getBytes());
            outputStream.close();
            System.out.println("File created successfully");
        } catch (IOException e) {
            System.out.println("An error occurred");
            e.printStackTrace();
        }
    }
}
```

**Answer**

Compile Time Error

**Status : Wrong**

**Marks : 0/1**



6. What will be the output of the following program?

Consider the local directory as: "F:\\program.txt"

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        File file = new File("nonexistent.txt");
        System.out.println(file.exists());
    }
}
```

**Answer**

false

**Status : Correct**

**Marks : 1/1**

7. Which data type does the length() function return to represent the length of a file in bits?

**Answer**

long

**Status : Correct**

**Marks : 1/1**

8. What will be the output of the following program?

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            BufferedWriter writer = new BufferedWriter(new
            FileWriter("output.txt"));
            writer.write("Hello, World!");
            writer.close();
            System.out.println("Data written to file.");
        } catch (IOException e) {
```

```
        System.out.println("An error occurred.");
        e.printStackTrace();
    }
}
}
```

**Answer**

Compile Time Error

**Status :** Wrong

**Marks :** 0/1

9. Which method is used to create a new file in Java?

**Answer**

createNewFile() method of the File class

**Status :** Correct

**Marks :** 1/1

10. In Java, which method is used to check if a file is a directory?

**Answer**

isDirectory() method of the File class

**Status :** Correct

**Marks :** 1/1

11. Which method can be used to check file accessibility?

**Answer**

All of the mentioned options

**Status :** Correct

**Marks :** 1/1

12. Which method is used to create a directory with file attributes?

**Answer**

Files.createDirectory(path, fileAttributes)

**Status :** Correct

**Marks :** 1/1

13. What do Files.lines(Path path) do?

**Answer**

It reads all the lines from a file as a Stream

**Status :** Correct

**Marks :** 1/1

14. What will be the output for the below code snippet?

```
import java.io.InputStream;
import java.io.FileInputStream;
import java.io.IOException;

class Main {
    public static void main(String args[]) {
        InputStream obj = new FileInputStream("inputoutput.java");
        System.out.print(obj.available());
    }
}
```

**Answer**

Compile Time Error

**Status :** Correct

**Marks :** 1/1

15. What will be the output of the following Java program?

```
import java.io.CharArrayReader;
import java.io.IOException;

class Main {
    public static void main(String[] args) {
        String obj = "abcdefgh";
        int length = obj.length();
        char c[] = new char[length];
        obj.getChars(0, length, c, 0);
    }
}
```

```

CharArrayReader input1 = new CharArrayReader(c);
CharArrayReader input2 = new CharArrayReader(c, 1, 4);
int i;
int j;
try {
    while ((i = input1.read()) == (j = input2.read())) {
        System.out.print((char) i);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

**Answer**

None of the mentioned options

**Status :** Correct

**Marks :** 1/1

16. What will be the output of the following Java program?

```

public class Main {
    public static void main(String[] args) {
        String obj = "abc";
        byte b[] = obj.getBytes();
        ByteArrayInputStream obj1 = new ByteArrayInputStream(b);
        for (int i = 0; i < 2; ++ i)
        {
            int c;
            while((c = obj1.read()) != -1)
            {
                if(i == 0)
                {
                    System.out.print(Character.toUpperCase((char)c));
                    obj2.write(1);
                }
            }
        }
        System.out.print(obj2);
    }
}

```

**Answer**

Compile Time Error

**Status :** Correct

**Marks :** 1/1

17. Which of the following is used to read characters from a file in Java?

**Answer**

FileReader

**Status :** Correct

**Marks :** 1/1

18. What is the output of the following program?

```
import java.io.*;
```

```
public class Main {  
    public static void main(String[] args) throws IOException {  
        FileWriter writer = new FileWriter("file1.txt");  
        writer.write("Hello, World!");  
        writer.close();  
  
        FileReader reader = new FileReader("file1.txt");  
        int character;  
        while ((character = reader.read()) != -1) {  
            System.out.print((char) character);  
        }  
        reader.close();  
    }  
}
```

**Answer**

Hello, World!

**Status :** Correct

**Marks :** 1/1

19. What is the output of the following program?

```
import java.io.*;
```

```
public class Main {  
    public static void main(String[] args) throws IOException {  
        FileWriter writer = new FileWriter("file2.txt");  
        writer.write("This is a test.");  
        writer.close();  
  
        FileReader reader = new FileReader("file2.txt");  
        char[] data = new char[100];  
        reader.read(data);  
        System.out.println(data);  
        reader.close();  
    }  
}
```

**Answer**

[C@1b6d3586

**Status :** Wrong

**Marks :** 0/1

20. What is the output of the following program?

```
import java.io.*;
```

```
public class Main {  
    public static void main(String[] args) throws IOException {  
        FileWriter writer = new FileWriter("file3.txt");  
        writer.write("NeoColab");  
        writer.close();  
  
        FileReader reader = new FileReader("file3.txt");  
        int i;  
        while ((i = reader.read()) != -1) {  
            if (i == 'o') break;  
            System.out.print((char) i);  
        }  
    }  
}
```

```
} reader.close();  
}
```

**Answer**

Neo

**Status :** Wrong

**Marks :** 0/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_FH\_Week 9\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

James, a software developer, is working on a program that processes text input. He needs to create a program that takes a string from the user, writes it to a file named output.txt, and then reads the file to count the number of uppercase and lowercase letters in the string. The program should then print the counts of uppercase and lowercase letters.

#### ***Input Format***

The input consists of a string.

#### ***Output Format***

The first line of output prints an integer representing the number of uppercase letters.

The second line of output prints an integer representing the number of



lowercase letters.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: Welcome to PS!!!

Output: 3

8

### **Answer**

```
import java.io.*;
import java.util.*;

class Sample {
    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);
        String ch;
        int upper = 0, lower = 0, number = 0, special = 0;
        FileWriter fw = new FileWriter("output.txt");

        public static void main(String[] args) throws Exception {

            Scanner sc = new Scanner(System.in);

            String ch = sc.nextLine();

            // write to file
            FileWriter fw = new FileWriter("output.txt");
            fw.write(ch);
            fw.close();

            // read from file
            BufferedReader br = new BufferedReader(new FileReader("output.txt"));
            String line = br.readLine();
            br.close();

            int upper = 0, lower = 0;
```

```

for (int i = 0; i < line.length(); i++) {
    char c = line.charAt(i);

    if (c >= 'A' && c <= 'Z') {
        upper++;
    } else if (c >= 'a' && c <= 'z') {
        lower++;
    }
}

System.out.println(upper);
System.out.println(lower);
}
}

```

**Status : Wrong**

**Marks : 0/10**

## 2. Problem Statement

Sophia is working on a text processing application where she needs to analyze user input to count the number of words, characters (excluding spaces), and special characters (non-alphanumeric characters) from a given sentence. The input is provided as a single line of text, and the results must be stored in an output file output.txt, and displayed on the console.

### **Input Format**

The first line of input consists of a string, representing the sentence entered by the user.

### **Output Format**

The first line of output prints "Word Count: " followed by an integer representing the number of words in the sentence.

The second line prints "Character Count: " followed by an integer representing the number of characters (excluding spaces and includes special characters also).

The third line prints "Special Character Count: " followed by an integer representing the number of special characters in the sentence(excluding space).

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: Hello, this is an easy level example.

Output: Word Count: 7

Character Count: 31

Special Character Count: 2

### **Answer**

```
import java.io.*;

class WordCounter {
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new
        InputStreamReader(System.in));
        FileWriter fw = new FileWriter("output.txt");

        String input = reader.readLine();
        String trimmed = input.trim();
        int wordCount = trimmed.isEmpty() ? 0 : trimmed.split("\\s+").length;
        int charCount = 0;
        int specialCount = 0;
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
            if (ch != ' ') {
                charCount++;
                if (!Character.isLetterOrDigit(ch)) {
                    specialCount++;
                }
            }
        }

        String result = "Word Count: " + wordCount + "\n"
            + "Character Count: " + charCount + "\n"
            + "Special Character Count: " + specialCount;
        System.out.println(result);
        fw.write(result);
        fw.close();
    }
}
```

}

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Ella is managing her expenses and wants to calculate the final prices of her purchases after applying a 10% tax. She keeps track of her item prices in a single file and wants a program that can:

Take a filename as input and store the original item prices in that file. Read the original prices from the file and display them. Apply a 10% tax to each item and display the new taxed prices.

Write a Java program that meets Ella's requirements and ensure that the program uses file-handling techniques to achieve the task efficiently.

**Formula :**

$\text{taxed price} = \text{original price} * 1.10$

#### ***Input Format***

The first line contains a string, representing the filename.

The second line contains an integer N, representing the number of items.

The third line contains N space-separated double values, representing the item prices.

#### ***Output Format***

The first line displays the message: "Reading content from the file:"

The second line prints the original prices in the format: "Original Prices: <price1> <price2> ...", rounded to two decimal places.

The third line prints the taxed prices in the format: "Calculated Taxed Prices: <taxed\_price1> <taxed\_price2> ...", rounded to two decimal places.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: sys.txt

1

140.5

Output: Reading content from the file:

Original Prices: 140.50

Calculated Taxed Prices: 154.55

### **Answer**

```
import java.io.*;
import java.util.Scanner;

class FileManager {

    public static void writeToFile(String fileName, double[] prices) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {
            for (double price : prices) {
                bw.write(price + " ");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static String readFromFile(String fileName) {
        StringBuilder sb = new StringBuilder();

        try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
            String line = br.readLine();
            if (line != null) {
                String[] parts = line.trim().split("\\s+");
                sb.append("Original Prices: ");
                for (String p : parts) {
                    double val = Double.parseDouble(p);
                    sb.append(String.format("%.2f ", val));
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

        return sb.toString();
    }
    public static double[] calculateTax(double[] prices) {
        double[] taxed = new double[prices.length];
        for (int i = 0; i < prices.length; i++) {
            taxed[i] = prices[i] * 1.10;
        }
        return taxed;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String fileName = scanner.nextLine();
        int n = scanner.nextInt();

        double[] prices = new double[n];
        for (int i = 0; i < n; i++) {
            prices[i] = scanner.nextDouble();
        }

        FileManager.writeToFile(fileName, prices);

        System.out.println("Reading content from the file:");
        String fileContent = FileManager.readFromFile(fileName);
        System.out.println(fileContent);

        double[] taxedPrices = FileManager.calculateTax(prices);
        System.out.print("Calculated Taxed Prices: ");
        for (double taxedPrice : taxedPrices) {
            System.out.printf("%.2f ", taxedPrice);
        }

        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Ram is working on a project where he needs to evaluate the performance of students based on their scores. He wants to calculate the total, maximum, and minimum scores of the students and store these results in a file for future reference.

Prompts the user to enter the number of students and their scores. Stores these scores in a file named student.txt. Reads the scores from "student.txt" and displays them. Calculates the total, maximum, and minimum scores. Stores the calculated results in a file named "performance.txt". Reads the performance.txt file and displays its contents.

Ensure that the program correctly handles file operations, manages potential errors gracefully, and properly formats the output.

##### ***Input Format***

The first line of input consists of an integer N, representing the number of students.

The second line consists of N space-separated integers, representing the score of each student.

##### ***Output Format***

The first line of output displays "Successfully written to student.txt".

The second line of output displays "Successfully read from student.txt".

The third line displays the student scores read from the file: "Student scores: <value> <value> ...".

The fourth line of output displays "Successfully written to performance.txt".

The fifth line of output displays "Successfully read from performance.txt".

The sixth line displays the total score as an integer: "Total Score: <value>".

The seventh line displays the maximum score as an integer: "Maximum Score: <value>".

The eighth line displays the minimum score as an integer: "Minimum Score: <value>".

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 3

85 90 78

Output: Successfully written to student.txt

Successfully read from student.txt

Student scores: 85 90 78

Successfully written to performance.txt

Successfully read from performance.txt

Total Score: 253

Maximum Score: 90

Minimum Score: 78

### **Answer**

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class FileManager {
```

```
    public static void writeToFile(String filename, int[] scores) {
```

```
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
```

```
            for (int s : scores) {
```

```
                bw.write(s + " ");
```

```
            }
```

```
            System.out.println("Successfully written to student.txt");
```

```
        } catch (IOException e) {
```

```
            System.out.println("Error writing to student.txt");
```

```
        }
```

```
    }
```

```
    public static int[] readFromFile(String filename) {
```

```
        int[] result = new int[100];
```

```
        int count = 0;
```

```
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
```



```
String line = br.readLine();

if (line != null) {
    String[] parts = line.trim().split("\\s+");
    for (String p : parts) {
        result[count++] = Integer.parseInt(p);
    }
}

if (filename.equals("student.txt")) {
    System.out.println("Successfully read from student.txt");
} else {
    System.out.println("Successfully read from performance.txt");
}

} catch (IOException e) {
    System.out.println("Error reading file");
}

int[] output = new int[count];
for (int i = 0; i < count; i++) {
    output[i] = result[i];
}

return output;
}

public static int calculateTotal(int[] scores) {
    int sum = 0;
    for (int s : scores) sum += s;
    return sum;
}

public static int findMax(int[] scores) {
    int max = scores[0];
    for (int s : scores) {
        if (s > max) max = s;
    }
    return max;
}

public static int findMin(int[] scores) {
```

```

    int min = scores[0];
    for (int s : scores) {
        if (s < min) min = s;
    }
    return min;
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();
        int[] scores = new int[n];

        for (int i = 0; i < n; i++) {
            scores[i] = scanner.nextInt();
        }

        FileManager.writeToFile("student.txt", scores);

        int[] studentScores = FileManager.readFromFile("student.txt");
        System.out.print("Student scores: ");
        for(int score : studentScores) {
            System.out.print(score + " ");
        }

        int[] performance = new int[3];
        performance[0] = FileManager.calculateTotal(scores);
        performance[1] = FileManager.findMax(scores);
        performance[2] = FileManager.findMin(scores);

        try (BufferedWriter writer = new BufferedWriter(new
        FileWriter("performance.txt"))) {
            for (int value : performance) {
                writer.write(value + " ");
            }
            System.out.println("\nSuccessfully written to performance.txt");
        } catch (IOException e) {
            System.out.println("Error writing to performance.txt: " + e.getMessage());
        }

        int[] performanceData = FileManager.readFromFile("performance.txt");
    }
}

```

```
System.out.println("Total Score: " + performanceData[0]);
System.out.println("Maximum Score: " + performanceData[1]);
System.out.println("Minimum Score: " + performanceData[2]);

    scanner.close();
}
}
```

**Status :** Correct

**Marks : 10/10**

## 5. Problem Statement

Sharon is tasked with analyzing network bandwidth data to calculate the average bandwidth from a list of values. She needs a quick tool to perform this task efficiently.

Create a file named "bandwidth.txt" to store the provided bandwidth value which is then written to bandwidth.txt. The program reads the values from bandwidth.txt and calculates the average bandwidth. The calculated result is formatted and written to a new file named "average.txt". Finally, print the average bandwidth from average.txt.

Assist Sharon in this task.

### **Input Format**

The first line of input consists of an integer N, representing the number of bandwidth values.

The second line consists of N space-separated decimal values, representing the individual bandwidth values.

### **Output Format**

The first line of output displays "Successfully written to bandwidth.txt".

The second line of output displays "Successfully read from bandwidth.txt".

The third line displays the bandwidth values read from the file: "Bandwidth values: <value> <value> ...".

The fourth line of output displays "Successfully written to average.txt".

The fifth line of output displays "Successfully read from average.txt".

The sixth line displays the calculated average bandwidth formatted to two decimal places: "Average Bandwidth: <value>".

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 1

30.5

Output: Successfully written to bandwidth.txt

Successfully read from bandwidth.txt

Bandwidth values: 30.5

Successfully written to average.txt

Successfully read from average.txt

Average Bandwidth: 30.50

### **Answer**

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class FileManager {
```

```
    public static void writeToFile(String filename, double[] bandwidth) {
```

```
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
```

```
            for (double b : bandwidth) {
```

```
                bw.write(b + " ");
```

```
            }
```

```
            System.out.println("Successfully written to bandwidth.txt");
```

```
        } catch (IOException e) {
```

```
            System.out.println("Error writing to bandwidth.txt: " + e.getMessage());
```

```
        }
```

```
    }
```

```
    public static String readFromFile(String filename) {
```

```
        StringBuilder sb = new StringBuilder();
```

```

try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
    String line = br.readLine();
    if (line != null) {
        sb.append(line);
    }

    if (filename.equals("bandwidth.txt")) {
        System.out.println("Successfully read from bandwidth.txt");
    } else if (filename.equals("average.txt")) {
        System.out.println("Successfully read from average.txt");
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
return sb.toString();
}

public static double calculateAverage(double[] bandwidth) {
    double sum = 0;
    for (double b : bandwidth) {
        sum += b;
    }
    return sum / bandwidth.length;
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();
        double[] bandwidth = new double[n];

        for (int i = 0; i < n; i++) {
            bandwidth[i] = scanner.nextDouble();
        }

        FileManager.writeToFile("bandwidth.txt", bandwidth);

        String fileContent = FileManager.readFromFile("bandwidth.txt");
        System.out.print("Bandwidth values: " + fileContent);
    }
}

```

```
double average = FileManager.calculateAverage(bandwidth);

try (BufferedWriter writer = new BufferedWriter(new
FileWriter("average.txt"))) {
    writer.write(String.format("%.2f", average));
    System.out.println("\nSuccessfully written to average.txt");
} catch (IOException e) {
    System.out.println("Error writing to average.txt: " + e.getMessage());
}

System.out.print("Average Bandwidth: " +
FileManager.readFromFile("average.txt"));

scanner.close();
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_FH\_Week 9\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Nick is developing a program to convert speed units from kilometers per hour (km/h) to meters per second (m/s). However, he needs assistance in creating a structured and user-friendly program.

Create a file named data.txt to enter the speed in km/h as given in the input. The program reads the speed from data.txt and then converts the speed from km/h to m/s using the formula. The converted speed is written to a new file named converted.txt and then displayed.

Formula: speed in meters per second = speed in km per hour \* 5.0 / 18.0.

#### ***Input Format***

The first line of input contains an integer N, representing the number of speed values.

The second line of input contains N space-separated integers, representing speeds in kilometers per hour (km/h).

### ***Output Format***

The first line of output prints "Successfully written to data.txt" after storing the input values in a file.

The second line of output prints "Successfully read from data.txt" after reading the stored values.

The third line of output prints "Speeds in km/h:" followed by the original speeds.

The fourth line of output prints "Successfully written to converted.txt" after storing the converted speeds.

The fifth line of output prints "Speeds in m/s:" followed by the converted speeds in meters per second (m/s), formatted to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1

180

Output: Successfully written to data.txt

Successfully read from data.txt

Speeds in km/h: 180

Successfully written to converted.txt

Speeds in m/s: 50.00

### ***Answer***

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class FileManager {
```

```
    public static void writeToFile(String filename, int[] speeds) {
```

```
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
```

```
            for (int i = 0; i < speeds.length; i++) {
```



```

        writer.write(speeds[i] + (i < speeds.length - 1 ? " ": ""));
    }
    System.out.println("Successfully written to data.txt");
} catch (IOException e) {
    System.out.println("Error writing to data.txt: " + e.getMessage());
}
}

public static int[] readFromFile(String filename) {
    int[] temp = new int[10];
    int count = 0;

    try (Scanner sc = new Scanner(new File(filename))) {
        while (sc.hasNextInt()) {
            temp[count++] = sc.nextInt();
        }
        System.out.println("Successfully read from data.txt");
    } catch (IOException e) {
        System.out.println("Error reading from data.txt: " + e.getMessage());
    }

    int[] result = new int[count];
    for (int i = 0; i < count; i++) {
        result[i] = temp[i];
    }
    return result;
}

public static double[] convertToMPS(int[] speeds) {
    double[] result = new double[speeds.length];
    for (int i = 0; i < speeds.length; i++) {
        result[i] = speeds[i] * 5.0 / 18.0;
    }
    return result;
}

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();
        int[] speeds = new int[n];
    }
}

```

```

    for (int i = 0; i < n; i++) {
        speeds[i] = scanner.nextInt();
    }

    FileManager.writeToFile("data.txt", speeds);

    int[] speedsKmh = FileManager.readFromFile("data.txt");
    System.out.print("Speeds in km/h: ");
    for(int speed : speedsKmh) {
        System.out.print(speed + " ");
    }

    double[] speedsMps = FileManager.convertToMPS(speedsKmh);

    try (BufferedWriter writer = new BufferedWriter(new
    FileWriter("converted.txt"))) {
        for (double speed : speedsMps) {
            writer.write(String.format("%.2f ", speed));
        }
        System.out.println("\nSuccessfully written to converted.txt");
    } catch (IOException e) {
        System.out.println("Error writing to converted.txt: " + e.getMessage());
    }

    System.out.print("Speeds in m/s: ");
    for(double speed : speedsMps) {
        System.out.printf("%.2f ", speed);
    }

    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Aaron, a software developer, is tasked with creating a program to manage student details using files.

Write a Java program that allows users to input a student's roll number, name, subject, and marks. The program should prompt the user to enter a file name, store the student's details in that file, and then read and display the stored details.

Ensure that the program properly handles file operations and user input.

### ***Input Format***

The first line of input consists of a string, representing the file name where the student's details will be stored.

The second line consists of an integer R, representing the student's roll number.

The third line consists of a string, representing the student's name.

The fourth line consists of a string, representing the subject the student is enrolled in.

The fifth line consists of an integer M, representing the student's marks.

### ***Output Format***

The first line of output prints "Content has been written to [file\_name]" confirming the write operation.

The second line of output prints "Reading content from the file:" before displaying the stored details.

The next lines print the student's details in the following format:

- "Roll Number: " followed by an integer representing the student's roll number.
- "Name: " followed by a string representing the student's name.
- "Subject: " followed by a string representing the subject the student is enrolled in.
- "Marks: " followed by an integer representing the student's marks.
- "ASCII Character of Marks: " followed by the corresponding ASCII character of the student's marks.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: student.txt

101

John

Math

95

Output: Content has been written to student.txt

Reading content from the file:

Roll Number: 101

Name: John

Subject: Math

Marks: 95

ASCII Character of Marks: \_

### **Answer**

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
class FileManager {
```

```
    public static void writeToFile(String fileName, String content, int marks) {  
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {
```

```
            bw.write(content);
```

```
            bw.newLine();
```

```
            bw.write("ASCII Character of Marks: " + (char) marks);
```

```
            System.out.println("Content has been written to " + fileName);
```

```
        } catch (IOException e) {
```

```
            System.out.println("Error writing file");
```

```
        }
```

```
    }
```

```
    public static String readFromFile(String fileName) {
```

```
        StringBuilder sb = new StringBuilder();
```

```
        try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
```

```
            String line;
```

```
            while ((line = br.readLine()) != null) {
```

```
                sb.append(line).append("\n");
```

```

    }
} catch (IOException e) {
    System.out.println("Error reading file");
}
return sb.toString();
}
}

class StudentDetails {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String fileName = scanner.nextLine();
        int rollNo = scanner.nextInt();
        scanner.nextLine();
        String name = scanner.nextLine();
        String subject = scanner.nextLine();
        int marks = scanner.nextInt();
        String content = "Roll Number: " + rollNo + "\n" +
            "Name: " + name + "\n" +
            "Subject: " + subject + "\n" +
            "Marks: " + marks;
        FileManager.writeToFile(fileName, content, marks);
        System.out.println("Reading content from the file:");
        String fileContent = FileManager.readFromFile(fileName);
        System.out.println(fileContent);
        scanner.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Ram is working on a project where he needs to evaluate the performance of students based on their scores. He wants to calculate the average score and total scores of the students.

Create a file named "student.txt" to store these scores, read the scores from the file and calculate the total and the average score. The results are formatted and written to a new file called "performance.txt". Finally, print the contents of the file performance.txt.

Help Ram to finish the project.

### ***Input Format***

The first line of input consists of an integer N, representing the number of students.

The second line consists of N space-separated integers, representing the score of each student.

### ***Output Format***

The first line of output prints "Total Score: " followed by an integer representing the total score of all students.

The second line prints "Average: " followed by a double value representing the average score, formatted to one decimal place.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 3  
85 90 78

Output: Total Score: 253  
Average: 84.3

### ***Answer***

```
import java.io.*;
import java.util.Scanner;

// You are using Java
class FileProcessor {
    public static void writeToFile(String fileName, String content) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {
            bw.write(content);
        } catch (IOException e) {
            System.out.println("Error writing file");
        }
    }
}
```

```

public static String readFromFile(String fileName) {
    StringBuilder sb = new StringBuilder();

    try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
        String line;
        while ((line = br.readLine()) != null) {
            sb.append(line).append(" ");
        }
    } catch (IOException e) {
        System.out.println("Error reading file");
    }

    return sb.toString().trim();
}

public static String calculatePerformance(String fileContent) {
    String[] parts = fileContent.trim().split("\\s+");

    int sum = 0;
    for (String p : parts) {
        sum += Integer.parseInt(p);
    }

    double avg = (double) sum / parts.length;

    String result = "Total Score: " + sum + "\n" +
        "Average: " + String.format("%.1f", avg);

    return result;
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();
        scanner.nextLine();
        StringBuilder ratings = new StringBuilder();

        for (int i = 0; i < n; i++) {
            ratings.append(scanner.nextInt()).append("\n");
        }
    }
}

```

```
String inputFile = "student.txt";
String outputFile = "performance.txt";

FileProcessor.writeToFile(inputFile, ratings.toString().trim());

String fileContent = FileProcessor.readFromFile(inputFile);
String performance = FileProcessor.calculatePerformance(fileContent);

FileProcessor.writeToFile(outputFile, performance);

System.out.println(performance);

scanner.close();
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Alice is concerned about the security of her confidential messages and wants to implement a simple encryption and decryption program.

Implement a simple encryption algorithm that increments the ASCII value of each character in the message. Implement a corresponding decryption algorithm that decrements the ASCII value of each character.

Create a program that takes user input for a confidential message, encrypts the message using a basic encryption algorithm, writes the encrypted message to a file named "encrypted\_data.txt", and then reads the encrypted data from the file to decrypt it. The decrypted message is then displayed to ensure the correctness of the encryption and decryption process.

Assist Alice in this task.

#### **Input Format**

The input consists of a string, representing a confidential message entered by



the user.

### **Output Format**

The first line of output displays "Encrypted Message: " followed by a string representing the encrypted message after applying the encryption algorithm.

The second line displays "Decrypted Message: " followed by a string representing the decrypted message after reading the encrypted data from the file and applying the decryption algorithm.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: Hello

Output: Encrypted Message: Ifmmp

Decrypted Message: Hello

### **Answer**

```
import java.io.*;
import java.util.Scanner;

// You are using Java
class EncryptionUtil {
    public static String encrypt(String message) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < message.length(); i++) {
            char c = message.charAt(i);
            sb.append((char)(c + 1));
        }

        return sb.toString();
    }

    // Decrypt: ASCII - 1
    public static String decrypt(String message) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < message.length(); i++) {
```

```

        char c = message.charAt(i);
        sb.append((char)(c - 1));
    }

    return sb.toString();
}

// Write encrypted data to file
public static void writeToFile(String filename, String data) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
        bw.write(data);
    } catch (IOException e) {
        System.out.println("Error writing file");
    }
}

// Read encrypted data from file
public static String readFromFile(String filename) {
    StringBuilder sb = new StringBuilder();

    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        int ch;
        while ((ch = br.read()) != -1) {
            sb.append((char) ch);
        }
    } catch (IOException e) {
        System.out.println("Error reading file");
    }

    return sb.toString();
}

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String message = scanner.nextLine();
        String filename = "encrypted_data.txt";

        String encryptedMessage = EncryptionUtil.encrypt(message);
        EncryptionUtil.writeToFile(filename, encryptedMessage);
    }
}

```

```
System.out.println("Encrypted Message: "+encryptedMessage);

String decryptedMessage =
EncryptionUtil.decrypt(EncryptionUtil.readFromFile(filename));
System.out.println("Decrypted Message: "+decryptedMessage);

scanner.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_FH\_Week 9\_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 46.5

### Section 1 : Coding

#### 1. Problem Statement

Vishal is working with a list of integers and wants to find numbers where the sum of their digits equals the product of their digits. Write a program that takes n integers as input, saves them in numbers.txt, reads the file to identify numbers with equal sum and product of digits, and saves the results in results.txt.

If no such numbers are found, write 'None' in results.txt.

#### ***Input Format***

The first line of input contains an integer n, representing the number of elements in the list.

The second line contains n space-separated integers.

### **Output Format**

The numbers that satisfy the condition should be written in the file results.txt and printed in the console. If no such numbers exist, the output will be 'None'.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 4  
981 123 234 231  
Output: 123 231

### **Answer**

```
import java.io.*;
import java.util.Scanner;

class Solution {
    public String findNumbersWithEqualSumAndProduct(String[] arr) {
        StringBuilder sb = new StringBuilder();
        boolean found = false;
        for (String numStr : arr) {
            int num = Integer.parseInt(numStr);
            int temp = Math.abs(num);
            int sum = 0;
            int product = 1;
            if (temp == 0) {
                sum = 0;
                product = 0;
            } else {
                while (temp > 0) {
                    int digit = temp % 10;
                    sum += digit;
                    product *= digit;
                    temp /= 10;
                }
            }
            if (sum == product) {
                sb.append(num).append(" ");
                found = true;
            }
        }
    }
}
```

```

    }
    }
    if (!found) {
        return "None";
    }
    return sb.toString().trim();
}
}

public class Main {
    public static void main(String[] args) throws IOException,
    ClassNotFoundException {
        Scanner scanner = new Scanner(System.in);

        int n = Integer.parseInt(scanner.nextLine());
        String numbersLine = scanner.nextLine();
        String[] numberArray = numbersLine.trim().split(" ");

        FileOutputStream fos = new FileOutputStream("numbers.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(numbersLine);
        oos.close();
        fos.close();

        FileInputStream fis = new FileInputStream("numbers.txt");
        ObjectInputStream ois = new ObjectInputStream(fis);
        String fileContent = (String) ois.readObject();
        ois.close();
        fis.close();

        String[] inputNumbers = fileContent.trim().split(" ");
        Solution solution = new Solution();
        String result =
        solution.findNumbersWithEqualSumAndProduct(inputNumbers);

        PrintWriter writer = new PrintWriter(new FileOutputStream("results.txt"));
        writer.println(result);
        writer.close();

        FileInputStream resultFis = new FileInputStream("results.txt");
        Scanner resultScanner = new Scanner(resultFis);
        if (resultScanner.hasNextLine()) {
            System.out.println(resultScanner.nextLine());
        }
    }
}

```

```
}
    resultScanner.close();
    resultFis.close();

    scanner.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Sophia, a software engineer specializing in data storage and transmission, seeks to optimize her storage by implementing a text compression method. She needs a program that takes multiple lines of text and compresses each line by replacing consecutive repeating characters with the character followed by the count of its occurrences.

The program should first create a file called "input\_file.txt" to store the provided text lines. It will then read this file, compress the text, and save the compressed results in a new file named "compressed\_file.txt." Finally, the program should print the compressed content to the console, aiding Sophia in effective storage management.

### ***Input Format***

The first line of input contains an integer N, representing the number of lines of text that Sophia will provide.

The next N lines contain the text as a string, one per line.

### ***Output Format***

For each line of input, the program compresses the text and outputs the compressed result. If a character appears only once, it prints the character followed by '1'. If the character repeats, it prints the character followed by the number of repetitions.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 3

aaabbbccc

dddddeee

fffffghhhh

Output: a3b3c3

d5e3

f5g1h4

### **Answer**

```
import java.io.*;
import java.util.Scanner;

class TextCompressor {
    public String compress(String line) {
        StringBuilder result = new StringBuilder();
        int count = 1;
        for (int i = 1; i < line.length(); i++) {
            if (line.charAt(i) == line.charAt(i - 1)) {
                count++;
            } else {
                result.append(line.charAt(i - 1)).append(count);
                count = 1;
            }
        }
        if (line.length() > 0) {
            result.append(line.charAt(line.length() - 1)).append(count);
        }
        return result.toString();
    }
}

public class Main {
    public static void main(String[] args) throws IOException {
        Scanner scanner = new Scanner(System.in);

        int N = Integer.parseInt(scanner.nextLine());

        PrintWriter writer = new PrintWriter(new FileOutputStream("input_file.txt"));
        for (int i = 0; i < N; i++) {
```



```

        String line = scanner.nextLine();
        writer.println(line);
    }
    writer.close();

    FileInputStream fis = new FileInputStream("input_file.txt");
    Scanner fileScanner = new Scanner(fis);

    TextCompressor compressor = new TextCompressor();

    PrintWriter compressedWriter = new PrintWriter(new
    FileOutputStream("compressed_file.txt"));

    while (fileScanner.hasNextLine()) {
        String line = fileScanner.nextLine();
        String compressedLine = compressor.compress(line);

        System.out.println(compressedLine);

        compressedWriter.println(compressedLine);
    }

    compressedWriter.close();
    fileScanner.close();
    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Sophia is an enthusiastic language learner who is keen on expanding her vocabulary. To aid her studies, she wants to analyze a list of words she has collected. Specifically, Sophia needs a program that can identify the longest and smallest words from her list.

The program should take a number of words as input, write them to a file named "words.txt", read the content from this file, and determine which word is the longest and which is the smallest. The results should be

written to a new file called "result\_word.txt" and also printed on the console for Sophia's convenience.

Implement a class WordFinder with a method that finds the longest and smallest words from an array of words. Help Sophia enhance her vocabulary analysis!

### ***Input Format***

The first line of input contains an integer N, representing the number of words.

The next N lines contain one word each as a string.

### ***Output Format***

The output should be displayed in the following format:

Longest word: <longest\_word>

Smallest word: <smallest\_word>

Where:

<longest\_word> is the longest word found in the input.

<smallest\_word> is the smallest word found in the input.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5  
apple  
banana  
grape  
pineapple  
kiwi

Output: Longest word: pineapple  
Smallest word: kiwi

### ***Answer***

```

import java.io.*;
import java.util.Scanner;
// You are using Java
class WordFinder {
    public String[] findLongestAndSmallestWords(String[] words) {

        String longest = words[0];
        String smallest = words[0];

        for (int i = 1; i < words.length; i++) {

            if (words[i].length() > longest.length()) {
                longest = words[i];
            }

            if (words[i].length() < smallest.length()) {
                smallest = words[i];
            }
        }

        return new String[]{longest, smallest};
    }
}

public class Main {
    public static void main(String[] args) throws IOException {
        Scanner scanner = new Scanner(System.in);

        int N = Integer.parseInt(scanner.nextLine());

        PrintWriter writer = new PrintWriter(new FileOutputStream("words.txt"));
        for (int i = 0; i < N; i++) {
            writer.println(scanner.nextLine());
        }
        writer.close();

        FileInputStream fis = new FileInputStream("words.txt");
        Scanner fileScanner = new Scanner(fis);
        StringBuilder content = new StringBuilder();

        while (fileScanner.hasNextLine()) {
            content.append(fileScanner.nextLine()).append(" ");
        }
    }
}

```

```

    }
    fileScanner.close();

    String[] words = content.toString().trim().split("\\s+");

    WordFinder finder = new WordFinder();
    String[] resultWords = finder.findLongestAndSmallestWords(words);

    PrintWriter resultWriter = new PrintWriter(new
    FileOutputStream("result_word.txt"));
    resultWriter.println("Longest word: " + resultWords[0]);
    resultWriter.println("Smallest word: " + resultWords[1]);
    resultWriter.close();

    System.out.println("Longest word: " + resultWords[0]);
    System.out.println("Smallest word: " + resultWords[1]);

    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Carlos, a biologist, is analyzing DNA sequences and needs a program that counts occurrences of specific patterns within a DNA sequence.

Your task is to create a program that should read a DNA sequence from user input, store it in a file named dna\_sequence.txt, then prompt for a pattern and count its occurrences in the sequence. Finally, the program should output the count in both a file named pattern\_count.txt and the console.

##### **Input Format**

The first line of input contains the DNA sequence.

The second line of input contains the DNA pattern to search for in the sequence.

##### **Output Format**

The output prints count of occurrences of the pattern in the sequence.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: ATCGATCGA

ATC

Output: 2

### **Answer**

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        String dna = sc.nextLine();
        String pattern = sc.nextLine();
        BufferedWriter bw = new BufferedWriter(new
        FileWriter("dna_sequence.txt"));
        bw.write(dna);
        bw.close();
        BufferedReader br = new BufferedReader(new
        FileReader("dna_sequence.txt"));
        String sequence = br.readLine();
        br.close();
        int count = 0;
        for (int i = 0; i <= sequence.length() - pattern.length(); i++) {
            if (sequence.substring(i, i + pattern.length()).equals(pattern)) {
                count++;
            }
        }
        BufferedWriter bw2 = new BufferedWriter(new
        FileWriter("pattern_count.txt"));
        bw2.write(String.valueOf(count));
        bw2.close();
        System.out.println(count);
        sc.close();
    }
}
```

}

Status : Partially correct

Marks : 6.5/10

## 5. Problem Statement

Sophia is tasked with analyzing numerical sequences for a project and needs to determine if a sequence of integers is strictly increasing, strictly decreasing, or neither, while also calculating the total sum of the numbers in the sequence. To simplify her task, she wants to develop a program that automates this analysis.

The program should read a space-separated list of integers, evaluate the pattern of the sequence, compute the sum of all integers, and report both the pattern and the sum. The results should be written to a file named "pattern\_check.txt" and displayed on the console for Sophia's reference. Assist Sophia by creating a class called PatternChecker that performs this analysis.

### **Input Format**

The input consists of a single line of input containing space-separated integers.

### **Output Format**

The output should be in the following format:

"The sequence is strictly increasing Total sum: <total\_sum>"

or

"The sequence is strictly decreasing Total sum: <total\_sum>"

or

"The sequence is neither increasing nor decreasing Total sum: <total\_sum>"

Refer to the sample output for the formatting specifications.

### Sample Test Case

Input: 1 2 3 4 5

Output: The sequence is strictly increasing Total sum: 15

### Answer

```
import java.io.FileOutputStream;
import java.io.PrintWriter;
import java.util.Scanner;

// You are using Java
class PatternChecker {
    public String checkPatternAndCalculateSum(Integer[] arr) {
        int sum = 0;
        boolean increasing = true;
        boolean decreasing = true;
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
        }

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > arr[i - 1]) {
                decreasing = false;
            }
            else if (arr[i] < arr[i - 1]) {
                increasing = false;
            }
            else {
                increasing = false;
                decreasing = false;
            }
        }
        if (increasing) {
            return "The sequence is strictly increasing Total sum: " + sum;
        }
        else if (decreasing) {
            return "The sequence is strictly decreasing Total sum: " + sum;
        }
        else {
            return "The sequence is neither increasing nor decreasing Total sum: " +
sum;
        }
    }
}
```

```
}  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Scanner scanner = new Scanner(System.in);  
  
        String input = scanner.nextLine();  
        String[] numberStrings = input.trim().split("\\s+");  
        Integer[] numbers = new Integer[numberStrings.length];  
  
        for (int i = 0; i < numberStrings.length; i++) {  
            numbers[i] = Integer.parseInt(numberStrings[i]);  
        }  
  
        PatternChecker checker = new PatternChecker();  
        String result = checker.checkPatternAndCalculateSum(numbers);  
  
        PrintWriter writer = new PrintWriter(new  
        FileOutputStream("pattern_check.txt"));  
        writer.println(result);  
        writer.close();  
  
        System.out.println(result);  
  
        scanner.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10



# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MT\_Week 10\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 17

#### Section 1 : MCQ

1. Which of the following methods is used to stop a thread in Java?

**Answer**

stop()

**Status : Wrong**

**Marks : 0/1**

2. What does the synchronized keyword in Java ensure?

**Answer**

It prevents two threads from executing the same method at the same time

**Status : Correct**

**Marks : 1/1**

3. Which of the following methods are used for thread synchronization in Java?

**Answer**

None of the mentioned options

**Status : Wrong**

**Marks : 0/1**

4. Which of the following is true regarding thread synchronization in Java?

**Answer**

Only one thread can execute a synchronized block of code at any given time

**Status : Correct**

**Marks : 1/1**

5. Which of the following classes implements the Runnable interface in Java?

**Answer**

Thread

**Status : Correct**

**Marks : 1/1**

6. Which of the following is the correct way to create a synchronized method?

**Answer**

```
synchronized void myMethod() {}
```

**Status : Correct**

**Marks : 1/1**

7. What is the result if a method is synchronized and two threads try to execute it simultaneously?

**Answer**

One thread will wait until the other completes execution.

**Status :** Correct

**Marks :** 1/1

8. Which of the following can be synchronized in Java?

**Answer**

Methods and blocks

**Status :** Correct

**Marks :** 1/1

9. How can you make sure that only one thread is executing a block of code at a time?

**Answer**

By using the synchronized keyword on the block.

**Status :** Correct

**Marks :** 1/1

10. What is the potential downside of using synchronization in Java?

**Answer**

It can cause performance bottlenecks due to excessive thread switching.

**Status :** Correct

**Marks :** 1/1

11. What happens when a thread enters a synchronized block in Java?

**Answer**

The thread locks the object associated with the block.

**Status :** Correct

**Marks :** 1/1

12. What is the purpose of the wait() method in Java?

**Answer**

To pause a thread and allow other threads to execute.

**Status :** Correct

**Marks :** 1/1

13. What does the notify() method do in Java synchronization?

**Answer**

It resumes execution of a single thread waiting on the same object.

**Status :** Correct

**Marks :** 1/1

14. What is a deadlock in Java synchronization?

**Answer**

A situation where two or more threads are blocked forever, waiting for each other to release resources.

**Status :** Correct

**Marks :** 1/1

15. What does the synchronized block achieve in Java?

**Answer**

It ensures that only one thread can execute the block at any given time.

**Status :** Correct

**Marks :** 1/1

16. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        try {  
            Thread t = new Thread();  
            t.start();  
            t.start();  
            System.out.print("Completed");  
        } catch (IllegalThreadStateException e) {  
            System.out.print("Exception");  
        }  
    }  
}
```

```
}
```

**Answer**

Exception

**Status : Correct**

**Marks : 1/1**

17. What will be the output of the following code?

```
class Test extends Thread {  
    public void run() {  
        System.out.print(getPriority());  
    }  
    public static void main(String[] args) throws InterruptedException {  
        Test t = new Test();  
        t.setPriority(8);  
        t.start();  
        t.join();  
    }  
}
```

**Answer**

8

**Status : Correct**

**Marks : 1/1**

18. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) throws InterruptedException {  
        Thread t1 = new Thread() -> System.out.print("A");  
        Thread t2 = new Thread() -> System.out.print("B");  
        t1.start();  
        t1.join();  
        t2.start();  
        t2.join();  
        System.out.print("C");  
    }  
}
```

**Answer**

ABC

**Status :** Correct

**Marks :** 1/1

19. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) throws InterruptedException {  
        Thread t = new Thread() -> {  
            System.out.print(Thread.currentThread().getName());  
        };  
        t.setName("Worker");  
        t.run();  
        System.out.print("-");  
        t.start();  
        t.join();  
    }  
}
```

**Answer**

main-main

**Status :** Wrong

**Marks :** 0/1

20. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) throws InterruptedException {  
        Thread t1 = new Thread() -> System.out.print("X");  
        Thread t2 = new Thread() -> System.out.print("Y");  
        t1.start();  
        t1.join();  
        t2.start();  
        t2.join();  
        System.out.print("Z");  
    }  
}
```

**Answer**

XYZ

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MT\_Week 10\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : Coding

#### 1. Problem Statement

Alexa is a developer tasked with an application that takes a range of numbers as input and employs two threads to concurrently calculate and print the squares and cubes of these numbers. If the number is odd, the program should print the square of the number. If the number is even, the program should print the cube of the number.

The goal is to test the correctness of the implementation and ensure proper synchronization between the square and cube threads.

Write the program to accomplish his task using multi-threading.

#### ***Input Format***

The input consists of an integer n, representing the upper limit for both square



and cube calculations.

### **Output Format**

The output displays the square and cube threads alternatively in the following format:

For square: "Square: " followed by the square value.

For cube: "Cube: " followed by the cube value.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 5

Output: Square: 1

Cube: 8

Square: 9

Cube: 64

Square: 25

### **Answer**

```
import java.util.Scanner;
class NumberPrinter {
    private int n;
    private int i = 1;
    private boolean squareTurn = true;
    NumberPrinter(int n) {
        this.n = n;
    }
    public synchronized void printSquare() {
        while (i <= n) {
            while (!squareTurn && i <= n) {
                try {
                    wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            if (i > n) break;
```

```

        System.out.println("Square: " + (i * i));
        squareTurn = false;
        notifyAll();
        i++;
    }
}

public synchronized void printCube() {
    while (i <= n) {
        while (squareTurn && i <= n) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        if (i > n) break;
        System.out.println("Cube: " + (i * i * i));
        squareTurn = true;
        notifyAll();
        i++;
    }
}

}

class SquareThread extends Thread {
    NumberPrinter printer;
    SquareThread(NumberPrinter printer) {
        this.printer = printer;
    }
    public void run() {
        printer.printSquare();
    }
}

class CubeThread extends Thread {
    NumberPrinter printer;

    CubeThread(NumberPrinter printer) {
        this.printer = printer;
    }

    public void run() {
        printer.printCube();
    }
}

```

```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        NumberPrinter printer = new NumberPrinter(n);

        SquareThread t1 = new SquareThread(printer);
        CubeThread t2 = new CubeThread(printer);

        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Sarah is working on a dynamic pattern printing application that generates a sequence of stars (\*) and dashes (-) based on user input.

The application uses two threads: one for printing stars and another for printing dashes. The task is to assess the correctness of the implementation and ensure proper synchronization between the star and dash threads.

Write a program to accomplish her task using multi-threading.

For example, if the input is 7, the output prints the star(\*) and dash(-) alternatively for 7 times as shown below:

### **Input Format**

The input consists of an integer n, representing the limit of the pattern.

### **Output Format**

The output displays a sequence of n stars and n dashes alternatively.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 7

Output: \*-\*-\*-\*-\*-\*

### **Answer**

```
import java.util.Scanner;
class PatternPrinter {
    private int n;
    private boolean starTurn = true;
    PatternPrinter(int n) {
        this.n = n;
    }
    public synchronized void printStar() {
        for (int i = 0; i < n; i++) {
            while (!starTurn) {
                try {
                    wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            System.out.print("*");
            starTurn = false;
            notify();
        }
    }
    public synchronized void printDash() {
        for (int i = 0; i < n; i++) {
            while (starTurn) {
                try {
                    wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }
    }
}
```

```

        System.out.print("-");
        starTurn = true;
        notify();
    }
}
}
class StarThread extends Thread {
    PatternPrinter p;
    StarThread(PatternPrinter p) {
        this.p = p;
    }
    public void run() {
        p.printStar();
    }
}
class DashThread extends Thread {
    PatternPrinter p;

    DashThread(PatternPrinter p) {
        this.p = p;
    }
    public void run() {
        p.printDash();
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        PatternPrinter printer = new PatternPrinter(n);
        Thread t1 = new StarThread(printer);
        Thread t2 = new DashThread(printer);
        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Bob is tasked with creating a multi-threaded application that prints a sequence of letters and digits alternatively. The application takes a limit as input and uses two threads: one for printing letters (A, B, C, ...) and another for printing digits (1, 2, 3, ...).

Write the program for the same that ensures proper synchronization between the letter and digit threads using multi-threading.

### ***Input Format***

The input consists of an integer n, representing the limit of characters to be printed.

### ***Output Format***

The output prints a sequence of letters(in uppercase) and digits alternatively separated by space.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 4

Output: A 1 B 2 C 3 D 4

### ***Answer***

```
import java.util.Scanner;

class Printer {
    private int n;
    private int i = 1;
    private char ch = 'A';
    private boolean letterTurn = true;

    Printer(int n) {
        this.n = n;
    }

    public synchronized void printLetter() {
        while (i <= n) {
```

```

        while (!letterTurn) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        if (i > n) break;

        System.out.print(ch + " ");
        ch++;
        letterTurn = false;
        notify();
    }
}

```

```

public synchronized void printDigit() {
    while (i <= n) {
        while (letterTurn) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        if (i > n) break;

        System.out.print(i + " ");
        i++;
        letterTurn = true;
        notify();
    }
}
}

```

```

class LetterThread extends Thread {
    Printer p;

    LetterThread(Printer p) {
        this.p = p;
    }
}

```

```

    }
    public void run() {
        p.printLetter();
    }
}

class DigitThread extends Thread {
    Printer p;

    DigitThread(Printer p) {
        this.p = p;
    }

    public void run() {
        p.printDigit();
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        Printer printer = new Printer(n);

        Thread t1 = new LetterThread(printer);
        Thread t2 = new DigitThread(printer);

        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Sharon is working on a multi-threaded program that prints geometric shapes concurrently.



The program takes an integer input n, and concurrently prints the following patterns using two separate threads:

Thread 1: Print a triangular " \* " pattern with 'n' lines. Thread 2: Print a square " # " pattern with 'n' lines.

Write a program using multi-thread to accomplish her task.

### ***Input Format***

The input consists of an integer n, representing the number of rows for both the right-angled triangle and square.

### ***Output Format***

The first n lines print the right-angled triangle pattern using '\*'.

The next n lines print the square pattern using '#'.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 5

Output: \*

\*\*

\*\*\*

\*\*\*\*

\*\*\*\*\*

#####

#####

#####

#####

#####

### ***Answer***

```
import java.util.Scanner;
class PatternPrinter {
    private int n;
    private int i = 1;
    private boolean triangleDone = false;
```

```

PatternPrinter(int n) {
    this.n = n;
}
public synchronized void printTriangle() {
    for (int i = 1; i <= n; i++) {
        System.out.println("*".repeat(i));
    }
    triangleDone = true;
    notify();
}
public synchronized void printSquare() {
    while (!triangleDone) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    for (int i = 0; i < n; i++) {
        System.out.println("#".repeat(n));
    }
}
}
class TriangleThread extends Thread {
    PatternPrinter p;
    TriangleThread(PatternPrinter p) {
        this.p = p;
    }
    public void run() {
        p.printTriangle();
    }
}
class SquareThread extends Thread {
    PatternPrinter p;
    SquareThread(PatternPrinter p) {
        this.p = p;
    }
    public void run() {
        p.printSquare();
    }
}

```

```

}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        PatternPrinter printer = new PatternPrinter(n);
        Thread t1 = new TriangleThread(printer);
        Thread t2 = new SquareThread(printer);
        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Develop a multi-threaded order processing system for a busy e-commerce warehouse with limited inventory. Each order runs in a separate thread, ensuring synchronized inventory updates to prevent data inconsistencies. If stock is available, the order is fulfilled; otherwise, it is rejected. ?

Note: inventory = 100

### **Input Format**

The first line of input contains an integer N, representing the number of orders.

The next N lines each contain an integer O, representing the quantity of items requested in the order.

### **Output Format**

The output displays whether each order was processed successfully or failed due to insufficient stock, along with the remaining inventory after each order.

At the end, the output displays the final inventory count.

Refer to the sample output for formatting specifications.

### Sample Test Case

Input: 2

70

29

Output: OrderThread-1 placed order: 70 items. Remaining inventory: 30

OrderThread-2 placed order: 29 items. Remaining inventory: 1

Final inventory: 1

### Answer

```
import java.util.Scanner;
class Inventory {
    private int stock = 100;
    public synchronized void processOrder(int qty, int id) {
        if (stock >= qty) {
            stock -= qty;
            System.out.println("OrderThread-" + id +
                " placed order: " + qty +
                " items. Remaining inventory: " + stock);
        } else {
            System.out.println("OrderThread-" + id +
                " order failed: Insufficient stock!");
        }
    }
    public int getStock() {
        return stock;
    }
}
class OrderThread extends Thread {
    Inventory inventory;
    int qty;
    int id;
    OrderThread(Inventory inventory, int qty, int id) {
        this.inventory = inventory;
        this.qty = qty;
        this.id = id;
    }
    public void run() {
        inventory.processOrder(qty, id);
    }
}
public class Main {
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    Inventory inventory = new Inventory();  
    OrderThread[] threads = new OrderThread[n];  
    for (int i = 0; i < n; i++) {  
        int qty = sc.nextInt();  
        threads[i] = new OrderThread(inventory, qty, i + 1);  
    }  
    for (int i = 0; i < n; i++) {  
        threads[i].start();  
        try {  
            threads[i].join();  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
    }  
    System.out.println("Final inventory: " + inventory.getStock());  
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MT\_Week 10\_PY

Attempt : 1

Total Mark : 30

Marks Obtained : 30

### Section 1 : Coding

#### 1. Problem Statement

Lisa is developing a program for a smart thermostat system that monitors and controls the different temperatures of a room in a building. Every room has its temperature sensor, and she wants to implement a multi-threaded approach to read the temperature values from these sensors concurrently.

Your task is to help her write a program that creates a separate thread for each temperature sensor.

Given the name of the building and the temperature range(Upper and Lower) of a room in the building. The program should create a thread that simulates reading the temperature values increased by 1 over time. The temperature values should be displayed with a time delay of 5 milliseconds between each value.

### ***Input Format***

The first line of input consists of a string, representing the name of the building.

The second line consists of an integer L, representing the lower-temperature range of the rooms.

The third line consists of an integer U, representing the upper-temperature range of the rooms.

### ***Output Format***

The first line of output prints "Name - " followed by a string, representing the building's name.

The second line prints space-separated integers, representing the temperature values from L to U.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: Central Plaza

10

15

Output: Name - Central Plaza

10 11 12 13 14 15

### ***Answer***

```
import java.util.Scanner;
class TemperatureSensor extends Thread {
    int l, u;
    TemperatureSensor(int l, int u) {
        this.l = l;
        this.u = u;
    }
    public void run() {
        for (int i = l; i <= u; i++) {
            System.out.print(i + " ");
            try {
                Thread.sleep(5);
            }
        }
    }
}
```





## **Output Format**

The output prints the words on the below format:

- Words are printed alternatively by two threads, preserving the original word order.
- Each word should be printed on a separate line, prefixed with the thread name (e.g., "Thread 1: word" or "Thread 2: word").

Refer to the sample output for formatting specifications.

## **Sample Test Case**

Input: I love java very much

Output: Thread 1: I

Thread 2: love

Thread 1: java

Thread 2: very

Thread 1: much

## **Answer**

```
import java.util.*;
```

```
class WordPrinter {  
    private String[] words;  
    private int index = 0;  
    private boolean turn = true;  
  
    public WordPrinter(String sentence) {  
        this.words = sentence.split(" ");  
    }  
  
    public void printWords(boolean isThread1) {  
        while (true) {  
            synchronized (this) {  
                while (index < words.length && turn != isThread1) {  
                    try {  
                        wait();  
                    } catch (InterruptedException e) {  
                        Thread.currentThread().interrupt();  
                    }  
                }  
            }  
            // Print the word  
            System.out.println(isThread1 ? "Thread 1: " + words[index] : "Thread 2: " + words[index]);  
            index++;  
            turn = !turn;  
        }  
    }  
}
```

```
    }  
}
```

```
    if (index >= words.length) {  
        notifyAll();  
        return;  
    }
```

```
    String threadName = isThread1 ? "Thread 1" : "Thread 2";  
    System.out.println(threadName + ": " + words[index]);  
    index++;  
    turn = !turn;
```

```
    notifyAll();  
}
```

```
    }  
}
```

```
class Thread1 extends Thread {  
    WordPrinter printer;
```

```
    Thread1(WordPrinter printer) {  
        this.printer = printer;  
    }
```

```
    public void run() {  
        printer.printWords(true);  
    }  
}
```

```
class Thread2 extends Thread {  
    WordPrinter printer;
```

```
    Thread2(WordPrinter printer) {  
        this.printer = printer;  
    }
```

```
    public void run() {  
        printer.printWords(false);  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sentence = sc.nextLine();

        WordPrinter printer = new WordPrinter(sentence);

        Thread t1 = new Thread1(printer);
        Thread t2 = new Thread2(printer);

        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

You are tasked with implementing the BankAccount class which maintains the account balance and ensures safe deposit and withdrawal operations using synchronized methods. The account operations will be executed by multiple threads, where:

Thread 1 will deposit money into the account. Thread 2 will withdraw money from the account. Thread 3 will withdraw money from the account again. Thread 4 will make another deposit into the account. Each operation must print the transaction details and the updated account balance. Finally, after all operations are performed, the program should print the final account balance.

#### ***Input Format***

The input contains 5 space-separated integers representing the initial balance in the account, deposit amount for Thread 1, withdrawal amount for Thread 2, withdrawal amount for Thread 3, and deposit amount for Thread 4.

#### ***Output Format***

The output prints,

For deposit: Deposited: <amount>, New Balance: <updated\_balance> where amount and updated\_balance rounded to 1 decimal place.

For withdrawal: Withdrew: <amount>, New Balance: <updated\_balance> where amount and updated\_balance rounded to 1 decimal place.

Final Balance: <final\_balance> where final\_balance rounded to 1 decimal place.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 800 150 50 100 200

Output: Deposited: 150.0, New Balance: 950.0

Withdrew: 50.0, New Balance: 900.0

Withdrew: 100.0, New Balance: 800.0

Deposited: 200.0, New Balance: 1000.0

Final Balance: 1000.0

### **Answer**

```
import java.util.*;
class BankAccount {
    private double balance;
    public BankAccount(double balance) {
        this.balance = balance;
    }
    public synchronized void deposit(double amount) {
        balance += amount;
        System.out.printf("Deposited: %.1f, New Balance: %.1f\n", amount, balance);
    }

    public synchronized void withdraw(double amount) {
        balance -= amount;
        System.out.printf("Withdrew: %.1f, New Balance: %.1f\n", amount, balance);
    }
    public double getBalance() {
        return balance;
    }
}
class DepositThread extends Thread {
```

```

BankAccount account;
double amount;
DepositThread(BankAccount account, double amount) {
    this.account = account;
    this.amount = amount;
}

public void run() {
    account.deposit(amount);
}
}

class WithdrawThread extends Thread {
    BankAccount account;
    double amount;
    WithdrawThread(BankAccount account, double amount) {
        this.account = account;
        this.amount = amount;
    }
    public void run() {
        account.withdraw(amount);
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        double initial = sc.nextDouble();
        double d1 = sc.nextDouble();
        double w1 = sc.nextDouble();
        double w2 = sc.nextDouble();
        double d2 = sc.nextDouble();
        BankAccount account = new BankAccount(initial);
        Thread t1 = new DepositThread(account, d1);
        Thread t2 = new WithdrawThread(account, w1);
        Thread t3 = new WithdrawThread(account, w2);
        Thread t4 = new DepositThread(account, d2);
        t1.start();
        t1.join();

        t2.start();
        t2.join();
    }
}

```

```
t3.start();
t3.join();

t4.start();
t4.join();

System.out.printf("Final Balance: %.1f", account.getBalance());

sc.close();
}
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### VelTech\_Java\_MT\_Week 10\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

Tim is working on a program that alternatively prints even numbers in binary and hexadecimal formats starting from 0. The program has two threads: one for printing numbers in hexadecimal and another for printing numbers in binary.

Both threads share a common lock to synchronize their execution. Write a program to accomplish his task using multi-threading.

#### ***Input Format***

The input consists of a single integer n, representing the number of even numbers to process.

#### ***Output Format***

The output prints the number in hexadecimal format and binary format alternatively in the following format:

For Hexadecimal: "Hexadecimal: " followed by the hexadecimal value.

For Binary: "Binary: " followed by the binary value.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 10

Output: Hexadecimal: 0

Binary: 1

Hexadecimal: 2

Binary: 11

Hexadecimal: 4

Binary: 101

Hexadecimal: 6

Binary: 111

Hexadecimal: 8

Binary: 1001

Hexadecimal: A

Binary: 1011

Hexadecimal: C

Binary: 1101

Hexadecimal: E

Binary: 1111

Hexadecimal: 10

Binary: 10001

Hexadecimal: 12

Binary: 10011

### **Answer**

```
import java.util.*;
class Printer {
    private int n;
    private int count = 0;
    private boolean hexTurn = true;
    public Printer(int n) {
```



```

    this.n = n;
}

public void printHex() {
    while (true) {
        synchronized (this) {
            while (!hexTurn && count < n) {
                try {
                    wait();
                } catch (Exception e) {}
            }
            if (count >= n) {
                notifyAll();
                return;
            }
            int value = 2 * count;
            String hex = Integer.toHexString(value).toUpperCase();
            System.out.println("Hexadecimal: " + hex);

            hexTurn = false;
            notifyAll();
        }
    }
}

public void printBinary() {
    while (true) {
        synchronized (this) {
            while (hexTurn && count < n) {
                try {
                    wait();
                } catch (Exception e) {}
            }
            if (count >= n) {
                notifyAll();
                return;
            }
            int value = 2 * count + 1;
            String bin = Integer.toBinaryString(value);
            System.out.println("Binary: " + bin);

            count++;
            hexTurn = true;
        }
    }
}

```

```

        notifyAll();
    }
}
}
}
class HexThread extends Thread {
    Printer p;
    HexThread(Printer p) {
        this.p = p;
    }
    public void run() {
        p.printHex();
    }
}
class BinThread extends Thread {
    Printer p;

    BinThread(Printer p) {
        this.p = p;
    }
    public void run() {
        p.printBinary();
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Printer printer = new Printer(n);
        Thread t1 = new HexThread(printer);
        Thread t2 = new BinThread(printer);
        t1.start();
        t2.start();
    }
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Imagine you are building a classroom participation tracker that aims to

engage students by asking questions and encouraging them to respond with either vowels (A, E, I, O, U) or consonants.

The participation tracker is implemented using a multi-threaded program that uses two threads: one representing vowels and the other representing consonants, to take turns to simulate student responses.

The program uses a common lock to ensure synchronized and alternating participation. Write a program to accomplish this task.

### ***Input Format***

The input consists of an integer n, representing the number of vowels to be printed alternatively.

### ***Output Format***

The output displays the participation tracker that has n vowels and n consonants printed alternatively, separated by space.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 10

Output: A B E C I D O F U G A H E J I K O L U M

### ***Answer***

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.Scanner;

public class Main {
    private static final char[] VOWELS = {'A', 'E', 'I', 'O', 'U'};
    private static final char[] CONSONANTS = {
        'B', 'C', 'D', 'F', 'G', 'H', 'J', 'K', 'L', 'M', 'N', 'P', 'Q', 'R', 'S', 'T', 'V', 'W', 'X', 'Y', 'Z'
    };

    private final Lock lock = new ReentrantLock();
```

```

private final Condition vowelTurn = lock.newCondition();
private final Condition consonantTurn = lock.newCondition();
private boolean isVowelTurn = true;

private int n;
private int vowelIndex = 0;
private int consonantIndex = 0;

public Main(int n) {
    this.n = n;
}

public void printVowels() {
    for (int i = 0; i < n; i++) {
        lock.lock();
        try {
            while (!isVowelTurn) {
                vowelTurn.await();
            }
            System.out.print(VOWELS[vowelIndex % VOWELS.length] + " ");
            vowelIndex++;
            isVowelTurn = false;
            consonantTurn.signal();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            lock.unlock();
        }
    }
}

public void printConsonants() {
    for (int i = 0; i < n; i++) {
        lock.lock();
        try {
            while (isVowelTurn) {
                consonantTurn.await();
            }
            System.out.print(CONSONANTS[consonantIndex %
CONSONANTS.length] + " ");
            consonantIndex++;
            isVowelTurn = true;

```

```

        vowelTurn.signal();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        lock.unlock();
    }
}
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    sc.close();

    Main tracker = new Main(n);

    Thread vowelThread = new Thread(tracker::printVowels);
    Thread consonantThread = new Thread(tracker::printConsonants);

    vowelThread.start();
    consonantThread.start();

    try {
        vowelThread.join();
        consonantThread.join();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Ram is working on a task to implement a thread-safe program where two threads access a shared resource. To ensure that only one thread accesses the resource at any given time, Ram is using synchronized blocks. The program alternates between printing vowels and consonants, with synchronization preventing race conditions. Each thread waits for its

turn to execute and once finished, notifies the other thread to proceed. This ensures proper order and thread safety while accessing the shared resource.

### ***Input Format***

The input consists of an integer n.

### ***Output Format***

The output displays n vowels and n consonants alternatively, separated by space (in uppercase characters).

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 7

Output: A B E C I D O F U G A H E J

### ***Answer***

```
import java.util.Scanner;
```

```
public class Main {  
    private static final char[] VOWELS = {'A', 'E', 'I', 'O', 'U'};  
    private static final char[] CONSONANTS = {  
        'B', 'C', 'D', 'F', 'G', 'H', 'J', 'K', 'L', 'M', 'N',  
        'P', 'Q', 'R', 'S', 'T', 'V', 'W', 'X', 'Y', 'Z'}  
};
```

```
    private final Object lock = new Object();  
    private boolean vowelTurn = true; // start with vowel  
    private int vowelIndex = 0, consonantIndex = 0;
```

```
    public void printVowel(int n) {  
        for (int i = 0; i < n; i++) {  
            synchronized (lock) {  
                while (!vowelTurn) {  
                    try {  
                        lock.wait();  
                    } catch (InterruptedException e) {
```

```

        Thread.currentThread().interrupt();
    }
}
System.out.print(VOWELS[vowelIndex % VOWELS.length] + " ");
vowelIndex++;
vowelTurn = false;
lock.notify();
}
}
}

```

```

public void printConsonant(int n) {
    for (int i = 0; i < n; i++) {
        synchronized (lock) {
            while (vowelTurn) {
                try {
                    lock.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            System.out.print(CONSONANTS[consonantIndex %
CONSONANTS.length] + " ");
            consonantIndex++;
            vowelTurn = true;
            lock.notify();
        }
    }
}
}
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    sc.close();

    Main tracker = new Main();

    Thread vowelThread = new Thread(() -> tracker.printVowel(n));
    Thread consonantThread = new Thread(() -> tracker.printConsonant(n));

    vowelThread.start();
    consonantThread.start();
}

```

```
    try {  
        vowelThread.join();  
        consonantThread.join();  
    } catch (InterruptedException e) {  
        Thread.currentThread().interrupt();  
    }  
}  
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Lisa is developing a program for a smart thermostat system that monitors and controls the different temperatures of a room in a building. Every room has its temperature sensor, and she wants to implement a multi-threaded approach to read the temperature values from these sensors concurrently.

Your task is to help her write a program that creates a separate thread for each temperature sensor.

Given the name of the building and the temperature range(Upper and Lower) of a room in the building. The program should create a thread that simulates reading the temperature values increased by 1 over time. The temperature values should be displayed with a time delay of 5 milliseconds between each value.

##### **Input Format**

The first line of input consists of a string, representing the name of the building.

The second line consists of an integer L, representing the lower-temperature range of the rooms.

The third line consists of an integer U, representing the upper-temperature range of the rooms.

##### **Output Format**

The first line of output prints "Name - " followed by a string, representing the



building's name.

The second line prints space-separated integers, representing the temperature values from L to U.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: Central Plaza

10

15

Output: Name - Central Plaza

10 11 12 13 14 15

### **Answer**

```
import java.util.Scanner;
class TemperatureSensor extends Thread {
    private int lower;
    private int upper;
    public TemperatureSensor(int lower, int upper) {
        this.lower = lower;
        this.upper = upper;
    }
    @Override
    public void run() {
        for (int temp = lower; temp <= upper; temp++) {
            System.out.print(temp + " ");
            try {
                Thread.sleep(5);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String buildingName = sc.nextLine();
```

```
int L = sc.nextInt();
int U = sc.nextInt();
sc.close();
System.out.println("Name - " + buildingName);
TemperatureSensor sensorThread = new TemperatureSensor(L, U);
sensorThread.start();
try {
    sensorThread.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 11\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 20

#### Section 1 : MCQ

1. What does JDBC stand for?

**Answer**

Java Database Connectivity

**Status : Correct**

**Marks : 1/1**

2. Which class/interface in JDBC is primarily used to obtain a database connection?

**Answer**

DriverManager

**Status : Correct**

**Marks : 1/1**

3. Which method is used to create a Statement object for sending SQL statements to the database?

**Answer**

`createStatement()`

**Status :** Correct

**Marks :** 1/1

4. Which of the following interfaces represents a table of data representing a database result set?

**Answer**

`ResultSet`

**Status :** Correct

**Marks :** 1/1

5. What is the correct syntax to execute a SQL query in JDBC?

**Answer**

`executeQuery("SELECT * FROM table_name")`

**Status :** Correct

**Marks :** 1/1

6. Which method of Statement interface is used to execute an SQL INSERT, UPDATE, or DELETE statement?

**Answer**

`executeUpdate()`

**Status :** Correct

**Marks :** 1/1

7. What is the purpose of using PreparedStatement in JDBC?

**Answer**

To execute dynamic SQL queries

**Status :** Correct

**Marks :** 1/1

8. What is the correct way to close a JDBC ResultSet, Statement, and Connection in Java?

**Answer**

ResultSet.close(); Statement.close(); Connection.close();

**Status : Correct**

**Marks : 1/1**

9. What happens if a JDBC connection is not closed properly?

**Answer**

All of the mentioned options

**Status : Correct**

**Marks : 1/1**

10. Which method is used to close a JDBC connection ?

**Answer**

close()

**Status : Correct**

**Marks : 1/1**

11. Which method is used to execute SELECT queries in JDBC?

**Answer**

executeQuery()

**Status : Correct**

**Marks : 1/1**

12. Which method is used to execute INSERT, UPDATE, DELETE queries?

**Answer**

executeUpdate()

**Status : Correct**

**Marks : 1/1**

13. Which object is used to retrieve data from a SELECT query execution?

**Answer**

ResultSet

**Status : Correct**

**Marks : 1/1**

14. Which of the following is NOT a type of JDBC statement?

**Answer**

ExecutableStatement

**Status : Correct**

**Marks : 1/1**

15. Which exception is thrown when a database access error occurs in JDBC?

**Answer**

SQLException

**Status : Correct**

**Marks : 1/1**

16. What does the DriverManager.getConnection() method return in JDBC?

**Answer**

A Connection object

**Status : Correct**

**Marks : 1/1**

17. What is the purpose of Statement in JDBC?

**Answer**

To execute SQL queries

**Status : Correct**

**Marks : 1/1**

18. Which interface is used to establish a connection with the database in JDBC?

**Answer**

Connection

**Status :** Correct

**Marks :** 1/1

19. What is the purpose of DriverManager in JDBC?

**Answer**

To manage database connections

**Status :** Correct

**Marks :** 1/1

20. What does the following Java code snippet do?

```
Connection connection = null;
try {
    connection = DriverManager.getConnection("jdbc:mysql://
localhost:3306/mydatabase", "username", "password");
} catch (SQLException e) {
    e.printStackTrace();
}
```

**Answer**

Establishes a connection to a MySQL database with provided credentials

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 11\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 30

### Section 1 : Coding

#### 1. Problem Statement

John is learning Java and needs to build a JDBC application that connects to his MySQL database. He must set up the connection using the given credentials and display the connection status.

Help him verify his setup by printing an appropriate message.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

#### ***Input Format***

No console input.

#### ***Output Format***



The first line of output prints, "Connected to the database successfully!" representing the connection status.

The second line of output prints the string representing the connection specific information.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: -

Output: Connected to the database successfully!  
com.mysql.jdbc.JDBC4Connection@4cdbe50f

### **Answer**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

class JDBCConnectionSetup {
    public static void main(String[] args) {
        Connection conn = null;

        try {
            // Load MySQL JDBC Driver
            Class.forName("com.mysql.jdbc.Driver");

            // Establish connection
            conn = DriverManager.getConnection(
                "jdbc:mysql://localhost/ri_db",
                "test",
                "test123"
            );

            // Print success message
            System.out.println("Connected to the database successfully!");

        }
        catch (ClassNotFoundException e) {
            System.out.println("MySQL JDBC Driver not found. Please include it in
```

```

your library path.");
    e.printStackTrace();
} catch (SQLException e) {
    System.out.println("Failed to connect to the database. Check the
connection details.");
    e.printStackTrace();
}
finally
{
    System.out.println(conn);
}
}
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

John, a budding software developer, needs to create a Java program that inserts a new user record into a MySQL database using JDBC. He should capture the username and email from the user via the console and display a success message.

Help him by ensuring his program confirms the record insertion clearly.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

### **Input Format**

The first line of input consists of a String representing the username.

The second line consists of a String representing the email address.

### **Output Format**

The output prints "Record inserted successfully for {username} with email {email}.", representing the confirmation that the record was inserted successfully along with the entered username and email.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: john\_doe

john@example.com

Output: Record inserted successfully for john\_doe with email  
john@example.com.

### **Answer**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

class Main {

    public static void insertUser(String username, String email) {
        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            // Establish connection
            conn = DriverManager.getConnection(
                "jdbc:mysql://localhost/ri_db",
                "test",
                "test123"
            );

            // SQL insert query
            String query = "INSERT INTO users(username, email) VALUES(?, ?)";

            pstmt = conn.prepareStatement(query);
            pstmt.setString(1, username);
            pstmt.setString(2, email);

            // Execute update
            pstmt.executeUpdate();
        }
    }
}
```

```
System.out.println("Record inserted successfully for " + username + " with  
email " + email + ".");
```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    } finally {  
        try {  
            if (pstmt != null) pstmt.close();  
            if (conn != null) conn.close();  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    String usernameInput = scanner.nextLine();  
    String emailInput = scanner.nextLine();  
    insertUser(usernameInput, emailInput);  
    scanner.close();  
}  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

John, a retail store manager, has pre-populated his MySQL database's products table with the following records:

(101, 'Widget', 19.99, 50)

(102, 'Gadget', 29.99, 40)

(103, 'Thingamajig', 9.99, 150)

(104, 'Contraption', 49.99, 20)

(105, 'Doohickey', 14.99, 70)

He now needs a Java program that takes a product ID as input and

retrieves the corresponding product details from the products table.

Help him fetch the product information accurately.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

### ***Input Format***

The input consists of an integer representing the product ID.

### ***Output Format***

If the product is found, the first line of output prints "Product Details:" followed by the product information (ID, Name, Price, and Quantity).

If the product is not found, the output prints an error message stating "No product found with ID <productId>".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 101

Output: Product Details:

ID: 101

Name: Widget

Price: 19.99

Quantity: 50

### ***Answer***

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

class FetchProductData {
    public static void main(String[] args) {
```

```

Scanner scanner = new Scanner(System.in);

int productId = scanner.nextInt();

Connection conn = null;
Statement stmt = null;

// You are using Java
try {
    Connection conn = DriverManager.getConnection(url, user, password);

    String query = "SELECT * FROM products WHERE product_id = ?";
    PreparedStatement ps = conn.prepareStatement(query);
    ps.setInt(1, productId);

    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        int id = rs.getInt("product_id");
        String name = rs.getString("name");
        double price = rs.getDouble("price");
        int quantity = rs.getInt("quantity");

        System.out.printf("Product Details: ID: %d Name: %s Price: %.2f\n",
            id, name, price, quantity);
    } else {
        System.out.print("No product found with ID " + productId);
    }

    rs.close();
    ps.close();
    conn.close();
}

catch (ClassNotFoundException e) {
    System.out.println("MySQL JDBC Driver not found. Please include it in your library path.");
    e.printStackTrace();
} catch (SQLException e) {
    System.out.println("Database error occurred.");
    e.printStackTrace();
}

```

```

    } finally {
        try {
            if (stmt != null) {
                stmt.close();
            }
            if (conn != null) {
                conn.close();
            }
        } catch (SQLException e) {
            System.out.println("Error closing the connection.");
            e.printStackTrace();
        }
    }
    scanner.close();
}
}

```

**Status : Wrong**

**Marks : 0/10**

#### 4. Problem Statement

April, a manager at a company, has pre-populated her MySQL database's employees table with the following records:

```

(101, 'John Doe', 50000.00)
(102, 'Jane Smith', 55000.00)
(103, 'Samuel Johnson', 45000.00)
(104, 'Emily White', 60000.00)
(105, 'David Brown', 65000.00)

```

April needs a Java program that takes an employee ID and a new salary as input and updates the corresponding employee's salary in the employees table. After the update, the program should display the updated details of that employee. If no employee with the given ID exists, the program should display an error message.

Help her fetch the employee information accurately.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

### ***Input Format***

The first line of input consists of an integer representing the employee ID.

The second line consists of a double value representing the new salary for the employee.

### ***Output Format***

If the employee is found and the salary is successfully updated, the output should be:

Salary updated successfully for Employee ID: <employeeId>

Updated Employee Details:

ID: <employeeId>

Name: <employeeName>

Salary: <newSalary>

If no employee with the given ID exists, the output should be:

No employee found with ID <employeeId>

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 101

20000.0

Output: Salary updated successfully for Employee ID: 101



Updated Employee Details:

ID: 101

Name: John Doe

Salary: 20000.0

**Answer**

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class UpdateEmployeeSalary {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int employeeId = scanner.nextInt();
```

```
        double newSalary = scanner.nextDouble();
```

```
        String url = "jdbc:mysql://localhost/ri_db";
```

```
        String username = "test";
```

```
        String password = "test123";
```

```
        Connection conn = null;
```

```
        PreparedStatement pstmt = null;
```

```
        Statement stmt = null;
```

```
        ResultSet rs = null;
```

```
    try {
```

```
        conn = DriverManager.getConnection(url, username, password);
```

```
        // Update salary
```

```
        String updateQuery = "UPDATE employees SET salary = ? WHERE  
employee_id = ?";
```

```
        pstmt = conn.prepareStatement(updateQuery);
```

```
        pstmt.setDouble(1, newSalary);
```

```
        pstmt.setInt(2, employeeId);
```

```
        int rowsUpdated = pstmt.executeUpdate();
```

```
        if (rowsUpdated > 0) {
```

```
            // Fetch updated record
```

```
            String selectQuery = "SELECT * FROM employees WHERE employee_id  
= ?";
```

```
            selectStmt = conn.prepareStatement(selectQuery);
```

```

        selectStmt.setInt(1, employeeId);

        rs = selectStmt.executeQuery();

        if (rs.next()) {
            System.out.println("Salary updated successfully for Employee ID: " +
employeeId);
            System.out.println("Updated Employee Details:");
            System.out.println("ID: " + rs.getInt("employee_id"));
            System.out.println("Name: " + rs.getString("name"));
            System.out.println("Salary: " + rs.getDouble("salary"));
        }
        else {
            System.out.println("No employee found with ID " + employeeId);
        }
    }

    catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            if (rs != null) rs.close();
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    scanner.close();
}
}

```

**Status :** Wrong

**Marks :** 0/10

## 5. Problem Statement

Create a JDBC-based School Management System that handles runtime input to manage student records. The system should allow users to:

Add a new student (student ID, name, grade level, GPA).

Update a student's GPA, ensuring the GPA value is within the valid range (0.0 - 4.0).

View a specific student's record by student ID.

Display all students in the database.

Exit the application.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_db

USER: test

PWD: test123

The students table has already been created with the following structure:

Table Name: students

### ***Input Format***

The first line of input consists of an integer choice, representing the operation to be performed:

(1 for Add Student, 2 for Update GPA, 3 for View Student Record, 4 for Display All Students, 5 for Exit)

For choice 1 (Add Student):

- The second line consists of an integer student\_id.
- The third line consists of a string name.
- The fourth line consists of a string grade\_level.
- The fifth line consists of a double gpa (must be between 0.0 and 4.0).

For choice 2 (Update GPA):

- The second line consists of an integer student\_id.
- The third line consists of a double new\_gpa (must be between 0.0 and 4.0).

For choice 3 (View Student Record):

- The second line consists of an integer student\_id.

For choice 4 (Display All Students):

- No additional inputs are required.

For choice 5 (Exit):

- No additional inputs are required.

### ***Output Format***

The output displays:

For choice 1 (Add Student):

- Print "Student added successfully" if the student was added.
- Print "Failed to add student." if the insertion failed.

For choice 2 (Update GPA):

- Print "GPA updated successfully" if the GPA update was successful.
- Print "Student not found." if the specified student ID does not exist.
- Print "GPA must be between 0.0 and 4.0." if the provided GPA is out of the valid range.

For choice 3 (View Student Record):

- Display the student details in the format:
- ID: [student\_id] | Name: [name] | Grade Level: [grade\_level] | GPA: [gpa]
- Print "Student not found." if the specified student ID does not exist.

For choice 4 (Display All Students):

- Display each student on a new line in the format:
- ID | Name | Grade Level | GPA
- If there are no records, print nothing (or handle with an appropriate message if desired).

For choice 5 (Exit):

- Print "Exiting School Management System."

For invalid input:

- Print "Invalid choice. Please try again."

### **Sample Test Case**

Input: 1

101

Alice Johnson

10

3.8

5

Output: Student added successfully  
Exiting School Management System.

### **Answer**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Scanner;

// You are using Java
class SchoolManagementSystem {
    public static void addStudent(Connection conn, Scanner scanner) {
        try {
            int id = scanner.nextInt();
            scanner.nextLine();
            String name = scanner.nextLine();
            String grade = scanner.nextLine();
            double gpa = scanner.nextDouble();

            if (gpa < 0.0 || gpa > 4.0) {
                System.out.println("Failed to add student.");
                return;
            }

            String query = "INSERT INTO students VALUES (?, ?, ?, ?)";
            PreparedStatement ps = conn.prepareStatement(query);
```

```

        ps.setInt(1, id);
        ps.setString(2, name);
        ps.setString(3, grade);
        ps.setDouble(4, gpa);

        int rows = ps.executeUpdate();
        if (rows > 0) {
            System.out.println("Student added successfully");
        } else {
            System.out.println("Failed to add student.");
        }

        ps.close();
    } catch (Exception e) {
        System.out.println("Failed to add student.");
    }
}

public static void updateGPA(Connection conn, Scanner scanner) {
    try {
        int id = scanner.nextInt();
        double gpa = scanner.nextDouble();

        if (gpa < 0.0 || gpa > 4.0) {
            System.out.println("GPA must be between 0.0 and 4.0.");
            return;
        }

        String query = "UPDATE students SET gpa=? WHERE student_id=?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setDouble(1, gpa);
        ps.setInt(2, id);

        int rows = ps.executeUpdate();
        if (rows > 0) {
            System.out.println("GPA updated successfully");
        } else {
            System.out.println("Student not found.");
        }

        ps.close();
    } catch (Exception e) {

```

```

        System.out.println("Student not found.");
    }
}

public static void viewStudent(Connection conn, Scanner scanner) {
    try {
        int id = scanner.nextInt();

        String query = "SELECT * FROM students WHERE student_id=?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, id);

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            System.out.printf(
                "ID: %d | Name: %s | Grade Level: %s | GPA: %.2f\n",
                rs.getInt("student_id"),
                rs.getString("name"),
                rs.getString("grade_level"),
                rs.getDouble("gpa")
            );
        } else {
            System.out.println("Student not found.");
        }

        rs.close();
        ps.close();
    } catch (Exception e) {
        System.out.println("Student not found.");
    }
}

```

```

public static void displayAllStudents(Connection conn) {
    try {
        String query = "SELECT * FROM students";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);

        boolean hasData = false;

        while (rs.next()) {

```

```

        if (!hasData) {
            System.out.println("ID | Name | Grade Level | GPA");
            hasData = true;
        }
        System.out.printf(
            "%d | %s | %s | %.2f\n",
            rs.getInt("student_id"),
            rs.getString("name"),
            rs.getString("grade_level"),
            rs.getDouble("gpa")
        );
    }

    rs.close();
    stmt.close();
} catch (Exception e) {
    // silent
}

}

public static void main(String[] args) {

    try (Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost/ri_db", "test", "test123");
        Scanner scanner = new Scanner(System.in)) {

        boolean running = true;

        while (running) {
            int choice = scanner.nextInt();
            switch (choice) {
                case 1: addStudent(conn, scanner); break;
                case 2: updateGPA(conn, scanner); break;
                case 3: viewStudent(conn, scanner); break;
                case 4: displayAllStudents(conn); break;
                case 5: System.out.println("Exiting School Management System.");
                running = false; break;
                default: System.out.println("Invalid choice. Please try again.");
            }
        }

    } catch (Exception e) {
        // Silent catch as per requirement
    }
}

```



vtu29201

**Marks : 10/10**

vtu29201

vtu29201

vtu29201

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 11\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 0

### Section 1 : Coding

#### 1. Problem Statement

Warren, a manager at a retail store, has pre-populated his MySQL database's products table with the following records:

('Widget', 19.99, 50)

('Gadget', 29.99, 40)

('Thingamajig', 9.99, 150)

('Contraption', 49.99, 20)

('Doohickey', 14.99, 70)

The productId is auto-incremented, so every time a new product is added, the productId is automatically generated. Warren needs a Java program that takes a productId as input and retrieves the corresponding product

details (name, price, and quantity) from the products table. If no product with the given productId exists, the program should display an error message indicating that the product is not found.

Help him fetch the product information accurately.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

***Input Format***

The input consists of an integer representing the productId.

***Output Format***

If the product is found, the output should display the following:

Product Name: <productName>

Price: <productPrice>

Quantity: <productQuantity>

If no product with the given productId exists, the output should be:

No product found with ID <productId>

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 1

Output: Product Name: Widget

Price: 19.99

***Answer***

```

import java.sql.*;
import java.util.Scanner;

class RetrieveProductById {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int productId = scanner.nextInt();

        String url = "jdbc:mysql://localhost/ri_db"; // Database URL
        String username = "test"; // Database username
        String password = "test123"; // Database password

        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        // You are using Java
        try {
            conn = DriverManager.getConnection(url, user, password);

            String query = "SELECT productName, price, quantity FROM products
WHERE productId = ?";
            pstmt = conn.prepareStatement(query);
            pstmt.setInt(1, productId);

            rs = pstmt.executeQuery();

            if (rs.next()) {
                System.out.println("Product Name: " + rs.getString("productName"));
                System.out.println("Price: " + rs.getDouble("price"));
                System.out.println("Quantity: " + rs.getInt("quantity"));
            } else {
                System.out.println("No product found with ID " + productId);
            }
        }

        catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                if (rs != null) rs.close();
            }
        }
    }
}

```

```

        if (pstmt != null) pstmt.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

scanner.close();
}
}

```

**Status : Wrong**

**Marks : 0/10**

## 2. Problem Statement

Oliver, a school administrator, has pre-populated his MySQL database's students table with the following records:

- (1, 'Alice Cooper', 19, 'A')
- (2, 'Bob Martin', 21, 'B')
- (3, 'Charlie Green', 22, 'C')
- (4, 'Diana White', 20, 'A')
- (5, 'Ethan Black', 23, 'B')

Oliver needs a Java program that takes a studentId as input and retrieves the corresponding student details (ID, Name, Age, Grade) from the students table. If no student with the given studentId exists, the program should display an error message indicating that no student is found.

Help him fetch the student information accurately.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

### **Input Format**

The input consists of an integer representing the studentId for which the details

need to be retrieved.

### **Output Format**

If the student is found, the output should display:

Student Details:

ID: <studentId>

Name: <studentName>

Age: <studentAge>

Grade: <studentGrade>

If no student with the given studentId exists, the output should be:

No student found with ID <studentId>

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 1

Output: Connection established successfully!

Student Details:

ID: 1

Name: Alice Cooper

Age: 19

Grade: A

### **Answer**

```
import java.sql.*;
import java.util.Scanner;

class RetrieveStudentDetails {
    public static void main(String[] args) {
```

```

Scanner scanner = new Scanner(System.in);

int studentId = scanner.nextInt();

String url = "jdbc:mysql://localhost/ri_db";
String username = "test";
String password = "test123";

Connection conn = null;
PreparedStatement pstmt = null;
ResultSet rs = null;

try {
    conn = DriverManager.getConnection(url, username, password);
    System.out.println("Connection established successfully!");

    String query = "SELECT * FROM students WHERE student_id = ?";
    pstmt = conn.prepareStatement(query);
    pstmt.setInt(1, studentId);

    rs = pstmt.executeQuery();

    if (rs.next()) {
        System.out.println("Student Details:");
        System.out.println("ID: " + rs.getInt("student_id"));
        System.out.println("Name: " + rs.getString("name"));
        System.out.println("Age: " + rs.getInt("age"));
        System.out.println("Grade: " + rs.getString("grade"));
    } else {
        System.out.println("No student found with ID " + studentId);
    }

}

catch (SQLException e) {
    System.out.println("SQL Exception occurred: " + e.getMessage());
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (pstmt != null) pstmt.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {

```

```
        System.out.println("Error closing resources: " + e.getMessage());
    }
}

scanner.close();
}
```

**Status : Wrong**

**Marks : 0/10**

### 3. Problem Statement

Create a JDBC-based Hospital Management System that handles runtime input to manage patient records. The system should allow users to:

Add a new patient (patient ID, name, age, status).

Update a patient's status.

View a specific patient's record by patient ID.

Display all patient records in the database.

Exit the application.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_db

USER: test

PWD: test123

The patients table has already been created with the following structure:

Table Name: patients

#### ***Input Format***

The first line of input consists of an integer choice, representing the operation to be performed:



(1 for Add Patient, 2 for Update Patient Status, 3 for View Patient Record, 4 for Display All Patients, 5 for Exit)

For choice 1 (Add Patient):

- The second line consists of an integer patient\_id.
- The third line consists of a string name.
- The fourth line consists of an integer age.
- The fifth line consists of a string status.

For choice 2 (Update Patient Status):

- The second line consists of an integer patient\_id.
- The third line consists of a string new\_status.

For choice 3 (View Patient Record):

- The second line consists of an integer patient\_id.

For choice 4 (Display All Patients):

- No additional inputs are required.

For choice 5 (Exit):

- No additional inputs are required.

### ***Output Format***

For choice 1 (Add Patient):

- Print "Patient added successfully" if the patient was added.
- Print "Failed to add patient." if the insertion failed.

For choice 2 (Update Patient Status):

- Print "Patient status updated successfully" if the update was successful.
- Print "Patient not found." if the specified patient ID does not exist.

For choice 3 (View Patient Record):

- Display the patient details in the format:
- ID: [patient\_id] | Name: [name] | Age: [age] | Status: [status]
- Print "Patient not found." if the specified patient ID does not exist.

For choice 4 (Display All Patients):

- Display each patient on a new line in the format:
- ID | Name | Age | Status
- If no records are available, print nothing (or handle it with an appropriate message if desired).

For choice 5 (Exit):

- Print "Exiting Hospital Management System."

For invalid input:

- Print "Invalid choice. Please try again."

### **Sample Test Case**

Input: 1

101

John Doe

45

Admitted

4

5

Output: Patient added successfully

ID | Name | Age | Status

101 | John Doe | 45 | Admitted

Exiting Hospital Management System.

### **Answer**

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
public static void addPatient(Connection conn, Scanner scanner) {
```

```
    try {
```

```
        int id = scanner.nextInt();
```

```
        scanner.nextLine();
```

```
String name = scanner.nextLine();
int age = scanner.nextInt();
scanner.nextLine();
String status = scanner.nextLine();
```

```
String query = "INSERT INTO patients (patient_id, name, age, status)
VALUES (?, ?, ?, ?)";
```

```
PreparedStatement pstmt = conn.prepareStatement(query);
pstmt.setInt(1, id);
pstmt.setString(2, name);
pstmt.setInt(3, age);
pstmt.setString(4, status);
```

```
int rows = pstmt.executeUpdate();
```

```
if (rows > 0) {
    System.out.println("Patient added successfully");
} else {
    System.out.println("Failed to add patient.");
}
```

```
} catch (Exception e) {
    System.out.println("Failed to add patient.");
}
}
```

```
public static void updatePatientStatus(Connection conn, Scanner scanner) {
    try {
        int id = scanner.nextInt();
        scanner.nextLine();
        String status = scanner.nextLine();
```

```
String query = "UPDATE patients SET status = ? WHERE patient_id = ?";
PreparedStatement pstmt = conn.prepareStatement(query);
pstmt.setString(1, status);
pstmt.setInt(2, id);
```

```
int rows = pstmt.executeUpdate();
```

```
if (rows > 0) {
    System.out.println("Patient status updated successfully");
} else {
```

```

        System.out.println("Patient not found.");
    }

    } catch (Exception e) {
        System.out.println("Patient not found.");
    }

}

public static void viewPatientRecord(Connection conn, Scanner scanner) {
    try {
        int id = scanner.nextInt();

        String query = "SELECT * FROM patients WHERE patient_id = ?";
        PreparedStatement pstmt = conn.prepareStatement(query);
        pstmt.setInt(1, id);

        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            System.out.println("ID: " + rs.getInt("patient_id") +
                " | Name: " + rs.getString("name") +
                " | Age: " + rs.getInt("age") +
                " | Status: " + rs.getString("status"));
        } else {
            System.out.println("Patient not found.");
        }

    } catch (Exception e) {
        System.out.println("Patient not found.");
    }

}

public static void displayAllPatients(Connection conn) {
    try {
        String query = "SELECT * FROM patients ORDER BY patient_id";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);

        boolean hasData = false;
    }

```

```

        while (rs.next()) {
            if (!hasData) {
                System.out.println("ID | Name | Age | Status");
                hasData = true;
            }
            System.out.println(rs.getInt("patient_id") + " | " +
                rs.getString("name") + " | " +
                rs.getInt("age") + " | " +
                rs.getString("status"));
        }
    }
}

```

```

    } catch (Exception e) {
    }
}

```

```

public static void main(String[] args) {
    try (Connection conn = DriverManager.getConnection(
        "jdbc:mysql://localhost/ri_db", "test", "test123");
        Scanner scanner = new Scanner(System.in)) {
    }
}

```

```

    boolean running = true;

```

```

    while (running) {
        int choice = scanner.nextInt();
    }

```

```

        switch (choice) {
            case 1: addPatient(conn, scanner); break;
            case 2: updatePatientStatus(conn, scanner); break;
            case 3: viewPatientRecord(conn, scanner); break;
            case 4: displayAllPatients(conn); break;
            case 5:
                System.out.println("Exiting Hospital Management System.");
                running = false;
                break;
            default:
                System.out.println("Invalid choice. Please try again.");
        }
    }
}

```

```

    } catch (SQLException e) {
        // No extra output
    }
}

```

**Status : Wrong**

**Marks : 0/10**

#### 4. Problem Statement

Create a JDBC-based Inventory Management System that handles runtime input to manage items in an inventory. The system should allow users to:

Add a new item (item ID, name, quantity, price).

Restock an item by increasing its quantity.

Reduce the stock of an item, ensuring sufficient quantity.

Display all items in the inventory in a sorted order by item ID.

Exit the application.

Half of the code is given here; Only the remaining part should be completed.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_db

USER: test

PWD: test123

The items table has already been created with the following structure:

Table Name: items

#### ***Input Format***

The first line contains an integer choice.

Input continues until choice = 5.

Each input must be given on a separate line.

No prompts must be printed.

No extra spaces must be printed.

If choice = 1 (Add Item):

Next line integer item\_id

Next line string name

Next line integer quantity

Next line double price

If choice = 2 (Restock Item):

Next line integer item\_id

Next line integer quantity\_to\_add

If choice = 3 (Reduce Stock):

Next line integer item\_id

Next line integer quantity\_to\_remove

If choice = 4 (Display Inventory):

No additional input

If choice = 5 (Exit):

No additional input

If invalid choice:

No additional input

Important Rules:

- Each input must be in a new line
- No blank lines in input
- No prompt text
- Double values must contain decimal point

***Output Format***

For choice = 1 (Add Item):

Print exactly one of:

Item added successfully

OR

Failed to add item.



For choice = 2 (Restock Item):

Print exactly one of:

Item restocked successfully

OR

Item not found.

For choice = 3 (Reduce Stock):

Print exactly one of:

Stock reduced successfully

OR

Not enough stock to remove.

OR

Item not found.

For choice = 4 (Display Inventory):

If at least one record exists:

First print:

ID | Name | Quantity | Price

Then print each record on a new line in the format:

item\_id | name | quantity | price

Price must be printed with exactly 2 decimal places.

If no records exist:

Print exactly:

No records found.

For choice = 5 (Exit):

Print exactly:

Exiting Inventory Management System.

For invalid choice:

Print exactly:

Invalid choice. Please try again.

Note

- Output must match character-by-character
- No records found. must start with capital N
- Must include period at end

- No extra spaces before or after sentence
- No blank lines before or after output
- Price must use exactly 2 decimal places

### **Sample Test Case**

Input: 1

101

Laptop

50

1200.00

4

5

Output: Item added successfully

ID | Name | Quantity | Price

101 | Laptop | 50 | 1200.00

Exiting Inventory Management System.

### **Answer**

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class BestSolution {
```

```
    public static void addItem(Connection conn, Scanner scanner) {
```

```
        try {
```

```
            int id = scanner.nextInt();
```

```
            scanner.nextLine();
```

```
            String name = scanner.nextLine();
```

```
            int quantity = scanner.nextInt();
```

```
            double price = scanner.nextDouble();
```

```
            String query = "INSERT INTO items (item_id, name, quantity, price) VALUES  
(?, ?, ?, ?)";
```

```
            PreparedStatement ps = conn.prepareStatement(query);
```

```
            ps.setInt(1, id);
```

```
            ps.setString(2, name);
```

```
            ps.setInt(3, quantity);
```

```
            ps.setDouble(4, price);
```

```
            int rows = ps.executeUpdate();
```

```

        if (rows > 0)
            System.out.println("Item added successfully");
        else
            System.out.println("Failed to add item.");
    }
}

```

```

public static void restockItem(Connection conn, Scanner scanner) {
    try {
        int id = scanner.nextInt();
        scanner.nextLine();
        String name = scanner.nextLine();
        int quantity = scanner.nextInt();
        double price = scanner.nextDouble();

        String query = "INSERT INTO items (item_id, name, quantity, price) VALUES
        (?, ?, ?, ?)";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, id);
        ps.setString(2, name);
        ps.setInt(3, quantity);
        ps.setDouble(4, price);

        int rows = ps.executeUpdate();

        if (rows > 0)
            System.out.println("Item added successfully");
        else
            System.out.println("Failed to add item.");
    }
}

```

```

public static void reduceStock(Connection conn, Scanner scanner) {

    try {
        int id = scanner.nextInt();
        int removeQty = scanner.nextInt();

        // Check current quantity
    }
}

```

```

String selectQuery = "SELECT quantity FROM items WHERE item_id = ?";
PreparedStatement ps1 = conn.prepareStatement(selectQuery);
ps1.setInt(1, id);
ResultSet rs = ps1.executeQuery();

if (rs.next()) {
    int currentQty = rs.getInt("quantity");

    if (currentQty >= removeQty) {
        String updateQuery = "UPDATE items SET quantity = quantity - ?
WHERE item_id = ?";
        PreparedStatement ps2 = conn.prepareStatement(updateQuery);
        ps2.setInt(1, removeQty);
        ps2.setInt(2, id);
        ps2.executeUpdate();

        System.out.println("Stock reduced successfully");
    } else {
        System.out.println("Not enough stock to remove.");
    }
} else {
    System.out.println("Item not found.");
}
}
}

```

```

public static void displayInventory(Connection conn) {
    try {
        String query = "SELECT * FROM items ORDER BY item_id";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);

        boolean hasData = false;

        while (rs.next()) {
            if (!hasData) {
                System.out.println("ID | Name | Quantity | Price");
                hasData = true;
            }
        }
    }
}

```

```

        System.out.printf("%d | %s | %d | %.2f%n",
            rs.getInt("item_id"),
            rs.getString("name"),
            rs.getInt("quantity"),
            rs.getDouble("price"));
    }

    if (!hasData) {
        System.out.println("No records found.");
    }
}

}
}

class InventoryManagementSystem {
    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://
localhost/ri_db", "test", "test123");
            Scanner scanner = new Scanner(System.in)) {

            while (true) {
                if (!scanner.hasNextInt())
                    break;

                int choice = scanner.nextInt();

                switch (choice) {
                    case 1:
                        BestSolution.addItem(conn, scanner);
                        break;
                    case 2:
                        BestSolution.restockItem(conn, scanner);
                        break;
                    case 3:
                        BestSolution.reduceStock(conn, scanner);
                        break;
                    case 4:
                        BestSolution.displayInventory(conn);
                        break;
                    case 5:
                        System.out.println("Exiting Inventory Management System.");
                        return;
                }
            }
        }
    }
}

```

```
        default:
            System.out.println("Invalid choice. Please try again.");
        }
    }
} catch (Exception e) {
}
}
```

**Status :** Wrong

**Marks :** 0/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 11\_CY

Attempt : 1

Total Mark : 30

Marks Obtained : 20

### Section 1 : Coding

#### 1. Problem Statement

Develop a JDBC-based Library Management System to manage book records stored in a MySQL database.

The database is already created with the following details:

Database URL: jdbc:mysql://localhost/ri\_db

Username: test

Password: test123

Table Name: books

Table Structure:

book\_id INT PRIMARY KEY



title VARCHAR(100)  
author VARCHAR(100)  
genre VARCHAR(50)  
publication\_year INT

</span>

The application must continuously accept user input until the user chooses Exit.

The system should support the following operations:

- 1 – Add a new book
- 2 – Update book details
- 3 – Delete a book
- 4 – Search books by partial title
- 5 – Display all books sorted by publication year
- 6 – Exit

Use PreparedStatement for all database operations except display (which may use Statement).

For searching, use the SQL LIKE operator with % wildcard.

For display operation, use ORDER BY publication\_year in ascending order.

Only the following attributes are allowed for update:

titleauthorgenreyear

If any other attribute is entered, print:

Invalid attribute.

If an invalid menu choice is entered, print:

Invalid choice. Please try again.

No additional text should be printed other than what is specified.

### ***Input Format***

The first line consists of an integer choice.

For choice 1 (Add):

Second line: integer book\_id

Third line: string title

Fourth line: string author

Fifth line: string genre

Sixth line: integer publication\_year

For choice 2 (Update):

Second line: integer book\_id

Third line: string attribute

Fourth line: string new\_value

For choice 3 (Delete):

Second line: integer book\_id

For choice 4 (Search):

Second line: string partial title

For choice 5 (Display):

No additional input

For choice 6 (Exit):

No additional input

The program continues until choice 6 is entered.

### ***Output Format***

For Add:

Book added successfully

Failed to add a book.

For Update:

Book details updated successfully

Failed to update book.

Invalid attribute.

For Delete:

Book deleted successfully

Book not found.

For Search:

Each matching book printed as:

book\_id | title | author | genre | publication\_year

If no records found:

No books found.

For Display:

Each book printed as:

book\_id | title | author | genre | publication\_year

For Exit:

Exiting Library Management System.

For Invalid Choice:

Invalid choice. Please try again.

Do not print extra spaces or extra lines.

### **Sample Test Case**

Input: 1

501

Java Programming

Author Z

Education

2020

5

6

Output: Book added successfully

501 | Java Programming | Author Z | Education | 2020

Exiting Library Management System

### **Answer**

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class BestSolution {
```

```
    static void addBook(Connection conn, Scanner scanner) throws SQLException  
{
```

```
        int book_id = Integer.parseInt(scanner.nextLine());
```

```
        String title = scanner.nextLine();
```

```
        String author = scanner.nextLine();
```

```
        String genre = scanner.nextLine();
```

```
        int year = Integer.parseInt(scanner.nextLine());
```

```
        String sql = "INSERT INTO books VALUES (?, ?, ?, ?, ?)";
```

```
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
            ps.setInt(1, book_id);
```

```
            ps.setString(2, title);
```

```
            ps.setString(3, author);
```

```
            ps.setString(4, genre);
```

```
            ps.setInt(5, year);
```

```
            ps.executeUpdate();
```

```

        System.out.println("Book added successfully");
    } catch (SQLException e) {
        System.out.println("Failed to add a book.");
    }
}

```

```

static void updateBook(Connection conn, Scanner scanner) throws
SQLException {

```

```

    int book_id = Integer.parseInt(scanner.nextLine());
    String attribute = scanner.nextLine();
    String new_value = scanner.nextLine();

```

```

    String column;
    boolean isInt = false;

```

```

    switch (attribute) {
        case "title":
            column = "title";
            break;
        case "author":
            column = "author";
            break;
        case "genre":
            column = "genre";
            break;
        case "year":
            column = "publication_year";
            isInt = true;
            break;
        default:
            System.out.println("Invalid attribute.");
            return;
    }

```

```

    String sql = "UPDATE books SET " + column + " = ? WHERE book_id = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        if (isInt) {
            ps.setInt(1, Integer.parseInt(new_value));
        } else {
            ps.setString(1, new_value);
        }
        ps.setInt(2, book_id);
    }

```

```

        int rows = ps.executeUpdate();
        if (rows > 0) {
            System.out.println("Book details updated successfully");
        } else {
            System.out.println("Failed to update book.");
        }
    }
}

```

```

static void deleteBook(Connection conn, Scanner scanner) throws
SQLException {
    int book_id = Integer.parseInt(scanner.nextLine());
    String sql = "DELETE FROM books WHERE book_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, book_id);
        int rows = ps.executeUpdate();
        if (rows > 0) {
            System.out.println("Book deleted successfully");
        } else {
            System.out.println("Book not found.");
        }
    }
}

```

```

static void searchBooks(Connection conn, Scanner scanner) throws
SQLException {
    String partial_title = scanner.nextLine();
    String sql = "SELECT * FROM books WHERE title LIKE ?";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, "%" + partial_title + "%");
        ResultSet rs = ps.executeQuery();
        boolean found = false;

        while (rs.next()) {
            found = true;
            System.out.println(rs.getInt("book_id") + " | " +
                rs.getString("title") + " | " +
                rs.getString("author") + " | " +
                rs.getString("genre") + " | " +

```

```

                rs.getInt("publication_year"));
            }
            if (!found) {
                System.out.println("No books found.");
            }
        }
    }
}

```

```

static void displayBooks(Connection conn) throws SQLException {
    String sql = "SELECT * FROM books ORDER BY publication_year ASC";
    try (Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {

```

```

        while (rs.next()) {
            System.out.println(rs.getInt("book_id") + " | " +
                rs.getString("title") + " | " +
                rs.getString("author") + " | " +
                rs.getString("genre") + " | " +
                rs.getInt("publication_year"));
        }
    }
}

```

```

class LibraryManagementSystem {
    public static void main(String[] args) {
        try {
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost/ri_db",
                "test",
                "test123"
            );

            Scanner scanner = new Scanner(System.in);

            while (true) {

                String input = scanner.nextLine();
                int choice;

                try {
                    choice = Integer.parseInt(input.trim());
                } catch (Exception e) {

```



```

        continue;
    }

    switch (choice) {
        case 1:
            BestSolution.addBook(conn, scanner);
            break;
        case 2:
            BestSolution.updateBook(conn, scanner);
            break;
        case 3:
            BestSolution.deleteBook(conn, scanner);
            break;
        case 4:
            BestSolution.searchBooks(conn, scanner);
            break;
        case 5:
            BestSolution.displayBooks(conn);
            break;
        case 6:
            System.out.println("Exiting Library Management System");
            conn.close();
            scanner.close();
            return;
        default:
            System.out.println("Invalid choice. Please try again.");
    }
}

} catch (Exception e) {
}
}
}

```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Create a JDBC-based Banking Management System that handles runtime input to manage customer bank accounts. The system should allow users to:

Add a new customer (customer ID, name, account number, balance).

Deposit money into a customer's account.

Withdraw money from a customer's account, ensuring sufficient balance.

Display all customer records in a sorted order by customer ID.

Exit the application.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_db

USER: test

PWD: test123

The customers table has already been created with the following structure:

Table Name: customers

### ***Input Format***

The first line of input consists of an integer choice, representing the operation to be performed (1 for Add, 2 for Deposit, 3 for Withdraw, 4 for Display, 5 for Exit).

The first line contains an integer choice.

If choice = 1:

Next line    integer customer\_id

Next line    string name

Next line    string account\_number

Next line    double balance

If choice = 2:

Next line integer customer\_id

Next line double amount

If choice = 3:

Next line integer customer\_id

Next line double amount

If choice = 4:

No additional input

If choice = 5:

No additional input

If invalid choice:

No additional input

Input continues until choice 5 is entered.

There must be:

- No prompts
- No extra text
- No extra spaces
- Each input on a new line

***Output Format***

For choice 1:

Customer added successfully

or

Failed to add customer.

For choice 2:

Deposit successful

or

Customer not found.

For choice 3:

Withdrawal successful

or

Insufficient balance.

or

Customer not found.

For choice 4:

| ID | Name | Account Number | Balance |
|----|------|----------------|---------|
|----|------|----------------|---------|

|     |      |            |          |
|-----|------|------------|----------|
| 101 | Raji | 1234567989 | 50000.00 |
|-----|------|------------|----------|

- Print header only if at least one record exists
- Balance must be printed with exactly 2 decimal places
- If no records exist print nothing

For choice 5:

Exiting Banking Management System.

For invalid choice:

Invalid choice. Please try again.

### **Sample Test Case**

Input: 1

101

Raji

1234567989

50000.00

4

5

Output: Customer added successfully

ID | Name | Account Number | Balance

101 | Raji | 1234567989 | 50000.00

Exiting Banking Management System.

### **Answer**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.SQLException;
import java.util.Scanner;

class BankingManagementSystem {
    public static void main(String[] args) {

        try (
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost/ri_db", "test", "test123");
            Scanner sc = new Scanner(System.in)
        ) {

            boolean running = true;

            while (running) {

                int choice = sc.nextInt();

                switch (choice) {

                    case 1:
                        BankingOperations.addCustomer(conn, sc);
                        break;

                    case 2:
                        BankingOperations.depositMoney(conn, sc);
                        break;
```

```

        case 3:
            BankingOperations.withdrawMoney(conn, sc);
            break;

        case 4:
            BankingOperations.displayCustomers(conn);
            break;

        case 5:
            System.out.println("Exiting Banking Management System.");
            running = false;
            break;

        default:
            System.out.println("Invalid choice. Please try again.");
    }
}

} catch (Exception e) {
    // no extra output
}
}
}

```

```

class BankingOperations {

    public static void addCustomer(Connection conn, Scanner sc) {
        try {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String accNo = sc.nextLine();
            double balance = sc.nextDouble();

            String query = "INSERT INTO customers (customer_id, name,
account_number, balance) VALUES (?, ?, ?, ?)";
            PreparedStatement ps = conn.prepareStatement(query);
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setString(3, accNo);
            ps.setDouble(4, balance);

```

```

int rows = ps.executeUpdate();

if (rows > 0)
    System.out.println("Customer added successfully");
else
    System.out.println("Failed to add customer.");

} catch (Exception e) {
    System.out.println("Failed to add customer.");
}
}

public static void depositMoney(Connection conn, Scanner sc) {
    try {
        int id = sc.nextInt();
        double amount = sc.nextDouble();

        String check = "SELECT balance FROM customers WHERE customer_id
= ?";
        PreparedStatement ps1 = conn.prepareStatement(check);
        ps1.setInt(1, id);
        ResultSet rs = ps1.executeQuery();

        if (rs.next()) {
            double current = rs.getDouble("balance");

            String update = "UPDATE customers SET balance = ? WHERE
customer_id = ?";
            PreparedStatement ps2 = conn.prepareStatement(update);
            ps2.setDouble(1, current + amount);
            ps2.setInt(2, id);
            ps2.executeUpdate();

            System.out.println("Deposit successful");
        } else {
            System.out.println("Customer not found.");
        }

    } catch (Exception e) {
        System.out.println("Customer not found.");
    }
}

```



```

    }

    public static void withdrawMoney(Connection conn, Scanner sc) {
        try {
            int id = sc.nextInt();
            double amount = sc.nextDouble();

            String check = "SELECT balance FROM customers WHERE customer_id
= ?";
            PreparedStatement ps1 = conn.prepareStatement(check);
            ps1.setInt(1, id);
            ResultSet rs = ps1.executeQuery();

            if (rs.next()) {
                double current = rs.getDouble("balance");

                if (current >= amount) {
                    String update = "UPDATE customers SET balance = ? WHERE
customer_id = ?";
                    PreparedStatement ps2 = conn.prepareStatement(update);
                    ps2.setDouble(1, current - amount);
                    ps2.setInt(2, id);
                    ps2.executeUpdate();

                    System.out.println("Withdrawal successful");
                } else {
                    System.out.println("Insufficient balance.");
                }
            } else {
                System.out.println("Customer not found.");
            }
        } catch (Exception e) {
            System.out.println("Customer not found.");
        }
    }

```

```

    public static void displayCustomers(Connection conn) {
        try {
            String query = "SELECT * FROM customers ORDER BY customer_id";
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query);

```

```

        boolean hasData = false;

        while (rs.next()) {
            if (!hasData) {
                System.out.println("ID | Name | Account Number | Balance");
                hasData = true;
            }

            System.out.println(
                rs.getInt("customer_id") + " | " +
                rs.getString("name") + " | " +
                rs.getString("account_number") + " | " +
                String.format("%.2f", rs.getDouble("balance"))
            );
        }

    } catch (Exception e) {
        // no output
    }
}

class BankingManagementSystem {
    public static void main(String[] args) {

        try (
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost/ri_db", "test", "test123");
            Scanner sc = new Scanner(System.in)
        ) {

            boolean running = true;

            while (running) {

                int choice = sc.nextInt();

                switch (choice) {

                    case 1:
                        BankingOperations.addCustomer(conn, sc);
                        break;

```

```

case 2:
    BankingOperations.depositMoney(conn, sc);
    break;

case 3:
    BankingOperations.withdrawMoney(conn, sc);
    break;

case 4:
    BankingOperations.displayCustomers(conn);
    break;

case 5:
    System.out.println("Exiting Banking Management System.");
    running = false;
    break;

default:
    System.out.println("Invalid choice. Please try again.");
}
}

} catch (Exception e) {
    // No extra debug output to maintain QC format
}
}
}

```

**Status : Wrong**

**Marks : 0/10**

### 3. Problem Statement

Peter, a store manager, has pre-populated his MySQL database's orders table with the following records:

(101, '2021-10-01', 250.50)

(102, '2021-10-02', 120.75)

(103, '2021-10-03', 315.20)

(104, '2021-10-04', 450.00)

(105, '2021-10-05', 700.80)

The order\_id is auto-incremented and will automatically be assigned when an order is inserted.

Peter needs a Java program that retrieves all orders made by a specific customer\_id from the orders table. The program should take the customer\_id as input and display the corresponding order details (order\_id, order\_date, and order\_amount). If no orders are found for the given customer\_id, the program should display a message indicating that no records were found.

The system should connect to a MYSQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri\_dbUsername: testPassword: test123

#### ***Input Format***

The input consists of an integer representing the customer\_id whose order details need to be fetched.

#### ***Output Format***

If orders for the given customer\_id are found, the output should display:

Order Details for Customer ID: <customerId>

Order ID: <orderId> // Auto-incremented

Order Date: <orderDate>

Order Amount: <orderAmount> with one decimal place.

Each order's details should be displayed on a separate line.

If no orders are found for the provided customer\_id, the output should display:

No orders found for Customer ID: <customerId>

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 101

Output: Order Details for Customer ID: 101

Order ID: 1

Order Date: 2021-10-01

Order Amount: 250.5

### **Answer**

```
import java.sql.*;
import java.util.Scanner;

class SelectQueryWithPreparedStatement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String url = "jdbc:mysql://localhost/ri_db";
        String username = "test";
        String password = "test123";

        int customerId = scanner.nextInt();

        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {
            conn = DriverManager.getConnection(url, username, password);

            String query = "SELECT order_id, order_date, order_amount FROM orders
WHERE customer_id = ?";
            pstmt = conn.prepareStatement(query);
            pstmt.setInt(1, customerId);

            rs = pstmt.executeQuery();

            boolean found = false;
```

```

while (rs.next()) {
    if (!found) {
        System.out.println("Order Details for Customer ID: " + customerId);
        found = true;
    }

    System.out.println("Order ID: " + rs.getInt("order_id"));
    System.out.println("Order Date: " + rs.getDate("order_date"));
    System.out.println("Order Amount: " + String.format("%.1f",
rs.getDouble("order_amount")));
}

    if (!found) {
        System.out.println("No orders found for Customer ID: " + customerId);
    }
}

catch (SQLException e) {
    System.out.println("SQL Exception occurred: " + e.getMessage());
    e.printStackTrace();
} finally {
    try {
        if (rs != null) rs.close();
        if (pstmt != null) pstmt.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {
        System.out.println("Error closing resources: " + e.getMessage());
    }
}

    scanner.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 12\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 20

#### Section 1 : MCQ

1. What is Collection in Java?

**Answer**

A group of objects

**Status : Correct**

**Marks : 1/1**

2. What is the correct way to create an ArrayList in Java?

**Answer**

`ArrayList<String> list = new ArrayList<>();`

**Status : Correct**

**Marks : 1/1**

3. How can you access the first element of an ArrayList named as list?

**Answer**

list.get(0);

**Status : Correct**

**Marks : 1/1**

4. What will be the output of the following code?

```
import java.util.ArrayList;
```

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Integer> list = new ArrayList<>();  
        list.add(10);  
        list.add(20);  
        list.add(30);  
        System.out.println("Size of the list: " + list.size());  
    }  
}
```

**Answer**

Size of the list: 3

**Status : Correct**

**Marks : 1/1**

5. Which of these is not an interface in the Collections Framework?

**Answer**

Group

**Status : Correct**

**Marks : 1/1**

6. Which of the following interfaces restricts duplicate elements?

**Answer**

Set



**Status :** Correct

**Marks :** 1/1

7. Which of the below does not implement Map interface?

**Answer**

Vector

**Status :** Correct

**Marks :** 1/1

8. Which allows the storage of a null key and many null values?

**Answer**

HashMap

**Status :** Correct

**Marks :** 1/1

9. Which of the following statements about the HashMap class is not correct?

**Answer**

The HashMap maintains the order of its elements

**Status :** Correct

**Marks :** 1/1

10. How do you remove duplicates from the List?

**Answer**

```
HashSet<String> listToSet = new HashSet<String>(duplicateList);
```

**Status :** Correct

**Marks :** 1/1

11. What will happen if you add a null element to a TreeSet?

**Answer**

An exception occurs

**Status :** Correct

**Marks :** 1/1

12. How can you ensure reverse sorting in a TreeSet?

**Answer**

Provide a custom Comparator

**Status :** Correct

**Marks :** 1/1

13. What is the output of the given code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        set.add(10);
        set.add(20);
        set.add(10);
        System.out.println(set.size());
    }
}
```

**Answer**

2

**Status :** Correct

**Marks :** 1/1

14. What is the output of the given code?

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        LinkedHashSet<String> set = new LinkedHashSet<>();
        set.add("B");
        set.add("A");
        set.add("C");
        System.out.println(set);
    }
}
```

**Answer**

[B, A, C]

**Status :** Correct

**Marks :** 1/1

15. What is the output of the given code?

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> set = new TreeSet<>();
        set.add(30);
        set.add(10);
        set.add(20);
        System.out.println(set);
    }
}
```

**Answer**

[10, 20, 30]

**Status :** Correct

**Marks :** 1/1

16. Which of the following is true about TreeMap?

**Answer**

It maintains natural ordering

**Status :** Correct

**Marks :** 1/1

17. Which method retrieves the lowest key in a TreeMap?

**Answer**

firstKey()

**Status :** Correct

**Marks :** 1/1

18. What happens if two keys have the same hash code in a HashMap?

**Answer**

A linked list is used to store values with the same hash

**Status :** Correct

**Marks :** 1/1

19. What will be the output of the following code?

```
import java.util.*;
class Test {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("A", 10);
        map.put("B", 20);
        map.put("C", 30);
        System.out.println(map.get("D"));
    }
}
```

**Answer**

null

**Status :** Correct

**Marks :** 1/1

20. What will be the output of the following code?

```
import java.util.*;
class Example {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(null, "Hello");
        map.put(1, null);
        map.put(2, "World");
        System.out.println(map);
    }
}
```

}

**Answer**

{null=Hello, 1=null, 2=World}

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 12\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

#### Section 1 : COD

##### 1. Problem Statement:

Arjun is working on a project where he needs to find all possible combinations of prime numbers that sum up to a given target. The prime numbers in the combinations should be greater than a specific value P and less than or equal to a limit S.

Arjun needs to select exactly N prime numbers that sum up to S. Help him implement the task using a List.

##### ***Input Format***

The first line of input consists of an integer value, S, representing the target sum.

The second line of input consists of an integer value, N, representing the number of prime numbers to be selected.

The third line of input consists of an integer value, P, representing the lower bound for the prime numbers to be considered.

### **Output Format**

The output prints: A space-separated list of prime number combinations that sum up to S, where exactly N primes are chosen from the range greater than P and less than or equal to S.

For each valid combination of primes, print them on a new line, separated by a space.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 54

2

3

Output: 7 47

11 43

13 41

17 37

23 31

### **Answer**

```
class PrimeCombinations {  
  
    public static void processPrimeCombinations(int S, int N, int P) {  
        List<Integer> primes = new ArrayList<>();  
  
        // Collect primes: P < prime <= S  
        for (int i = P + 1; i <= S; i++) {  
            if (isPrime(i)) {  
                primes.add(i);  
            }  
        }  
    }  
}
```

```

List<Integer> current = new ArrayList<>();
findCombos(primes, S, N, 0, current);
}

private static void findCombos(List<Integer> primes, int target, int count, int
idx, List<Integer> current) {
    if (count == 0) {
        if (target == 0) {
            for (int i = 0; i < current.size(); i++) {
                System.out.print(current.get(i));
                if (i < current.size() - 1) System.out.print(" ");
            }
            System.out.println();
        }
        return;
    }

    if (target <= 0 || idx >= primes.size()) return;

    for (int i = idx; i < primes.size(); i++) {
        int prime = primes.get(i);
        if (prime > target) break;

        current.add(prime);
        findCombos(primes, target - prime, count - 1, i + 1, current);
        current.remove(current.size() - 1);
    }

    private static boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }
}

```

**Status :** Correct

**Marks :** 10/10



## 2. Problem Statement

Rahul is analyzing student grades in a university. He wants to find out the highest score obtained by each student in multiple subjects. The student names should be stored in a TreeMap so that the output is sorted alphabetically by student names automatically.

You are required to build a Java program that:

Uses a `TreeMap<String, Integer>` to store the highest score of each student. Updates the score if a higher score is encountered for the same student. Outputs the student names and their highest scores in sorted order.

You must use a `TreeMap` in the class named `GradeAnalyzer`.

### ***Input Format***

- The first line contains an integer `n`, the number of records.
- The next `n` lines contain the student's name followed by the score, separated by a space.

### ***Output Format***

- The first line prints: "Student Highest Scores:"
- Then print each student's name and highest score in the format:  
`<student_name>: <highest_score>`

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5

Alice 78

Bob 85

Alice 90

Charlie 70

Bob 88

Output: Student Highest Scores:

Alice: 90

Bob: 88

Charlie: 70

**Answer**

```
import java.util.*;

class GradeAnalyzer {
    private TreeMap<String, Integer> highestScores;

    public GradeAnalyzer() {
        highestScores = new TreeMap<>();
    }

    public void analyzeStudentGrades(List<String> records) {
        for (String record : records) {
            String[] parts = record.split(" ");
            String name = parts[0];
            int score = Integer.parseInt(parts[1]);

            // Update if student not present or current score is higher
            if (!highestScores.containsKey(name) || highestScores.get(name) < score)
            {
                highestScores.put(name, score);
            }
        }

        printResults();
    }

    private void printResults() {
        System.out.println("Student Highest Scores:");
        for (Map.Entry<String, Integer> entry : highestScores.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());
        List<String> records = new ArrayList<>();

        for (int i = 0; i < n; i++) {
```

```

        records.add(sc.nextLine());
    }

    GradeAnalyzer analyzer = new GradeAnalyzer();
    analyzer.analyzeStudentGrades(records);
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

In an e-commerce platform, you are tasked with analyzing customer reviews. Each review consists of a product ID and a rating. The goal is to calculate the average rating for each product and store the results in a TreeMap. The TreeMap will automatically sort the products by their product ID.

You need to build a Java program that:

Uses a TreeMap<Integer, Double> to store the product ID and its average rating. Reads the product IDs and ratings for multiple products. For each product ID, it computes the average of all its ratings. Outputs the product IDs and their corresponding average ratings in ascending order of product ID.

#### **Input Format**

- The first line contains an integer n, representing the number of reviews.
- The next 5 lines each line has product\_id and rating separated by a space

For example:

- 101 4.5    product\_id = 101, rating = 4.5
- 102 3.0    product\_id = 102, rating = 3.0

#### **Output Format**

- The first line of output should print: "Product ID and Average Rating:"

- Then print each product ID and its average rating in the following format:  
<product\_id>: <average\_rating>
- The output should be sorted by product\_id in ascending order.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5

101 4.5

102 3.0

101 5.0

102 4.5

103 2.5

Output: Product ID and Average Rating:

101: 4.75

102: 3.75

103: 2.50

### **Answer**

```
import java.util.*;
```

```
// You are using Java
```

```
class ProductAnalyzer {
```

```
    public void analyzeProductRatings(List<String> lines) {
```

```
        TreeMap<Integer, Double> sumMap = new TreeMap<>();
```

```
        TreeMap<Integer, Integer> countMap = new TreeMap<>();
```

```
        for (String line : lines) {
```

```
            String[] parts = line.split(" ");
```

```
            int productId = Integer.parseInt(parts[0]);
```

```
            double rating = Double.parseDouble(parts[1]);
```

```
            sumMap.put(productId, sumMap.getOrDefault(productId, 0.0) + rating);
```

```
            countMap.put(productId, countMap.getOrDefault(productId, 0) + 1);
```

```
        }
```

```
        System.out.println("Product ID and Average Rating:");
```

```
        for (Integer productId : sumMap.keySet()) {
```

```
            double avg = sumMap.get(productId) / countMap.get(productId);
```

```
            // Format to 2 decimal places
```

```
        System.out.printf("%d: %.2f%n", productId, avg);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());
        List<String> lines = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            lines.add(sc.nextLine());
        }

        ProductAnalyzer analyzer = new ProductAnalyzer();
        analyzer.analyzeProductRatings(lines);

        sc.close();
    }
}
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Misha is planning a birthday party and wants to create a list of guests for the party. She decided to use an ArrayList to store the names of the guests who were invited.

She writes a program that takes input for the number of guests and then reads and stores their names in an ArrayList. Finally, the program should display the list of guests invited.

##### ***Input Format***

The first line of input is an integer N, representing the number of guests Misha wants to invite.

The next N lines each of input consists of a string, representing the name of a guest.

### **Output Format**

The output displays the names of the guests, each on a new line.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 1

Sri

Output: Sri

### **Answer**

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// You are using Java
class GuestListManager {
    public static void manageGuestList(int n, Scanner sc) {
        ArrayList<String> guestList = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            String name = sc.nextLine();
            guestList.add(name);
        }

        // Display the list of guests invited
        for (String guest : guestList) {
            System.out.println(guest);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();
        GuestListManager.manageGuestList(n, sc);
    }
}
```

```
        sc.close();  
    }  
}
```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

Assist Pranitha in developing a program that takes an integer N as input, representing the number of names to be read. Then read N names and store them in an ArrayList. Finally, input a search string and output the frequency of that string in the list of names.

Note: Some parts of the code are provided as snippets, and you need to complete the remaining sections by writing the necessary code.

### **Input Format**

The first line of input consists of an integer N, representing the number of names to be read.

The following N lines consist of N names, as a string.

The last line consists of a string, representing the name to be searched.

### **Output Format**

The output prints a single integer, representing the frequency of the specified name in the given list.

If the specified name is not found, print 0.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5  
Alice  
Bob  
Ankit

Alice  
Pranitha  
Alice

Output: 2

**Answer**

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// You are using Java
public class Main {

    public static void processNames() {
        Scanner sc = new Scanner(System.in);

        ArrayList<String> nameList = new ArrayList<>();

        int n = sc.nextInt();
        sc.nextLine(); // consume newline after integer

        // Read N names
        for (int i = 0; i < n; i++) {
            nameList.add(sc.nextLine());
        }

        // Read search string
        String searchName = sc.nextLine();

        // Count frequency - case sensitive
        int count = 0;
        for (String name : nameList) {
            if (name.equals(searchName)) {
                count++;
            }
        }

        System.out.println(count);
        sc.close();
    }
}
```



```
public static void main(String[] args) {  
    processNames();  
}  
}
```

**Status :** Correct

**Marks : 10/10**

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 12\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : COD

#### 1. Problem Statement

Sanjay is working on a program to merge two sorted linked lists into a single sorted list using Java's LinkedList class from the Collections framework. Given two sorted linked lists, he wants to merge them while maintaining the sorted order.

Write a Java program that:

Reads two sorted linked lists. Merges them into a single sorted linked list. Prints the merged list in ascending order.

#### ***Input Format***

The first line contains an integer m (the size of the first linked list).

The second line contains m space-separated integers (sorted).

The third line contains an integer n (the size of the second linked list).

The fourth line contains n space-separated integers (sorted).

### **Output Format**

The output prints the merged linked list as space-separated integers.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 2

5 10

3

1 3 8

Output: 1 3 5 8 10

### **Answer**

```
import java.util.*;
```

```
class MergeUtility {
```

```
    public static LinkedList<Integer> readList(Scanner sc, int size) {
```

```
        LinkedList<Integer> list = new LinkedList<>();
```

```
        for (int i = 0; i < size; i++) {
```

```
            list.add(sc.nextInt());
```

```
        }
```

```
        return list;
```

```
    }
```

```
    public static LinkedList<Integer> mergeLists(LinkedList<Integer> list1,  
    LinkedList<Integer> list2) {
```

```
        LinkedList<Integer> merged = new LinkedList<>();
```

```
        int i = 0, j = 0;
```

```
        while (i < list1.size() && j < list2.size()) {
```

```
            if (list1.get(i) <= list2.get(j)) {
```

```
                merged.add(list1.get(i));
```

```
                i++;
```

```

    } else {
        merged.add(list2.get(j));
        j++;
    }
}

while (i < list1.size()) {
    merged.add(list1.get(i));
    i++;
}

while (j < list2.size()) {
    merged.add(list2.get(j));
    j++;
}

return merged;
}

public static void printList(LinkedList<Integer> list) {
    for (int i = 0; i < list.size(); i++) {
        System.out.print(list.get(i));
        if (i < list.size() - 1) {
            System.out.print(" ");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int m = sc.nextInt();
        LinkedList<Integer> list1 = MergeUtility.readList(sc, m);

        int n = sc.nextInt();
        LinkedList<Integer> list2 = MergeUtility.readList(sc, n);

        LinkedList<Integer> merged = MergeUtility.mergeLists(list1, list2);
        MergeUtility.printList(merged);

        sc.close();
    }
}

```

**Status : Correct**

**Marks : 10/10**

## 2. Problem Statement

Amit is analyzing a collection of email addresses in a database. He needs to organize and count the occurrence of each domain name (e.g., gmail.com, yahoo.com, etc.) in the email addresses. He wants to use a TreeMap to automatically store the domains in sorted order and count how many times each domain appears.

You are required to build a Java program that:

Uses a `TreeMap<String, Integer>` to store the domain names and count how many times each domain appears in the list of email addresses. Sorts the domains in ascending order automatically using the `TreeMap`. The input is a list of email addresses, and the program must extract the domain from each email address. The program should output the domains and their corresponding frequency.

You must use a `TreeMap` in the class named `EmailAnalyzer`.

### ***Input Format***

The first line of input contains an integer `n`, the number of email addresses.

The next `n` lines each contain an email address.

### ***Output Format***

The first line of output prints: "Domain Frequency:"

Then print each domain and its frequency in the format: "<domain>: <count>" per line.

Refer to the sample output for formatting specifications.

### Sample Test Case

Input: 3

john.doe@gmail.com

jane.doe@yahoo.com

alice.smith@gmail.com

Output: Domain Frequency:

gmail.com: 2

yahoo.com: 1

### Answer

```
import java.util.Scanner;  
import java.util.TreeMap;  
import java.util.Map;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        // Read the number of email addresses  
        if (sc.hasNextInt()) {  
            int n = sc.nextInt();  
            sc.nextLine();
```

```
        // TreeMap automatically sorts keys in ascending (alphabetical) order  
        TreeMap<String, Integer> domainMap = new TreeMap<>();
```

```
        for (int i = 0; i < n; i++) {  
            if (sc.hasNextLine()) {  
                String email = sc.nextLine().trim();  
                String[] parts = email.split("@");  
                if (parts.length == 2) {  
                    String domain = parts[1];
```

```
                    // Increment the count for this domain  
                    domainMap.put(domain, domainMap.getOrDefault(domain, 0) + 1);
```

```
                }  
            }  
        }
```

```
        // Output the results  
        System.out.println("Domain Frequency:");
```

```

int count = 0;
int totalEntries = domainMap.size();
for (Map.Entry<String, Integer> entry : domainMap.entrySet()) {
    count++;
    System.out.print(entry.getKey() + ": " + entry.getValue());
    if (count < totalEntries) {
        System.out.println();
    }
}
}
}
}
sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Raman, a computer science teacher, is responsible for registering students for his programming class. To streamline the registration process, he wants to develop a program that stores students' names and allows him to retrieve a student's name based on their index in the list.

Raman has decided to use an ArrayList to store the names of students, as it provides efficient dynamic resizing and indexing.

Write a program that enables Raman to input the names of students and fetch a student's name using the specified index. If the entered index is invalid, the program should return an appropriate message.

#### ***Input Format***

The first line of input consists of an integer  $n$ , representing the number of students to register.

The next  $n$  lines of input consist of the names of each student, one by one.

The last line of input is an integer, representing the index (0-indexed) of the element to retrieve.

#### ***Output Format***

If the index is valid (within the bounds of the ArrayList), print "Element at index [index]: " followed by the element (student name as string).

If the index is invalid, print "Invalid index".

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 5

Alice

Bob

Ankit

Alice

Prajit

2

Output: Element at index 2: Ankit

### **Answer**

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
class NameManager {
```

```
    private ArrayList<String> nameList;
```

```
    public NameManager() {  
        nameList = new ArrayList<>();  
    }
```

```
    public void addName(String name) {  
        nameList.add(name);  
    }
```

```
    public String getNameAtIndex(int index) {  
        if (index >= 0 && index < nameList.size()) {  
            return nameList.get(index);  
        } else {  
            return null;  
        }  
    }
```



```

}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        NameManager manager = new NameManager();

        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < n; i++) {
            String name = sc.nextLine();
            manager.addName(name);
        }

        int index = sc.nextInt();
        String result = manager.getNameAtIndex(index);

        if (result != null) {
            System.out.println("Element at index " + index + ": " + result);
        } else {
            System.out.println("Invalid index");
        }

        sc.close();
    }
}

```

**Status :** Correct

**Marks : 10/10**

#### 4. Problem Statement

Sarah, a warehouse manager, is managing a list of product names in her store's inventory system. She needs to perform basic operations like adding (inserting) new products, removing products that are sold out or discontinued, displaying all the products in stock, and searching for a specific product in the inventory list.

Sarah's goal is to manage the inventory using a list of product names (strings). The system allows her to perform the following operations using ArrayList:

Insert a Product: Sarah adds a new product to the inventory. Delete a Product: Sarah removes a product from the inventory when it's sold or discontinued. Display the Inventory: Sarah checks all the products currently available in the inventory. Search for a Product: Sarah searches for a specific product in the inventory to check if it's available.

### ***Input Format***

The input consists of multiple space-separated values representing different operations on a product list. Each operation follows a specific format:

- 1 <product\_name> - Adds <product\_name> to the product list.
- 2 <product\_name> - Removes <product\_name> from the product list if it exists.
- 3 - Print all products currently on the list.
- 4 <product\_name> - Checks if <product\_name> exists in the list.

### ***Output Format***

The output displays,

For (choice 1) prints, " <item> has been added to the list."

For (choice 2) prints, " <item> has been removed from the list."

For (choice 3) prints, "Items in the list:" followed by each item in the list on a new line, or "The list is empty." if the list is empty.

For (choice 4) prints, " <item> is found in the list." or " <item> not found in the list."

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 1 apple 1 banana 2 apple 3 4 apple

Output: apple has been added to the list.

banana has been added to the list.

apple has been removed from the list.

Items in the list:

banana  
apple not found in the list.

**Answer**

```
import java.util.ArrayList;
import java.util.Scanner;

class StringListOperations {

    public static void processCommands(String input) {
        ArrayList<String> productList = new ArrayList<>();
        Scanner sc = new Scanner(input);

        while (sc.hasNext()) {
            int choice = sc.nextInt();

            if (choice == 1) {
                String product = sc.next();
                productList.add(product);
                System.out.println(product + " has been added to the list.");

            } else if (choice == 2) {
                String product = sc.next();
                if (productList.contains(product)) {
                    productList.remove(product);
                    System.out.println(product + " has been removed from the list.");
                }

            } else if (choice == 3) {
                if (productList.isEmpty()) {
                    System.out.println("The list is empty.");
                } else {
                    System.out.println("Items in the list:");
                    for (int i = 0; i < productList.size(); i++) {
                        System.out.println(productList.get(i));
                    }
                }

            } else if (choice == 4) {
                String product = sc.next();
                if (productList.contains(product)) {
                    System.out.println(product + " is found in the list.");
                } else {
```

```
        System.out.println(product + " not found in the list.");
    }
}
}
}
sc.close();
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();
        StringListOperations.processCommands(input);
        sc.close();
    }
}
```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 12\_CY

Attempt : 1

Total Mark : 30

Marks Obtained : 29

### Section 1 : COD

#### 1. Problem Statement

Mesa, a store manager, needs a program to manage inventory items. Define a class ItemType with private attributes for name, deposit, and cost per day. Create an ArrayList in the Main class to store ItemType objects, allowing input and display.

Note: Use "%-20s%-20s%-20s" for formatting output in tabular format, display double values with 1 decimal place.

#### ***Input Format***

The first line of input consists of an integer n, representing the number of items.

For each of the n items, there are three lines:

1. The name of the item (a string)

2. The deposit amount (a double value)
3. The cost per day (a double value)

### **Output Format**

The output prints a formatted table with columns for name, deposit and cost per day.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 3  
Laptop  
10000.0  
250.0  
Light  
1000.0  
50.0  
Fan  
1000.0  
100.0

| Output: Name | Deposit | Cost Per Day |
|--------------|---------|--------------|
| Laptop       | 10000.0 | 250.0        |
| Light        | 1000.0  | 50.0         |
| Fan          | 1000.0  | 100.0        |

### **Answer**

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class ItemType {
    private String name;
    private double deposit;
    private double costPerDay;

    public ItemType(String name, double deposit, double costPerDay) {
        this.name = name;
        this.deposit = deposit;
        this.costPerDay = costPerDay;
    }
}
```

```

    }

    public String getName() {
        return name;
    }

    public double getDeposit() {
        return deposit;
    }

    public double getCostPerDay() {
        return costPerDay;
    }
}

// InventoryManager to process
class InventoryManager {
    public static void processInventory(Scanner sc) {
        int n = sc.nextInt();
        List<ItemType> items = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            sc.nextLine(); // consume leftover newline
            String name = sc.nextLine();
            double deposit = sc.nextDouble();
            double costPerDay = sc.nextDouble();
            items.add(new ItemType(name, deposit, costPerDay));
        }

        // Print header
        System.out.printf("%-20s%-20s%-20s\n", "Name", "Deposit", "CostPerDay");

        // Print each item
        for (ItemType item : items) {
            System.out.printf("%-20s%-20.1f%-20.1f\n",
                item.getName(),
                item.getDeposit(),
                item.getCostPerDay());
        }
    }
}

public class Main {

```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    InventoryManager.processInventory(sc);  
    sc.close();  
}  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

John is organizing a fruit festival, and the quantities of various fruits are stored in a HashMap where fruit names are keys and quantities are values.

Help him develop a program to find the total quantity of fruits for the festival by summing up the values in the HashMap.

### ***Input Format***

The input consists of fruit quantities in the format 'fruitName:quantity', where fruitName is the name of the fruit(a string), and quantity is a double value representing the quantity.

The input is terminated by entering "done".

### ***Output Format***

The output prints a double value, representing the sum of values in the HashMap, rounded off to two decimal places.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are entered, print "Invalid format".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: Banana:15.2

Orange:56.3



Mango:47.3

done

Output: 118.80

### **Answer**

```
import java.util.Scanner;
import java.util.Map;
import java.util.HashMap;

class ValueProcessor {
    public static Map<String, Double> readValues(Scanner scanner) {
        Map<String, Double> map = new HashMap<>();
        while (scanner.hasNextLine()) {
            String input = scanner.nextLine().trim();
            if (input.equalsIgnoreCase("done")) {
                break;
            }

            // Check format: must contain only one :
            if (!input.contains(":") || input.indexOf(":") != input.lastIndexOf(":")) {
                System.out.println("Invalid format");
                return null;
            }
            String[] parts = input.split(":");
            if (parts.length != 2) {
                System.out.println("Invalid format");
                return null;
            }

            String fruit = parts[0].trim();
            String valueStr = parts[1].trim();
            if (fruit.length() < 1 || fruit.length() > 20 || !fruit.matches("[a-zA-Z]+")) {
                System.out.println("Invalid format");
                return null;
            }

            try {
                double value = Double.parseDouble(valueStr);
                if (value < 1.0 || value > 100.0) {
                    System.out.println("Invalid input");
                    return null;
                }
            }
        }
    }
}
```

```

        map.put(fruit, value);
    } catch (NumberFormatException e) {
        System.out.println("Invalid input");
        return null;
    }
}
return map;
}
public static double calculateSum(Map<String, Double> map) {
    double sum = 0;
    for (double val : map.values()) {
        sum += val;
    }
    return sum;
}
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Map<String, Double> valueMap = ValueProcessor.readValues(scanner);
        if (valueMap != null) {
            double sum = ValueProcessor.calculateSum(valueMap);
            System.out.printf("%.2f\n", sum);
        }
        scanner.close();
    }
}

```

**Status :** Partially correct

**Marks :** 9/10

### 3. Problem Statement

David is managing an employee database where each employee has a unique ID, name, and department. He wants to ensure that duplicate employee IDs are not added to the system. Implement a Java program that allows adding employees to the system, displaying all employees, and checking if an employee exists based on the given ID.

Implement a class EmployeeDatabase that contains a HashSet to store

employee records. The Employee class should be a user-defined object containing employee details. The main class should handle user operations and interact with the EmployeeDatabase class.

### ***Input Format***

The first line contains an integer  $n$  representing the number of employees to be added.

The next  $n$  lines follow, each containing:

1. An integer `employee_id`
2. A string `name`
3. A string `department`

The next line contains an integer  $m$  representing the number of queries.

The next  $m$  lines follow, each containing an employee ID to check for existence.

### ***Output Format***

The output prints a list of all employees added in the format:

"ID: <employee\_id>, Name: <name>, Department: <department>"

For each query, output "Employee exists" if the ID is found, otherwise "Employee not found".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 3

101 John IT

102 Alice HR

103 Bob Finance

2

101

104

Output: ID: 101, Name: John, Department: IT

ID: 102, Name: Alice, Department: HR

ID: 103, Name: Bob, Department: Finance  
Employee exists  
Employee not found

**Answer**

```
import java.util.*;

// You are using Java
class Employee {
    private int id;
    private String name;
    private String department;

    public Employee(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Department: " + department;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Employee employee = (Employee) obj;
        return id == employee.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

class EmployeeDatabase {
    private HashSet<Employee> employees;

    public EmployeeDatabase() {
        employees = new HashSet<>();
    }
}
```

```

    public void addEmployee(int id, String name, String department) {
        Employee emp = new Employee(id, name, department);
        employees.add(emp);
    }

    public void displayEmployees() {
        for (Employee emp : employees) {
            System.out.println(emp);
        }
    }

    public boolean checkEmployee(int id) {
        // Check using a dummy Employee with same id
        return employees.contains(new Employee(id, "", ""));
    }
}

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        EmployeeDatabase db = new EmployeeDatabase();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            String department = sc.next();
            db.addEmployee(id, name, department);
        }
        db.displayEmployees();
        int m = sc.nextInt();
        for (int i = 0; i < m; i++) {
            int id = sc.nextInt();
            if (db.checkEmployee(id))
                System.out.println("Employee exists");
            else
                System.out.println("Employee not found");
        }
        sc.close();
    }
}

```

**Status :** Correct

**Marks :** 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 13\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 20

#### Section 1 : MCQ

1. What type of object is returned on completion of a test?

**Answer**

org.junit.runner.Result

**Status : Correct**

**Marks : 1/1**

2. JUnit test methods must compulsorily return what value?

**Answer**

void

**Status : Correct**

**Marks : 1/1**

3. Which of the following checks if the object is NOT NULL?

**Answer**

`void assertNotNull(Object obj)`

**Status :** Correct

**Marks :** 1/1

4. Which of the following checks if the condition is FALSE?

**Answer**

`void assertFalse(boolean condition)`

**Status :** Correct

**Marks :** 1/1

5. Which of the following checks if the condition is TRUE?

**Answer**

`void assertTrue(boolean condition)`

**Status :** Correct

**Marks :** 1/1

6. A primitive or object can be checked to be equal by calling \_\_\_\_\_.

**Answer**

`void assertEquals(boolean expected,boolean actual)`

**Status :** Correct

**Marks :** 1/1

7. Which methods cannot be tested by the JUnit test class?

**Answer**

Private methods

**Status :** Correct

**Marks :** 1/1

8. JUnit runners are available in which package?

**Answer**

org.junit.runners

**Status :** Correct

**Marks :** 1/1

9. The invocation of a method after all tests have been completed is specified by the \_\_\_\_\_ annotation.

**Answer**

@AfterClass

**Status :** Correct

**Marks :** 1/1

10. Which of the following is correct about Test Suite in JUnit?

**Answer**

All of the mentioned options

**Status :** Correct

**Marks :** 1/1

11. Which annotation implies that a method is a JUnit test case?

**Answer**

@org.junit.Test

**Status :** Correct

**Marks :** 1/1

12. The JUnit testing framework is an \_\_\_\_-source tool for Java developers.

**Answer**

Open

**Status :** Correct

**Marks :** 1/1

13. \_\_\_\_, After, and others are some of the classes and interfaces



contained in the org.junit package for junit testing.

**Answer**

All of the mentioned options

**Status : Correct**

**Marks : 1/1**

14. Which annotation is used to indicate that a test method should throw a specific exception?

**Answer**

@Test(expected = Exception.class)

**Status : Correct**

**Marks : 1/1**

15. What is the purpose of the @Before annotation in JUnit?

**Answer**

Runs before each test method

**Status : Correct**

**Marks : 1/1**

16. Which method is used to compare arrays for equality in JUnit assertions?

**Answer**

assertArrayEquals()

**Status : Correct**

**Marks : 1/1**

17. Which of the following is NOT a valid JUnit assertion method?

**Answer**

assertDifferent()

**Status : Correct**

**Marks : 1/1**

18. What does the @Ignore annotation do in JUnit?

**Answer**

Skips execution of a test method

**Status :** Correct

**Marks :** 1/1

19. Which JUnit runner is commonly used for parameterized tests?

**Answer**

parameterized

**Status :** Correct

**Marks :** 1/1

20. Which JUnit annotation can be used to specify test execution order?

**Answer**

@TestMethodOrder

**Status :** Correct

**Marks :** 1/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 14\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 17

#### Section 1 : MCQ

1. What is a lambda expression in Java?

**Answer**

A way to define anonymous methods

**Status : Correct**

**Marks : 1/1**

2. What is the syntax for a basic lambda expression in Java?

**Answer**

(parameters) -> expression

**Status : Correct**

**Marks : 1/1**

3. Which of the following is a valid lambda expression in Java?

**Answer**

$(x) \rightarrow x * 2$

**Status : Wrong**

**Marks : 0/1**

4. Can a lambda expression have more than one parameter?

**Answer**

Yes, it can have multiple parameters

**Status : Correct**

**Marks : 1/1**

5. What is the return type of a lambda expression in Java?

**Answer**

The return type is inferred from the context

**Status : Correct**

**Marks : 1/1**

6. Which functional interface is commonly used with lambda expressions for comparisons?

**Answer**

Comparator

**Status : Correct**

**Marks : 1/1**

7. Can a lambda expression in Java have a body with multiple statements?

**Answer**

Yes, if the statements are enclosed in curly braces

**Status : Correct**

**Marks : 1/1**

8. What is the correct lambda syntax for multiplying two integers x and y?

**Answer**

`(x, y) -> x * y`

**Status : Correct**

**Marks : 1/1**

9. Which statement about lambda expressions is true?

**Answer**

They can replace short method implementations.

**Status : Correct**

**Marks : 1/1**

10. Which of the following operations returns the smallest element in a stream?

**Answer**

`min()`

**Status : Correct**

**Marks : 1/1**

11. Which of the following operations is an intermediate operation in Streams?

**Answer**

`map()`

**Status : Correct**

**Marks : 1/1**

12. Which terminal operation in Stream returns a boolean value based on the predicate?

**Answer**

`allMatch()`

**Status : Correct**

**Marks : 1/1**

13. What is the correct usage of the distinct() method?

**Answer**

It limits the stream to a specified number of elements.

**Status : Wrong**

**Marks : 0/1**

14. Which of the following methods is used for sorting in Streams?

**Answer**

sorted()

**Status : Correct**

**Marks : 1/1**

15. Which terminal operation in Streams is used to transform the elements into a collection?

**Answer**

collect()

**Status : Correct**

**Marks : 1/1**

16. What does the limit() operation in a stream do?

**Answer**

It limits the size of the stream to a specified number of elements.

**Status : Correct**

**Marks : 1/1**

17. Which of the following will NOT work with Streams in Java?

**Answer**

add()

**Status : Correct**

**Marks : 1/1**

18. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("apple", "banana", "cherry", "date",
"elderberry");
        list.stream().distinct().forEach(System.out::println);
    }
}
```

**Answer**

applebananacherrydateelderberry

**Status : Correct**

**Marks : 1/1**

19. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
        int result = list.stream().reduce(0, Integer::sum);
        System.out.println(result);
    }
}
```

**Answer**

15

**Status : Correct**

**Marks : 1/1**

20. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(10, 20, 30, 40);
        Optional<Integer> result = list.stream().max(Integer::compare);
    }
}
```

```
} System.out.println(result.get());  
}  
}
```

**Answer**

10

**Status :** Wrong

**Marks :** 0/1



# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 14\_MCQ

Attempt : 1

Total Mark : 20

Marks Obtained : 17

#### Section 1 : MCQ

1. What is a lambda expression in Java?

**Answer**

A way to define anonymous methods

**Status : Correct**

**Marks : 1/1**

2. What is the syntax for a basic lambda expression in Java?

**Answer**

(parameters) -> expression

**Status : Correct**

**Marks : 1/1**

3. Which of the following is a valid lambda expression in Java?

**Answer**

$(x) \rightarrow x * 2$

**Status : Wrong**

**Marks : 0/1**

4. Can a lambda expression have more than one parameter?

**Answer**

Yes, it can have multiple parameters

**Status : Correct**

**Marks : 1/1**

5. What is the return type of a lambda expression in Java?

**Answer**

The return type is inferred from the context

**Status : Correct**

**Marks : 1/1**

6. Which functional interface is commonly used with lambda expressions for comparisons?

**Answer**

Comparator

**Status : Correct**

**Marks : 1/1**

7. Can a lambda expression in Java have a body with multiple statements?

**Answer**

Yes, if the statements are enclosed in curly braces

**Status : Correct**

**Marks : 1/1**

8. What is the correct lambda syntax for multiplying two integers x and y?

**Answer**

`(x, y) -> x * y`

**Status : Correct**

**Marks : 1/1**

9. Which statement about lambda expressions is true?

**Answer**

They can replace short method implementations.

**Status : Correct**

**Marks : 1/1**

10. Which of the following operations returns the smallest element in a stream?

**Answer**

`min()`

**Status : Correct**

**Marks : 1/1**

11. Which of the following operations is an intermediate operation in Streams?

**Answer**

`map()`

**Status : Correct**

**Marks : 1/1**

12. Which terminal operation in Stream returns a boolean value based on the predicate?

**Answer**

`allMatch()`

**Status : Correct**

**Marks : 1/1**

13. What is the correct usage of the distinct() method?

**Answer**

It limits the stream to a specified number of elements.

**Status : Wrong**

**Marks : 0/1**

14. Which of the following methods is used for sorting in Streams?

**Answer**

sorted()

**Status : Correct**

**Marks : 1/1**

15. Which terminal operation in Streams is used to transform the elements into a collection?

**Answer**

collect()

**Status : Correct**

**Marks : 1/1**

16. What does the limit() operation in a stream do?

**Answer**

It limits the size of the stream to a specified number of elements.

**Status : Correct**

**Marks : 1/1**

17. Which of the following will NOT work with Streams in Java?

**Answer**

add()

**Status : Correct**

**Marks : 1/1**

18. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("apple", "banana", "cherry", "date",
"elderberry");
        list.stream().distinct().forEach(System.out::println);
    }
}
```

**Answer**

applebananacherrydateelderberry

**Status : Correct**

**Marks : 1/1**

19. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
        int result = list.stream().reduce(0, Integer::sum);
        System.out.println(result);
    }
}
```

**Answer**

15

**Status : Correct**

**Marks : 1/1**

20. What is the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(10, 20, 30, 40);
        Optional<Integer> result = list.stream().max(Integer::compare);
    }
}
```

```
} System.out.println(result.get());  
}  
}
```

**Answer**

10

**Status :** Wrong

**Marks :** 0/1

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 14\_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

### Section 1 : COD

#### 1. Problem Statement

Abi is working on a text analysis project where she needs to categorize words based on their length.

Words that have three or fewer characters are considered "Short," while Words with more than three characters are classified as "Long."

Create a program that takes a sentence as input, analyzes each word, and outputs a list indicating whether each word is "Short" or "Long," using lambda expressions for the categorization.

#### ***Input Format***

The input consists of a string, representing a sentence that Abi wants to analyze.

#### ***Output Format***

The output displays a string with each word categorized as "Short" or "Long," separated by spaces.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: Cat Dog Rat

Output: Short Short Short

### **Answer**

```
// You are using Java
import java.util.*;
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read full sentence
        String sentence = sc.nextLine().trim();

        // Split into words
        String[] words = sentence.split("\\s+");

        // Lambda expression to categorize words
        Function<String, String> categorize = w -> (w.length() <= 3) ? "Short" : "Long";

        // Process and print result
        StringBuilder result = new StringBuilder();
        for (String word : words) {
            result.append(categorize.apply(word)).append(" ");
        }

        // Print output (trim to remove trailing space)
        System.out.print(result.toString().trim());
    }
}
```

**Status : Correct**

**Marks : 10/10**



## 2. Problem Statement

### Problem Statement

Sophia, a data analyst, is studying experimental results collected from various lab sensors. Each sensor provides a list of numeric readings, and Sophia wants to calculate the average of these readings to analyze consistency.

She decides to use lambda expressions and the Function functional interface to compute the average of all the recorded values efficiently.

### Your Task

Write a Java program that:

Reads the total number of measurements. Reads all the measurement values as doubles. Uses a `Function<double[], Double>` lambda expression to calculate the average value. Displays the final average, formatted to two decimal places.

### ***Input Format***

The first line of input consists of an integer N, representing the number of measurements.

The second line contains N space-separated double values.

### ***Output Format***

Print the average of the entered values, rounded to two decimal places.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 6

2.2 1.2 5.4 4.6 2.9 55.7

Output: 12.00

### ***Answer***

// You are using Java

```

import java.util.*;
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read number of measurements
        int n = sc.nextInt();

        // Read values
        double[] values = new double[n];
        for (int i = 0; i < n; i++) {
            values[i] = sc.nextDouble();
        }

        // Lambda expression to calculate average
        Function<double[], Double> average = arr -> {
            double sum = 0;
            for (double v : arr) {
                sum += v;
            }
            return sum / arr.length;
        };

        // Compute and print result
        double result = average.apply(values);
        System.out.printf("%.2f", result);
    }
}

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Sharmi is working on a project that involves analyzing experimental data. As part of her analysis, she needs to determine the length of the longest and shortest strings from the given set of words.

Create a Java program that allows Sharmi to check and display the length

of the longest and shortest words from the given array.

Note: Use lambda expressions

### ***Input Format***

The first line of input consists of an integer N, representing the size of the array.

The next N line consists of strings, representing the elements of the array.

### ***Output Format***

The first line of output displays an integer representing the length of the longest string.

The second line displays an integer representing the length of the shortest string.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 5

red

blue

black

green

white

Output: 5

3

### ***Answer***

```
// You are using Java
```

```
import java.util.*;
```

```
import java.util.function.Function;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        // Read number of strings
```

```

int n = sc.nextInt();

String[] words = new String[n];
for (int i = 0; i < n; i++) {
    words[i] = sc.next();
}

// Lambda to find longest length
Function<String[], Integer> longest = arr -> {
    int max = arr[0].length();
    for (String s : arr) {
        if (s.length() > max) {
            max = s.length();
        }
    }
    return max;
};

// Lambda to find shortest length
Function<String[], Integer> shortest = arr -> {
    int min = arr[0].length();
    for (String s : arr) {
        if (s.length() < min) {
            min = s.length();
        }
    }
    return min;
};

int maxLen = longest.apply(words);
int minLen = shortest.apply(words);

// Print results (same line, space separated)
System.out.print(maxLen + " " + minLen);
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Sabrina is working on a project that involves analyzing a set of numbers. She wants to calculate the sum of even numbers from a given array using a lambda expression and a custom functional interface.

Do not use Predicate, Consumer, Function, Supplier, or Java Streams.

***Input Format***

First line: integer N (size of the array)

Second line: N space-separated integers (array elements)

***Output Format***

Single line: sum of all even numbers in the array

Refer to the sample output for formatting specifications.

***Sample Test Case***

Input: 3

29 37 45

Output: 0

***Answer***

```
// You are using Java
import java.util.*;
```

```
// Custom functional interface
interface EvenSum {
    int calculate(int[] arr);
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        // Read input
        int n = sc.nextInt();
        int[] arr = new int[n];
```

```

for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

// Lambda expression to calculate sum of even numbers
EvenSum sumEven = (array) -> {
    int sum = 0;
    for (int num : array) {
        if (num % 2 == 0) {
            sum += num;
        }
    }
    return sum;
};

// Output result
System.out.print(sumEven.calculate(arr));
}
}

```

**Status :** Correct

**Marks :** 10/10

## 5. Problem Statement

On his property, a farmer is sowing potato seeds. He thinks that everything about his plants' growth and good output comes from nature. If the weight of each potato is divisible by five, he can plant it in the ground for growth. Otherwise, change the seed to acquire a good harvest.

Write the array into a file named output.txt and create a method to monitor seed quality and assist the seed company in making more money.

### **Input Format**

The first line of input consists of an integer N, representing the number of seeds.

The second line consists of N space-separated integers, representing the values of their grams.

### **Output Format**

The output prints the type of seed that can be planted or changed for each of the N values in one of the following formats:

1. If the seed is changed, print "Change X gram seed" where X is the value of the grams.
2. If the seed is planted, print "Plant X gram seed" where X is the value of the grams.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 5

23 12 7 2 5

Output: Change 23 gram seed

Change 12 gram seed

Change 7 gram seed

Change 2 gram seed

Plant 5 gram seed

### **Answer**

```
import java.io.*;
import java.util.*;
```

```
// You are using Java
```

```
class SeedProcessor {
    public void writeArrayToFile(FileWriter fw, int[] arr) throws IOException {
        for (int i = 0; i < arr.length; i++) {
            fw.write(arr[i] + "");
            if (i < arr.length - 1) {
                fw.write(" ");
            }
        }
        fw.close();
    }
}
```

```
// Method to process seeds from file
```

```
public void processSeeds(FileReader fr) throws IOException {
    BufferedReader br = new BufferedReader(fr);
    String line = br.readLine();
}
```

```

String[] parts = line.split(" ");

for (String part : parts) {
    int weight = Integer.parseInt(part);
    if (weight % 5 == 0) {
        System.out.println("Plant " + weight + " gram seed");
    } else {
        System.out.println("Change " + weight + " gram seed");
    }
}
br.close();
}
}

class Main {
    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        FileWriter fw = new FileWriter("output.txt");
        SeedProcessor processor = new SeedProcessor();
        processor.writeArrayToFile(fw, arr);

        FileReader fr = new FileReader("output.txt");
        processor.processSeeds(fr);
    }
}

```

**Status :** Correct

**Marks :** 10/10



# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 14\_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 37.5

### Section 1 : COD

#### 1. Problem Statement

Rohit is given the task of developing a program that calculates the difference between two dates using the java.time class. The program should input two dates in the format "yyyy-mm-dd" and display the calculated difference in days.

The program makes use of the java.time.LocalDate class for date manipulation, java.time.format.DateTimeFormatter for parsing the input dates, and java.util.Scanner for reading the user input.

Implement a program to help Rohit that inputs two dates in the format "yyyy-mm-dd", and displays the calculated difference in days.

**Input Format**

The first line of input consists of the start date as a string, in the format 'yyyy-mm-dd'.

The second line of input consists of the end date as string, in the format 'yyyy-mm-dd'.

### **Output Format**

The output prints "X days", where X represents the calculated difference between the given two dates.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 2024-12-02

2000-12-02

Output: 8766 days

### **Answer**

```
import java.util.Scanner;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Check if there is input available to avoid NoSuchElementException
        if (sc.hasNextLine()) {
            String startDateStr = sc.nextLine().trim();
            if (sc.hasNextLine()) {
                String endDateStr = sc.nextLine().trim();
                DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
                LocalDate startDate = LocalDate.parse(startDateStr, formatter);
                LocalDate endDate = LocalDate.parse(endDateStr, formatter);

                long daysBetween = ChronoUnit.DAYS.between(startDate, endDate);
```

```
        // Use Math.abs() because "difference" is typically represented as a
        positive magnitude
        System.out.print(Math.abs(daysBetween) + " days");
    }
}
sc.close();
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Alex needs to plan a series of events and wants to determine how many complete weeks lie between two important dates. Write a program for him that calculates the number of complete weeks between two given dates, provided in the format yyyy-mm-dd.

Use java.time class to handle date parsing and calculations accurately.

### **Input Format**

The first line of input consists of the start date as a string, in the format 'yyyy-mm-dd'.

The second line consists of the end date as a string, in the format 'yyyy-mm-dd'.

### **Output Format**

The output prints "X weeks", where X represents the number of complete weeks between the given dates.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 2020-12-02

2000-12-02

Output: 1043 weeks

### Answer

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read input dates
        String startInput = sc.nextLine();
        String endInput = sc.nextLine();

        // Define formatter
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

        // Parse dates
        LocalDate startDate = LocalDate.parse(startInput, formatter);
        LocalDate endDate = LocalDate.parse(endInput, formatter);

        // Calculate total days (excluding both dates)
        long days = Math.abs(ChronoUnit.DAYS.between(startDate, endDate)) - 1;

        // Calculate complete weeks
        long weeks = days / 7;

        // Output result
        System.out.println(weeks + " weeks");

        sc.close();
    }
}
```

**Status :** Partially correct

**Marks :** 7.5/10

### 3. Problem Statement

In the mystical realm of programming, there exists a magical incantation to

reveal hidden words.

Elara, the skilled enchantress, wishes to summon a word using her spell and then reverse its characters to uncover its enchanted reflection.

Write a program that uses the predefined functional interface `Supplier<String>` and a lambda expression to:

Supply (generate) a string, and Display its reversed form.

### ***Input Format***

The input consist of single supplies string.

### ***Output Format***

The output prints the reversed version of the supplied string.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: Wizard!!

Output: !!draziW

### ***Answer***

```
import java.util.Scanner;
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read the input string from the user
        if (sc.hasNextLine()) {
            String input = sc.nextLine();

            // Using Supplier functional interface and a lambda expression
            // The lambda supplies the captured input string
            Supplier<String> stringSupplier = () -> input;
```

```

// Get the string from the supplier
String word = stringSupplier.get();

// Reverse the string using StringBuilder
String reversed = new StringBuilder(word).reverse().toString();

// Print the result - using print to avoid extra newline issues
System.out.print(reversed);
}

sc.close();
}
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Jeevitha is working on a project that involves analyzing a set of strings. In her exploration, she encounters scenarios where extracting the strings in alphabetical order is essential.

Create a program that implements a lambda expression to sort a list of strings in alphabetical order.

##### **Input Format**

The first line of input consists of an integer N, representing the size of the array.

The second line consists of N space-separated strings, representing the elements of the array.

##### **Output Format**

The first line of output displays "Sorted strings:".

The second line displays the strings in alphabetical order, separated by a space.

Refer to the sample output for formatting specifications.

### Sample Test Case

Input: 5

Banana apple orange grape berry

Output: Sorted strings:

apple Banana berry grape orange

### Answer

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (sc.hasNextInt()) {
            int n = sc.nextInt();
            ArrayList<String> strings = new ArrayList<>();

            // Reading N strings
            for (int i = 0; i < n; i++) {
                if (sc.hasNext()) {
                    strings.add(sc.next());
                }
            }
            Collections.sort(strings, (s1, s2) -> s1.compareToIgnoreCase(s2));

            // Matching the exact output format
            System.out.println("Sorted strings:");
            for (int i = 0; i < strings.size(); i++) {
                System.out.print(strings.get(i));
                System.out.print(" ");
            }
        }
        sc.close();
    }
}
```

Status : Correct

Marks : 10/10

# Vel Tech Group of Institutions

Name: DEGANIPALLE REVANTH REDDY Revanth Reddy

Email: vtu29201@veltech.edu.in

Roll no: vtu29201

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



## 2028\_NeoColab\_Java Programming\_S2-1L7

### Neocolab\_VTU\_JAVA\_Week 14\_CY

Attempt : 1

Total Mark : 30

Marks Obtained : 30

### Section 1 : COD

#### 1. Problem Statement

Abi is working on a text analysis project. She needs to categorize words based on their length:

Words with 3 or fewer characters "Short"

Words with more than 3 characters "Long"

Write a Java program that reads a sentence, analyzes each word, and prints a list showing whether each word is "Short" or "Long" using a custom functional interface and lambda expression.

Do not use Function, Consumer, Predicate, Supplier, or Java Streams.

**Input Format**



A single line containing a sentence (words separated by spaces).

### **Output Format**

A single line with each word categorized as "Short" or "Long", separated by spaces.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: I love my cat

Output: Short Long Short Short

### **Answer**

```
import java.util.Scanner;
interface WordAnalyzer {
    String analyze(String word);
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (sc.hasNextLine()) {
            String sentence = sc.nextLine();
            // Split the sentence into words based on spaces
            String[] words = sentence.split(" ");
            WordAnalyzer analyzer = (word) -> {
                if (word.length() <= 3) {
                    return "Short";
                } else {
                    return "Long";
                }
            };

            // Process each word and print the result
            for (int i = 0; i < words.length; i++) {
                if (!words[i].isEmpty()) {
                    System.out.print(analyzer.analyze(words[i]) + " ");
                }
            }
        }
    }
}
```

```
}  
}  
    sc.close();  
}  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Nethra is a researcher working on a project that involves analyzing experimental data. As part of her analysis, she needs to determine whether a given word is a palindrome or not.

Create a Java program that allows Nethra to input a word, and then check and display whether the entered word is a palindrome. Use lambda expressions to perform the palindrome check.

### ***Input Format***

The first line of input consists of a word.

### ***Output Format***

The output prints whether the given word is a palindrome or not in the following format:

"<input> is palindrome" or "<input> is not palindrome".

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: malayalam

Output: malayalam is palindrome

### ***Answer***

```
import java.util.Scanner;  
interface PalindromeChecker {
```

```

        boolean check(String str);
    }

    public class Main {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            if (sc.hasNext()) {
                String word = sc.next();
                PalindromeChecker checker = (str) -> {
                    String reversed = new StringBuilder(str).reverse().toString();
                    return str.equalsIgnoreCase(reversed);
                };

                // Use the checker and format the output
                if (checker.check(word)) {
                    System.out.print(word + " is palindrome");
                } else {
                    System.out.print(word + " is not palindrome");
                }
            }

            sc.close();
        }
    }

```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Emily, an analyst at a data processing firm, is tasked with cleaning up datasets to remove duplicate values from lists of integers.

Create a Java program that allows Emily to input a series of integers, with the program then utilizing a lambda expression to efficiently remove any duplicates.

#### **Input Format**

The first line of input consists of an integer N, representing the size of the array.

The second line consists of N space-separated integers, each denoting an array

element.

### **Output Format**

The output prints the array elements after removing the duplicates inside the square bracket separated by a comma and space.

Refer to the sample output for formatting specifications.

### **Sample Test Case**

Input: 15

1 2 3 4 3 2 1 2 3 4 4 4 5 5 6

Output: [1, 2, 3, 4, 5, 6]

### **Answer**

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (sc.hasNextInt()) {
            int n = sc.nextInt();
            List<Integer> inputList = new ArrayList<>();

            for (int i = 0; i < n; i++) {
                if (sc.hasNextInt()) {
                    inputList.add(sc.nextInt());
                }
            }

            LinkedHashSet<Integer> set = new LinkedHashSet<>();
            List<Integer> result = new ArrayList<>();
```

```
// Using forEach with a lambda to filter unique elements into the result list
inputList.forEach(element -> {
    if (!set.contains(element)) {
        set.add(element);
    }
});
```

```
        result.add(element);
    }
});
System.out.print(result.toString());
}
sc.close();
}
}
```

**Status :** Correct

**Marks :** 10/10